SQL Handbook — Vol 1 (Ch 1–11)

Queries · joins · window functions · data modeling · normalization · indexes · transactions · scaling · 40+ interview problems — PostgreSQL first

11 parts · one place to study: DesignGurus courses + classic books + the web, combined, beginner to intermediate

what it is | problem | how it works | trade-offs | real world | interview | self-check | TL;DR

Personal study notes. Course diagrams belong to DesignGurus. Not for redistribution.

Contents

1. **Chapter 1 — Databases and the Relational Model**
   - 1.1 Before databases: the file processing era
   - 1.2 What a database is
   - 1.3 What a DBMS does
   - 1.4 Inside a DBMS: the core components
   - 1.5 Database architectures
   - 1.6 The relational model
   - 1.7 RDBMS — relational database management systems
   - 1.8 Relational vs non-relational databases
   - 1.9 What SQL is
   - 1.10 Your private PostgreSQL server and psql
   - 1.11 The sample database
   - Key terms (chapter 1)
   - Chapter 1 TL;DR
2. **Chapter 2 — SELECT Basics: Reading, Filtering, Sorting and Paging**
   - 2.1 SELECT: choosing what comes back
   - 2.2 WHERE: filtering rows
   - 2.3 Operators
   - 2.4 Ranges, lists and patterns: BETWEEN, IN, LIKE/ILIKE, regex
   - 2.5 NULL and three-valued logic
   - 2.6 DISTINCT: removing duplicate rows
   - 2.7 ORDER BY: sorting results
   - 2.8 LIMIT, OFFSET and FETCH: returning a page
   - 2.9 How a SELECT is evaluated (logical processing order)
   - Putting it together
   - Chapter 2 TL;DR
3. **Chapter 3 — Functions and Expressions**
   - 3.1 Data types and conversions
   - 3.2 Number functions
   - 3.3 String functions
   - 3.4 Date and time functions
   - 3.5 Conditional expressions: CASE, COALESCE, NULLIF, GREATEST, LEAST
   - 3.6 Worked exercises
   - Chapter 3 TL;DR
4. **Chapter 4 — Aggregation: Summarising Many Rows**
   - 4.1 Aggregate functions
   - 4.2 GROUP BY
   - 4.3 HAVING
   - 4.4 Conditional aggregation: FILTER and CASE
   - 4.5 The logical order of query processing
   - 4.6 Subtotals: GROUPING SETS, ROLLUP and CUBE
   - Chapter 4 TL;DR
5. **Chapter 5 — Joins: Combining Tables**
   - 5.1 Join theory: the Cartesian product and CROSS JOIN
   - 5.2 The four main join types at a glance
   - 5.3 INNER JOIN
   - 5.4 LEFT OUTER JOIN
   - 5.5 RIGHT OUTER JOIN
   - 5.6 FULL OUTER JOIN
   - 5.7 SELF JOIN
   - 5.8 Semi-joins and anti-joins
   - 5.9 Join pitfalls
   - 5.10 USING, NATURAL JOIN, multi-table joins, and how joins are executed
   - Chapter 5 TL;DR
6. **Chapter 6 — Subqueries and Set Operations**
   - 6.1 Subqueries: queries inside queries
   - 6.2 Multi-row subqueries: IN and NOT IN
   - 6.3 Correlated subqueries
   - 6.4 EXISTS and NOT EXISTS
   - 6.5 ANY, SOME and ALL
   - 6.6 Table subqueries: derived tables and row subqueries
   - 6.7 LATERAL subqueries
   - 6.8 Set operations and UNION
   - 6.9 INTERSECT
   - 6.10 EXCEPT
   - 6.11 Precedence, sorting, and choosing the right tool
   - Chapter 6 TL;DR
7. **Chapter 7 — CTEs, Views and Recursion: Naming Queries and Walking Trees**
   - 7.1 Common table expressions (WITH)
   - 7.2 Data-modifying CTEs (PostgreSQL)
   - 7.3 Recursive CTEs: the mechanics
   - 7.4 Hierarchies: trees stored as parent pointers
   - 7.5 Graphs and cycles
   - 7.6 generate_series: making rows (PostgreSQL)
   - 7.7 Views
   - 7.8 Materialized views
   - 7.9 Choosing the right tool
   - Chapter 7 TL;DR
8. **Chapter 8 — Window Functions: Rankings, Running Totals and Row-to-Row Comparisons**
   - 8.1 What a window function is
   - 8.2 PARTITION BY: per-group values next to each row
   - 8.3 ORDER BY inside OVER: running calculations and the default frame
   - 8.4 Frames: ROWS, RANGE, GROUPS and EXCLUDE
   - 8.5 Ranking functions
   - 8.6 Value (offset) functions: LAG, LEAD, FIRST_VALUE, LAST_VALUE, NTH_VALUE

- 8.7 Running totals, moving averages and growth
- 8.8 Top-N per group and de-duplication
- 8.9 Gaps and islands
- 8.10 Named windows, FILTER, evaluation order and performance
- Chapter 8 TL;DR

9. **Chapter 9 — DDL, DML and Constraints: Building Tables and Changing Data Safely**
   - 9.1 Data types
   - 9.2 CREATE: databases, schemas and tables
   - 9.3 Keys and constraints
   - 9.4 Generated keys: identity, sequences, serial, UUID, generated columns
   - 9.5 ALTER: changing existing tables
   - 9.6 DROP and TRUNCATE (vs DELETE)
   - 9.7 Indexes (in brief)
   - 9.8 INSERT
   - 9.9 UPDATE
   - 9.10 DELETE
   - 9.11 Upsert: INSERT … ON CONFLICT, and MERGE
   - 9.12 Backup and restore
   - Chapter 9 TL;DR
10. **Chapter 10 — Data Modeling and ER Diagrams**
   - 10.1 What data modeling is
   - 10.2 The three levels of a data model
   - 10.3 The data modeling process
   - 10.4 Best practices in data modeling
   - 10.5 The Entity-Relationship (ER) model and its notations
   - 10.6 Entities and the five kinds of attribute
   - 10.7 Relationships: degree, cardinality and participation
   - 10.8 Attributes of relationships
   - 10.9 Weak entities
   - 10.10 Worked example — an employee-management ER diagram
   - 10.11 The relational model
   - 10.12 Relations, tuples, attributes — the precise vocabulary
   - 10.13 Keys in relational databases
   - 10.14 Relational integrity constraints
   - 10.15 Converting an ER diagram to tables
   - 10.16 Best practices for ER → relational mapping
   - Key terms
   - Chapter 10 TL;DR
11. **Chapter 11 — Normalization**
   - 11.1 Functional dependencies
   - 11.2 Types of functional dependencies
   - 11.3 Armstrong's inference rules
   - 11.4 Attribute closure and finding candidate keys
   - 11.5 Minimal (canonical) cover
   - 11.6 Anomalies — why normalization exists
   - 11.7 First normal form (1NF)
   - 11.8 Second normal form (2NF)
   - 11.9 Third normal form (3NF)
   - 11.10 Decomposition done right: lossless joins and dependency preservation
   - 11.11 Boyce–Codd normal form (BCNF)
   - 11.12 Fourth normal form (4NF) — independent multivalued facts
   - 11.13 Fifth normal form (5NF) — facts that need three tables
   - 11.14 Putting it together — a normalization procedure
   - 11.15 Exercises (with solutions)
   - 11.16 Denormalization — when and how to break the rules
   - Chapter 11 TL;DR

Chapter 1 — Databases and the Relational Model

Where do an application's facts live, and why is the answer usually "in a relational database"? This chapter covers the problems that existed before databases, what a database management system (DBMS) does, how it is built inside, how relational databases compare with NoSQL stores (databases that keep data in shapes other than tables, 1.8), and what SQL is. It ends with a working PostgreSQL server (PostgreSQL is a free, open-source relational database; a *server* is a background program that answers requests from other programs) and the **sample database** used from chapter 2 on.

Sections: 1.1 Life before databases · 1.2 What a database is · 1.3 What a DBMS does · 1.4 Inside a DBMS · 1.5 Database architectures · 1.6 The relational model · 1.7 RDBMS products · 1.8 Relational vs non-relational · 1.9 What SQL is · 1.10 Your private PostgreSQL and psql · 1.11 The sample database.

All code is in `code/`. Every output was pasted from a real run on the book's private PostgreSQL 14 server (1.10).

> **Dialect note.** This book writes PostgreSQL first. Where MySQL, SQLite or SQL Server (compared in 1.7) behave differently, a small table says so. Each product speaks its own *dialect* of SQL (1.9); the differences are mainly in functions (chapter 3), limits/paging (chapter 2) and data types (chapter 9).

1.1 Before databases: the file processing era

What it is

Until the late 1960s most business software stored data in **files owned by one program each** — payroll files for the payroll program, billing files for billing. The file layout (which bytes meant "name", which "salary") was written into the program's code, not stored with the file. This is a **file processing system**.

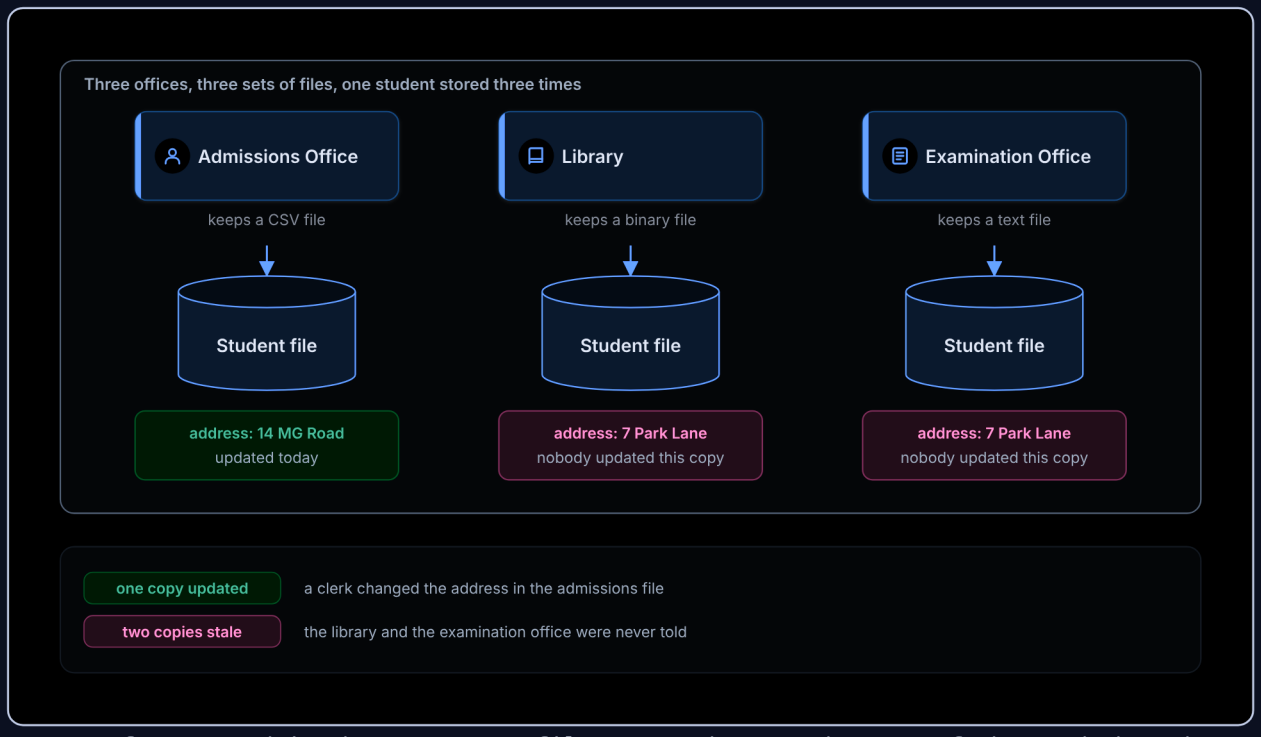


Figure — One fact stored in three separate files: an update reaches one of them and the other two go stale (course diagram).

Records were found by **sequential access** (read from the start until the record appears) or **indexed access** (a second, hand-maintained file mapping a key to a byte position — faster, but one more thing to

keep in sync).

The problem it solves — seven recurring failures

Each of these later became a DBMS feature.

#	Problem	What went wrong	What a DBMS does instead
1	**Redundancy → inconsistency**	A customer's address is copied into several files; an update reaches one copy, the copies disagree, and nothing notices.	Store each fact once; every program reads the same copy.
2	**Data isolation**	Files sit on different machines in different formats (CSV = text with commas between values; fixed-width binary = raw bytes at set positions); a cross-department question needs custom parsing code.	One store with one self-describing format.
3	**No ad-hoc questions**	No query language: "which customers owe more than ₹500?" means days of writing a new program.	A declarative query language (SQL — you state *what* you want, not *how*): a new question is a new query.
4	**Integrity rules buried in code**	"Marks must be 0–100" lives as an `if` in one program; any other writer can skip it.	Constraints (rules declared on the table) enforced for every writer.
5	**Unsafe concurrent access**	Two clerks both read "1 copy left" and both save 0: two copies go out, the count dropped once — a **lost update**. Locking the whole file blocks everybody.	Row-level locks (others wait while one program changes a row) and transactions (all-or-nothing groups of changes, 1.4).
6	**No crash recovery**	A transfer is two writes; power fails after the first and the money has left one account and reached none.	A log (append-only diary of every change) that lets the system undo or redo half-finished work.
7	**All-or-nothing security**	File permissions cover the whole file; a clerk who needs addresses can also read salaries.	Permissions per table, column and even row.

How the DBMS changed things

A DBMS is a layer of software that **every application must go through**; no program touches the stored files directly. Because every read and write passes through one place, that place can apply all seven fixes.

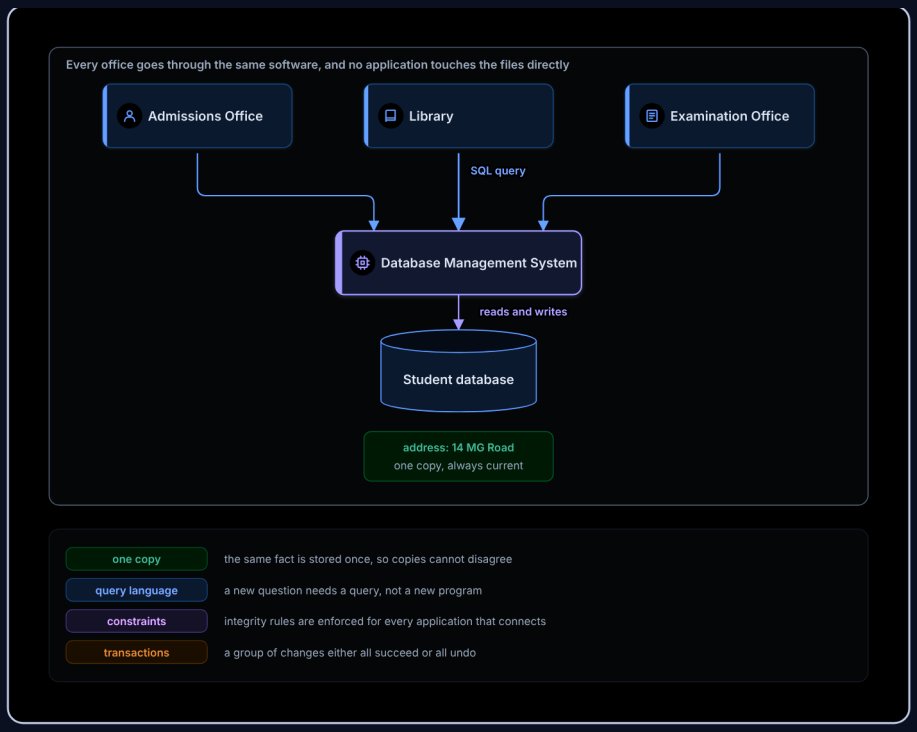


Figure — One shared copy of the data alongside its protecting rules (course diagram).

Year	Milestone
1960s	Navigational databases: IBM's IMS (hierarchical, a tree of records) and Charles Bachman's IDS / the CODASYL network model (records linked in a web). Programs followed stored pointers (saved disk addresses) from record to record.
1970	Edgar F. Codd (IBM) publishes *A Relational Model of Data for Large Shared Data Banks*.
1974–79	IBM's System R and SEQUEL (later SQL); UC Berkeley's Ingres; Oracle ships the first commercial SQL database (1979).
1986–87	SQL becomes an ANSI (1986), then ISO (1987) standard (US and international standards bodies). In 1986 UC Berkeley starts POSTGRES, the ancestor of PostgreSQL.
2000s	Open-source RDBMSs (relational DBMSs, 1.7) dominate the web (MySQL, PostgreSQL); SQLite runs inside phones and browsers.
late 2000s	"NoSQL" systems (key-value, document, wide-column, graph) for web-scale workloads.
2010s–now	Cloud-managed databases (run for you by a provider such as AWS) and distributed SQL ("NewSQL": relational databases spread over many machines).

Self-check

1. *Why does storing a fact in three files give wrong answers, not just wasted space?* → Updates reach some copies and not others, and nothing checks that they match.
2. *What is a lost update?* → Two programs read the same value, each computes a new value from it, and the second write overwrites the first.
3. *Which file-system problem does a transaction log solve?* → Recovery from a crash in the middle of a multi-step change.

1.2 What a database is

What it is

A **database** is an organised, persistent collection of related data that you can query and update, stored *along with a description of its own structure*. A phone's contact list and a bank's decade of

transactions both qualify; size changes the hardware, not the definition.

Property	Meaning	Example
Related	Everything describes one slice of reality, the *mini-world* (a school, a shop).	A shop database holds customers, products and orders — not holiday photos.
Self-describing	Stores **metadata** (data about data: table and column names, types, rules) in its **catalog**, also called the **data dictionary**.	Any program can ask "which columns does `employees` have?"
Purposeful	Built for specific people and their questions; requirements come before tables.	A hospital schema is shaped by what doctors, nurses and billing need.
Persistent	Survives program exit and machine restart.	An in-memory list that disappears on exit is not a database.

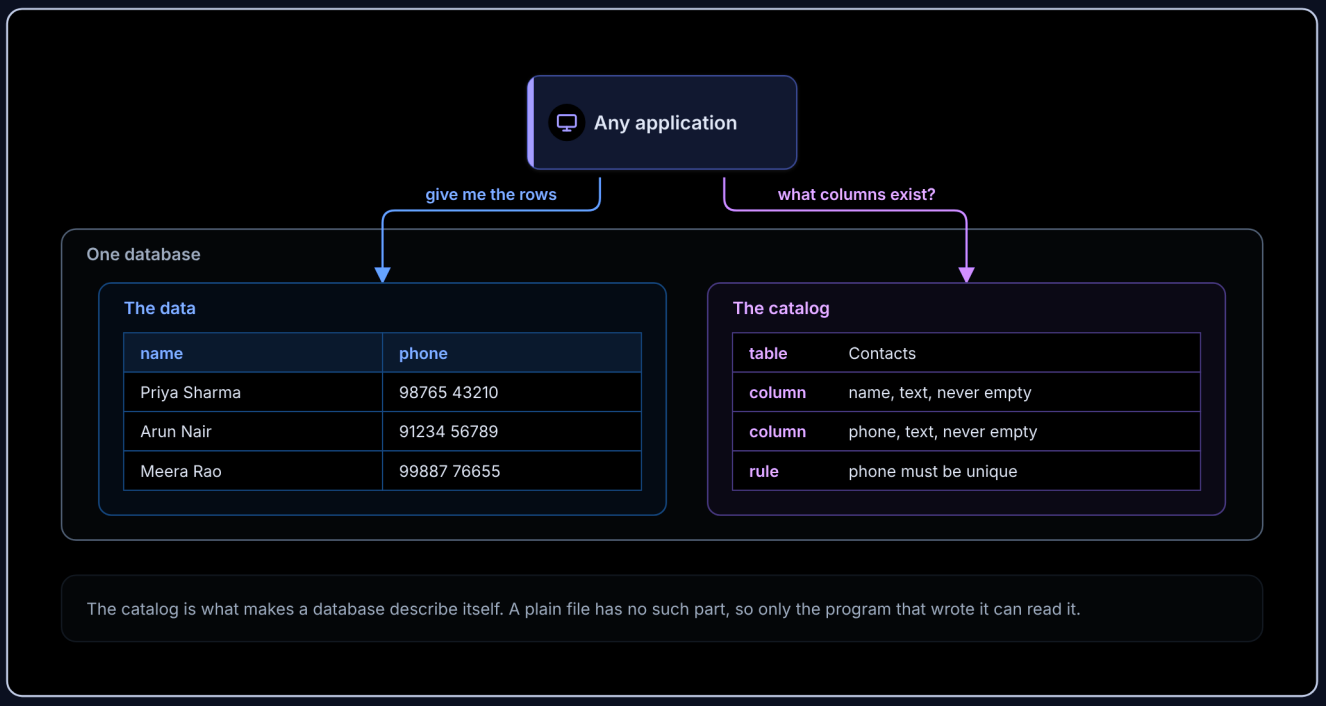


Figure — The catalog sits inside the database next to the data it describes (course diagram).

PostgreSQL exposes self-description through the standard `information_schema` views (a **view** is a saved query you read like a table — chapter 7) and its own `pg_catalog` tables.

```sql
-- code/02_catalog_metadata.sql (part)
SELECT column_name, data_type, is_nullable
FROM information_schema.columns
WHERE table_name = 'departments'
ORDER BY ordinal_position;
```

Output:

```
 column_name | data_type | is_nullable
-------------+-----------+-------------
 dept_id     | integer   | NO
 name        | text      | NO
 location    | text      | YES
 budget      | numeric   | NO
(4 rows)
```

```sql
SELECT conname AS constraint_name, contype AS kind, conrelid::regclass AS on_table
FROM pg_constraint
WHERE conrelid = 'employees'::regclass
ORDER BY conname;
```

Output:

```
     constraint_name      | kind | on_table
--------------------------+------+-----------
 employees_dept_id_fkey   | f    | employees
 employees_email_key      | u    | employees
 employees_manager_id_fkey | f    | employees
 employees_pkey           | p    | employees
 employees_salary_check   | c    | employees
(5 rows)
```

A **constraint** is a rule attached to a table that the DBMS checks on every write: `p` *primary key* (the column identifying each row), `f` *foreign key* (value must exist in another table; both defined in 1.6), `u` *unique* (no repeats), `c` *check* (a yes/no test every row must pass — here, salary positive). Because the rules live in the catalog, the DBMS enforces them for every program.

Database vs DBMS vs database system

Often mixed up, including in interviews.

Term	What it is	Example
Database	The stored data plus its description	the `company` tables in this book
DBMS (database management system)	The software that creates, stores, queries and protects databases	PostgreSQL, MySQL, Oracle
Database system	DBMS + databases + the applications that use them	PostgreSQL + `company` + the web shop that talks to it

Why it matters

Databases make large data fast to find, keep it accurate (integrity rules) and private (access control — who may see what; encryption — only key holders can read it), let many users work at once, and feed business reports. A good one is well structured, scalable (handles more data and users by adding hardware), reliable through failures and richly queryable. A shop must never lose a paid order, a bank never double-apply a transfer — each such requirement maps to a DBMS job in 1.3.

Self-check

1. *What is metadata and where is it kept?* → Data describing the data (tables, columns, types, constraints, permissions), in the catalog inside the database.
2. *Is MySQL a database?* → Strictly no — it is a DBMS; the tables you create inside it form a database.

1.3 What a DBMS does

What it is

A **DBMS** stands between applications and stored data: applications send requests ("give me these rows", "store this order"); the DBMS decides where bytes go on disk, how to find them, and how to keep them safe.

How it works — the six jobs

Job	Meaning	In SQL / PostgreSQL
Define	Describe tables, columns, types and rules; saved in the catalog	`CREATE TABLE`, `ALTER TABLE`
Construct	Physically write the data; you never choose file names or offsets	files under PostgreSQL's data directory
Manipulate	Read, add, change, remove data	`SELECT`, `INSERT`, `UPDATE`, `DELETE`
Share	Many users work on the same data at once without corrupting it	transactions, locks, MVCC (several row versions so readers and writers don't block each other; chapter 13)
Protect	*Security* (who may read/write what) and *failure protection* (correct after a crash)	`GRANT` / `REVOKE`, the write-ahead log (change diary saved to disk *before* the data, used to repair after a crash)
Maintain	Let schema and storage evolve for years without rewriting applications	add a column, add an index (below), move to faster disks

File processing systems managed the first three poorly and the last three not at all.

Who talks to a DBMS

Role	What they do
Application programs	Issue SQL for their users (a shopper clicks buttons, never writes SQL).
Database administrators (DBAs)	Users and permissions, backups, tuning and monitoring.
Database designers	Decide which tables exist and how they connect (chapter 10).
Analysts	Query the data themselves for reports.

Data independence

Data independence means a change at one level of the system's description does not force changes at the level above — why a 20-year-old report still runs after a hardware move. The **logical level** is what you query (tables, columns, views); the **physical level** is how bytes are laid out (data files, 8 kB pages — fixed-size chunks read and written as one unit — index trees).

Kind	You change…	…and this keeps working	Example
Physical data independence	how data is stored (indexes, files, disks, page layout)	the tables and every query	add an index; move to SSDs
Logical data independence	the tables (add a column or a table)	applications that don't use the changed part	add `phone` to `employees`; a report reading names is unaffected

> **Key term — index:** an extra, sorted structure beside a table that finds rows by a column's value without reading every row — like a book's index. Faster reads, at the cost of disk space and extra work on every write. PostgreSQL's default is a *B-tree*, a balanced tree that stays sorted as rows change (chapter 12). SQL names only logical objects, so the DBMS may reorganise the physical side freely.

The demo changes both levels and runs the same "application" query each time:

```sql
-- code/06_data_independence.sql
SELECT first_name, last_name, job_title FROM employees WHERE last_name = 'Verma';
EXPLAIN (COSTS OFF) SELECT first_name, last_name, job_title FROM employees WHERE last_name = 'Verma';

-- Physical change: add an index. The query text is unchanged.
CREATE INDEX employees_last_name_idx ON employees (last_name);
SET enable_seqscan = off;   -- the table is tiny; nudge the planner so we can SEE it use the index
EXPLAIN (COSTS OFF) SELECT first_name, last_name, job_title FROM employees WHERE last_name = 'Verma';
SELECT first_name, last_name, job_title FROM employees WHERE last_name = 'Verma';
RESET enable_seqscan;

-- Logical change: add a column the query does not use. Still unchanged.
ALTER TABLE employees ADD COLUMN phone text;
SELECT first_name, last_name, job_title FROM employees WHERE last_name = 'Verma';
```

Output (abridged):

```
 first_name | last_name | job_title
------------+-----------+-----------
 Rahul      | Verma     | Engineer
(1 row)

              QUERY PLAN
---------------------------------------
 Seq Scan on employees
   Filter: (last_name = 'Verma'::text)
(2 rows)

                      QUERY PLAN
------------------------------------------------------
 Index Scan using employees_last_name_idx on employees
   Index Cond: (last_name = 'Verma'::text)
(2 rows)

...
ALTER TABLE
 first_name | last_name | job_title
------------+-----------+-----------
 Rahul      | Verma     | Engineer
(1 row)
```

`EXPLAIN` shows the **plan** — the method chosen by the **planner** (PostgreSQL's name for the optimizer). Before the index it reads every row (`Seq Scan`); after, it jumps through the index (`Index Scan`). Result and query text never changed. (`enable_seqscan = off` is a teaching switch only; on 15 rows the full scan is rightly preferred. Chapter 12 covers plans.)

Self-check

1. *Name the six jobs of a DBMS.* → Define, construct, manipulate, share, protect, maintain.
2. *Adding an index without touching any query is an example of…?* → Physical data independence.

1.4 Inside a DBMS: the core components

What it is

A DBMS is several cooperating parts, each with one job. Knowing them explains why a query's first run is slower than its second, why a crash does not lose committed data, and why two users can update different rows at once.

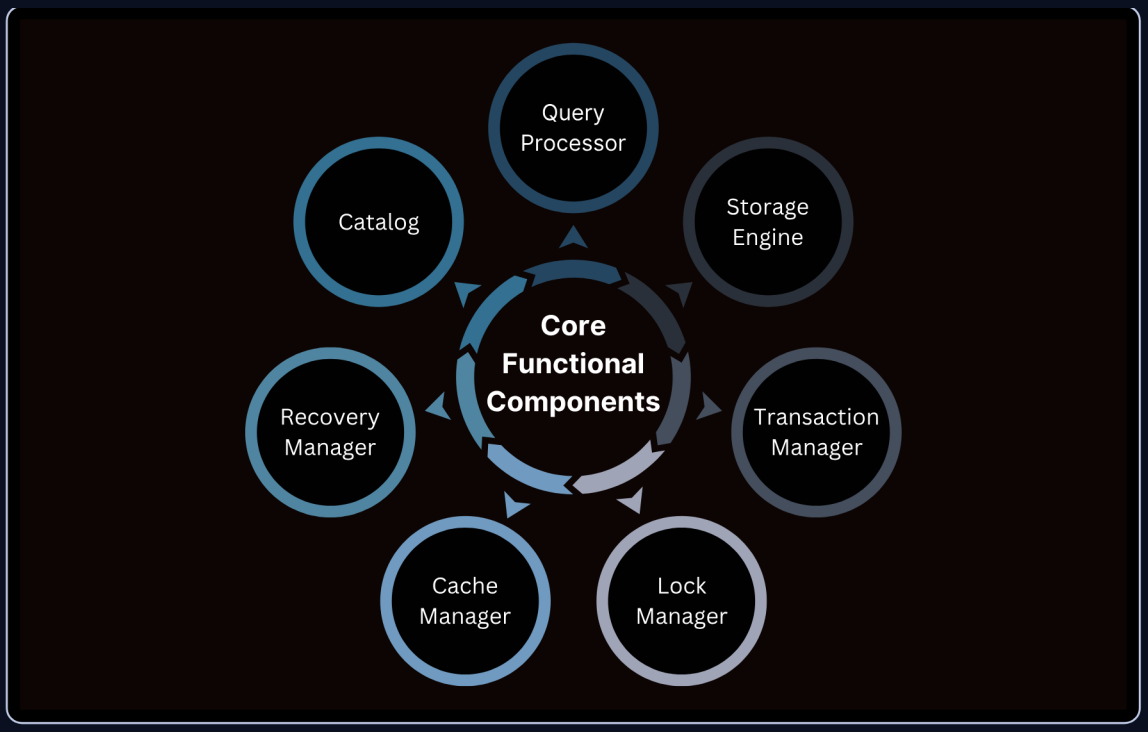
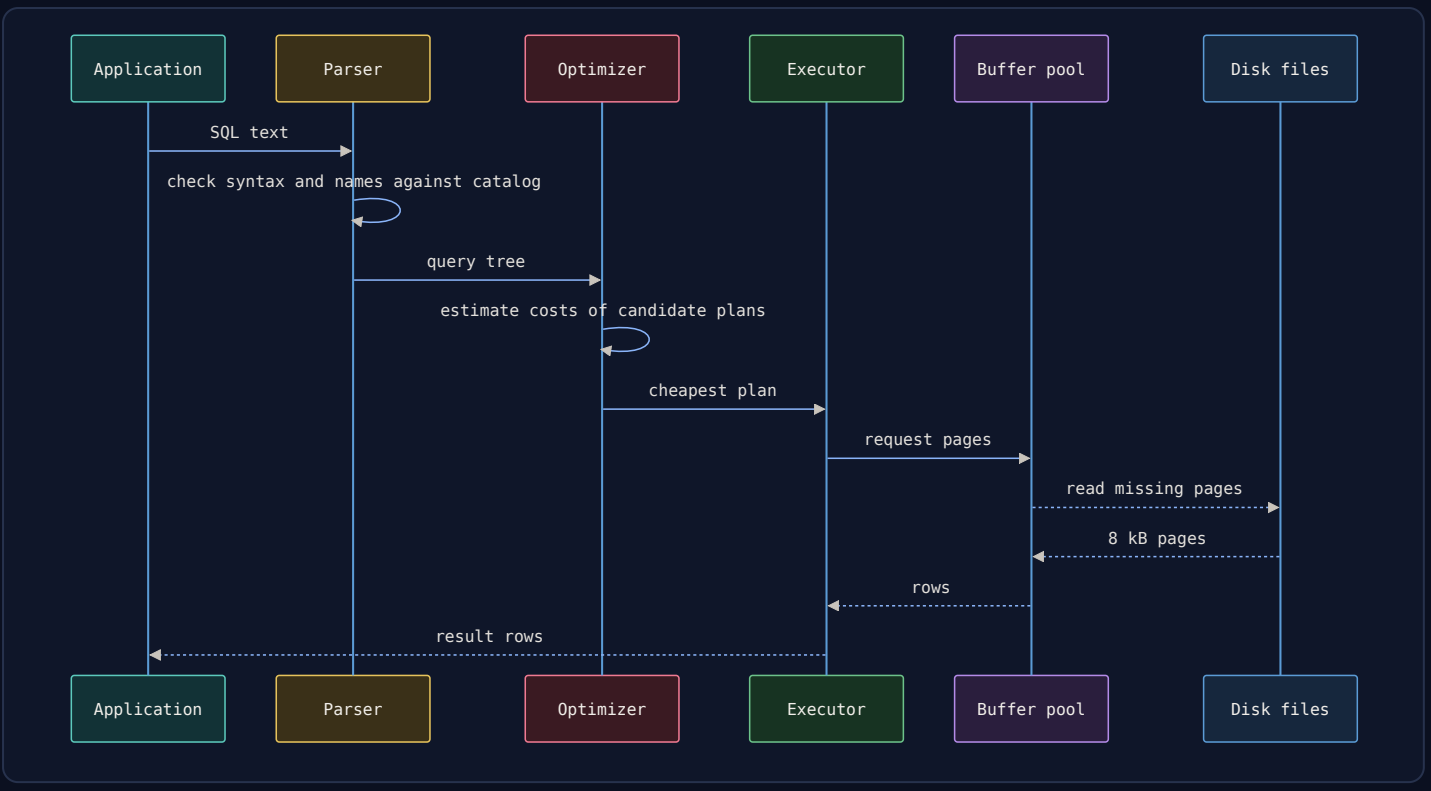


Figure — Query processor, storage engine, transaction manager, lock manager, cache manager, recovery manager and catalog (course diagram).

Component	Job	Sub-parts / tools
Query processor	Turn SQL text into results	**Parser** checks syntax and meaning (do these tables/columns exist? may you use them?) and builds an internal tree. **Optimizer** picks the cheapest of alternative plans by estimated cost. **Execution engine** runs the plan, asking storage for rows.
Storage engine (storage manager)	Store and fetch bytes	Data in fixed-size **pages** (blocks) inside files; maintains indexes; row store or column store. Examples: InnoDB (MySQL), WiredTiger (MongoDB), PostgreSQL's heap + index access methods.
Cache / buffer manager	Keep hot pages in memory	The **buffer pool** holds recently used pages; cached pages avoid disk reads.
Transaction manager	Make groups of operations all-or-nothing and isolated	Begins, commits (makes permanent) and rolls back (cancels) transactions; guarantees ACID (1.8, chapter 13).
Lock manager	Coordinate concurrent access	**Shared (S)** locks: many readers at once. **Exclusive (X)** locks: one writer, nobody else. Prevents lost updates and dirty reads (seeing another transaction's uncommitted change); detects **deadlocks** (two transactions each waiting for the other's lock).
Recovery manager	Restore a consistent state after a crash	Writes a **log** of every change *before* applying it; after a crash **redoes** committed work (roll forward) and **undoes** uncommitted work (roll back).
Catalog (data dictionary)	Describe everything	Tables, columns, types, constraints, indexes, views, users, permissions, sizes. Every other component reads it.

A **transaction** bundles several operations into one all-or-nothing unit — a bank transfer's debit and credit succeed together or not at all.

How it works — the journey of one query



Candidate plans differ in scan method (full scan or index), join order and join algorithm; the optimizer costs each from **statistics** about the data. That division of labour is what **declarative** means: your SQL names the result, the DBMS picks the algorithm. For writes, the change is first logged, the lock manager holds off conflicting writers, and the change becomes durable at `COMMIT`.

Real-world example — the same parts in PostgreSQL

Generic component	PostgreSQL name
Parser / optimizer / executor	parser → rewriter → **planner** → executor (each connection gets its own server process)
Pages	8 kB blocks (`SHOW block_size` → `8192`) in files under the data directory
Buffer pool	`shared_buffers` (default `128MB`) plus the operating-system file cache
Recovery log	**WAL** (write-ahead log) in `pg_wal/`
Lock manager + concurrency	table/row locks **plus MVCC** (multi-version concurrency control: readers see a snapshot and don't block writers)
Catalog	`pg_catalog` tables and the standard `information_schema` views

(Both numbers were read on the book's server by `code/02_catalog_metadata.sql`.)

Common mistakes

- Assuming SQL runs "top to bottom as written" — the optimizer may reorder joins and filters if the result is the same.
- Judging a query by its first (cold-cache) run — the second may be served from the buffer pool.
- Confusing the cache (in-memory copy, lost in a crash) with durability (committed data survives — provided by the log on disk).

In the interview

30-second answer: "A DBMS has a query processor — parser, optimizer, executor — that turns SQL into a plan and runs it; a storage engine that keeps data in fixed-size pages and indexes; a buffer pool

that caches pages; transaction and lock managers that give ACID and safe concurrency; a recovery manager that uses a write-ahead log to redo or undo work after a crash; and a catalog that describes everything."

- *Why does the optimizer need statistics?* → To estimate row counts per step, which decides index vs full scan and the join order.
- *What is write-ahead logging?* → The log record of a change is flushed to disk before the changed data page, so after a crash the system can replay or undo from the log.
- *Shared vs exclusive lock?* → Shared allows concurrent readers; exclusive allows one writer and blocks others.
- *Which component decides between an index scan and a full scan?* → The optimizer (planner).
- *A committed transaction's pages weren't yet written when the server crashed — what happens?* → The recovery manager redoes it from the log (roll forward).

1.5 Database architectures

What it is

An **architecture** describes where the database runs relative to its users and how requests travel. The same PostgreSQL fits any of these.

Two-tier: client–server

The **client** application talks **directly** to the database server: it opens a **connection** (a logged-in, two-way network channel), sends SQL, gets rows back.

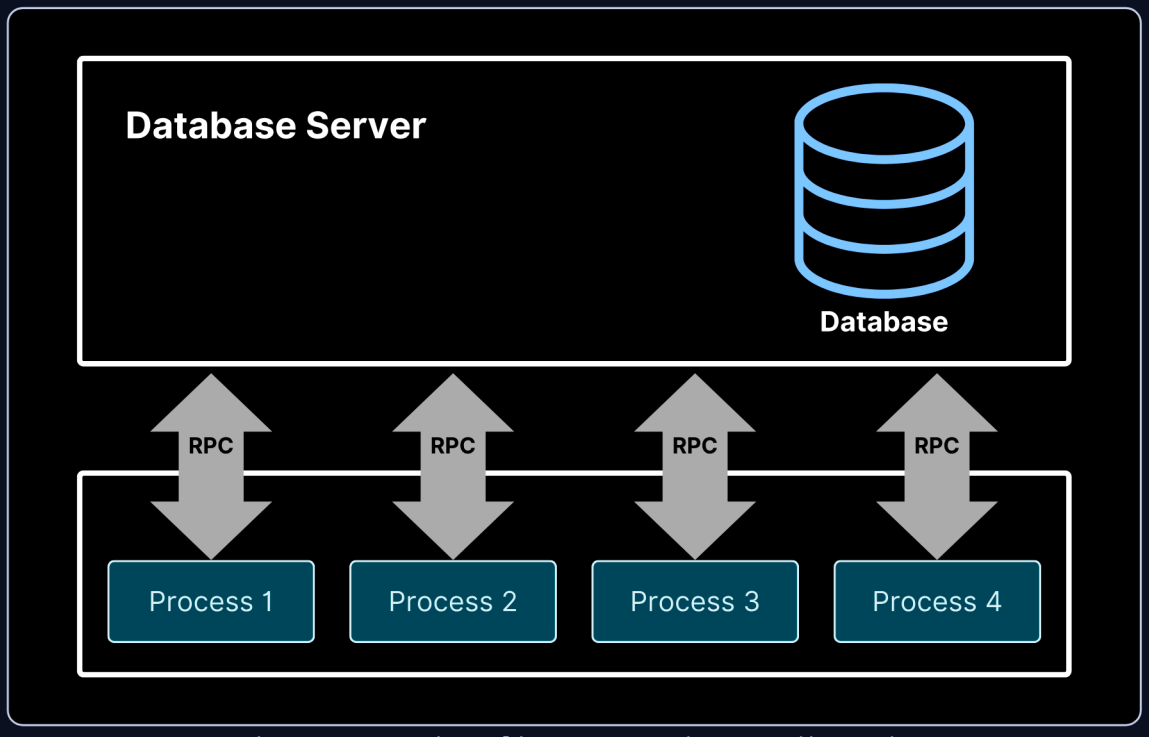


Figure — Two-tier client–server (course diagram).

- **Good:** simple; fine for small teams and internal tools (a desktop accounting app, `psql` itself).
- **Bad:** one server struggles with many connections; business rules live in every client, so a change means redeploying them all; clients hold database credentials.

Three-tier

A middle **application tier** sits between the user interface and the database.

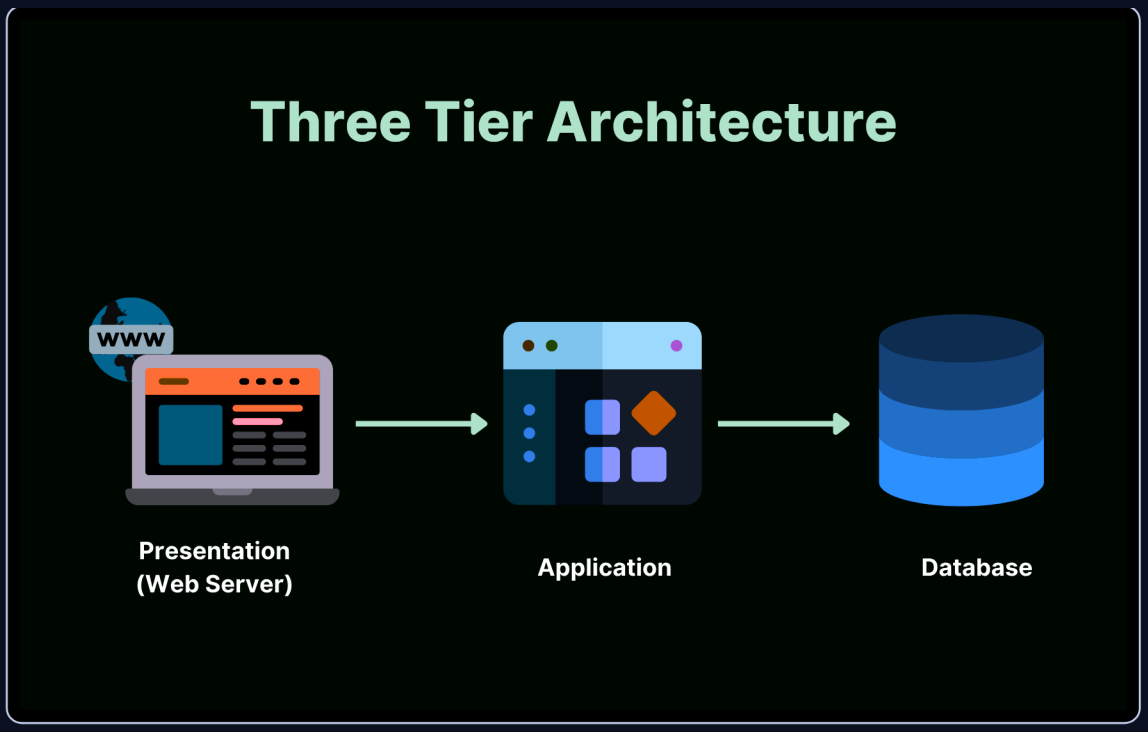


Figure — Three-tier: presentation, application and data layers (course diagram).

Tier	Job	Example
Presentation	User interface	browser page or mobile app
Application (middle tier)	Business rules, validation, coordination, connection pooling (reusing a few open connections for many users)	a Java/Python/Node web service
Data	Storage and queries	PostgreSQL

- **Good:** tiers scale separately (more app servers behind a **load balancer**, which spreads requests across servers); rules change in one place; the database stays off the internet. **Bad:** more moving parts and a little **latency** (delay) per network hop. See System Design Fundamentals.
- **Typical use:** almost every web application — shops, ERP (enterprise resource planning) and CRM (customer relationship management) systems.

N-tier

Further specialised layers: authentication, payments, a cache tier (e.g. Redis, a fast in-memory store), search, message queues (buffers where one service drops work for another).

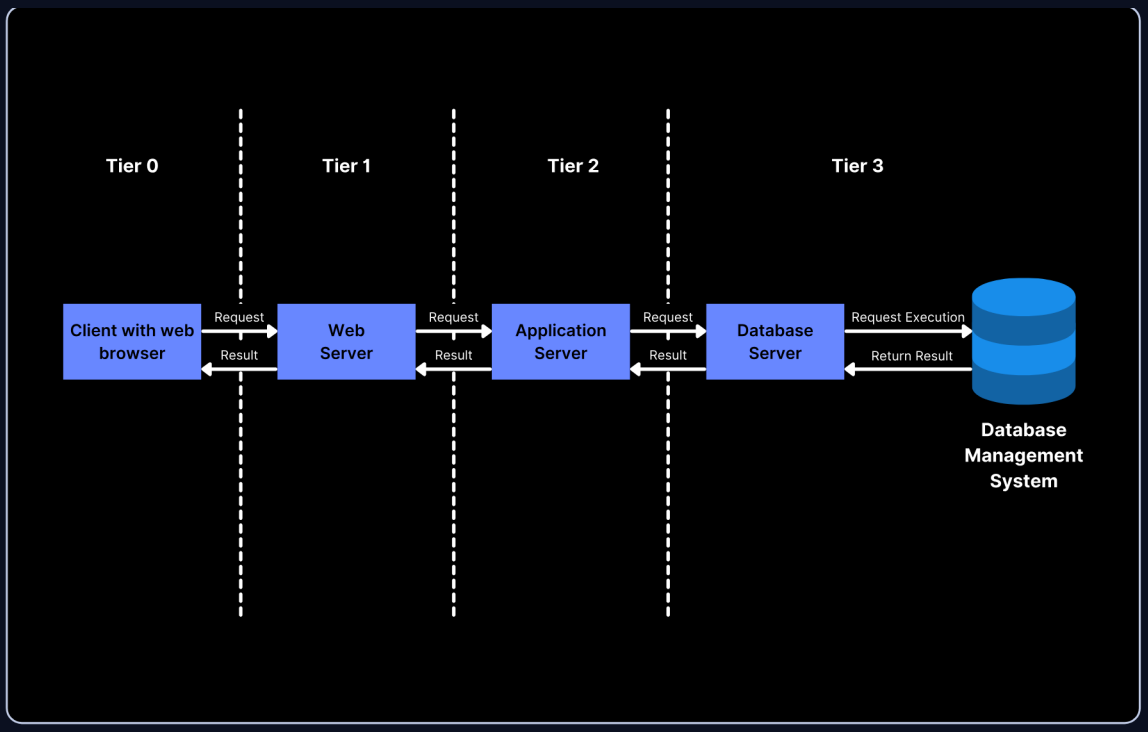


Figure — N-tier (course diagram).

- **Good:** layers scale, deploy and fail independently. **Bad:** harder to operate and debug; more hops. Microservices (many small, independently deployed services) are an extreme form.

Cloud-hosted databases

Model	You manage	Provider manages	Example
IaaS (Infrastructure as a Service)	OS, DBMS install, backups, upgrades	hardware, virtual machines	PostgreSQL you install on an AWS EC2 or Azure VM
PaaS / DBaaS (Platform / Database as a Service)	schema, queries, some settings	install, patching, backups, failover (switching to a standby when the main copy dies)	Amazon RDS, Google Cloud SQL, Azure Database for PostgreSQL
SaaS (Software as a Service)	only your data inside the app	everything	Salesforce, Google Workspace

- **Good:** elastic capacity, less operational work, global reach. **Bad:** network-dependent; cost can grow quietly; data location must satisfy data-residency rules.

Distributed databases

Data spread over several machines (**nodes**), often in several regions.

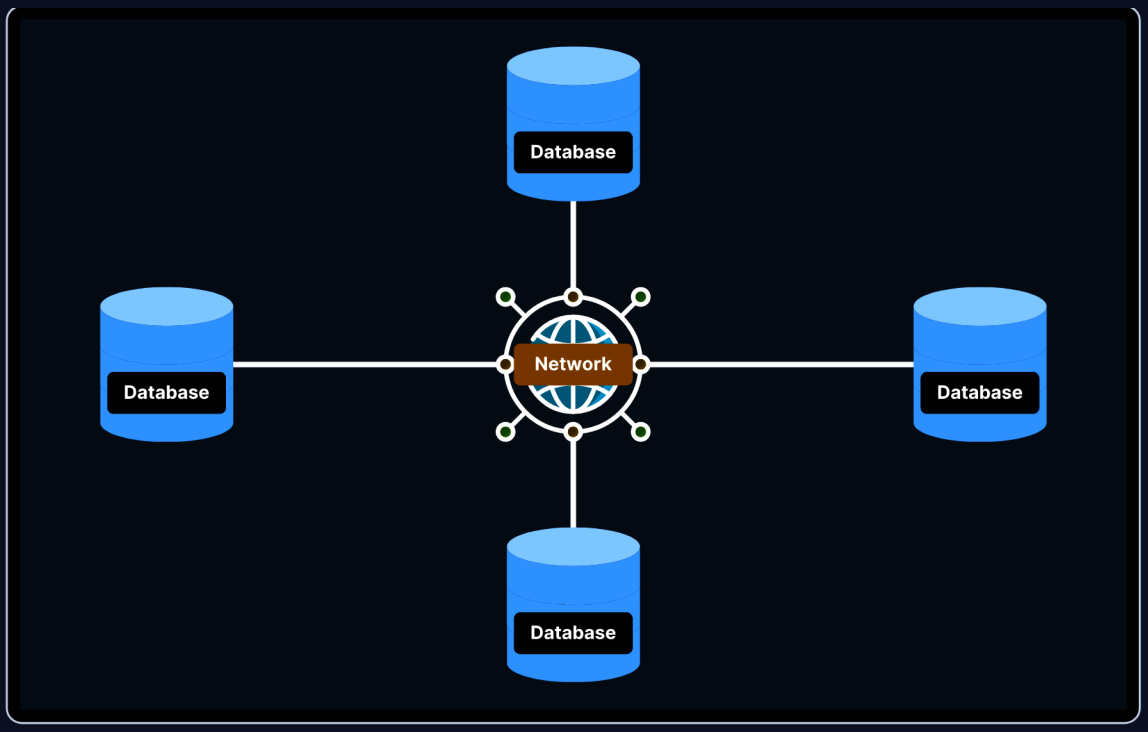


Figure — Nodes in different locations, data partitioned and replicated (course diagram).

- **Partitioning / sharding** — different data on different nodes (customers A–M on node 1, N–Z on node 2).
- **Replication** — copies (**replicas**) of the same data on several nodes for availability and read scaling.
- **Consistency models** — how quickly copies agree, from **strong** (every read sees the latest write) to **eventual** (copies converge after a while).
- **Good:** scales past one machine, survives node failures, data near users. **Bad:** harder to reason about; cross-node queries and transactions pay network costs.

More in chapter 15 (Scaling databases) and the DDIA notes (*Designing Data-Intensive Applications*; `../../../ddia/`).

Self-check

1. *Why not expose the database directly to thousands of browsers?* → Too many connections, credentials on clients, rules duplicated in every client; a middle tier pools connections and centralises rules.
2. *Amazon RDS is which cloud model?* → PaaS / managed database service.
3. *Partitioning vs replication?* → Partitioning splits different data across nodes; replication copies the same data to several nodes.

1.6 The relational model

What it is

The **relational model** stores *all* data in tables and connects them by **shared values**, not pointers or file addresses. Codd proposed it in 1970 because navigational databases forced programs to follow stored pointers, so reorganising data meant rewriting programs. Values in tables plus a system-chosen retrieval strategy fixed that.

"Relational" does **not** come from relationships between tables; it comes from **relation**, the mathematical word for a table (a set of tuples).

The vocabulary

Formal term	Everyday term	Meaning
Relation	table	a named set of rows with the same columns
Tuple	row, record	one item — one employee, one order
Attribute	column, field	one property — a name, a price
Domain	data type (+ rules)	the allowed values for a column (a *data type* is the kind of value: integer, text, date…)
Degree	number of columns	`departments` has degree 4
Cardinality	number of rows	`departments` has cardinality 6
Schema	table/database definition	the tables, columns, types and constraints

("Cardinality" also means one-to-many / many-to-many in ER modelling — chapter 10.)

The four rules every table obeys

1. **Same columns in every row.** A missing value is `NULL`, "unknown" or "not applicable" (chapter 2).
2. **One value per cell (atomic values)** — not a list or nested record. The heart of *First Normal Form* (chapter 11).
3. **Row order means nothing.** The table is a set; ask for an order with `ORDER BY`.
4. **No duplicate rows — in theory.** SQL is looser: a table without a primary key or unique constraint will store identical rows, so good designs always declare one.

Keys and referential integrity

- A **primary key (PK)** is a column (or columns) whose value identifies exactly one row; never repeated, never `NULL`. Example: `customers.customer_id`.
- A **foreign key (FK)** is a column whose values must match a primary (or unique) key in another table: `orders.customer_id` → `customers.customer_id`.
- **Referential integrity**: every FK value points at a real row. An order for a non-existent customer is rejected; deleting a customer who has orders is refused or **cascades** (repeats the delete on dependent rows), per the `ON DELETE` rule (chapter 9).
- **Entity integrity**: a primary key is never `NULL`.

The sample database enforces all of these (errors are deliberate):

```
-- code/04_integrity.sql
INSERT INTO orders (order_id, customer_id, placed_at, status)
VALUES (200, 99, '2025-07-01 10:00', 'pending');          -- no customer 99
DELETE FROM customers WHERE customer_id = 1;               -- customer 1 has orders
INSERT INTO orders (order_id, customer_id, placed_at, status)
VALUES (201, 1, '2025-07-01 10:00', 'lost');              -- not an allowed status
INSERT INTO departments VALUES (1, 'Legal', 'Pune', 1000);  -- duplicate key
INSERT INTO departments VALUES (NULL, 'Legal', 'Pune', 1000);
SELECT count(*) AS orders_still FROM orders;
```

Output:

```
ERROR:  insert or update on table "orders" violates foreign key constraint "orders_customer_id_fkey"
DETAIL:  Key (customer_id)=(99) is not present in table "customers".
ERROR:  update or delete on table "customers" violates foreign key constraint "orders_customer_id_fkey" on table
"orders"
DETAIL:  Key (customer_id)=(1) is still referenced from table "orders".
ERROR:  new row for relation "orders" violates check constraint "orders_status_check"
DETAIL:  Failing row contains (201, 1, null, 2025-07-01 10:00:00, lost, null).
ERROR:  duplicate key value violates unique constraint "departments_pkey"
DETAIL:  Key (dept_id)=(1) already exists.
ERROR:  null value in column "dept_id" of relation "departments" violates not-null constraint
DETAIL:  Failing row contains (null, Legal, Pune, 1000.00).
 orders_still
--------------
           14
(1 row)
```

No application code checked anything — the rules live in the schema, so they hold for every program.

Connecting tables with a join

A **join** combines rows from two tables whose shared values match (chapter 5). Which products did customer 1 buy?

```sql
-- code/01_first_queries.sql (part)
SELECT c.first_name, c.last_name, o.order_id, p.name AS product
FROM customers   c
JOIN orders      o  ON o.customer_id = c.customer_id
JOIN order_items oi ON oi.order_id   = o.order_id
JOIN products    p  ON p.product_id  = oi.product_id
WHERE c.customer_id = 1
ORDER BY o.order_id, p.name;
```

Output:

```
 first_name | last_name | order_id |       product
------------+-----------+----------+---------------------
 Anita      | Shah      |      101 | Laptop Pro 14
 Anita      | Shah      |      101 | Wireless Mouse
 Anita      | Shah      |      104 | Ergonomic Chair
 Anita      | Shah      |      104 | Standing Desk
 Anita      | Shah      |      113 | Laptop Pro 14
 Anita      | Shah      |      113 | Mechanical Keyboard
(6 rows)
```

Nothing "points" anywhere: the value `1` in both `customer_id` columns is the whole connection. Because queries name only tables and columns, the optimizer is free to choose the method; because the model is mathematical, it gives exact tests for design flaws — the normal forms of chapter 11.

Common mistakes

- Relying on the order rows came back in without `ORDER BY` — it can change at any time.
- Storing lists in one cell (`'red,green,blue'`), breaking rule 2.
- Tables without a primary key, letting exact duplicates creep in.

In the interview

30-second answer: "The relational model, from Codd in 1970, stores data as relations — tables of tuples with typed attributes — linked only by values: a foreign-key column stores another row's key, and the DBMS guarantees that row exists. Rows are unordered and cells hold single values. Queries are declarative and never mention storage, so the DBMS can optimise and reorganise freely."

- *Where does 'relational' come from?* → *Relation*, the mathematical name for a table — not relationships.

- *Deleting a parent row that has children?* → Depends on the FK's `ON DELETE`: `NO ACTION` / `RESTRICT` (error, the default), `CASCADE`, `SET NULL` or `SET DEFAULT`.

Self-check

1. *Degree and cardinality of `products`?* → Degree 5, cardinality 9.
2. *Why can't a primary key be NULL?* → An unknown identifier identifies nothing (entity integrity).
3. *Which rule does `'555-1234, 555-9876'` in one phone column break?* → One value per cell (1NF).

1.7 RDBMS — relational database management systems

What it is

An **RDBMS** is a DBMS that implements the relational model and speaks SQL.

Feature	What it gives you
Tables of rows and columns	one uniform structure for all data
Primary and foreign keys	identity and enforced relationships
SQL	one standard language to define, change, query and secure data
Transactions with **ACID** guarantees	Atomicity, Consistency, Isolation, Durability (1.8, chapter 13)
Integrity constraints	types, `NOT NULL`, `UNIQUE`, `CHECK`, keys — bad data refused at the door
Separation of logical and physical	data independence (1.3)

Advantages: the same tables answer countless questions without reorganisation, SQL is far shorter than file-access code, ACID and constraints keep data correct, and modern products handle large volumes and many concurrent users.

The main products

Product	Licence	Known for	Notes for this book
PostgreSQL	open source (PostgreSQL licence)	standards compliance, correctness, extensibility (JSONB = stored JSON documents, PostGIS = geography, custom types)	this book's main dialect
MySQL (and fork MariaDB)	open source (GPL, keeps derived code open) + commercial	ubiquity on the web, simple operations	the SQL course's original dialect; differences noted per chapter
Oracle Database	commercial	enterprise features, PL/SQL (Oracle's procedural language)	mentioned where syntax differs notably
Microsoft SQL Server	commercial	Microsoft ecosystem, T-SQL (Transact-SQL, its dialect)	differences noted in small tables
SQLite	public domain	serverless: a whole database in one file, embedded in apps, phones and browsers	`*.sqlite.sql` examples run in `sqlite3`

Self-check

1. *What makes SQLite different?* → No separate server process; a library storing the whole database in one file.

1.8 Relational vs non-relational databases

What it is

Non-relational databases store data in shapes other than tables. The umbrella name **NoSQL** is now usually expanded as "**not only SQL**", since many gained SQL-like query languages. Neither family is "better": the shape of the data decides which questions are cheap. In a food-delivery app, orders and payments fit tables perfectly; restaurant menus (thirty dishes with toppings here, four fixed meals there, rewritten weekly) do not.

The four NoSQL families



Figure — One dataset, four non-relational shapes, next to the relational version (course diagram).

Family	Data shape	Strength	Weakness	Examples	Typical use
Document	self-contained JSON/BSON (or XML) documents (JSON = nested `{"key": value}` text; BSON = binary JSON); documents in one collection may differ in fields	reading/writing one whole object; varied shapes	cross-document joins limited	MongoDB, Couchbase, CouchDB	catalogs, content, user profiles, menus
Key-value	a value under a key; mainly `put` and `get`	extremely fast key lookups	cannot search inside values	Redis, Amazon DynamoDB, Riak	caches, sessions, feature flags
Wide-column (column-family)	rows grouped into column families; rows may have different columns; partitioned by key	very high write throughput across many machines	queries must follow the partition key (the column deciding which machine holds a row)	Apache Cassandra, HBase	time series, event logs, IoT
Graph	nodes, edges and properties	multi-hop relationship questions ("friends of friends who…")	not built for bulk aggregation	Neo4j, Amazon Neptune	social networks, recommendations, fraud detection

Chapter 16 (Beyond relational) builds each in practice.

ACID vs BASE

ACID — the guarantees of a relational transaction:

Letter	Guarantee
Atomicity	all operations happen, or none do
Consistency	after the transaction, every declared rule (keys, checks, constraints) still holds
Isolation	concurrent transactions behave as if run alone; half-done changes stay invisible
Durability	once committed, changes survive crashes and power loss

BASE — how many distributed NoSQL systems describe themselves:

Letter(s)	Meaning
Basically Available	requests get answers while some nodes fail, though data may be stale
Soft state	state may change without new writes as replicas catch up
Eventual consistency	if writes stop, replicas converge; until then readers may see different values

BASE gives up immediate agreement to stay available — fine for a product catalogue, not for an account balance.

Three claims that need care

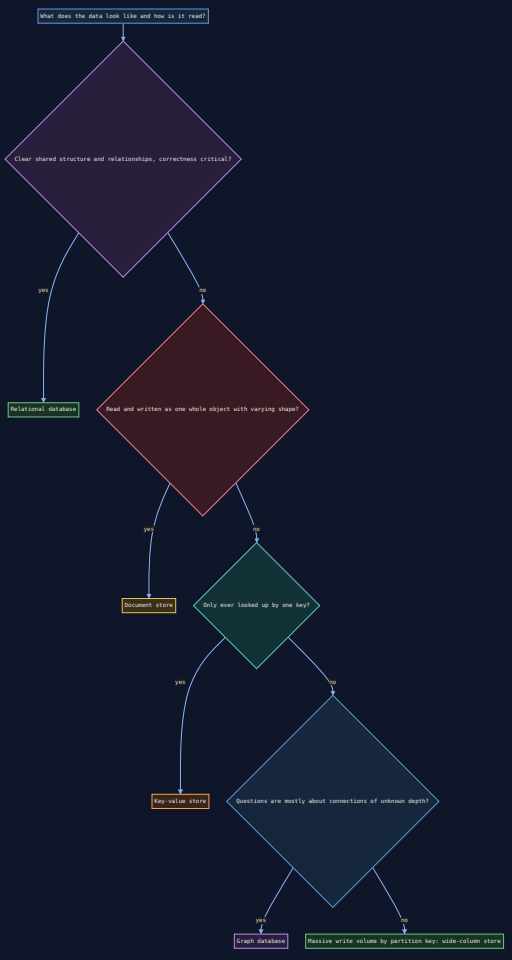
1. **"Relational databases only scale vertically."** *False.* **Vertical scaling** = a bigger machine; **horizontal scaling** = more machines. Relational systems scale out with read replicas, partitioning/sharding and distributed SQL (CockroachDB, Google Spanner, YugabyteDB, Citus for PostgreSQL). The real issue is *cost*: cross-machine joins and transactions are expensive, and NoSQL models avoid them by design.

2. **"NoSQL databases don't support transactions."** *Out of date.* MongoDB has multi-document ACID transactions since 4.0 (2018); DynamoDB added transactions in 2018; Cassandra offers lightweight (compare-and-set: "write only if the value is still what I last saw") transactions per partition. Limits vary by product.
3. **"NoSQL databases have no schema."** *Misleading.* The database doesn't *enforce* one, but your code still expects `price` to be a number — the schema moved into the application. **Schema-on-write** (database checks data as written — relational) vs **schema-on-read** (the reader interprets whatever is stored): the latter makes change cheap and bad data easy.

Comparison

Aspect	Relational	Non-relational
Schema	schema-on-write	mostly schema-on-read
Query language	SQL, standardised	product-specific (MongoDB query API, Cassandra's CQL, Neo4j's Cypher…), some SQL-like
Joins	built in, optimised	few or none; join in code or embed related data
Transactions	full ACID across tables	often scoped (one document / partition) or slower
Best fit	structured, related data where correctness matters (orders, payments, bookings, inventory, ERP, CRM)	huge volumes, varied shapes, one dominant access pattern (catalogues, caches, feeds, telemetry, real-time analytics)

How to choose



Using different stores for different jobs (orders in PostgreSQL, menus in a document store, sessions in Redis) is **polyglot persistence**. Every extra store must be run, backed up, monitored and kept in step, so start with one (relational is the safe default) and add another only for a concrete requirement.

In the interview

30-second answer: "I pick from the access pattern, not the fashion. Orders need multi-table transactions and constraints, so relational. Menus differ per restaurant and are fetched whole — a document store. Sessions are key lookups — Redis. Relational databases do scale horizontally — replicas, sharding, distributed SQL — the real cost is cross-node joins and transactions."

- *Why not one database for everything?* → Often you should; each extra store adds operational cost and cross-store consistency problems.

Self-check

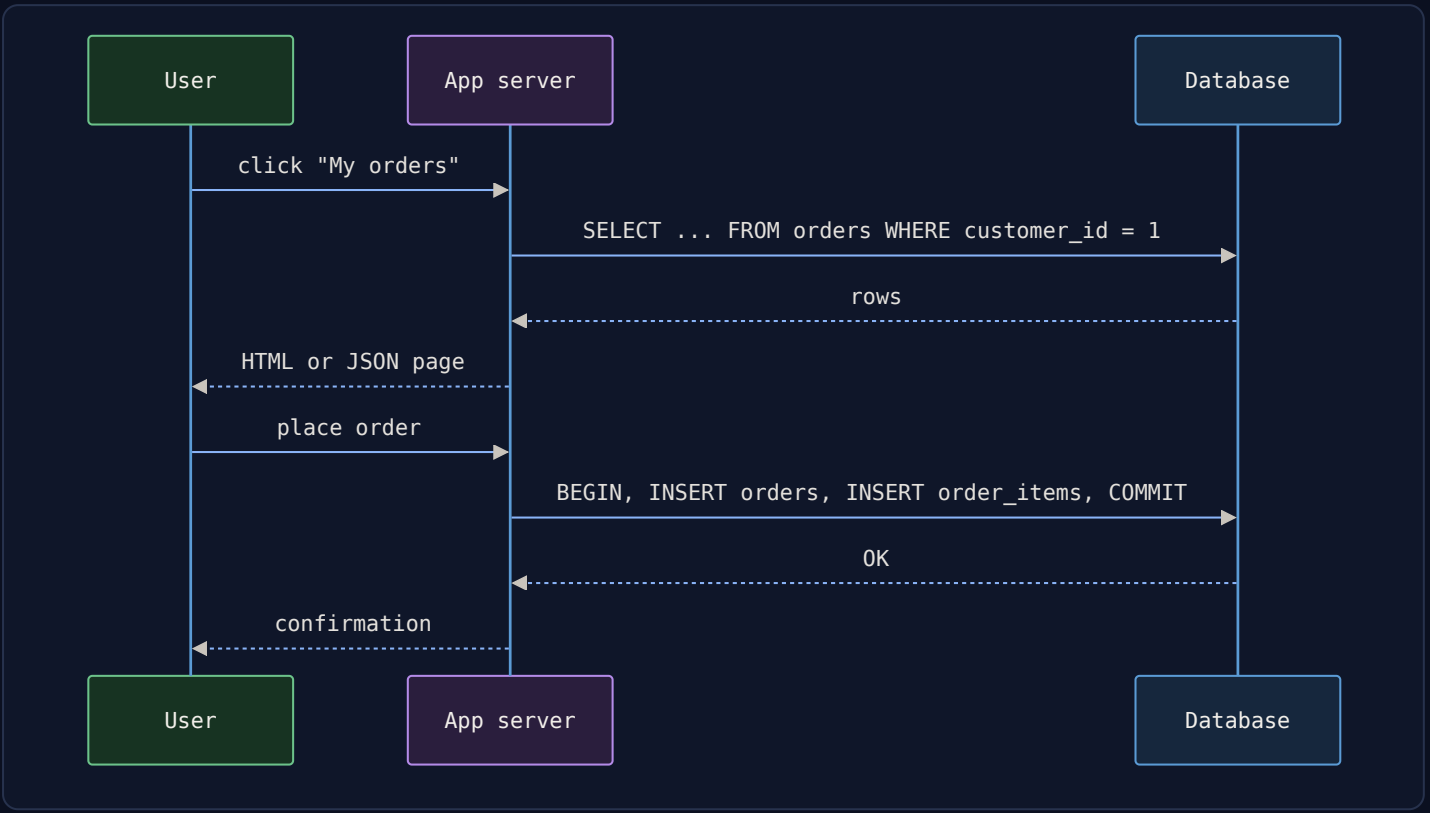
1. *Feature flags read by name millions of times per second?* → Key-value.
2. *"Friends of my friends who also like jazz"?* → Graph.
3. *Horizontal vs vertical scaling?* → More machines vs a bigger machine.

1.9 What SQL is

What it is

SQL (Structured Query Language) is the standard language for defining, changing, querying and securing relational data. It is **declarative**: you describe the result, not the steps. Developed at IBM in the 1970s as SEQUEL (renamed for trademark reasons), it became an ANSI standard in 1986 and ISO in 1987; the current edition is **SQL:2023**. Every RDBMS implements the core plus extensions — its **dialect** (PostgreSQL's, MySQL's, T-SQL, PL/SQL-flavoured Oracle SQL).

SQL is the traffic between applications and databases:



The five sub-languages

(Some texts fold DQL into DML; know both.)

Sub-language	Purpose	Main statements
DDL — Data Definition Language	define and change *structure*	`CREATE`, `ALTER`, `DROP`, `TRUNCATE`, `COMMENT`, `RENAME` (via `ALTER`)
DML — Data Manipulation Language	change *data*	`INSERT`, `UPDATE`, `DELETE`, `MERGE` (PostgreSQL 15+), `COPY` (PostgreSQL)
DQL — Data Query Language	read data	`SELECT`
DCL — Data Control Language	control *access*	`GRANT`, `REVOKE` (and role management such as `CREATE ROLE`)
TCL — Transaction Control Language	group changes into transactions	`BEGIN` / `START TRANSACTION`, `COMMIT`, `ROLLBACK`, `SAVEPOINT`, `ROLLBACK TO SAVEPOINT`, `RELEASE SAVEPOINT`

Frequent mix-up: **granting and revoking is DCL, not TCL.** TCL is only transaction boundaries.

How it works — one statement of each kind

```sql
-- code/03_sql_sublanguages.sql
-- DDL: define structure
CREATE TABLE suppliers (
    supplier_id integer PRIMARY KEY,
    name        text NOT NULL,
    country     text
);

-- DML: change data
INSERT INTO suppliers VALUES (1, 'Deskworks', 'India'), (2, 'Clicky Ltd', 'UK');
UPDATE suppliers SET country = 'Ireland' WHERE supplier_id = 2;

-- DQL: read data
SELECT * FROM suppliers ORDER BY supplier_id;

-- DCL: control who may do what
CREATE ROLE sql01_report_reader;
GRANT SELECT ON suppliers TO sql01_report_reader;
SELECT grantee, privilege_type
FROM information_schema.role_table_grants
WHERE table_name = 'suppliers' AND grantee = 'sql01_report_reader';
REVOKE SELECT ON suppliers FROM sql01_report_reader;

-- TCL: group changes into a transaction, with a savepoint
BEGIN;
INSERT INTO suppliers VALUES (3, 'Chairmakers', 'India');
SAVEPOINT before_mistake;
DELETE FROM suppliers;                -- oops: no WHERE clause
ROLLBACK TO SAVEPOINT before_mistake;  -- undo only the delete
COMMIT;
SELECT * FROM suppliers ORDER BY supplier_id;
```

Output (psql echoes a *command tag* such as `INSERT 0 2` — `0` is a legacy OID (object identifier) field, `2` the row count):

```
CREATE TABLE
INSERT 0 2
UPDATE 1
 supplier_id |    name    | country
-------------+------------+---------
           1 | Deskworks  | India
           2 | Clicky Ltd | Ireland
(2 rows)

CREATE ROLE
GRANT
       grantee       | privilege_type
---------------------+----------------
 sql01_report_reader | SELECT
(1 row)

REVOKE
BEGIN
INSERT 0 1
SAVEPOINT
DELETE 3
ROLLBACK
COMMIT
 supplier_id |    name     | country
-------------+-------------+---------
           1 | Deskworks   | India
           2 | Clicky Ltd  | Ireland
           3 | Chairmakers | India
(3 rows)
```

`DELETE 3` wiped every row, but `ROLLBACK TO SAVEPOINT` undid just that step; the insert before the savepoint survived the `COMMIT`.

SQL syntax basics you'll use from chapter 2

Rule	Detail
Statements end with `;`	psql waits for the semicolon before running anything
Keywords are case-insensitive	`select` = `SELECT`; this book capitalises keywords
Unquoted identifiers fold to lower case in PostgreSQL	`Employees` means `employees`; `"Employees"` keeps the case
Strings use single quotes	`'Mumbai'`; a quote inside is doubled: `'O''Brien'`
Comments	`-- to end of line` and `/* block */`
Whitespace is free	break long queries over lines

Dialect differences at a glance

Feature	PostgreSQL	MySQL	SQLite	SQL Server
Limit rows	`LIMIT n` / `FETCH FIRST n ROWS ONLY`	`LIMIT n`	`LIMIT n`	`TOP n` / `OFFSET … FETCH`
Identifier quotes	`"name"`	`` `name` `` (or `"` in ANSI mode)	`"name"`	`[name]` or `"name"`
String concat	`||`, `concat()`	`concat()` (`||` is OR by default)	`||`	`+`, `concat()`
Auto-increment	`GENERATED … AS IDENTITY` / `serial`	`AUTO_INCREMENT`	`INTEGER PRIMARY KEY`	`IDENTITY(1,1)`
DDL inside transactions	yes, can be rolled back	no (implicit commit)	yes	mostly yes

Self-check

1. *Classify:* `TRUNCATE`, `GRANT`, `SAVEPOINT`, `MERGE`, `SELECT`. → DDL, DCL, TCL, DML, DQL.
2. *What does "declarative" mean?* → You state the result; the DBMS decides the algorithm.
3. *Why did* `ROLLBACK TO SAVEPOINT` *not remove supplier 3?* → The insert came before the savepoint; only later work is undone.

1.10 Your private PostgreSQL server and psql

What it is

The script `tools_pg.sh` in the project root runs a **private PostgreSQL 14 server** for this book only: data in `~/.pg-books-system-design/` (outside the iCloud-synced Documents folder), listening on port **5544** (a *port* is a numbered network door) through a Unix socket in `/tmp` (a special file for same-machine communication). It never touches another PostgreSQL install (normally port 5432). **psql** is PostgreSQL's command-line client: you type SQL, it prints results.

How it works — step by step

From the project root (`system_design_notes/`):

1. **Start the server** (first run creates the data directory):

```
bash tools_pg.sh start
```

```
postgres ready on port 5544 (socket /tmp)
```

2. **Create the database**:

```
bash tools_pg.sh psql -d postgres -c "CREATE DATABASE company ENCODING 'UTF8' TEMPLATE template0"
```

```
CREATE DATABASE
```

   An **encoding** turns characters into bytes; **UTF-8** covers every language. Asking for `UTF8` explicitly guarantees `café` and `मुंबई` are stored and measured correctly whatever the server default; `TEMPLATE template0` is required when the encoding differs from the default template's.

3. **Load the sample schema** (1.11):

```
bash tools_pg.sh psql -d company -q -f sql/chapters/01-databases-and-the-relational-model/code/00_schema.sql
```

4. **Check it**:

```
bash tools_pg.sh psql -d company -c '\conninfo' -c 'SELECT count(*) FROM employees'
```

```
You are connected to database "company" as user "postgres" via socket in "/tmp" at port "5544".
 count
-------
    15
(1 row)
```

5. **Interactive session**: `bash tools_pg.sh psql -d company`. Prompt `company=#` means connected as a superuser (an account allowed everything); `=>` for ordinary users. `company-#` means psql is waiting for the rest of a statement (no `;` yet). `\q` leaves.
6. **Stop the server**: `bash tools_pg.sh stop`.

`tools_pg.sh psql` wraps `psql -h /tmp -p 5544 -U postgres -X`: socket directory, port, user, and `-X` to skip a personal `~/.psqlrc` so output looks the same for every reader.

psql meta-commands

Lines starting with a backslash are **meta-commands**, handled by psql itself (no semicolon needed).

Command	Does
`\?` / `\h SELECT`	help on meta-commands / SQL syntax for a statement
`\l`	list databases
`\c company`	connect to another database
`\conninfo`	show current connection
`\dt` / `\dt+`	list tables (with sizes)
`\d employees`	describe a table: columns, types, indexes, constraints, references
`\dn` , `\df` , `\dv` , `\di` , `\du`	schemas, functions, views, indexes, roles
`\x` (or `\x auto`)	expanded display: one field per line
`\pset null '(null)'`	show NULLs visibly (used throughout this book)
`\timing`	print each statement's duration
`\i file.sql`	run a file of SQL
`\e`	edit the last query in `$EDITOR`
`\copy`	import/export CSV from your machine (client side)
`\q`	quit

They also work inside script files:

```
-- code/05_psql_meta_commands.sql
\dt
\d departments
\x on
SELECT * FROM employees WHERE emp_id = 7;
\x off
```

Output (abridged; `\dt` lists the 8 tables):

```
               Table "public.departments"
  Column  |     Type      | Collation | Nullable | Default
----------+---------------+-----------+----------+---------
 dept_id  | integer       |           | not null |
 name     | text          |           | not null |
 location | text          |           |          |
 budget   | numeric(12,2) |           | not null |
Indexes:
    "departments_pkey" PRIMARY KEY, btree (dept_id)
    "departments_name_key" UNIQUE CONSTRAINT, btree (name)
Check constraints:
    "departments_budget_check" CHECK (budget >= 0::numeric)
Referenced by:
    TABLE "employees" CONSTRAINT "employees_dept_id_fkey" FOREIGN KEY (dept_id) REFERENCES departments(dept_id)
    TABLE "projects" CONSTRAINT "projects_dept_id_fkey" FOREIGN KEY (dept_id) REFERENCES departments(dept_id)

Expanded display is on.
-[ RECORD 1 ]--+---------------------------
emp_id         | 7
first_name     | Priya
...
hire_date      | 2020-02-10
```

Running the book's code

- **One file**: `bash tools_pg.sh psql -d company -f sql/chapters/02-select-basics/code/02_where.sql`. Run files that change data (like 1.3's demo) on a fresh copy — re-run step 3 to reset.
- **Everything**, as the book was checked: `bash tools_run_sql.sh sql` — per file it creates a fresh database, loads that chapter's `00_schema.sql`, runs the file, prints `OK` or `FAIL`.

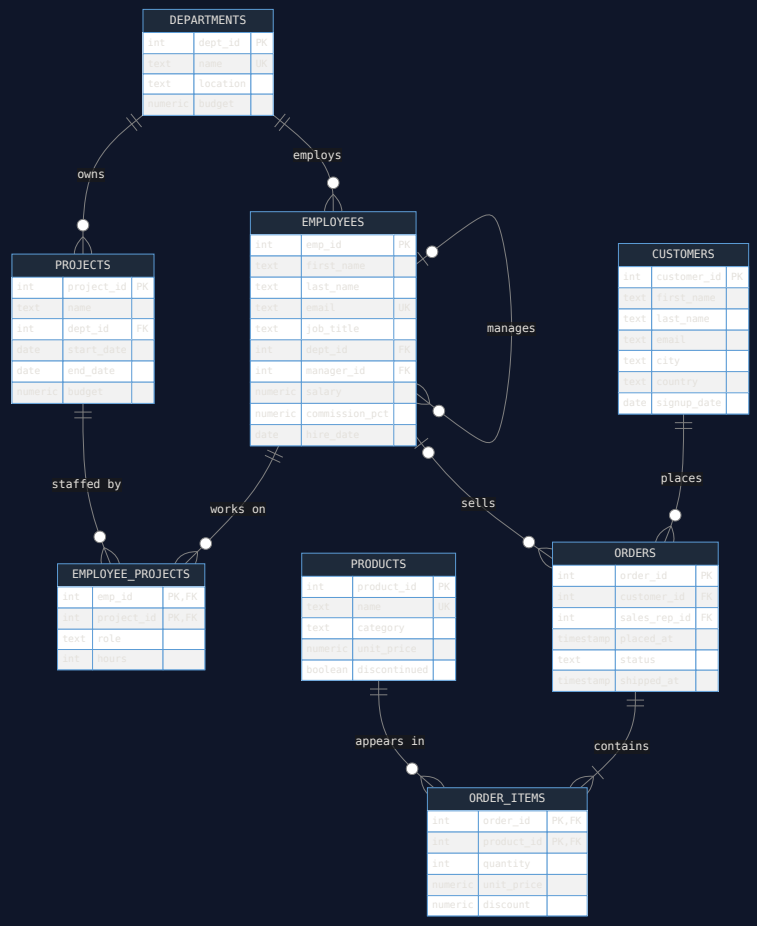
Common mistakes

- Forgetting the `;` — psql shows `company-#` and waits.
- Typing SQL at the shell prompt (`$`) instead of inside psql.
- Connecting to the wrong server: use `tools_pg.sh psql` (port 5544), not a bare `psql` (5432).
- A typo inside a transaction aborts it (`current transaction is aborted…`); type `ROLLBACK;`.

1.11 The sample database

What it is

Chapters 2 onward query **`company`**, a fictional firm selling office equipment: staff in departments and projects, plus a small shop with customers, products and orders. Every chapter's `code/00_schema.sql` is a copy of `code/00_schema.sql`.



(Crow's-foot symbols — `||` exactly one, `o{` zero or more, `|{` one or more, `|o` zero or one — are explained in chapter 10.)

The tables

Table	Rows	What a row is	Notable columns
`departments`	6	a department	`location` may be NULL, `budget`
`employees`	15	a member of staff	`manager_id` → another employee (a self-reference: an FK to its own table), `commission_pct` only for some sales staff
`projects`	5	an internal project owned by a department	`end_date` NULL = still running
`employee_projects`	13	"employee X works on project Y" (many-to-many link)	composite key (several columns together) `(emp_id, project_id)`, `role`, `hours`
`customers`	10	a shop customer	`city`, `email` may be NULL
`products`	9	a product for sale	`category` may be NULL, `discontinued`
`orders`	14	an order header	`placed_at`, `status` ∈ pending / shipped / delivered / cancelled, `shipped_at`, `sales_rep_id` NULL for online orders
`order_items`	22	one product line in an order	`quantity`, `unit_price` at time of sale, `discount` (0.10 = 10 %)

Deliberate quirks (so later chapters have something to find)

Quirk	Where	Useful for
CEO (Asha) and a new intern (Rohan) have no department	`employees.dept_id` NULL	NULL handling, outer joins
Research has no employees; Support has no location	`departments`	outer joins (keep unmatched rows), anti-joins (rows with *no* match), NULL sorting
Salary ties: Rahul & Meera 98,000; Arjun & Sara 68,000	`employees.salary`	`DISTINCT`, `WITH TIES`, ranking functions
Missing emails (Meera, Ananya) and commissions	`employees`	`IS NULL`, `COALESCE`, `count(col)`
One email in mixed case (`Neha.Gupta@Company.example`)	`employees.email`	case-insensitive matching
Anita Shah entered twice, second time with spaces and capitals in the email and city `mumbai`	`customers` 1 and 8	de-duplication, `trim`, `lower`
Customers 9 and 10 never ordered; "Desk Lamp" never sold; "Gift Card" has no category	`customers`, `products`	anti-joins, NULL groups
Orders across Jan–Jun 2025, one at 23:55 on 31 January	`orders.placed_at`	date ranges, month boundaries
Pending/cancelled orders have no `shipped_at`	`orders`	NULLs in date arithmetic

```sql
SELECT * FROM departments;
```

Output:

```
 dept_id |    name     | location  |  budget
---------+-------------+-----------+-----------
       1 | Engineering | Bengaluru | 900000.00
       2 | Sales       | Mumbai    | 400000.00
       3 | Marketing   | Mumbai    | 250000.00
       4 | Finance     | Pune      | 200000.00
       5 | Support     | (null)    | 150000.00
       6 | Research    | Pune      | 300000.00
(6 rows)
```

Key terms (chapter 1)

Defined where first used (in **bold**): file processing system, redundancy/inconsistency, lost update, metadata, catalog/data dictionary, DBMS, database system, data independence (physical/logical), buffer pool, page, WAL, transaction, ACID, BASE, relation/tuple/attribute/domain, degree, cardinality, primary key, foreign key, entity and referential integrity, schema-on-write/-on-read, horizontal/vertical scaling, polyglot persistence, DDL/DML/DQL/DCL/TCL, declarative. Book-wide glossary: chapter 20.

Chapter 1 TL;DR

- File-based systems suffered seven problems (redundancy, isolation, no queries, rules in code, lost updates, no recovery, coarse security); the **DBMS** fixes them by mediating every access.
- A **database** is related, self-describing (catalog), purposeful, persistent data; the **DBMS defines, constructs, manipulates, shares, protects and maintains** it, with **physical and logical data independence**.
- Inside: **parser** → **optimizer** → **executor**, a **storage engine** with pages, a **buffer pool**, **transaction/lock managers**, a **recovery manager** with a write-ahead log, and a **catalog**.
- Architectures: two-tier, three-tier, n-tier, cloud (IaaS/PaaS/SaaS), distributed (partitioning + replication).
- The **relational model**: relations of tuples and attributes; same columns per row, atomic cells, no row order, no duplicates; **primary keys** identify, **foreign keys** link, the DBMS enforces **referential integrity**.
- NoSQL families (document, key-value, wide-column, graph) trade enforced structure and joins for flexible shapes and easier scale-out; choose by access pattern.
- **SQL** is declarative and standard: **DDL, DML, DQL, DCL, TCL**; each product speaks a dialect.
- Lab: `bash tools_pg.sh start` → create UTF-8 `company` → load `00_schema.sql` → `bash tools_pg.sh psql -d company` ; `\dt` , `\d` , `\x` , `\pset null` , `\timing` , `\q` ; `tools_run_sql.sh sql` runs all examples on fresh databases.

Next: Chapter 2 — SELECT basics.

Chapter 2 — SELECT Basics: Reading, Filtering, Sorting and Paging

This chapter covers the single-table `SELECT` completely: choosing and computing columns, filtering with `WHERE`, operators, the rules of `NULL`, `DISTINCT`, `ORDER BY`, and paging with `LIMIT` / `OFFSET` or the standard `FETCH`. Joins, grouping and subqueries (queries nested inside another query) build on these pieces in chapters 4–6.

Sections: 2.1 `SELECT` and aliases · 2.2 `WHERE` and logical operators · 2.3 Operators · 2.4 `BETWEEN`, `IN`, `LIKE` / `ILIKE`, regex · 2.5 `NULL` and three-valued logic · 2.6 `DISTINCT` / `DISTINCT ON` · 2.7 `ORDER BY` · 2.8 `LIMIT` / `OFFSET` / `FETCH` · 2.9 Logical processing order.

Setup. Examples use the `company` database from chapter 1, section 1.11. Files are in `code/`; each starts with `\pset null '(null)'` so missing values print as `(null)`. To follow along: `bash tools_pg.sh psql -d company`, then `\pset null '(null)'`.

2.1 SELECT: choosing what comes back

What it is

`SELECT` reads rows and returns a new, temporary table called the **result set**: it chooses which columns you see (and, with `WHERE`, which rows).

```
SELECT column1, column2, ...     -- what to show
FROM   table_name;               -- where to read it
```

How it works

`FROM` names the table; the `SELECT` list says which columns (or computed values) to return, in order; `*` means "every column, in table order".

```
-- code/01_select_columns.sql
SELECT * FROM products;
```

Output:

```
 product_id |           name             |  category   | unit_price | discontinued
------------+----------------------------+-------------+------------+--------------
          1 | Laptop Pro 14              | Electronics |    1299.00 | f
          2 | Wireless Mouse             | Electronics |      24.99 | f
          ...
          8 | Desk Lamp                  | Furniture   |      35.00 | t
          9 | Gift Card                  | (null)      |      50.00 | f
(9 rows)
```

(`f` / `t` are psql's display of a **boolean**, a true/false value.) To choose columns, list them: `SELECT first_name, last_name, job_title FROM employees;` returns those three columns for all 15 rows.

Expressions and column aliases

The `SELECT` list can hold any **expression** — arithmetic, function calls (a *function* is a named built-in operation such as `round(x, 2)`), constants. An **alias** (`AS new_name`) names the output column; double-quote it to keep spaces or capitals.

```sql
SELECT name,
       unit_price,
       unit_price * 1.18        AS price_with_gst,
       'per unit'               AS note,
       unit_price * 1.18 AS "Price incl. GST"
FROM products
WHERE product_id <= 3;
```

Output:

```
      name      | unit_price | price_with_gst |   note   | Price incl. GST
----------------+------------+----------------+----------+-----------------
 Laptop Pro 14  |    1299.00 |      1532.8200 | per unit |       1532.8200
 Wireless Mouse |      24.99 |        29.4882 | per unit |         29.4882
 USB-C Hub      |      49.50 |        58.4100 | per unit |         58.4100
(3 rows)
```

(GST is India's 18 % sales tax.) The table is not changed — computed columns exist only in the result. (Rounding: `round(x, 2)`, chapter 3.)

Table aliases give a table a short name, essential once queries involve several tables. `AS` is optional for both kinds, but writing it for column aliases avoids a nasty bug (*Common mistakes*).

```sql
SELECT e.first_name, e.salary / 12 monthly
FROM employees AS e
WHERE e.emp_id IN (1, 2);
```

Output:

```
 first_name |      monthly
------------+--------------------
 Asha       | 20833.333333333333
 Vikram     | 15416.666666666667
(2 rows)
```

SELECT without FROM — PostgreSQL, MySQL, SQLite and SQL Server evaluate expressions directly (psql as a calculator). Oracle required `FROM dual` (a one-row dummy table) until 23ai made `FROM` optional. `now()` returns the current date-time; `::date` **casts** (converts) it to a plain date.

```sql
SELECT 2 + 3 AS five, now()::date >= DATE '2025-01-01' AS after_2025;
```

Output:

```
 five | after_2025
------+------------
    5 | t
(1 row)
```

Common mistakes

- `SELECT *` **in application code** — fetches unneeded columns (more I/O — disk input/output — and network traffic) and breaks or silently changes when a column is added. Fine for exploring in psql.
- **A missing comma turns a column into an alias.** `SELECT first_name last_name FROM employees` returns *one* column named `last_name` holding first names. Writing `AS` makes this visible.
- **Single-quoting aliases.** `'total'` is a string literal (a fixed text value); aliases use double quotes: `"Total"`.

Self-check

1. *Does* `SELECT unit_price * 2 FROM products` *change prices?* → No; it only computes a result value.
2. *How do you name an output column* `Net Price` *exactly?* → `AS "Net Price"`.

2.2 WHERE: filtering rows

What it is

`WHERE condition` keeps only rows for which the condition is **true**. It works identically in `SELECT`, `UPDATE` and `DELETE` (chapter 9), so a filter tested with `SELECT` can be reused safely in a `DELETE` (without `WHERE`, a `DELETE` removes all rows).

How it works

Per row, the condition evaluates to `TRUE`, `FALSE` or — with `NULL` — `UNKNOWN` (2.5). Only `TRUE` rows pass.

```sql
-- code/02_where.sql
SELECT name, category, unit_price
FROM products
WHERE category = 'Electronics';
```

Output:

```
           name             |  category   | unit_price
----------------------------+-------------+------------
 Laptop Pro 14              | Electronics |    1299.00
 Wireless Mouse             | Electronics |      24.99
 Noise-Cancelling Headphones | Electronics |     199.95
(3 rows)
```

Comparisons work on numbers, text and dates. Date literals are ISO-format strings `'YYYY-MM-DD'`, converted to the column's type:

```sql
SELECT first_name, hire_date FROM employees WHERE hire_date >= '2023-01-01' ORDER BY hire_date;
```

Output:

```
 first_name | hire_date
------------+------------
 Li         | 2023-02-14
 Arjun      | 2023-11-20
 Sara       | 2024-04-01
 Rohan      | 2025-06-02
(4 rows)
```

Combining conditions: AND, OR, NOT

Operator	True when	Example
`AND`	both sides are true	`dept_id = 2 AND salary > 70000`
`OR`	at least one side is true	`city = 'Delhi' OR city = 'Mumbai'`
`NOT`	the condition is false	`NOT discontinued`

Precedence: `NOT` binds tightest, then `AND`, then `OR` (like `*` before `+`), so `a OR b AND c` means `a OR (b AND c)`:

```sql
-- Intended: Engineering or Sales staff earning over 100k
SELECT first_name, dept_id, salary
FROM employees
WHERE dept_id = 1 OR dept_id = 2 AND salary > 100000
ORDER BY emp_id;
```

Output:

```
 first_name | dept_id |  salary
------------+---------+-----------
 Vikram     |       1 | 185000.00
 Neha       |       1 | 142000.00
 Rahul      |       1 |  98000.00
 Meera      |       1 |  98000.00
 Daniel     |       2 | 150000.00
(5 rows)
```

Rahul and Meera earn under 100k but appear: the query says "*everyone* in dept 1, or dept-2 staff over 100k". Writing `WHERE (dept_id = 1 OR dept_id = 2) AND salary > 100000` returns only Vikram, Neha and Daniel. Rule: whenever `AND` and `OR` mix, add parentheses.

`NOT` negates the condition after it:

```sql
SELECT name FROM products WHERE NOT discontinued AND NOT unit_price < 100 ORDER BY name;
```

Output:

```
           name
----------------------------
 Ergonomic Chair
 Laptop Pro 14
 Noise-Cancelling Headphones
 Standing Desk
(4 rows)
```

Case sensitivity

In PostgreSQL `=` on text is **case-sensitive**: `'Mumbai' = 'mumbai'` is false, so `WHERE city = 'Mumbai'` finds one of our two Mumbai customers:

```
 exact_mumbai
--------------
            1
```

MySQL's default collations (text comparison and sorting rules, 2.7) are case-*insensitive*, so the same query returns 2 there — a portability trap. In PostgreSQL use `lower(city) = 'mumbai'` or `ILIKE` (2.4).

Common mistakes

- Mixing `AND` and `OR` without parentheses.
- Comparing with `= NULL` — never true; use `IS NULL` (2.5).
- `"Mumbai"` (double quotes) for a string: PostgreSQL reads a column name and errors *column "Mumbai" does not exist*.

Self-check

1. *How is `a AND b OR c` evaluated?* → `(a AND b) OR c`.

2.3 Operators

What it is

An **operator** is a symbol or keyword combining one or more values (its **operands**) into a new value: `+`, `=`, `AND`, `||`.

Arithmetic operators

Operator	Meaning	Example	Result
`+`	add	`5 + 3`	8
`-`	subtract	`10 - 4`	6
`*`	multiply	`3 * 7`	21
`/`	divide	`20 / 5`	4
`%`	remainder (modulo)	`17 % 3`	2
`^`	power (PostgreSQL)	`2 ^ 10`	1024
`|/`, `@`	square root, absolute value (PostgreSQL)	`|/ 25`, `@ -5`	5, 5

```sql
-- code/08_operators.sql (part)
SELECT 7 / 2 AS int_div, 7 / 2.0 AS num_div, 7::numeric / 2 AS cast_div, -7 / 2 AS neg_int_div;
SELECT 2 + 3 * 4 AS no_parens, (2 + 3) * 4 AS parens;
```

Output:

```
 int_div |      num_div       |      cast_div      | neg_int_div
---------+--------------------+--------------------+-------------
       3 | 3.5000000000000000 | 3.5000000000000000 |          -3
(1 row)

 no_parens | parens
-----------+--------
        14 |     20
(1 row)
```

Integer division truncates toward zero (drops the fraction) in PostgreSQL, SQL Server and SQLite: `7 / 2` = `3`, `-7 / 2` = `-3`. A decimal operand (`2.0`) or a cast to `numeric` (PostgreSQL's exact-decimal type) keeps the fraction. MySQL differs: `/` always returns a decimal (`7 / 2` = `3.5000`) and `DIV` does integer division — a classic "my average is wrong" bug when porting.

Operators work row by row on columns — the value of each order line:

```sql
SELECT order_id, product_id, quantity, unit_price, discount,
       quantity * unit_price * (1 - discount) AS line_total
FROM order_items WHERE order_id IN (102, 104) ORDER BY order_id, product_id;
```

Output:

```
 order_id | product_id | quantity | unit_price | discount | line_total
----------+------------+----------+------------+----------+------------
      102 |          7 |        2 |     199.95 |     0.10 |   359.9100
      104 |          5 |        1 |     479.00 |     0.00 |   479.0000
      104 |          6 |        1 |     349.99 |     0.05 |   332.4905
(3 rows)
```

Comparison operators

Operator	Meaning	Example
`=`	equal	`unit_price = 49.50`
`<>` or `!=`	not equal (`<>` is standard)	`status <> 'cancelled'`
`<` , `>` , `<=` , `>=`	ordering	`salary >= 100000`
`BETWEEN a AND b`	inclusive range	2.4
`IN (…)`	membership in a list	2.4
`LIKE` , `ILIKE` , `~`	pattern match	2.4
`IS [NOT] NULL` , `IS [NOT] DISTINCT FROM`	NULL-aware tests	2.5

Logical operators

`AND` , `OR` , `NOT` — precedence in 2.2, NULL behaviour in 2.5.

String concatenation

`||` joins strings in PostgreSQL, SQLite and Oracle: `SELECT first_name || ' ' || last_name AS full_name FROM employees` → `Asha Rao` , `Vikram Menon` , …. In MySQL `||` is logical OR by default (use `CONCAT()`); SQL Server uses `+` or `CONCAT()` . Chapter 3 covers `concat()` and NULL handling.

Compound assignment operators

T-SQL (SQL Server) has shorthand `SET @x += 3` , also `-=` , `*=` , `/=` , `%=` , for **procedural variables** (named values in step-by-step code such as a stored procedure) and in `UPDATE … SET col += 1` . They are **not** standard SQL; PostgreSQL writes `SET col = col + 1` , or `x := x + 3` inside a PL/pgSQL function (PostgreSQL's built-in procedural language; chapter 14).

Dialect	Add-and-assign
PostgreSQL	`UPDATE t SET n = n + 3` ; PL/pgSQL `x := x + 3`
SQL Server	`SET @x += 3` ; `UPDATE t SET n += 3`
MySQL	`SET @x = @x + 3` (no `+=`)

Operator precedence (highest first, simplified)

Level	Operators
1	`::` (cast)
2	unary `-` / `+`
3	`^`
4	`*`, `/`, `%`
5	binary `+`, `-`
6	other operators such as `||`
7	`BETWEEN`, `IN`, `LIKE`, `ILIKE`
8	`<`, `>`, `=`, `<=`, `>=`, `<>`
9	`IS NULL`, `IS DISTINCT FROM` …
10	`NOT`
11	`AND`
12	`OR`

(*Unary* operators take one operand, like `-5`; *binary* take two.) Because unary minus binds tighter than `^`, `SELECT -2 ^ 2` returns `4` in PostgreSQL, not `-4`. When in doubt, parenthesise.

Self-check

1. *`SELECT 9 / 4` in PostgreSQL?* → `2`; `9 / 4.0` → `2.25`.
2. *Why might `first_name || ' ' || last_name` misbehave in MySQL?* → `||` is logical OR there by default.

2.4 Ranges, lists and patterns: BETWEEN, IN, LIKE/ILIKE, regex

BETWEEN — inclusive ranges

`x BETWEEN a AND b` means `x >= a AND x <= b`. **Both ends are included.**

```sql
-- code/03_between_in_like.sql
SELECT name, unit_price FROM products WHERE unit_price BETWEEN 49.50 AND 199.95 ORDER BY unit_price;
```

Output:

```
           name             | unit_price
----------------------------+------------
 USB-C Hub                  |      49.50
 Gift Card                  |      50.00
 Mechanical Keyboard        |      89.00
 Noise-Cancelling Headphones |     199.95
(4 rows)
```

`NOT BETWEEN` is the complement. `a` must be the smaller value — `BETWEEN 200 AND 50` matches nothing (PostgreSQL's `BETWEEN SYMMETRIC` accepts either order).

The timestamp trap. `placed_at` is a timestamp (date *and* time). `'2025-01-31'` means midnight at the *start* of 31 January, so an order at 23:55 that day is excluded:

```sql
SELECT order_id, placed_at FROM orders
WHERE placed_at BETWEEN '2025-01-01' AND '2025-01-31'
ORDER BY order_id;
```

Output:

```
 order_id |      placed_at
----------+---------------------
      101 | 2025-01-03 10:15:00
      102 | 2025-01-17 22:40:00
(2 rows)
```

Order 103 (31 Jan, 23:55) is missing. The robust pattern is **half-open**: `>=` the start and `<` the next period's start.

```
SELECT order_id, placed_at FROM orders
WHERE placed_at >= '2025-01-01' AND placed_at < '2025-02-01'
ORDER BY order_id;
```

Output:

```
 order_id |      placed_at
----------+---------------------
      101 | 2025-01-03 10:15:00
      102 | 2025-01-17 22:40:00
      103 | 2025-01-31 23:55:00
(3 rows)
```

IN — membership in a list

`x IN (a, b, c)` is shorthand for `x = a OR x = b OR x = c`; `NOT IN` negates it.

```
SELECT order_id, status FROM orders WHERE status IN ('pending', 'cancelled') ORDER BY order_id;
```

Output:

```
 order_id |  status
----------+-----------
      105 | cancelled
      111 | cancelled
      113 | pending
      114 | pending
(4 rows)
```

`WHERE job_title NOT IN ('Engineer', 'Sales Rep', 'Intern')` likewise returns the 9 other employees. `IN` also takes a subquery (`WHERE customer_id IN (SELECT …)`, chapter 6). **Warning:** `NOT IN` with a `NULL` in the list returns no rows (2.5).

LIKE and ILIKE — simple patterns

Wildcard	Matches	Example	Matches…
`%`	any sequence of zero or more characters	`'%Desk%'`	"Standing Desk", "Desk Lamp"
`_`	exactly one character	`'_a%'`	"Daniel", "Rahul", "Sara" (second letter *a*)

```
SELECT name FROM products WHERE name LIKE '%Desk%' ORDER BY name;
```

Output:

```
     name
---------------
 Desk Lamp
 Standing Desk
(2 rows)
```

`LIKE` is **case-sensitive** in PostgreSQL; `ILIKE` (PostgreSQL-only) ignores case:

```
SELECT email FROM employees WHERE email LIKE 'neha%';
SELECT email FROM employees WHERE email ILIKE 'neha%';
```

Output:

```
 email
-------
(0 rows)

           email
---------------------------
 Neha.Gupta@Company.example
(1 row)
```

Portable alternative: `lower(email) LIKE 'neha%'`.

Matching a literal `%` or `_` — use an **escape character** ("treat the next character as plain text"). PostgreSQL's default is backslash; `ESCAPE` chooses another:

```
SELECT label FROM (VALUES ('10% off'), ('100 off'), ('10_off')) AS t(label)
WHERE label LIKE '10\%%' ORDER BY label;
SELECT label FROM (VALUES ('10% off'), ('100 off'), ('10_off')) AS t(label)
WHERE label LIKE '10!_%' ESCAPE '!' ORDER BY label;
```

Output:

```
  label
---------
 10% off
(1 row)

 label
--------
 10_off
(1 row)
```

(`VALUES (…), (…) AS t(label)` builds a tiny inline table — handy for experiments.)

Regular expressions (PostgreSQL)

A **regular expression** (regex) is a mini-language for text patterns — `^` "start of text", `[A-Z]` "one capital letter", `+` "one or more of the previous"; POSIX is the Unix standard fixing this syntax. Operators: `~` (case-sensitive match), `~*` (case-insensitive), `!~` and `!~*` (no match). Names made of a capitalised word, a space, then another capital:

```
SELECT name FROM products WHERE name ~ '^[A-Z][a-z]+ [A-Z]' ORDER BY name;
```

Output:

```
        name
--------------------
 Desk Lamp
 Ergonomic Chair
 ...
 Wireless Mouse
(7 rows)
```

`USB-C Hub` fails (`S` isn't lower case) and `Noise-Cancelling…` fails (hyphen, not space). The standard's `SIMILAR TO` exists but is rarely used.

Performance note

`LIKE 'abc%'` (fixed prefix) can use a B-tree **index** (a sorted lookup structure beside the table that lets the database jump to matching rows; **B-tree** is the default, ordered kind) — in PostgreSQL the index needs the `text_pattern_ops` operator class (tells the index how to compare values) unless the database

uses the `C` collation (byte-by-byte comparison). `LIKE '%abc'` (leading wildcard) cannot, and scans every row. Chapter 12 shows trigram (`pg_trgm`) indexes for "contains" searches (a *trigram* is every 3-character slice of the text).

Dialect notes

Feature	PostgreSQL	MySQL	SQLite	SQL Server
`LIKE` case-sensitivity	sensitive	depends on collation (default insensitive)	insensitive for ASCII (plain English letters) by default	depends on collation (default insensitive)
Case-insensitive LIKE	`ILIKE`	`LIKE` (default)	`LIKE`	`LIKE` (default)
Regex	`~`, `~*`	`REGEXP` / `RLIKE`	`REGEXP` only when a `regexp()` function is loaded (the `sqlite3` shell includes one)	none built in (SQL Server 2025 adds `REGEXP_LIKE`)

Common mistakes

- `BETWEEN` on timestamps with date-only upper bounds (use half-open ranges).
- Forgetting `_` is a wildcard: `LIKE 'user_%'` also matches `userA…`.

Self-check

1. *Pattern for "exactly 4 characters ending in `n`"?* → `LIKE '___n'`.
2. *Names containing `%`?* → `LIKE '%\%%'` (or `ESCAPE`).

2.5 NULL and three-valued logic

What it is

`NULL` is a **marker for a missing value** — unknown ("we don't know Meera's email") or not applicable ("interns earn no commission"). It is not zero, not an empty string, and not a value, so it cannot *equal* anything — not even another `NULL`.

Why it matters

Real data is full of gaps, and `NULL` changes every comparison, filter and calculation. Most "missing rows" bugs come from it.

How it works — three-valued logic

A comparison involving `NULL` yields the third truth value **UNKNOWN** (displayed as `NULL`). Arithmetic and concatenation with `NULL` yield `NULL`.

```
-- code/04_nulls.sql
SELECT NULL = NULL AS null_eq_null, NULL <> 1 AS null_ne_1, NULL IS NULL AS is_null,
       1 + NULL AS plus, 'a' || NULL AS concat;
```

Output:

```
 null_eq_null | null_ne_1 | is_null |  plus  | concat
--------------+-----------+---------+--------+--------
 (null)       | (null)    | t       | (null) | (null)
(1 row)
```

Truth tables, generated by the database:

```
SELECT a, b, a AND b AS "a AND b", a OR b AS "a OR b", NOT a AS "NOT a"
FROM (VALUES (true), (false), (NULL::boolean)) AS x(a)
CROSS JOIN (VALUES (true), (false), (NULL::boolean)) AS y(b);
```

Output:

```
   a    |   b    | a AND b | a OR b | NOT a
--------+--------+---------+--------+--------
 t      | t      | t       | t      | f
 t      | f      | f       | t      | f
 t      | (null) | (null)  | t      | f
 f      | t      | f       | t      | t
 f      | f      | f       | f      | t
 f      | (null) | f       | (null) | t
 (null) | t      | (null)  | t      | (null)
 (null) | f      | f       | (null) | (null)
 (null) | (null) | (null)  | (null) | (null)
(9 rows)
```

Read UNKNOWN as "could be either": `FALSE AND UNKNOWN` is `FALSE`, `TRUE OR UNKNOWN` is `TRUE`, `NOT UNKNOWN` is UNKNOWN, anything else with UNKNOWN stays UNKNOWN. `WHERE` **keeps only TRUE** — FALSE and UNKNOWN rows are dropped.

Testing for NULL: IS NULL / IS NOT NULL

```
SELECT count(*) AS eq_null  FROM employees WHERE email = NULL;
SELECT count(*) AS is_null  FROM employees WHERE email IS NULL;
SELECT first_name, last_name FROM employees WHERE email IS NULL ORDER BY emp_id;
```

Output:

```
 eq_null
---------
       0
(1 row)

 is_null
---------
       2
(1 row)

 first_name | last_name
------------+-----------
 Meera      | Iyer
 Ananya     | Das
(2 rows)
```

`= NULL` is always UNKNOWN, so it never matches. (SQL Server has a legacy `ANSI_NULLS OFF` mode where `= NULL` works; don't rely on it.)

The "missing rows" trap

Sales reps have commission `0.10`, `0.10` and `NULL`. `= 0.10` and `<> 0.10` should cover everyone — but Sara (NULL) is in neither:

```
SELECT first_name, commission_pct FROM employees WHERE job_title = 'Sales Rep' AND commission_pct =  0.10 ORDER BY
emp_id;
SELECT first_name, commission_pct FROM employees WHERE job_title = 'Sales Rep' AND commission_pct <> 0.10 ORDER BY
emp_id;
```

Output:

```
 first_name | commission_pct
------------+----------------
 Priya      |           0.10
 Arjun      |           0.10
(2 rows)

 first_name | commission_pct
------------+----------------
(0 rows)
```

`IS DISTINCT FROM` is the NULL-safe "not equal": two NULLs are equal, NULL vs a value differ, and the result is always TRUE or FALSE.

```
SELECT first_name, commission_pct FROM employees
WHERE job_title = 'Sales Rep' AND commission_pct IS DISTINCT FROM 0.10 ORDER BY emp_id;
```

Output:

```
 first_name | commission_pct
------------+----------------
 Sara       |         (null)
(1 row)
```

(MySQL's NULL-safe equality is `<=>`; SQL Server supports `IS [NOT] DISTINCT FROM` from 2022.)

The NOT IN trap

`x NOT IN (1, 2, NULL)` expands to `x <> 1 AND x <> 2 AND x <> NULL`; the last part is always UNKNOWN, so the condition is never TRUE:

```
SELECT count(*) AS not_in_with_null FROM departments WHERE dept_id NOT IN (1, 2, NULL);
SELECT count(*) AS not_in_no_null   FROM departments WHERE dept_id NOT IN (1, 2);
```

Output:

```
 not_in_with_null
------------------
                0
(1 row)

 not_in_no_null
----------------
              4
(1 row)
```

It bites hardest with subqueries: `WHERE id NOT IN (SELECT manager_id FROM employees)` returns nothing because the CEO's `manager_id` is NULL. Use `NOT EXISTS` (chapter 6).

NULL is not zero or ''

```
SELECT count(*) AS rows, count(commission_pct) AS non_null_commissions FROM employees;
SELECT '' IS NULL AS empty_string_is_null;
```

Output:

```
 rows | non_null_commissions
------+----------------------
   15 |                    3
(1 row)

 empty_string_is_null
----------------------
 f
(1 row)
```

`count(*)` counts rows; `count(column)` counts non-NULL values (chapter 4). An empty string is a real value — except in **Oracle**, which stores `''` as NULL.

Allowing or forbidding NULL in a column

Columns accept NULL **unless** declared `NOT NULL`; writing `NULL` in a definition just states the default.

```sql
CREATE TABLE students (
    id    integer PRIMARY KEY,
    email text NULL,          -- explicit "NULL allowed" (the default anyway)
    name  text NOT NULL       -- a value is required
);
INSERT INTO students VALUES (1, NULL, 'Ravi');
INSERT INTO students VALUES (2, 'kim@school.example', NULL);
SELECT * FROM students WHERE email IS NULL;
```

Output:

```
CREATE TABLE
INSERT 0 1
ERROR:  null value in column "name" of relation "students" violates not-null constraint
DETAIL:  Failing row contains (2, kim@school.example, null).
 id | email  | name
----+--------+------
  1 | (null) | Ravi
(1 row)
```

Replacing NULLs in output (`COALESCE` , `NULLIF`): chapter 3; NULLs in sorting 2.7, in `DISTINCT` 2.6, in aggregates chapter 4, in joins chapter 5.

In the interview

30-second answer: "NULL means unknown, so comparisons with it give UNKNOWN, and WHERE only keeps TRUE rows. Use IS NULL / IS NOT NULL, or IS DISTINCT FROM for NULL-safe comparison. NOT IN with a NULL in the list or subquery returns nothing, so prefer NOT EXISTS. Aggregates like COUNT(col) and SUM skip NULLs."

- *What does `NULL = NULL` return?* → NULL (UNKNOWN), not TRUE.
- *Why can `WHERE x <> 5` miss rows?* → Rows where `x` is NULL give UNKNOWN and are filtered out.
- *Is `''` NULL?* → Not in PostgreSQL, MySQL, SQL Server or SQLite; yes in Oracle.

Self-check

1. *`TRUE OR NULL` ? `FALSE AND NULL` ? `NOT NULL` (boolean)?* → TRUE, FALSE, NULL.
2. *Rows returned by `WHERE email = NULL` ?* → Zero, always.

2.6 DISTINCT: removing duplicate rows

What it is

`SELECT DISTINCT` removes duplicate rows from the **result** (the table is untouched) — "which categories exist?", "which cities do customers live in?".

How it works

```sql
-- code/05_distinct.sql
SELECT DISTINCT category FROM products ORDER BY category;
```

Output (the 9 rows collapse to 4):

```
 category
-------------
 Accessories
 Electronics
 Furniture
 (null)
(4 rows)
```

Two rules:

1. **NULLs count as duplicates of each other** for `DISTINCT` (unlike `=`) — NULL categories collapse into one row.
2. `DISTINCT` **applies to the whole selected row**, not the first column — with two columns you get distinct *pairs*:

```
SELECT DISTINCT country, city FROM customers ORDER BY country, city;
```

Output:

```
 country |   city
---------+-----------
 India   | Bengaluru
 India   | Delhi
 India   | Mumbai
 India   | mumbai
 India   | (null)
 UK      | London
 USA     | New York
(7 rows)
```

`Mumbai` and `mumbai` both survive — `DISTINCT` removes only **exact** duplicates. Clean dirty data first (`trim` strips leading/trailing spaces; `lower` lower-cases). Anita Shah was entered twice with differently formatted emails:

```
SELECT DISTINCT first_name, last_name, email FROM customers WHERE last_name = 'Shah';
SELECT DISTINCT first_name, last_name, lower(trim(email)) AS email FROM customers WHERE last_name = 'Shah';
```

Output:

```
 first_name | last_name |           email
------------+-----------+---------------------------
 Anita      | Shah      |   Anita.Shah@Mail.example
 Anita      | Shah      | anita.shah@mail.example
(2 rows)

 first_name | last_name |          email
------------+-----------+------------------------
 Anita      | Shah      | anita.shah@mail.example
(1 row)
```

Counting distinct values: `count(DISTINCT expr)`, which ignores NULLs (aggregates — functions such as `count` and `sum` that combine many rows into one value — are chapter 4):

```
SELECT count(*) AS rows, count(city) AS with_city, count(DISTINCT city) AS distinct_cities,
       count(DISTINCT lower(city)) AS distinct_cities_ci
FROM customers;
```

Output:

```
 rows | with_city | distinct_cities | distinct_cities_ci
------+-----------+-----------------+--------------------
   10 |         9 |               6 |                  5
(1 row)
```

DISTINCT ON (PostgreSQL)

`DISTINCT ON (expr)` keeps the **first row of each group** of equal `expr` values, "first" being decided by `ORDER BY` (which must start with the same expressions). Top earner per department:

```
SELECT DISTINCT ON (dept_id) dept_id, first_name, salary
FROM employees
WHERE dept_id IS NOT NULL
ORDER BY dept_id, salary DESC, emp_id;
```

Output:

```
 dept_id | first_name |  salary
---------+------------+-----------
       1 | Vikram     | 185000.00
       2 | Daniel     | 150000.00
       3 | Lucas      | 120000.00
       4 | Kiran      | 135000.00
       5 | Li         |  64000.00
(5 rows)
```

Other databases use window functions (which look at related rows without merging them, e.g. `ROW_NUMBER()` — chapter 8).

Trade-offs and when not to use it

- `DISTINCT` sorts or hashes the whole result (hashing turns each row into a number used to bucket equal rows) — expensive on millions of rows.
- **Don't use `DISTINCT` to hide a bad join.** Unexpected duplicates usually mean a wrong join condition (*fan-out*, chapter 5); `DISTINCT` masks the symptom and can hide real duplicates.
- For counts per value use `GROUP BY` (chapter 4), not `DISTINCT`.

Self-check

1. *Does `SELECT DISTINCT a, b` make `a` unique?* → No; each `(a, b)` pair.
2. *Why did `count(DISTINCT city)` give 6 but the case-insensitive version 5?* → `Mumbai` and `mumbai` match only after `lower()`.

2.7 ORDER BY: sorting results

What it is

Without `ORDER BY`, rows come back in **no guaranteed order** — whatever is fastest at that moment. `ORDER BY` is the only way to get a defined order. `ASC` (smallest first) is the default; `DESC` is descending.

```
ORDER BY column1 [ASC | DESC], column2 [ASC | DESC];
```

How it works

```
-- code/06_order_by.sql
SELECT first_name, salary FROM employees ORDER BY salary DESC;
```

Output (abridged):

```
 first_name |  salary
------------+-----------
 Asha       | 250000.00
 Vikram     | 185000.00
 ...
 Meera      |  98000.00
 Rahul      |  98000.00
 ...
 Rohan      |  30000.00
(15 rows)
```

Meera and Rahul tie; their relative order is **not guaranteed**. Add a unique **tie-breaker** column whenever order matters (especially for paging) so the order is **deterministic** — the same every run.

Several sort keys — the second key only matters within ties of the first, each with its own direction:

```
SELECT dept_id, first_name, salary FROM employees
ORDER BY dept_id, salary DESC, first_name;
```

Output (abridged):

```
 dept_id | first_name |  salary
---------+------------+-----------
       1 | Vikram     | 185000.00
       1 | Neha       | 142000.00
       1 | Meera      |  98000.00
       1 | Rahul      |  98000.00
       2 | Daniel     | 150000.00
 ...
       5 | Li         |  64000.00
  (null) | Asha       | 250000.00
  (null) | Rohan      |  30000.00
(15 rows)
```

Where do NULLs go?

PostgreSQL treats NULL as **larger than any value**: **last in ASC**, **first in DESC**. Override with `NULLS FIRST` / `NULLS LAST`:

```
SELECT name, location FROM departments ORDER BY location NULLS FIRST;
```

Output:

```
    name     | location
-------------+-----------
 Support     | (null)
 Engineering | Bengaluru
 Sales       | Mumbai
 Marketing   | Mumbai
 Finance     | Pune
 Research    | Pune
(6 rows)
```

(Plain `ORDER BY location` puts Support last; `ORDER BY location DESC NULLS LAST` gives Pune first and Support last.)

Database	NULLs in ASC	NULLS FIRST/LAST supported?
PostgreSQL, Oracle	last	yes
MySQL, SQL Server, SQLite	first	SQLite yes (3.30+); MySQL/SQL Server no — use `ORDER BY col IS NULL, col` (MySQL) or a `CASE` expression

Sorting by alias, expression or position

```
SELECT name, unit_price * 1.18 AS gross FROM products ORDER BY gross DESC LIMIT 3;
SELECT first_name, last_name FROM employees ORDER BY length(last_name) DESC, last_name LIMIT 3;
SELECT first_name, hire_date FROM employees ORDER BY 2 LIMIT 3;
```

Output:

```
     name       |   gross
----------------+----------
 Laptop Pro 14  | 1532.8200
 Standing Desk  |  588.8200
 Ergonomic Chair |  412.9882
(3 rows)

 first_name | last_name
------------+-----------
 Daniel     | Brooks
 Omar       | Farouk
 Lucas      | Martin
(3 rows)

 first_name | hire_date
------------+------------
 Asha       | 2015-01-05
 Kiran      | 2016-12-01
 Daniel     | 2017-09-25
(3 rows)
```

`ORDER BY 2` means "the second select-list column" — quick in psql, fragile in code (reorder the list and the sort silently changes).

Custom order with CASE

Sort statuses by workflow, not alphabetically (`CASE` is chapter 3):

```
SELECT order_id, status FROM orders
ORDER BY CASE status WHEN 'pending' THEN 1 WHEN 'shipped' THEN 2
                    WHEN 'delivered' THEN 3 ELSE 4 END, order_id;
```

Output (abridged):

```
 order_id |  status
----------+-----------
      113 | pending
      114 | pending
      107 | shipped
      110 | shipped
      112 | shipped
      101 | delivered
      ...
      109 | delivered
      105 | cancelled
      111 | cancelled
(14 rows)
```

Text order depends on collation

A **collation** is the rule set for comparing and sorting text in a language. The book's server uses the `C` collation (byte order), so all capitals sort before all lower-case letters:

```
SELECT city FROM customers WHERE city IS NOT NULL ORDER BY city;
```

Output:

```
   city
-----------
 Bengaluru
 Delhi
 Delhi
 London
 London
 Mumbai
 New York
 New York
 mumbai
(9 rows)
```

`ORDER BY lower(city), city` puts `mumbai` next to `Mumbai`; so would a database created with a language collation such as `en_US.UTF-8`, or per query `ORDER BY city COLLATE "en-x-icu"` (needs ICU — International Components for Unicode, a library of language-aware text rules).

Common mistakes

- Relying on insertion order or "it always came back sorted".
- No unique tie-breaker, then paging (2.8) shows some rows twice and others never.
- Numbers stored as text sort alphabetically: `'10' < '9'`.

Self-check

1. *Where does PostgreSQL put NULLs with `ORDER BY x DESC`?* → First.
2. *Why add `emp_id` to `ORDER BY salary DESC`?* → Unique tie-breaker for a deterministic order.

2.8 LIMIT, OFFSET and FETCH: returning a page

What it is

`LIMIT n` returns at most `n` rows; `OFFSET m` skips the first `m`. Together they return "page *k*". The SQL-standard spelling is `OFFSET m ROWS FETCH FIRST n ROWS ONLY`. **Always pair them with `ORDER BY`** — "the first 3 rows" means nothing without an order.

How it works

```sql
-- code/07_limit_offset.sql
-- Top 3 earners
SELECT first_name, salary FROM employees ORDER BY salary DESC LIMIT 3;

-- Page 2 with 5 rows per page (skip 5)
SELECT emp_id, first_name FROM employees ORDER BY emp_id LIMIT 5 OFFSET 5;

-- Standard SQL spelling of the same thing
SELECT emp_id, first_name FROM employees ORDER BY emp_id
OFFSET 5 ROWS FETCH FIRST 5 ROWS ONLY;
```

Output:

```
 first_name |  salary
------------+----------
 Asha       | 250000.00
 Vikram     | 185000.00
 Daniel     | 150000.00
(3 rows)

 emp_id | first_name
--------+------------
      6 | Daniel
      7 | Priya
      8 | Arjun
      9 | Sara
     10 | Lucas
(5 rows)
```

(The `FETCH` version returns the same 5 rows.) Page *p* (from 1) of size *s*: `LIMIT s OFFSET (p - 1) * s`.

Ties: FETCH … WITH TIES

`LIMIT 1` on a tie picks one row arbitrarily. `FETCH FIRST n ROWS WITH TIES` (PostgreSQL 13+, requires `ORDER BY`) also returns every row tying with the last:

```
SELECT first_name, salary FROM employees WHERE dept_id IN (1, 2)
ORDER BY salary LIMIT 1;
SELECT first_name, salary FROM employees WHERE dept_id IN (1, 2)
ORDER BY salary FETCH FIRST 1 ROWS WITH TIES;
```

Output:

```
 first_name |  salary
------------+----------
 Arjun      | 68000.00
(1 row)

 first_name |  salary
------------+----------
 Arjun      | 68000.00
 Sara       | 68000.00
(2 rows)
```

"Lowest-paid person" has two correct answers; `WITH TIES` returns both (chapter 8's ranking functions generalise this).

Why OFFSET gets slow — keyset pagination

`OFFSET 100000` still produces and discards 100,000 rows, so deep pages slow down; rows inserted or deleted between page loads shift pages, so users see duplicates or miss rows. **Keyset** ("seek") pagination remembers the last key shown and asks for rows after it:

```
-- page after the one that ended at emp_id 10
SELECT emp_id, first_name FROM employees WHERE emp_id > 10 ORDER BY emp_id LIMIT 5;
```

Output:

```
 emp_id | first_name
--------+------------
     11 | Ananya
     12 | Kiran
     13 | Omar
     14 | Li
     15 | Rohan
(5 rows)
```

With an index on the key, every page costs the same however deep. Trade-off: "next/previous" only, no jump to page 500. Chapter 12 has details.

Dialect differences

Database	First 10 rows	Skip 20, take 10
PostgreSQL	`LIMIT 10` or `FETCH FIRST 10 ROWS ONLY`	`LIMIT 10 OFFSET 20` or `OFFSET 20 ROWS FETCH NEXT 10 ROWS ONLY`
MySQL	`LIMIT 10`	`LIMIT 10 OFFSET 20` or `LIMIT 20, 10` (offset first!)
SQLite	`LIMIT 10`	`LIMIT 10 OFFSET 20`
SQL Server	`SELECT TOP (10) …`	`ORDER BY … OFFSET 20 ROWS FETCH NEXT 10 ROWS ONLY`
Oracle 12c+	`FETCH FIRST 10 ROWS ONLY`	`OFFSET 20 ROWS FETCH NEXT 10 ROWS ONLY`

Common mistakes

- `LIMIT` without `ORDER BY` — rows that can change between runs.
- Reading MySQL's `LIMIT 20, 10` as "20 rows starting at 10" — it's the reverse.
- Paging on a non-unique sort key — boundary rows repeat or vanish.

In the interview

> **30-second answer:** "`ORDER BY` plus `LIMIT n OFFSET m`, or the standard `OFFSET … FETCH`. Deep OFFSET is O(offset) (work grows in proportion to the offset) and unstable under inserts, so I'd use keyset pagination — `WHERE (sort_key, id) > (:last_key, :last_id) ORDER BY sort_key, id LIMIT n` — backed by an index." (`:last_key` / `:last_id` are placeholders the application fills with the last row shown.)

- *Top 3 including ties?* → `FETCH FIRST 3 ROWS WITH TIES`, or `DENSE_RANK()` (chapter 8).
- *SQL Server equivalent of LIMIT?* → `TOP (n)` or `OFFSET … FETCH`.

Self-check

1. `LIMIT 5 OFFSET 12` *over 15 rows?* → 3 rows (13–15).
2. *Page 4, size 20?* → `LIMIT 20 OFFSET 60`.
3. *What does keyset pagination need?* → A stable, unique ordering key, ideally indexed.

2.9 How a SELECT is evaluated (logical processing order)

What it is

You *write* `SELECT … FROM … WHERE … ORDER BY … LIMIT`, but the database *evaluates* it logically in a different order, which explains many errors.



(Chapter 4 inserts `GROUP BY` and `HAVING` between `WHERE` and `SELECT`, and window functions after them.)

- `WHERE` **cannot use a column alias** from the `SELECT` list — `SELECT` hasn't happened yet.
- `ORDER BY` **can use aliases** (it runs after `SELECT`) — but in PostgreSQL only a bare alias: `ORDER BY gross` works, `ORDER BY gross + 0` fails with *column "gross" does not exist* (inside an expression, names refer to input columns).
- `LIMIT` is applied last, to the sorted result.

```sql
-- code/01_select_columns.sql (end)
SELECT name, unit_price * 1.18 AS gross FROM products WHERE gross > 500;
SELECT name, unit_price * 1.18 AS gross FROM products WHERE unit_price * 1.18 > 500 ORDER BY gross DESC;
```

Output:

```
ERROR:  column "gross" does not exist
LINE 1: ...e, unit_price * 1.18 AS gross FROM products WHERE gross > 50...
                                                             ^
     name      |   gross
---------------+-----------
 Laptop Pro 14 | 1532.8200
 Standing Desk |  588.8200
(2 rows)
```

This *logical* order defines the meaning; the optimizer (the component choosing how to run a query) may physically differ — e.g. read rows already sorted from an index — as long as the result is the same.

Putting it together

"The three most recent non-cancelled 2025 orders placed through a sales rep":

```sql
-- code/09_putting_it_together.sql
SELECT order_id, customer_id, sales_rep_id, placed_at, status
FROM orders
WHERE status <> 'cancelled'
  AND sales_rep_id IS NOT NULL
  AND placed_at >= '2025-01-01' AND placed_at < '2026-01-01'
ORDER BY placed_at DESC, order_id DESC
LIMIT 3;
```

Output:

```
 order_id | customer_id | sales_rep_id |      placed_at      | status
----------+-------------+--------------+---------------------+---------
      113 |           1 |            7 | 2025-06-02 10:30:00 | pending
      112 |           5 |            9 | 2025-05-20 21:05:00 | shipped
      110 |           7 |            8 | 2025-04-25 11:10:00 | shipped
(3 rows)
```

Chapter 2 TL;DR

- `SELECT cols FROM t` — name your columns (`*` only to explore); any expression can be a column; `AS` aliases name it; table aliases shorten references.
- `WHERE` keeps TRUE rows only. Precedence `NOT` > `AND` > `OR`: parenthesise. Text `=` is case-sensitive in PostgreSQL.
- Integer division truncates in PostgreSQL (`7/2 = 3`); `<>` is not-equal; `||` concatenates; compound `+=` is T-SQL/procedural, not standard SQL.
- `BETWEEN` is inclusive (use half-open `>=` / `<` ranges for timestamps); `IN` = ORs; `LIKE` (`%` , `_`) is case-sensitive, `ILIKE` isn't; `~` for regex.
- NULL means unknown → three-valued logic. Use `IS [NOT] NULL` , `IS [NOT] DISTINCT FROM` ; never put a nullable list inside `NOT IN` .
- `DISTINCT` removes exact duplicate rows over the whole select list (NULLs grouped); clean with `lower` / `trim` first; `DISTINCT ON` keeps the first row per group.
- Only `ORDER BY` gives order; add unique tie-breakers; `NULLS FIRST/LAST` ; collation affects text order.
- `LIMIT/OFFSET` or `OFFSET … FETCH` ; `WITH TIES` keeps ties; keyset pagination for deep pages.
- Logical order FROM → WHERE → SELECT → DISTINCT → ORDER BY → LIMIT; aliases exist from SELECT onward.

Next: Chapter 3 — Functions and expressions.

Chapter 3 — Functions and Expressions

Reports need to *transform* stored values: round a price, tidy an email, compute days-to-ship, label a product "premium", show "no email on file" instead of a blank. This chapter covers SQL's **type system** and conversions, **number**, **string** and **date/time** functions, and the **conditional expressions** `CASE`, `COALESCE`, `NULLIF`, `GREATEST` and `LEAST`. Function names differ between databases more than anything else in SQL (the source course used MySQL), so each section ends with a **dialect map** from MySQL (and SQL Server, SQLite) to PostgreSQL.

Sections: 3.1 Types and casts · 3.2 Number functions · 3.3 String functions · 3.4 Date and time functions · 3.5 Conditional expressions · 3.6 Worked exercises.

Setup. Examples use the `company` sample database (chapter 1, 1.11); code is in `code/`. Outputs come from PostgreSQL 14.

Functions in one paragraph

A **function** takes zero or more **arguments** and returns one value: `round(9.876, 1)` → `9.9`. **Scalar functions** work on one row at a time (this chapter); **aggregate functions** squash many rows into one (`sum`, `count`, `avg` — chapter 4). Functions nest (`lower(trim(email))`) and go anywhere an expression can: `SELECT`, `WHERE`, `ORDER BY`, `GROUP BY`, `UPDATE … SET`.

3.1 Data types and conversions

What it is

Every value has a **data type** deciding what it holds and which operators apply: `'42' || '1'` is `'421'`, `42 + 1` is `43`. PostgreSQL is **strict** — it rarely converts types silently, which catches bugs early.

The core PostgreSQL types

Family	Type	Holds	Notes
Integer	`smallint`, `integer` (`int`), `bigint`	2-, 4-, 8-byte whole numbers	`integer` max is 2,147,483,647
Exact decimal	`numeric(p, s)` / `decimal(p, s)`	exact numbers, *p* total digits, *s* after the point	use for **money**; slower than floats
Floating point	`real`, `double precision`	approximate binary fractions	science/measurements, never money
Text	`text`, `varchar(n)`, `char(n)`	strings	in PostgreSQL `text` and `varchar` perform the same; `char(n)` pads with spaces
Boolean	`boolean`	`true`, `false`, `NULL`	
Date/time	`date`, `time`, `timestamp`, `timestamptz`, `interval`	calendar dates, clock times, instants, durations	3.4
Other	`uuid`, `json` / `jsonb`, arrays (`int[]`), `bytea`, enums	uuid = random 128-bit ID; jsonb = stored JSON; array = list in one cell; bytea = raw bytes; enum = fixed set of labels	JSON and arrays in chapter 14

Concept	PostgreSQL	MySQL	SQL Server	SQLite
Exact decimal	`numeric(10,2)`	`DECIMAL(10,2)`	`DECIMAL(10,2)`	stored as REAL/INTEGER (type *affinity*)
Variable text	`text` / `varchar(n)`	`VARCHAR(n)`, `TEXT`	`VARCHAR(n)` / `NVARCHAR(n)`	`TEXT`
Boolean	`boolean`	`TINYINT(1)` alias `BOOLEAN`	`BIT`	integer 0/1
Timestamp with zone	`timestamptz`	`TIMESTAMP` (converted to UTC, the world reference time)	`DATETIMEOFFSET`	text or number, by convention

(SQLite's column type is only a *preference* — it will store text in an integer column. DDL (Data Definition Language, `CREATE` / `ALTER`) types in full: chapter 9.)

How it works — asking for and changing types

`pg_typeof(x)` reports a value's type:

```sql
-- code/01_types_and_casts.sql
SELECT pg_typeof(42) AS int_lit, pg_typeof(42.0) AS dec_lit, pg_typeof(4.2e1) AS sci_lit,
       pg_typeof('hi') AS str_lit, pg_typeof(true) AS bool_lit, pg_typeof(now()) AS now_type;
SELECT pg_typeof(salary) AS salary, pg_typeof(hire_date) AS hire_date, pg_typeof(email) AS email
FROM employees LIMIT 1;
```

Output:

```
 int_lit | dec_lit | sci_lit | str_lit | bool_lit |         now_type
---------+---------+---------+---------+----------+--------------------------
 integer | numeric | numeric | unknown | boolean  | timestamp with time zone
(1 row)

 salary  | hire_date | email
---------+-----------+-------
 numeric | date      | text
(1 row)
```

`42.0` is `numeric` (exact), not a float; a quoted literal starts as `unknown` and takes its type from context — why `WHERE hire_date >= '2023-01-01'` reads the string as a `date`.

Casting (explicit conversion) has three spellings:

Form	Standard?	Example
`CAST(expr AS type)`	yes — works everywhere	`CAST('2025-03-18' AS date)`
`expr::type`	PostgreSQL shorthand	`'2025-03-18'::date`
`type 'literal'`	standard, literals only	`date '2025-03-18'`

```sql
SELECT CAST('2025-03-18' AS date) AS std_cast, '2025-03-18'::date AS pg_cast, date '2025-03-18' AS typed_literal;
SELECT CAST(100 AS numeric(5,2)) AS to_numeric, 3.99::integer AS rounds_not_truncates,
       '42'::integer + 1 AS text_to_int, 123::text || 'abc' AS int_to_text;
```

Output:

```
  std_cast  |  pg_cast   | typed_literal
------------+------------+--------------
 2025-03-18 | 2025-03-18 | 2025-03-18
(1 row)

 to_numeric | rounds_not_truncates | text_to_int | int_to_text
------------+----------------------+-------------+-------------
     100.00 |                    4 |          43 | 123abc
(1 row)
```

Casting a decimal to `integer` **rounds** (3.99 → 4); use `trunc()` or `floor()` if you want to cut off the fraction.

Exact vs approximate numbers

`double precision` stores binary fractions, and most decimal fractions (like 0.1) have no exact binary form:

```
SELECT 0.1 + 0.2 = 0.3 AS numeric_equal,
       0.1::double precision + 0.2::double precision = 0.3 AS float_equal,
       0.1::double precision + 0.2::double precision AS float_sum;
SELECT 12.345::numeric(5,2) AS rounded_to_scale;
```

Output:

```
 numeric_equal | float_equal |      float_sum
---------------+-------------+---------------------
 t             | f           | 0.30000000000000004
(1 row)

 rounded_to_scale
------------------
            12.35
(1 row)
```

Rule: **money and anything that must add up exactly** → `numeric`. A `numeric(p,s)` value is rounded to *s* decimal places when stored.

Errors a strict type system gives you

```
SELECT 2147483647::integer + 1;
SELECT 'abc'::integer;
SELECT 1000::numeric(5,2);
SELECT first_name + 1 FROM employees LIMIT 1;
```

Output:

```
ERROR:  integer out of range
ERROR:  invalid input syntax for type integer: "abc"
LINE 1: SELECT 'abc'::integer;
               ^
ERROR:  numeric field overflow
DETAIL:  A field with precision 5, scale 2 must round to an absolute value less than 10^3.
ERROR:  operator does not exist: text + integer
LINE 1: SELECT first_name + 1 FROM employees LIMIT 1;
                          ^
HINT:  No operator matches the given name and argument types. You might need to add explicit type casts.
```

MySQL would quietly turn `'abc' + 1` into `1` (with a warning) and wrap or clip some overflows (values too big for their type) depending on its SQL mode (a server strictness setting). PostgreSQL refuses — annoying for a minute, valuable for years. (Booleans accept several input spellings: `'yes'::boolean`, `'on'` → `t`; `'f'` → `f`.)

Common mistakes

- Storing money in `real` / `double precision`.
- Storing numbers or dates in text columns — sorting and comparisons become alphabetical (`'10' < '9'`).
- Expecting `::integer` to truncate; it rounds.

Self-check

1. *Portable way to convert text to a date?* → `CAST('2025-03-18' AS date)`.
2. *Why is `0.1::float8 + 0.2::float8 = 0.3` false?* → Binary floating point can't represent 0.1 or 0.2 exactly.

3.2 Number functions

What it is

Rounding, absolute differences, powers and logarithms (a **logarithm** answers "to what power must I raise the base to get this number?" — $\log_{10} 1000 = 3$), modulo for "every n-th", random numbers for sampling.

The functions

Function (PostgreSQL)	Does	Example → result
`abs(x)`	absolute value	`abs(-10)` → 10
`round(x)` / `round(x, d)`	round to *d* decimals (negative *d* rounds left of the point)	`round(9.876, 1)` → 9.9; `round(1234.567, -2)` → 1200
`trunc(x, d)`	cut to *d* decimals, no rounding	`trunc(9.876, 1)` → 9.8
`ceil(x)` / `ceiling(x)`	smallest integer ≥ x (round **up**)	`ceil(4.3)` → 5; `ceil(-4.3)` → -4
`floor(x)`	largest integer ≤ x (round **down**)	`floor(4.8)` → 4; `floor(-4.3)` → -5
`sqrt(x)` , `cbrt(x)`	square / cube root	`sqrt(25)` → 5
`power(a, b)` , `a ^ b`	a to the power b	`power(2, 3)` → 8
`mod(a, b)` , `a % b`	remainder	`mod(17, 5)` → 2
`div(a, b)`	integer quotient	`div(17, 5)` → 3
`exp(x)`	e^x (e ≈ 2.718, the natural growth constant)	`exp(2)` → 7.389…
`ln(x)`	natural log (base e)	`ln(10)` → 2.302585…
`log(x)` , `log10(x)`	**base-10** log in PostgreSQL	`log(100)` → 2
`log(b, x)`	log of x in base b	`log(2, 8)` → 3
`sign(x)`	-1, 0 or 1	`sign(-15)` → -1
`pi()` , `degrees(r)` , `radians(d)`	π and angle conversion	`radians(60)` → 1.047…
`greatest(…)` , `least(…)`	largest / smallest argument	`greatest(3, 9, 4)` → 9
`gcd(a, b)` , `lcm(a, b)`	greatest common divisor / least common multiple (PostgreSQL 13+)	`gcd(12, 18)` → 6
`random()` , `setseed(s)`	random `double precision` in [0, 1) (0 included, 1 excluded); seed it (give a fixed starting value) for repeatable output	
`to_char(x, fmt)`	format a number as text	`to_char(1234567, 'FM9,999,999.00')` → `1,234,567.00`

How it works — run them

Every table example is run in `code/02_number_functions.sql` with the results shown. With negative numbers `ceil` moves toward zero and `floor` away from it — `floor(-4.3)` is -5, not -4; `trunc` always moves toward zero.

Halfway cases depend on the type:

```
SELECT round(2.5) AS num_2_5, round(-2.5) AS num_neg_2_5,
       round(2.5::double precision) AS dbl_2_5, round(3.5::double precision) AS dbl_3_5;
```

Output:

```
 num_2_5 | num_neg_2_5 | dbl_2_5 | dbl_3_5
---------+-------------+---------+---------
       3 |          -3 |       2 |       4
(1 row)
```

`numeric` rounds halves **away from zero** (2.5 → 3); `double precision` typically rounds halves **to even** ("banker's rounding", 2.5 → 2, 3.5 → 4), following the platform's floating-point rounding. Another reason to keep money in `numeric`.

Remember: the remainder keeps the sign of the **dividend** (the number divided; `mod(-17, 5)` = -2), and `log(x)` is **base 10** in PostgreSQL but **natural log** in MySQL and SQL Server — write `ln()` for natural log.

Random numbers

`random()` returns a new value each call; `setseed()` makes the sequence repeatable for the rest of the session (your current connection) — for tests and reproducible samples.

```
SELECT setseed(0.42);
SELECT round(random()::numeric, 4) AS r1, round(random()::numeric, 4) AS r2;
SELECT floor(random() * 6)::int + 1 AS dice_roll;
```

Output:

```
   r1   |   r2
--------+--------
 0.5699 | 0.2954
(1 row)

 dice_roll
-----------
         6
(1 row)
```

`floor(random() * n)::int + 1` gives a whole number from 1 to *n*. (Sampling rows randomly — `ORDER BY random() LIMIT k` or `TABLESAMPLE` — comes up in chapter 12.)

Formatting numbers for people

`to_char(number, pattern)` turns numbers into display strings. `9` is a digit position, `0` a digit that's always shown, `,` a group separator, `.` the decimal point, and `FM` ("fill mode") strips the padding spaces `to_char` otherwise adds.

```
SELECT to_char(1234567, 'FM9,999,999.00') AS with_commas,
       to_char(0.0725, 'FM990.00%') AS naive_pct,
       to_char(0.0725 * 100, 'FM990.00') || '%' AS pct,
       to_char(42, '00000') AS zero_padded;
```

Output:

```
 with_commas  | naive_pct |  pct  | zero_padded
--------------+-----------+-------+-------------
 1,234,567.00 | 0.07%     | 7.25% |  00042
(1 row)
```

`%` prints literally — it does *not* multiply by 100. Without `FM`, a leading space is reserved for the sign (`' 00042'`). Format only at the very end: the result is text and no longer sorts or sums as a number.

Real use on the sample data

```
SELECT order_id, product_id,
       quantity * unit_price * (1 - discount)           AS exact,
       round(quantity * unit_price * (1 - discount), 2)  AS rounded
FROM order_items WHERE order_id IN (104, 107, 109) ORDER BY order_id, product_id;

-- Salary per month, rounded up to the next hundred
SELECT first_name, salary, ceil(salary / 12 / 100) * 100 AS monthly_rounded_up
FROM employees WHERE emp_id <= 4 ORDER BY emp_id;
```

Output:

```
 order_id | product_id |  exact   | rounded
----------+------------+----------+---------
      104 |          5 | 479.0000 |  479.00
      104 |          6 | 332.4905 |  332.49
      107 |          6 | 629.9820 |  629.98
      109 |          4 | 151.3000 |  151.30
(4 rows)

 first_name |  salary   | monthly_rounded_up
------------+-----------+--------------------
 Asha       | 250000.00 |              20900
 Vikram     | 185000.00 |              15500
 Neha       | 142000.00 |              11900
 Rahul      |  98000.00 |               8200
(4 rows)
```

"Divide, round, multiply back" (`ceil(x / 100) * 100`) works for any step — 5 minutes, 1,000 rupees.

Dialect map (numbers)

Task	MySQL	PostgreSQL	SQL Server	SQLite
Truncate decimals	`TRUNCATE(x, d)`	`trunc(x, d)`	`ROUND(x, d, 1)`	— (cast tricks)
Random 0–1	`RAND()`	`random()`	`RAND()`	`random()` returns a 64-bit integer
Natural log	`LOG(x)` or `LN(x)`	`ln(x)`	`LOG(x)`	`ln(x)` (3.35+ math functions)
Base-10 log	`LOG10(x)`	`log(x)` / `log10(x)`	`LOG10(x)`	`log(x)` / `log10(x)`
Power	`POWER(a,b)` / `POW`	`power(a,b)` / `a ^ b`	`POWER(a,b)` (`^` is XOR, a bit-flipping operator!)	`pow(a,b)`
Number with commas	`FORMAT(x, 2)`	`to_char(x, 'FM999,999,990.00')`	`FORMAT(x, 'N2')`	no built-in (`printf('%,d', x)` groups integers only)
Convert type	`CONVERT(x, DECIMAL(5,2))` / `CAST`	`CAST(x AS numeric(5,2))` / `x::numeric(5,2)`	`CONVERT(DECIMAL(5,2), x)` / `CAST`	`CAST(x AS REAL)`
Integer division of ints	`DIV`	`/` or `div()`	`/`	`/`

Common mistakes

- Using `log()` expecting natural log in PostgreSQL.
- Formatting (`to_char`) inside calculations or before sorting.
- `round()` on `double precision` expecting "half up".

Self-check

1. `floor(-2.5)`, `ceil(-2.5)`, `trunc(-2.5)`? → -3, -2, -2.
2. `round(123.456, -1)`? → 120.
3. *PostgreSQL for MySQL's* `LOG(x)`? → `ln(x)`.

3.3 String functions

What it is

String functions join, cut, measure, clean and search text — turning messy input (stray spaces, mixed case, combined fields) into consistent values.

The functions

Function (PostgreSQL)	Does	Example → result
`concat(a, b, …)` / `a || b`	join strings	`concat('Hello', ' ', 'World')` → `Hello World`
`concat_ws(sep, a, b, …)`	join with a separator, skipping NULLs	`concat_ws('-', '2024', '01', '01')` → `2024-01-01`
`substring(s FROM start FOR len)` / `substr(s, start, len)`	part of a string (positions start at 1)	`substring('SQL Functions' FROM 1 FOR 3)` → `SQL`
`left(s, n)`, `right(s, n)`	first / last *n* characters (negative *n* = all but)	`left('Database', 3)` → `Dat`
`length(s)` / `char_length(s)`	number of characters	`length('Data')` → 4
`octet_length(s)`	number of bytes	`octet_length('Data')` → 4
`upper(s)`, `lower(s)`, `initcap(s)`	change case	`initcap('desk lamp')` → `Desk Lamp`
`replace(s, from, to)`	replace every occurrence of a substring	`replace('I love SQL', 'SQL', 'Postgres')`
`translate(s, from, to)`	replace character by character	`translate('2025/01/03', '/', '-')`
`trim(s)`, `ltrim`, `rtrim`, `trim(BOTH 'x' FROM s)`	remove spaces (or given characters) from ends	`trim(' SQL ')` → `SQL`
`position(sub IN s)` / `strpos(s, sub)`	1-based position of first match, 0 if absent	`strpos('Hello World', 'World')` → 7
`lpad(s, n, fill)`, `rpad(s, n, fill)`	pad (or cut) to length *n*	`lpad('7', 3, '0')` → `007`
`repeat(s, n)`, `reverse(s)`	repeat / reverse	`repeat('La', 3)` → `LaLaLa`
`split_part(s, delim, n)`	*n*-th field of a delimited string (fields separated by a marker such as `.` or `,`) (negative *n* from the end, PostgreSQL 14+)	`split_part('a@b.com', '@', 2)` → `b.com`
`format(fmt, …)`	printf-style building (fill `%s` slots in a template); `%I` quotes identifiers (table/column names), `%L` literals	
`regexp_replace`, `regexp_match`	regular-expression replace / capture	

How it works — concatenation and NULL

```
-- code/03_string_functions.sql
SELECT concat('Hello', ' ', 'World') AS concat, 'Hello' || ' ' || 'World' AS pipes,
       concat_ws('-', '2024', '01', '01') AS concat_ws;
SELECT concat('a', NULL, 'b') AS concat_null, 'a' || NULL || 'b' AS pipes_null,
       concat_ws(', ', 'Pune', NULL, 'India') AS ws_null;
```

Output:

```
   concat    |    pipes    | concat_ws
-------------+-------------+------------
 Hello World | Hello World | 2024-01-01
(1 row)

 concat_null | pipes_null |   ws_null
-------------+------------+-------------
 ab          | (null)     | Pune, India
(1 row)
```

In PostgreSQL `concat()` **ignores** NULL arguments while `||` returns NULL if any part is NULL; `concat_ws` skips NULLs (ideal for addresses with optional parts). **MySQL differs:** `CONCAT()` returns NULL if any argument is NULL (`CONCAT_WS` skips them).

Extracting parts

String positions in SQL start at **1**, not 0: `substring('SQL Functions', 5)` → `Functions`, `substr('SQL Functions', 5, 4)` → `Func` , `right('Postgres', 3)` → `res`, `left('Database', -2)` → `Databa` (all run in `code/03_string_functions.sql`).

Measuring

`length` counts spaces too (`length(' padded ')` → 10). For ASCII (128 basic English letters, digits and symbols, one byte each), characters = bytes. In a UTF-8 database (the universal encoding, 1 to 4 bytes per character — what the book's runner creates, with `C` collation) `é` is one character but two bytes:

```
SELECT current_setting('server_encoding') AS encoding, char_length('café') AS chars,
       octet_length('café') AS bytes, upper('café') AS upper_c_collation,
       upper('café' COLLATE "en-x-icu") AS upper_icu;
```

Output:

```
 encoding | chars | bytes | upper_c_collation | upper_icu
----------+-------+-------+-------------------+-----------
 UTF8     |     4 |     5 | CAFé              | CAFÉ
(1 row)
```

Case conversion follows the **collation**: plain `C` only knows ASCII letters, so `é` stays lower case; the ICU collation `en-x-icu` (available when PostgreSQL is built with ICU — International Components for Unicode) converts it. A legacy `SQL_ASCII` database doesn't understand multibyte characters and would report 5 characters — always create databases as UTF-8.

Portability trap: MySQL's `LENGTH()` counts **bytes** (use `CHAR_LENGTH()` for characters); SQL Server's `LEN()` counts characters but ignores trailing spaces (`DATALENGTH()` gives bytes).

Changing case, replacing, repeating

`initcap('noise-cancelling headphones')` → `Noise-Cancelling Headphones` (capitalises after hyphens too); `reverse('stressed')` → `desserts`. `upper` / `lower` are the standard case-insensitive comparison: `WHERE lower(email) = lower(:input)` (`:input` = an application-supplied value). `replace` swaps whole substrings; `translate` maps single characters one-to-one.

Trimming

`trim` removes leading and trailing characters (spaces by default), never middle ones: `'[' || ltrim(' SQL ') || ']'` → `[SQL ]`, `rtrim` → `[ SQL]`, `trim(BOTH '*' FROM '**SQL**')` → `SQL`.

Searching and padding

`position('zzz' IN 'Hello')` → 0 (not found). `lpad('SQL', 5, '*')` → `**SQL`, `rpad` → `SQL**` ; `lpad` also **truncates** to the requested length — `lpad('Postgres', 3)` → `Pos` — careful with long values.

Splitting delimited text

```
SELECT split_part('www.example.com', '.', 1) AS first, split_part('www.example.com', '.', -1) AS last,
       array_to_string((string_to_array('www.example.com', '.'))[1:2], '.') AS first_two;
```

Output:

```
 first | last |  first_two
-------+------+-------------
 www   | com  | www.example
(1 row)
```

`split_part` replaces MySQL's `SUBSTRING_INDEX` : `SUBSTRING_INDEX(s, '.', -1)` (after the last dot) is `split_part(s, '.', -1)` ; "everything before the second dot" (`SUBSTRING_INDEX(s, '.', 2)`) needs the array form (an **array** is an ordered list in one cell; `[1:2]` takes its first two elements). Splitting into *rows*: `string_to_table` / `unnest`, chapter 14.

format() and regular expressions

```
SELECT format('%s earns %s', first_name, salary) AS sentence,
       format('SELECT * FROM %I WHERE name = %L', 'my table', 'O''Brien') AS safe_sql
FROM employees WHERE emp_id = 7;
SELECT regexp_replace('(022) 555-0199', '[^0-9]', '', 'g') AS digits_only,
       (regexp_match('order #A-1042 shipped', '#([A-Z])-(\d+)'))[2] AS order_number;
```

Output:

```
      sentence       |                    safe_sql
---------------------+-------------------------------------------------
 Priya earns 72000.00 | SELECT * FROM "my table" WHERE name = 'O''Brien'
(1 row)

 digits_only | order_number
-------------+--------------
 0225550199  | 1042
(1 row)
```

`%I` and `%L` quote an identifier and a literal safely — the basis of safe dynamic SQL (SQL built as a string at run time; quoting stops *SQL injection*, where crafted input changes the query's meaning — chapter 14). `regexp_replace(…, 'g')` replaces all matches (global); `regexp_match` returns an array of captured groups (text matched by each bracketed part), `[2]` picks the second.

Cleaning real data

The two Anita Shah rows differ only in formatting:

```
SELECT customer_id, email AS raw, lower(trim(email)) AS clean, initcap(city) AS city_fixed
FROM customers WHERE last_name = 'Shah' ORDER BY customer_id;
```

Output:

```
 customer_id |           raw              |          clean          | city_fixed
-------------+----------------------------+-------------------------+------------
           1 | anita.shah@mail.example    | anita.shah@mail.example | Mumbai
           8 |   Anita.Shah@Mail.example  | anita.shah@mail.example | Mumbai
(2 rows)
```

Better: clean on the way *in* (application validation, a `CHECK` constraint — a rule tested on every insert — or a unique index on `lower(email)`, chapter 12).

Full names and email domains:

```
SELECT first_name || ' ' || last_name AS full_name,
       split_part(email, '@', 2) AS domain
FROM employees WHERE emp_id IN (1, 3, 5) ORDER BY emp_id;
```

Output:

```
 full_name  |     domain
------------+-----------------
 Asha Rao   | company.example
 Neha Gupta | Company.example
 Meera Iyer | (null)
(3 rows)
```

Meera has no email, so the domain is NULL — functions given NULL generally return NULL.

Dialect map (strings)

Task	MySQL	PostgreSQL	SQL Server	SQLite
Concatenate	`CONCAT(a,b)` (NULL if any NULL)	`a || b`, `concat()` (skips NULL)	`a + b`, `CONCAT()` (NULL → '')	`a || b`, `concat()` (3.44+)
Characters	`CHAR_LENGTH(s)`	`length(s)` / `char_length(s)`	`LEN(s)` (no trailing spaces)	`length(s)`
Bytes	`LENGTH(s)`	`octet_length(s)`	`DATALENGTH(s)`	`length(CAST(s AS BLOB))` (BLOB = raw binary data)
Find position	`LOCATE(sub, s)`, `INSTR(s, sub)`, `POSITION(sub IN s)`	`position(sub IN s)`, `strpos(s, sub)`	`CHARINDEX(sub, s)`	`instr(s, sub)`
Split field	`SUBSTRING_INDEX(s, d, n)`	`split_part(s, d, n)`	`STRING_SPLIT` (rows) / `PARSENAME`	— (use `instr` + `substr`)
Left/right	`LEFT`, `RIGHT`	`left`, `right`	`LEFT`, `RIGHT`	`substr(s, 1, n)`, `substr(s, -n)`
Pad	`LPAD`, `RPAD`	`lpad`, `rpad`	`RIGHT(REPLICATE('0', n) + s, n)`	`printf('%05d', n)`
Repeat	`REPEAT`	`repeat`	`REPLICATE`	—
Regex replace	`REGEXP_REPLACE` (8.0+)	`regexp_replace`	`REGEXP_REPLACE` (SQL Server 2025)	extension

Common mistakes

- Assuming positions start at 0.
- Using `||` when a part may be NULL and getting a NULL result; use `concat` / `concat_ws` or `COALESCE`.
- Counting bytes instead of characters (MySQL `LENGTH`).
- Comparing case-insensitively without `lower()` / `ILIKE` in PostgreSQL.

Self-check

1. *`'a' || NULL` vs `concat('a', NULL)`?* → NULL vs `'a'`.
2. *Pad 42 to `00042`?* → `lpad(42::text, 5, '0')` (or `to_char(42, 'FM00000')`).

3.4 Date and time functions

What it is

Time values cause more subtle bugs than any other data: months differ in length, time zones shift, and "now" depends on when you ask.

The types

Type	Holds	Example
`date`	a calendar date	`2025-03-18`
`time`	a time of day	`19:30:00`
`timestamp` (without time zone)	date + time, *no* zone — a wall-clock reading	`2025-03-18 19:30:00`
`timestamptz` (with time zone)	an absolute **instant** (one exact moment, the same everywhere on Earth); stored in UTC, shown in the session's time zone	`2025-03-18 14:00:00+00`
`interval`	a duration	`3 days`, `1 mon`, `02:30:00`

Use `timestamptz` for "when did it happen" (events, orders, logins), `date` for calendar facts (birthdays, hire dates). Sample `orders.placed_at` is a plain `timestamp` so outputs match on every machine.

"Now"

A **transaction** is a group of statements that succeed or fail together (`BEGIN` … `COMMIT` ; a lone statement is one too). Several "now" functions freeze at its start:

Function	Returns	Changes within a transaction?
`current_date`	today's `date`	no
`current_time`	time of day with zone (`timetz`)	no
`localtime`	time of day without zone	no
`now()` = `current_timestamp` = `transaction_timestamp()`	`timestamptz` at transaction start	**no**
`localtimestamp`	`timestamp` at transaction start	no
`statement_timestamp()`	start of the current statement	per statement
`clock_timestamp()`	the actual current time	**yes, every call**

```sql
-- code/04_date_functions.sql (the file sets SET TIME ZONE 'UTC' first)
SELECT current_date AS today, current_time AS time_with_tz, localtime AS local_time,
       now() AS now_tstz, localtimestamp AS local_ts;
```

Output (from the run that produced this book; yours will show your current time):

```
   today    |    time_with_tz    |   local_time    |           now_tstz            |          local_ts
------------+--------------------+-----------------+-------------------------------+---------------------------
 2026-09-28 | 19:15:38.702642+00 | 19:15:38.702642 | 2026-09-28 19:15:38.702642+00 | 2026-09-28 19:15:38.702642
(1 row)
```

`now()` is frozen for the whole transaction, so rows inserted together share one timestamp — usually what you want. `clock_timestamp()` keeps moving:

```
BEGIN;
SELECT now() AS tx_start, clock_timestamp() AS wall_clock;
SELECT pg_sleep(0.2);
SELECT now() AS tx_start_again, clock_timestamp() AS wall_clock_later;
COMMIT;
```

Output:

```
          tx_start            |          wall_clock
------------------------------+------------------------------
 2026-09-28 19:15:38.702968+00 | 2026-09-28 19:15:38.703095+00
(1 row)

        tx_start_again        |       wall_clock_later
------------------------------+------------------------------
 2026-09-28 19:15:38.702968+00 | 2026-09-28 19:15:38.905532+00
(1 row)
```

Taking the date part of a timestamp

MySQL's `DATE(ts)` is a cast in PostgreSQL:

```
SELECT placed_at, placed_at::date AS just_date, date_trunc('day', placed_at) AS truncated
FROM orders WHERE order_id = 103;
```

Output:

```
     placed_at      | just_date  |      truncated
--------------------+------------+---------------------
 2025-01-31 23:55:00 | 2025-01-31 | 2025-01-31 00:00:00
(1 row)
```

`::date` returns a `date`; `date_trunc('day', …)` returns a timestamp at midnight.

Date arithmetic

Expression	Result type	Meaning
`date + integer`	date	add days
`date - date`	integer	days between
`date/timestamp ± interval`	timestamp	add/subtract a duration
`timestamp - timestamp`	interval	elapsed time
`age(later, earlier)`	interval	"years mons days" difference

```
SELECT date '2024-01-01' + 3 AS plus_3_days, date '2024-01-01' + interval '3 days' AS plus_interval,
       date '2024-01-01' - interval '1 month' AS minus_month, date '2024-01-05' - date '2024-01-01' AS
days_between;
```

Output:

```
 plus_3_days |    plus_interval    |     minus_month     | days_between
-------------+---------------------+---------------------+-------------
 2024-01-04  | 2024-01-04 00:00:00 | 2023-12-01 00:00:00 |           4
(1 row)
```

Month arithmetic clamps to the last valid day, so it is not reversible:

```
SELECT date '2025-01-31' + interval '1 month' AS jan31_plus_month,
       date '2024-02-29' + interval '1 year' AS leap_plus_year,
       date '2025-01-31' + interval '1 month' - interval '1 month' AS round_trip;
```

Output:

```
  jan31_plus_month   |   leap_plus_year    |     round_trip
---------------------+---------------------+---------------------
 2025-02-28 00:00:00 | 2025-02-28 00:00:00 | 2025-01-28 00:00:00
(1 row)
```

Back a month from 28 February lands on 28 January, not the 31st — billing systems must define "one month later" explicitly.

Extracting fields: EXTRACT and date_part

`extract(field FROM value)` is standard SQL; `date_part('field', value)` is PostgreSQL's function form. Since PostgreSQL 14, `extract` returns `numeric`.

```
SELECT extract(YEAR FROM date '2024-01-01') AS year, extract(MONTH FROM date '2024-01-01') AS month,
       extract(DAY FROM date '2024-01-01') AS day, extract(DOW FROM date '2024-01-01') AS dow_0_sun,
       extract(ISODOW FROM date '2024-01-01') AS isodow_1_mon, extract(QUARTER FROM date '2024-08-15') AS quarter,
       date_part('week', date '2024-01-01') AS iso_week;
```

Output:

```
 year | month | day | dow_0_sun | isodow_1_mon | quarter | iso_week
------+-------+-----+-----------+--------------+---------+----------
 2024 |     1 |   1 |         1 |            1 |       3 |        1
(1 row)
```

1 January 2024 was a **Monday**: `DOW` is Sunday 0 … Saturday 6, `ISODOW` Monday 1 … Sunday 7. Also: `HOUR`, `MINUTE`, `SECOND`, `DOY` (day of year), `EPOCH` (seconds since 1970-01-01 UTC).

Truncating to a period: date_trunc

`date_trunc(unit, ts)` rounds *down* to the start of the unit — the workhorse of time-series reports ("sales per month"):

```
SELECT placed_at, date_trunc('month', placed_at) AS month_start, date_trunc('week', placed_at) AS week_start,
       date_trunc('hour', placed_at) AS hour_start
FROM orders WHERE order_id = 107;
```

Output:

```
     placed_at      |     month_start     |     week_start      |     hour_start
--------------------+---------------------+---------------------+---------------------
 2025-03-18 19:30:00 | 2025-03-01 00:00:00 | 2025-03-17 00:00:00 | 2025-03-18 19:00:00
(1 row)
```

Weeks start on **Monday** (ISO 8601, the international date-time standard). Grouping by `date_trunc('month', …)` gives one sortable key per month (exercise E5, 3.6).

Formatting and parsing

`to_char(value, pattern)` formats; `to_date` / `to_timestamp` parse; `make_date` / `make_timestamp` build from numbers.

Pattern	Meaning	Example
`YYYY` , `MM` , `DD`	year, month number, day	`2025` , `03` , `18`
`Month` , `Mon`	month name, abbreviation	`March` , `Mar`
`Day` , `Dy`	weekday name, abbreviation	`Tuesday` , `Tue`
`HH24` , `HH12` , `MI` , `SS` , `AM`	hour, minute, second, meridiem	`19` , `07` , `30` , `00` , `PM`
`Q`	quarter	`1`
`FM` prefix	drop padding spaces/zeros	`FMDay` → `Monday`
`"text"`	literal text	`"Q"`

```
SELECT to_char(date '2024-01-01', 'FMDay, FMMonth FMDD, YYYY') AS long_form,
       to_char(timestamp '2025-03-18 19:30', 'DD Mon YYYY HH12:MI AM') AS short_form,
       to_char(timestamp '2025-03-18 19:30', 'YYYY-"Q"Q') AS quarter_label;
SELECT to_date('18/03/2025', 'DD/MM/YYYY') AS parsed_date,
       to_timestamp('2025-03-18 19:30', 'YYYY-MM-DD HH24:MI') AS parsed_ts,
       make_date(2025, 3, 18) AS made_date;
```

Output:

```
        long_form        |      short_form      | quarter_label
-------------------------+----------------------+---------------
 Monday, January 1, 2024 | 18 Mar 2025 07:30 PM | 2025-Q1
(1 row)

 parsed_date |       parsed_ts        | made_date
-------------+------------------------+------------
 2025-03-18  | 2025-03-18 19:30:00+00 | 2025-03-18
(1 row)
```

`to_timestamp` returns a `timestamptz` in the session time zone (UTC here). ISO strings just cast; use `to_date` for layouts like `18/03/2025`, which are otherwise ambiguous.

Differences between moments

```
SELECT order_id, placed_at, shipped_at,
       shipped_at - placed_at AS time_to_ship,
       round(extract(EPOCH FROM shipped_at - placed_at) / 3600, 1) AS hours_to_ship,
       shipped_at::date - placed_at::date AS calendar_days
FROM orders WHERE shipped_at IS NOT NULL AND order_id <= 104 ORDER BY order_id;
```

Output:

```
 order_id |      placed_at      |     shipped_at      |  time_to_ship  | hours_to_ship | calendar_days
----------+---------------------+---------------------+----------------+---------------+---------------
      101 | 2025-01-03 10:15:00 | 2025-01-04 09:00:00 | 22:45:00       |          22.8 |             1
      102 | 2025-01-17 22:40:00 | 2025-01-19 11:30:00 | 1 day 12:50:00 |          36.8 |             2
      103 | 2025-01-31 23:55:00 | 2025-02-02 08:10:00 | 1 day 08:15:00 |          32.3 |             2
      104 | 2025-02-11 09:05:00 | 2025-02-12 16:45:00 | 1 day 07:40:00 |          31.7 |             1
(4 rows)
```

Three answers to "how long did shipping take?": an `interval`, hours (`EPOCH` seconds ÷ 3600), and calendar-date boundaries crossed (order 101 took under a day but crossed a midnight). Decide which the business means.

`age()` gives a human-style difference:

```
SELECT first_name, hire_date, age(date '2025-07-01', hire_date) AS tenure_on_2025_07_01
FROM employees WHERE emp_id IN (1, 9, 15) ORDER BY emp_id;
```

Output:

```
 first_name | hire_date  |  tenure_on_2025_07_01
------------+------------+------------------------
 Asha       | 2015-01-05 | 10 years 5 mons 27 days
 Sara       | 2024-04-01 | 1 year 3 mons
 Rohan      | 2025-06-02 | 29 days
(3 rows)
```

`age(x)` with one argument measures from `current_date` at midnight. Whole years of service: `extract(YEAR FROM age(current_date, hire_date))`. Expected delivery ten days after ordering: `(placed_at + interval '10 days')::date` (order 105 of 2025-02-28 → 2025-03-10).

Time zones

A `timestamptz` is one instant; how it *prints* depends on the session's `TimeZone` setting. `AT TIME ZONE` converts between instants and wall-clock readings:

Expression	Gives
`timestamptz AT TIME ZONE 'Zone'`	the wall-clock `timestamp` in that zone at that instant
`timestamp AT TIME ZONE 'Zone'`	the `timestamptz` instant when that zone's clock showed that reading

```
SET TIME ZONE 'UTC';
SELECT timestamptz '2025-03-18 19:30:00+05:30' AS shown_in_utc;
SET TIME ZONE 'Asia/Kolkata';
SELECT timestamptz '2025-03-18 19:30:00+05:30' AS shown_in_ist,
       timestamptz '2025-03-18 19:30:00+05:30' AT TIME ZONE 'America/New_York' AS wall_clock_ny,
       timestamp '2025-03-18 19:30' AT TIME ZONE 'Europe/London' AS london_wall_clock_as_instant;
```

Output:

```
     shown_in_utc
-----------------------
 2025-03-18 14:00:00+00
(1 row)

       shown_in_ist        |    wall_clock_ny    | london_wall_clock_as_instant
---------------------------+---------------------+------------------------------
 2025-03-18 19:30:00+05:30 | 2025-03-18 10:00:00 | 2025-03-19 01:00:00+05:30
(1 row)
```

7:30 pm in India is 14:00 UTC and 10:00 in New York (already on daylight-saving time on 18 March). 7:30 pm in London (still on GMT — Greenwich Mean Time, the UK winter clock, equal to UTC) is 01:00 next day in India. Use named zones (`'Asia/Kolkata'`), not fixed offsets, so daylight-saving rules apply.

A calendar of dates

`generate_series` produces rows — one per day here — to build calendars or fill report gaps (chapter 7):

```
SELECT d::date AS day, to_char(d, 'Dy') AS weekday
FROM generate_series(date '2025-03-01', date '2025-03-05', interval '1 day') AS d;
```

Output:

```
    day     | weekday
------------+---------
 2025-03-01 | Sat
 2025-03-02 | Sun
 2025-03-03 | Mon
 2025-03-04 | Tue
 2025-03-05 | Wed
(5 rows)
```

Dialect map (dates)

Task	MySQL	PostgreSQL	SQL Server	SQLite
Now	`NOW()`	`now()`	`SYSDATETIME()` / `GETDATE()`	`datetime('now')`
Today	`CURRENT_DATE` / `CURDATE()`	`current_date`	`CAST(GETDATE() AS date)`	`date('now')`
Date part of datetime	`DATE(ts)`	`ts::date`	`CAST(ts AS date)`	`date(ts)`
Add 3 days	`DATE_ADD(d, INTERVAL 3 DAY)`	`d + interval '3 days'` (or `d + 3` for a date)	`DATEADD(day, 3, d)`	`date(d, '+3 days')`
Subtract a month	`DATE_SUB(d, INTERVAL 1 MONTH)`	`d - interval '1 month'`	`DATEADD(month, -1, d)`	`date(d, '-1 month')`
Year / month / day	`YEAR(d)`, `MONTH(d)`, `DAY(d)`, `EXTRACT`	`extract(YEAR FROM d)` …	`YEAR(d)`, `DATEPART(month, d)`	`strftime('%Y', d)`
Format	`DATE_FORMAT(d, '%W, %M %e, %Y')`	`to_char(d, 'FMDay, FMMonth FMDD, YYYY')`	`FORMAT(d, 'dddd, MMMM d, yyyy')`	`strftime(…)`
Days between	`DATEDIFF(end, start)`	`end::date - start::date`	`DATEDIFF(day, start, end)`	`julianday(end) - julianday(start)`
Difference in a unit	`TIMESTAMPDIFF(HOUR, start, end)`	`extract(EPOCH FROM end - start) / 3600`	`DATEDIFF(hour, start, end)`	`(julianday(end) - julianday(start)) * 24`
Truncate to month	`DATE_FORMAT(d, '%Y-%m-01')`	`date_trunc('month', d)`	`DATETRUNC(month, d)` (2022+)	`date(d, 'start of month')`

Note the argument orders: MySQL `DATEDIFF(end, start)` vs SQL Server `DATEDIFF(unit, start, end)`. SQL Server's `DATEDIFF` counts *boundaries crossed* (like `calendar_days` above), not elapsed time.

Common mistakes

- Comparing a timestamp column to a date-only upper bound (see chapter 2, 2.4) — use half-open ranges.
- Wrapping an indexed column (an **index** is a sorted lookup structure for finding rows without reading them all) in a function in `WHERE` (`WHERE date_trunc('month', placed_at) = …`) — can stop the index being used; rewrite as a range (chapter 12).
- Storing local times in `timestamp` for a multi-time-zone app — use `timestamptz`.
- Assuming "+1 month" can be undone with "−1 month".

In the interview

> **30-second answer:** "I store instants as timestamptz, truncate with date_trunc for per-day or per-month reports, filter with half-open ranges so indexes still work, and compute durations with timestamp subtraction or EXTRACT(EPOCH …). I'm careful that month arithmetic clamps and that now() is fixed per transaction."

- *Orders per month including months with zero orders?* → Generate the months with `generate_series` and left-join the counts (a *left join* keeps every row of the first table even when the second has no match; chapters 5 and 7).
- *Difference between `timestamp` and `timestamptz`?* → `timestamptz` is an absolute instant converted to/from the session zone; `timestamp` is a zone-less wall-clock reading.

Self-check

1. *`date '2025-03-31' - interval '1 month'`?* → `2025-02-28 00:00:00`.

2. *Why are two `now()` calls in one transaction equal?* → `now()` is the transaction start time.
3. *Hours between two timestamps as a number?* → `extract(EPOCH FROM b - a) / 3600`.

3.5 Conditional expressions: CASE, COALESCE, NULLIF, GREATEST, LEAST

What it is

If-then-else logic inside a query; being expressions, they return a value and go wherever a column can.

CASE

Searched CASE tests conditions in order and returns the result of the first true one:

```
-- code/05_case_coalesce_nullif.sql
SELECT name, unit_price,
       CASE WHEN unit_price >= 300 THEN 'premium'
            WHEN unit_price >= 50  THEN 'standard'
            ELSE 'budget' END AS price_band
FROM products ORDER BY unit_price DESC;
```

Output:

```
           name             | unit_price | price_band
----------------------------+------------+------------
 Laptop Pro 14              |    1299.00 | premium
 ...
 Noise-Cancelling Headphones |     199.95 | standard
 ...
 USB-C Hub                  |      49.50 | budget
 ...
(9 rows)
```

The first matching branch wins (the 1,299 laptop is also `>= 50`), so order branches most to least specific.

Simple CASE compares one expression with a list of values:

```
SELECT order_id, status,
       CASE status WHEN 'pending' THEN 'Awaiting dispatch'
                   WHEN 'shipped' THEN 'On the way'
                   WHEN 'delivered' THEN 'Done'
       END AS customer_message
FROM orders WHERE order_id IN (101, 105, 107, 113) ORDER BY order_id;
```

Output:

```
 order_id |  status   | customer_message
----------+-----------+------------------
      101 | delivered | Done
      105 | cancelled | (null)
      107 | shipped   | On the way
      113 | pending   | Awaiting dispatch
(4 rows)
```

No `ELSE` means `ELSE NULL` — the cancelled order gets NULL. Simple `CASE x WHEN NULL` never matches (it uses `=`); use searched `CASE WHEN x IS NULL`.

`CASE` appears in `ORDER BY` for custom orders (2.7), inside aggregates for conditional counts (`count(*) FILTER (…)` / `sum(CASE …)`, chapter 4), in `UPDATE … SET` (chapter 9), and in pivots (row values into columns, e.g. one per month; chapter 14).

COALESCE — first non-NULL

`coalesce(a, b, c, …)` returns the first non-NULL argument (NULL if all are) — the standard way to supply defaults.

```
SELECT first_name, email, coalesce(email, 'no email on file') AS contact,
       commission_pct, coalesce(commission_pct, 0) AS commission_or_zero
FROM employees WHERE emp_id IN (5, 6, 9, 11) ORDER BY emp_id;
```

Output:

```
 first_name |            email             |           contact            | commission_pct | commission_or_zero
------------+------------------------------+------------------------------+----------------+--------------------
 Meera      | (null)                       | no email on file             |         (null) |                  0
 Daniel     | daniel.brooks@company.example | daniel.brooks@company.example |           0.05 |               0.05
 Sara       | sara.khan@company.example    | sara.khan@company.example    |         (null) |                  0
 Ananya     | (null)                       | no email on file             |         (null) |                  0
(4 rows)
```

Why it matters — NULL "poisons" arithmetic:

```
SELECT first_name, salary, commission_pct,
       salary * (1 + commission_pct)             AS naive_total,
       salary * (1 + coalesce(commission_pct, 0)) AS total
FROM employees WHERE dept_id = 2 ORDER BY emp_id;
```

Output:

```
 first_name |  salary   | commission_pct | naive_total |    total
------------+-----------+----------------+-------------+-------------
 Daniel     | 150000.00 |           0.05 | 157500.0000 | 157500.0000
 Priya      |  72000.00 |           0.10 |  79200.0000 |  79200.0000
 Arjun      |  68000.00 |           0.10 |  74800.0000 |  74800.0000
 Sara       |  68000.00 |         (null) |      (null) |    68000.00
(4 rows)
```

Without `coalesce`, Sara's total is unknown. Arguments must share a compatible type — `coalesce(salary, 'n/a')` fails; use `coalesce(salary::text, 'n/a')`.

NULLIF — turn a value into NULL

`nullif(a, b)` returns NULL if `a = b`, otherwise `a`. Two classic uses:

```
-- 1. Avoid division by zero
SELECT a, b, a / NULLIF(b, 0) AS safe_ratio
FROM (VALUES (10, 2), (10, 0)) AS t(a, b);

-- 2. Treat blank strings as missing
SELECT email, NULLIF(trim(email), '') AS email_or_null
FROM (VALUES ('x@y.example'), ('   '), (NULL)) AS t(email);
```

Output:

```
 a  | b | safe_ratio
----+---+------------
 10 | 2 |          5
 10 | 0 |     (null)
(2 rows)

    email    | email_or_null
-------------+---------------
 x@y.example | x@y.example
             | (null)
 (null)      | (null)
(3 rows)
```

`x / 0` raises *division by zero* and aborts the query; `x / NULLIF(0, 0)` = NULL. Use `coalesce(a / nullif(b, 0), 0)` if zero is more useful than NULL.

CASE with dates and NULLs together

```
SELECT name, start_date, end_date,
       coalesce(end_date, date '2025-12-31') AS planned_or_open_end,
       CASE WHEN end_date IS NULL THEN 'running' ELSE 'finished' END AS state
FROM projects ORDER BY project_id;
```

Output:

```
      name       | start_date |  end_date  | planned_or_open_end |  state
-----------------+------------+------------+---------------------+----------
 Checkout Revamp  | 2024-09-01 | 2025-03-31 | 2025-03-31          | finished
 Mobile App       | 2025-01-15 | (null)     | 2025-12-31          | running
 Spring Campaign  | 2025-02-01 | 2025-04-30 | 2025-04-30          | finished
 Year-End Audit   | 2024-11-01 | 2025-01-31 | 2025-01-31          | finished
 Helpdesk Chatbot | 2025-04-01 | (null)     | 2025-12-31          | running
(5 rows)
```

GREATEST and LEAST

Largest / smallest of their arguments (across rows is `max` / `min`, chapter 4). In PostgreSQL they **ignore NULLs**:

```
SELECT greatest(1, NULL, 3) AS greatest, least(1, NULL, 3) AS least;
```

```
 greatest | least
----------+-------
        3 |     1
(1 row)
```

MySQL and Oracle return NULL if any argument is NULL — wrap arguments in `coalesce` for portable code. Typical use: `greatest(price - discount, 0)` to floor a value at zero.

Dialect map (conditionals)

Task	Standard / PostgreSQL	MySQL	SQL Server	Oracle
If-then-else	`CASE WHEN … THEN … ELSE … END`	also `IF(cond, a, b)`	also `IIF(cond, a, b)`	also `DECODE(…)`
First non-NULL	`COALESCE(a, b, …)`	also `IFNULL(a, b)`	also `ISNULL(a, b)`	also `NVL(a, b)`
Value → NULL	`NULLIF(a, b)`	same	same	same
Greatest/least	`GREATEST`, `LEAST` (ignore NULLs)	same names (NULL if any NULL)	`GREATEST` / `LEAST` (2022+, ignore NULLs)	same names (NULL if any NULL)

Prefer the standard forms (`CASE`, `COALESCE`, `NULLIF`) — they work everywhere.

Common mistakes

- Forgetting `ELSE` and getting unexpected NULLs.
- Overlapping `WHEN` conditions in the wrong order.
- `CASE x WHEN NULL …` — never true.
- Mixing result types in branches (`THEN 1 ELSE 'none'`) — PostgreSQL errors; make them the same type.

In the interview

30-second answer: "CASE is SQL's if-else and works anywhere an expression does — SELECT, ORDER BY, inside SUM for conditional aggregation. COALESCE returns the first non-null, so I use it for

defaults and to stop NULL wiping out arithmetic. NULLIF turns a value into NULL, mainly to avoid division by zero."

- *Count delivered and cancelled orders in one row?* → `sum(CASE WHEN status='delivered' THEN 1 ELSE 0 END)`, or `count(*) FILTER (WHERE status='delivered')` in PostgreSQL (chapter 4).
- *`ISNULL` vs `COALESCE` in SQL Server?* → `ISNULL` takes two arguments and uses the first argument's type; `COALESCE` is standard, takes many, and uses type precedence (the rule for which type wins when arguments differ).

Self-check

1. *Safe average price per item when quantity may be 0?* → `total / NULLIF(quantity, 0)`.
2. *`coalesce(NULL, NULL, 'x', 'y')` ?* → `'x'`.

3.6 Worked exercises

Try each first. They preview aggregates (`sum`, `count`, `GROUP BY` — chapter 4) and a simple join (`JOIN … USING` pairs rows of two tables whose shared column, here `order_id`, matches; chapter 5).

E1. Total value of all delivered orders, rounded to 2 decimals and shown with thousands separators.

```sql
-- code/06_function_exercises.sql
SELECT to_char(round(sum(quantity * unit_price * (1 - discount)), 2), 'FM999,999,990.00') AS total_sales
FROM order_items oi
JOIN orders o USING (order_id)
WHERE o.status = 'delivered';
```

```
 total_sales
-------------
 4,508.11
(1 row)
```

Compute with numbers, round, and only then format — `to_char` is the last step.

E2. The square root of the number of order lines, and the natural log of total revenue.

```sql
SELECT round(sqrt(count(*))::numeric, 3) AS sqrt_lines,
       round(ln(sum(quantity * unit_price * (1 - discount))), 4) AS ln_revenue
FROM order_items;
```

```
 sqrt_lines | ln_revenue
------------+------------
      4.690 |     9.2130
(1 row)
```

`sqrt` returns `double precision`, and `round(x, d)` needs `numeric` — hence the cast. `ln`, not `log`, for natural log.

E3. Customer display names like "Shah, A." and lower-case email domains.

```sql
SELECT customer_id,
       last_name || ', ' || left(first_name, 1) || '.' AS display_name,
       lower(split_part(trim(email), '@', 2))          AS domain
FROM customers ORDER BY customer_id LIMIT 5;
```

```
 customer_id | display_name |    domain
-------------+--------------+--------------
           1 | Shah, A.     | mail.example
           2 | Carter, B.   | mail.example
           3 | Li, C.       | mail.example
           4 | Menon, D.    | (null)
           5 | Walker, E.   | mail.example
(5 rows)
```

E4. Revenue per year and month, excluding cancelled orders.

```sql
SELECT extract(YEAR FROM o.placed_at)  AS year,
       extract(MONTH FROM o.placed_at) AS month,
       round(sum(oi.quantity * oi.unit_price * (1 - oi.discount)), 2) AS revenue
FROM orders o
JOIN order_items oi USING (order_id)
WHERE o.status <> 'cancelled'
GROUP BY 1, 2
ORDER BY 1, 2;
```

```
 year | month | revenue
------+-------+---------
 2025 |     1 | 1871.90
 2025 |     2 |  811.49
 2025 |     3 |  804.95
 2025 |     4 | 2148.75
 2025 |     5 |  249.93
 2025 |     6 | 1516.60
(6 rows)
```

E5. The same idea with one sortable month key (orders per month):

```sql
SELECT to_char(date_trunc('month', o.placed_at), 'YYYY-MM') AS month,
       count(DISTINCT o.order_id) AS orders
FROM orders o
WHERE o.status <> 'cancelled'
GROUP BY date_trunc('month', o.placed_at)
ORDER BY date_trunc('month', o.placed_at);
```

```
  month  | orders
---------+--------
 2025-01 |      3
 2025-02 |      1
 2025-03 |      2
 2025-04 |      3
 2025-05 |      1
 2025-06 |      2
(6 rows)
```

Grouping by `date_trunc` keeps a real timestamp for sorting; `to_char` only dresses it for display. (Expected delivery, order date + 10 days, is in 3.4.)

Chapter 3 TL;DR

- **Types:** everything has one; `numeric` for money, `text` for strings, `date` / `timestamptz` for time; convert with `CAST(x AS t)` or `x::t`; PostgreSQL refuses silent coercions (automatic conversions).
- **Numbers:** `round` / `trunc` / `ceil` / `floor` (numeric rounds half away from zero, floats to even), `power`, `sqrt`, `exp`, `ln`, `log` (base 10 in PostgreSQL!), `mod` / `%`, `random` + `setseed`, `to_char` for display.
- **Strings:** `||` vs `concat` (NULL handling), `substring` / `left` / `right` / `split_part`, `length` vs `octet_length`, `lower` / `upper` / `initcap`, `trim`, `replace` / `translate`, `position` / `strpos`, `lpad` / `rpad`, `format`, `regexp_replace`.

- **Dates:** `now()` vs `clock_timestamp()`, `+ interval`, `date - date`, `age`, `extract`, `date_trunc`, `to_char` / `to_date`, `AT TIME ZONE`; month arithmetic clamps.
- **Conditionals:** `CASE` (no ELSE → NULL), `COALESCE` (defaults), `NULLIF` (safe division), `GREATEST` / `LEAST`.
- MySQL names differ a lot (`DATE_ADD`, `DATE_FORMAT`, `SUBSTRING_INDEX`, `IFNULL`, `LOG` = ln, `LENGTH` = bytes) — use the dialect maps.

Next: Chapter 4 — Aggregation.

Chapter 4 — Aggregation: Summarising Many Rows

Chapters 2 and 3 worked **row by row**. Most business questions — "how many orders shipped last month?", "average salary per department?", "which customers spent over 1,000?" — squeeze **many rows into one answer**. That is **aggregation**.

Sections: 4.1 aggregate functions (and their `NULL`/empty-input rules) · 4.2 `GROUP BY` · 4.3 `HAVING` · 4.4 conditional aggregation (`FILTER`, `CASE`) and pivot-style reports · 4.5 the logical query processing order · 4.6 subtotals with `GROUPING SETS`, `ROLLUP`, `CUBE`.

Running the examples. All queries are in `code/`, run on the book's PostgreSQL 14 server (`bash tools_run_sql.sh sql '04-*'` runs this chapter on fresh databases); `(null)` is psql's `\pset null` display. Data: the `company` database (chapter 1, 1.11; `code/00_schema.sql`). Quirks that matter here: Asha (CEO) and Rohan (intern) have **no department**; Research has **no** employees; only 3 people have a commission; two salaries are shared by two people each; `sales_rep_id` is `NULL` for online orders; one product has no category; two customers never ordered.

4.1 Aggregate functions

What it is

An **aggregate function** takes a *set of rows* and returns *one value* summarising them — like a till's end-of-day count of sales, total taken and largest sale.

Function	Returns	Works on	Notes
`COUNT(*)`	number of **rows**	any rows	Counts rows, even rows that are all `NULL`.
`COUNT(expr)`	number of rows where `expr` is **not NULL**	any type	The usual way to count "how many have a value".
`COUNT(DISTINCT expr)`	number of **different** non-NULL values	any type	Duplicates and `NULL`s are dropped first.
`SUM(expr)`	total	numbers, intervals (durations)	`NULL` if there are no non-NULL inputs.
`AVG(expr)`	arithmetic mean = `SUM / COUNT(expr)`	numbers, intervals	Divides by the count of **non-NULL** values.
`MIN(expr)` / `MAX(expr)`	smallest / largest	anything that can be sorted: numbers, text, dates	Text uses the column's collation (alphabetical order).

A **scalar function** (chapter 3) returns one value *per row*; an aggregate returns one value *per group*. Without `GROUP BY`, the whole table (after `WHERE`) is one group, so the query returns exactly one row.

Why it matters

Reports, dashboards, KPIs (key performance indicators — a business's headline numbers), billing, fraud checks and most SQL interview questions are aggregations. Summarising inside the database is far cheaper than shipping every row to the application: it reads rows where they live and returns one small result.

How it works

1. Build the input rows (`FROM` and any joins — a **join** combines rows of several tables by matching values, Chapter 5); drop rows failing `WHERE`.
2. Per aggregate, walk the rows keeping a small running **state** (counter, running total, smallest so far…), **skipping** `NULL`**s** — every aggregate except `COUNT(*)` does.
3. Turn the state into the result (for `AVG`, total ÷ count).

```sql
SELECT count(*)              AS employees,
       sum(salary)           AS payroll,
       round(avg(salary), 2) AS avg_salary,
       min(salary)           AS lowest,
       max(salary)           AS highest
FROM employees;
```

Output:

```
 employees |  payroll   | avg_salary |  lowest  |  highest
-----------+------------+------------+----------+-----------
        15 | 1617000.00 |  107800.00 | 30000.00 | 250000.00
(1 row)
```

The three flavours of COUNT

```sql
SELECT count(*)                AS all_rows,
       count(email)            AS with_email,
       count(dept_id)          AS with_dept,
       count(DISTINCT dept_id) AS distinct_depts,
       count(DISTINCT job_title) AS distinct_titles
FROM employees;
```

Output:

```
 all_rows | with_email | with_dept | distinct_depts | distinct_titles
----------+------------+-----------+----------------+-----------------
       15 |         13 |        13 |              5 |              12
(1 row)
```

`count(*)` = 15 rows; `count(email)` = 13 (two have no email); `count(dept_id)` = 13 people with a department, spread over `count(DISTINCT dept_id)` = 5 departments — the two `NULL`s are not a sixth "value".

NULLs are skipped — and that changes averages

```sql
SELECT count(commission_pct)            AS non_null,
       avg(commission_pct)              AS avg_ignoring_nulls,
       avg(coalesce(commission_pct, 0)) AS avg_nulls_as_zero
FROM employees;
```

Output:

```
 non_null |   avg_ignoring_nulls   |   avg_nulls_as_zero
----------+------------------------+------------------------
        3 | 0.08333333333333333333 | 0.01666666666666666667
(1 row)
```

`AVG` divided by **3** (people with a commission), not 15. "Average rate *among people who earn commission*" → 0.083 is right; "across all staff, no commission = zero" → say so with `COALESCE(commission_pct, 0)`, giving 0.017. Neither is wrong; the mistake is not knowing which you asked.

Empty input

```sql
SELECT count(*) AS n, sum(salary) AS total, avg(salary) AS mean, max(salary) AS top
FROM employees WHERE salary > 1000000;
```

Output:

```
 n | total  |  mean  |  top
---+--------+--------+--------
 0 | (null) | (null) | (null)
(1 row)
```

With zero input rows `COUNT` answers `0`; `SUM`, `AVG`, `MIN`, `MAX` answer `NULL` ("no total of nothing"). You still get **one row** — an aggregate query without `GROUP BY` always returns exactly one. For `0` from an empty sum write `COALESCE(SUM(x), 0)`.

Other useful points

`MIN` / `MAX` work on any sortable type, and an aggregate can take an **expression**, not just a column:

```
SELECT min(hire_date) AS first_hire, max(hire_date) AS latest_hire,
       min(last_name) AS first_alpha, max(last_name) AS last_alpha
FROM employees;

SELECT round(sum(quantity * unit_price * (1 - discount)), 2) AS gross_revenue
FROM order_items;
```

Output:

```
 first_hire | latest_hire | first_alpha | last_alpha
------------+-------------+-------------+------------
 2015-01-05 | 2025-06-02  | Brooks      | Wei
(1 row)

 gross_revenue
---------------
      10026.61
(1 row)
```

A `WHERE` clause simply shrinks the set being aggregated (`WHERE status = 'delivered'` → 7 orders, first 2025-01-03, last 2025-04-09).

Beyond the big five (PostgreSQL)

The extra PostgreSQL aggregates you will actually use:

```
SELECT string_agg(first_name, ', ' ORDER BY first_name) AS engineers,
       array_agg(emp_id ORDER BY emp_id)                AS ids,
       bool_and(salary > 90000)                         AS all_above_90k,
       bool_or(email IS NULL)                           AS any_missing_email
FROM employees WHERE dept_id = 1;

SELECT round(stddev_samp(salary), 2) AS stddev,
       percentile_cont(0.5) WITHIN GROUP (ORDER BY salary) AS median,
       mode() WITHIN GROUP (ORDER BY salary)               AS most_common
FROM employees;
```

Output:

```
         engineers          |    ids    | all_above_90k | any_missing_email
----------------------------+-----------+---------------+-------------------
 Meera, Neha, Rahul, Vikram | {2,3,4,5} | t             | t
(1 row)

  stddev  | median | most_common
----------+--------+-------------
 57217.88 |  98000 |    68000.00
(1 row)
```

Aggregate	What it gives	Typical use
`string_agg(x, sep ORDER BY …)`	all values joined into one string	"list the products in each order"
`array_agg(x ORDER BY …)`	all values as an array (an ordered list stored as one value)	feed an app, or `unnest` (expand back into rows) later
`bool_and` / `bool_or`	TRUE if all / any inputs are true	"did every item ship?"
`stddev_samp`, `variance`	spread of the values (standard deviation ≈ how far values typically sit from the average)	analytics
`percentile_cont(f) WITHIN GROUP (ORDER BY x)`	interpolated percentile (0.5 = median)	median salary, p95 latency (the time 95 % of requests finish within)
`percentile_disc(f) …`	percentile that is an actual input value	"the median order", not an average of two
`mode() WITHIN GROUP (ORDER BY x)`	most frequent value	most common salary / status

Key term — percentile and median: the *f*-th percentile is the value below which a fraction *f* of sorted values fall; the **median** is the 50th percentile, which (unlike the average) is not dragged by a few huge values. *Interpolated* = blended from the two values either side of the cut, so it may never occur in the data. **Latency** is how long one request takes; "p95 latency" is its 95th percentile (System Design Fundamentals).

`percentile_cont`, `percentile_disc` and `mode` are **ordered-set aggregates**: `WITHIN GROUP (ORDER BY …)` says which order to read the values in.

Dialect differences

Feature	PostgreSQL	MySQL 8	SQLite	SQL Server
Join values into a string	`string_agg(x, ',')`	`GROUP_CONCAT(x SEPARATOR ',')`	`group_concat(x, ',')`	`STRING_AGG(x, ',')` (2017+)
`AVG` of an integer column	exact `numeric`	decimal	floating point	**integer** (truncated!) — cast first: `AVG(1.0 * x)`
Median	`percentile_cont(0.5) WITHIN GROUP (…)`	no built-in	no built-in	`PERCENTILE_CONT(0.5) WITHIN GROUP (…) OVER ()` (window form, Chapter 8)
Boolean aggregates	`bool_and`, `bool_or`	`MIN` / `MAX` of 0/1	`min` / `max` of 0/1	`MIN` / `MAX` of a `CASE`

Common mistakes

- **`COUNT(column)` vs `COUNT(*)` mixed up** — they differ by the number of `NULL`s in that column.
- **Averaging with NULLs without thinking** (commission example).
- **`SUM` of an empty set used as a number** in application code — it is `NULL`, not `0`.
- **Integer averages in SQL Server** (`AVG` of `int` returns an `int`).
- **`COUNT(DISTINCT a, b)`** — PostgreSQL does not accept two arguments; count distinct *rows* instead: `COUNT(DISTINCT (a, b))` or a subquery with `SELECT DISTINCT a, b`.

In the interview

30-second answer. "Aggregate functions reduce a set of rows to one value: `COUNT`, `SUM`, `AVG`, `MIN`, `MAX`. All of them ignore `NULL` inputs except `COUNT(*)`, which counts rows. On empty input `COUNT` returns 0 and the others return `NULL`. Without `GROUP BY` the whole filtered table is one group, so you get one row."

- *Q: What is the difference between `COUNT(*)`, `COUNT(1)` and `COUNT(col)`?* — `COUNT(*)` and `COUNT(1)` both count rows (1 is never NULL), and PostgreSQL treats them the same way; `COUNT(col)` counts non-NULL values of `col`.
- *Q: `AVG(x)` vs `SUM(x)/COUNT(*)`?* — They differ when `x` has NULLs: `AVG` divides by `COUNT(x)`. Also `SUM(int)/COUNT(*)` is integer division in PostgreSQL.
- *Q: How do you compute a median in PostgreSQL?* — `percentile_cont(0.5) WITHIN GROUP (ORDER BY x)`.

Self-check

1. Values 10, NULL, 20: `COUNT(*)`, `COUNT(col)`, `SUM(col)`, `AVG(col)`? → 3, 2, 30, 15.
2. `SELECT MAX(salary) FROM employees WHERE 1 = 0` — zero rows or one? → One row containing `NULL`.
3. Count employees with *no* email? → `COUNT(*) - COUNT(email)`, or `COUNT(*) FILTER (WHERE email IS NULL)` (§4.4).

4.2 GROUP BY

What it is

`GROUP BY` splits rows into **groups** — one per distinct value (or combination) of the listed expressions — and runs the aggregates **once per group**, giving **one row per group** (like sorting exam papers into piles by class and writing count and average mark on each pile).

How it works

1. Build and filter the rows (`FROM`, `WHERE`).
2. Put rows with the **same values** in all `GROUP BY` expressions into the same group. For grouping, all `NULL`s count as equal, so every row with a `NULL` key lands in **one** `NULL` group.
3. For each group, compute each aggregate over just that group's rows.
4. Output one row per group. The `SELECT` list may contain only the grouping expressions, aggregates, constants, or expressions built from those.

```
SELECT dept_id, count(*) AS headcount, sum(salary) AS payroll
FROM employees
GROUP BY dept_id
ORDER BY dept_id;
```

Output:

```
 dept_id | headcount |  payroll
---------+-----------+-----------
       1 |         4 | 523000.00
       2 |         4 | 358000.00
       3 |         2 | 181000.00
       4 |         2 | 211000.00
       5 |         1 |  64000.00
  (null) |         2 | 280000.00
(6 rows)
```

Five departments plus a `NULL` **group** (Asha and Rohan). Research does not appear: groups come only from existing rows. Departments with zero employees need an outer join (keeps rows with no match; Chapter 5).

Grouping by several columns

With two grouping columns, each **distinct pair** is a group:

```
SELECT dept_id, job_title, count(*) AS n, round(avg(salary)) AS avg_salary
FROM employees
WHERE dept_id IN (1, 2)
GROUP BY dept_id, job_title
ORDER BY dept_id, job_title;
```

Output:

```
 dept_id |      job_title      | n | avg_salary
---------+---------------------+---+------------
       1 | Engineer            | 2 |      98000
       1 | Engineering Manager | 1 |     185000
       1 | Senior Engineer     | 1 |     142000
       2 | Sales Manager       | 1 |     150000
       2 | Sales Rep           | 3 |      69333
(5 rows)
```

`GROUP BY a, b` and `GROUP BY b, a` give the same groups; output order comes only from `ORDER BY`.

Grouping by an expression

Group by a computed value, such as a timestamp's month:

```
SELECT date_trunc('month', placed_at)::date AS month, count(*) AS orders
FROM orders
GROUP BY date_trunc('month', placed_at)
ORDER BY month;
```

Output:

```
   month    | orders
------------+--------
 2025-01-01 |      3
 2025-02-01 |      2
 2025-03-01 |      2
 2025-04-01 |      3
 2025-05-01 |      2
 2025-06-01 |      2
(6 rows)
```

PostgreSQL also accepts a `SELECT` alias or column position in `GROUP BY` (non-standard; if an alias clashes with a real column name, the real column wins):

```
SELECT extract(year FROM hire_date) AS hire_year, count(*) AS hires
FROM employees
GROUP BY hire_year
ORDER BY 1;
```

Output:

```
 hire_year | hires
-----------+-------
      2015 |     1
      2016 |     1
 ...
      2025 |     1
(11 rows)
```

Grouping usually follows a join (chapter 5) — revenue per category:

```sql
SELECT p.category, sum(oi.quantity) AS units,
       round(sum(oi.quantity * oi.unit_price * (1 - oi.discount)), 2) AS revenue
FROM order_items oi JOIN products p ON p.product_id = oi.product_id
GROUP BY p.category
ORDER BY revenue DESC;
```

Output:

```
  category   | units | revenue
-------------+-------+---------
 Electronics |    16 | 7314.79
 Furniture   |     5 | 1940.47
 Accessories |    11 |  671.35
 (null)      |     2 |  100.00
(4 rows)
```

A `GROUP BY` with no aggregates returns each distinct group once: `SELECT status FROM orders GROUP BY status` = `SELECT DISTINCT status FROM orders` (4 rows).

The single-value rule

Each output row stands for a whole group, so a column neither grouped nor aggregated could have **several values** per group. SQL refuses to guess:

```sql
SELECT dept_id, first_name, count(*) FROM employees GROUP BY dept_id;
```

Output:

```
ERROR:  column "employees.first_name" must appear in the GROUP BY clause or be used in an aggregate function
LINE 1: SELECT dept_id, first_name, count(*) FROM employees GROUP BY...
                        ^
```

Exception: grouping by a table's **primary key** (the column uniquely identifying each row) means every other column of that table has one value per group (it is *functionally dependent* on the key — Chapter 11), so PostgreSQL (9.1+) allows it:

```sql
SELECT d.dept_id, d.name, count(e.emp_id) AS headcount
FROM departments d JOIN employees e ON e.dept_id = d.dept_id
GROUP BY d.dept_id
ORDER BY d.dept_id;
```

Output:

```
 dept_id |    name     | headcount
---------+-------------+-----------
       1 | Engineering |         4
       2 | Sales       |         4
       3 | Marketing   |         2
       4 | Finance     |         2
       5 | Support     |         1
(5 rows)
```

Database	Non-grouped, non-aggregated column in SELECT
PostgreSQL	Error, unless the column depends on a grouped primary key.
MySQL 8	Error by default (`ONLY_FULL_GROUP_BY` mode, on since 5.7.5); also detects primary-key dependency. With the mode off it returns an arbitrary value.
SQLite	Allowed ("bare column"); the value comes from an arbitrary row of the group (or from the row holding the `min` / `max` if the query has exactly one of those).
SQL Server, Oracle	Always an error.

Trade-offs and performance

Two physical strategies (shown by `EXPLAIN`, which prints the query plan — Chapter 12):

Strategy	How	Trade-off
Hash aggregation	in-memory hash table (turns each key into a slot number, ~one-step lookup) keyed by group values	fast, no sort; needs memory for all groups
Sort + group aggregation	sort by the grouping key, summarise each run of equal keys	can use an index (sorted lookup structure beside the table) already in that order; streams with little memory

Grouping by high-cardinality columns (many distinct values) creates many groups and costs more memory.

When not to use it

- Each **row** *and* its group total (salary next to department average)? `GROUP BY` collapses the rows; use a **window function** (computes over related rows but keeps every row): `avg(salary) OVER (PARTITION BY dept_id)` — Chapter 8.
- Only distinct values? `SELECT DISTINCT` states the intent more clearly.

Common mistakes

- "Fixing" the non-grouped-column error by adding the column to `GROUP BY`, silently creating *smaller* groups. Ask "one row per *what*?" and group by exactly that.
- Expecting groups for values with no rows (Research).
- Forgetting `NULL` keys form their own group — label with `COALESCE(dept_id::text, 'none')`.
- Relying on `GROUP BY` to sort — hash aggregation returns groups in no order; add `ORDER BY`.

In the interview

30-second answer. "`GROUP BY` partitions the filtered rows by the grouping columns and computes the aggregates once per partition, returning one row per group. Every selected column must be grouped or aggregated — unless it depends on a grouped primary key. `NULL`s form one group. For per-row output with group totals, use window functions instead."

- *Q: Does `GROUP BY` guarantee sorted output?* — No. Only `ORDER BY` does.
- *Q: `GROUP BY` vs `DISTINCT`?* — Without aggregates they give the same rows; `GROUP BY` is needed when you want to compute something per group.
- *Q: How do you find duplicate emails?* — `SELECT lower(email), count(*) FROM customers GROUP BY lower(email) HAVING count(*) > 1;` (the `HAVING` part is next).

Self-check

1. `GROUP BY dept_id` gives 6 rows and there are 6 departments. Is Research one? → No; the sixth is the `NULL` group.
2. Does `GROUP BY a, b` give more, fewer or the same groups as `GROUP BY a`? → The same or more — each `a` group is split by `b`.

4.3 HAVING

What it is

`HAVING` filters **groups**, **after** aggregation, so it can use aggregates (`count(*) > 2`). `WHERE` filters **rows** *before* grouping, so it cannot.

	WHERE	HAVING
Filters	individual rows	whole groups
Runs	before `GROUP BY`	after `GROUP BY` and aggregation
Can use aggregates?	**No**	Yes
Can use plain columns?	Yes	Only grouping columns (or inside aggregates)
Effect on work	fewer rows to group → cheaper	groups are already computed

How it works

```sql
SELECT dept_id, count(*) AS headcount
FROM employees
GROUP BY dept_id
HAVING count(*) > 2
ORDER BY dept_id;
```

Output:

```
 dept_id | headcount
---------+-----------
       1 |         4
       2 |         4
(2 rows)
```

Group 15 employees into 6 groups, count each, keep groups over 2 — Engineering and Sales.

`WHERE` removes *rows*, then `HAVING` removes *groups*. "Among non-cancelled orders, which customers ordered at least twice?"

```sql
SELECT customer_id, count(*) AS orders
FROM orders
WHERE status <> 'cancelled'
GROUP BY customer_id
HAVING count(*) >= 2
ORDER BY customer_id;
```

Output:

```
 customer_id | orders
-------------+--------
           1 |      3
           2 |      2
           5 |      2
           6 |      2
(4 rows)
```

Customer 3 (Chen) has two orders but one is cancelled: `WHERE` drops it, the group has 1 row, `HAVING` drops the group.

`HAVING` may use an aggregate not in the `SELECT` list, and combine conditions:

```sql
SELECT dept_id
FROM employees
GROUP BY dept_id
HAVING max(salary) > 140000
ORDER BY dept_id;

SELECT dept_id, count(*) AS n, round(avg(salary)) AS avg_salary
FROM employees
WHERE dept_id IS NOT NULL
GROUP BY dept_id
HAVING avg(salary) > 90000 AND count(*) >= 2
ORDER BY avg_salary DESC;
```

Output:

```
 dept_id
---------
       1
       2
  (null)
(3 rows)

 dept_id | n | avg_salary
---------+---+------------
       1 | 4 |     130750
       4 | 2 |     105500
       3 | 2 |      90500
(3 rows)
```

The `NULL` group qualifies in the first result because it holds the CEO's salary.

An aggregate in WHERE is an error

```
SELECT dept_id, count(*) FROM employees WHERE count(*) > 2 GROUP BY dept_id;
```

Output:

```
ERROR:  aggregate functions are not allowed in WHERE
LINE 1: SELECT dept_id, count(*) FROM employees WHERE count(*) > 2 G...
                                                      ^
```

When `WHERE` runs, groups don't exist yet (§4.5).

Row conditions belong in WHERE

A condition on a grouping column gives the same answer in either place: `… GROUP BY dept_id HAVING dept_id = 2` and `… WHERE dept_id = 2 GROUP BY dept_id` both return `2 | 4` . Prefer `WHERE`: it removes rows *before* the expensive grouping, can use an index, and reads as a row filter. (PostgreSQL's planner — the component deciding how to execute a query — often pushes such a `HAVING` down into `WHERE`, but not every database does.)

HAVING without GROUP BY

`HAVING` without `GROUP BY` treats the whole input as **one group**: one row if the condition holds, none otherwise:

```
SELECT count(*) AS products FROM products HAVING count(*) > 5;
SELECT count(*) AS products FROM products HAVING count(*) > 50;
```

Output:

```
 products
----------
        9
(1 row)

 products
----------
(0 rows)
```

Common mistakes

- Aggregate condition in `WHERE` (error) or row condition in `HAVING` (works, slower, confusing).
- `SELECT` alias in `HAVING`: `HAVING headcount > 2` fails in PostgreSQL (`column "headcount" does not exist`) — `HAVING` runs before `SELECT` names outputs; repeat the aggregate. (MySQL allows the alias.)
- Forgetting `WHERE` changes what `HAVING` sees (Chen).

In the interview

> **30-second answer.** "`WHERE` filters rows before grouping and can't use aggregates; `HAVING` filters groups after aggregation and can. Use `WHERE` for anything that doesn't need an aggregate, because it shrinks the input early."

- *Q: Find departments with more than 3 employees.* — `SELECT dept_id FROM employees GROUP BY dept_id HAVING COUNT(*) > 3;`
- *Q: Can you use `HAVING` without `GROUP BY`?* — Yes; the whole table is one group, so you get one row or none.
- *Q: Find duplicate rows.* — `GROUP BY` all the columns that define "duplicate" and `HAVING COUNT(*) > 1`.

Self-check

1. Rewrite `HAVING dept_id <> 5` in the better place. → `WHERE dept_id <> 5` — it also drops `NULL` rows, just as the `HAVING` version drops the `NULL` group.
2. `SELECT count(*) FROM t HAVING count(*) > 0` on an empty table? → No rows (count 0 filters out the one group).

4.4 Conditional aggregation: FILTER and CASE

What it is

Conditional aggregation aggregates only rows meeting a condition, *inside* one query, so several differently-filtered numbers sit side by side. Standard clause: `aggregate(...) FILTER (WHERE condition)` (PostgreSQL 9.4+, SQLite 3.30+); every database supports a `CASE` inside the aggregate.

The problem it solves

"Per customer: total, delivered, cancelled, online orders." Plain `WHERE` needs four queries (or subqueries — queries nested inside another), each scanning `orders` again; conditional aggregation does it in **one pass**.

How it works

`FILTER (WHERE cond)` feeds only matching rows into that one aggregate; other aggregates still see all rows of the group.

```sql
SELECT customer_id,
       count(*)                                        AS total,
       count(*) FILTER (WHERE status = 'delivered')    AS delivered,
       count(*) FILTER (WHERE status = 'cancelled')    AS cancelled,
       count(*) FILTER (WHERE sales_rep_id IS NULL)    AS online
FROM orders
GROUP BY customer_id
ORDER BY customer_id;
```

Output:

```
 customer_id | total | delivered | cancelled | online
-------------+-------+-----------+-----------+--------
           1 |     3 |         2 |         0 |      1
           2 |     2 |         2 |         0 |      1
           3 |     2 |         1 |         1 |      1
           4 |     1 |         0 |         1 |      0
           5 |     2 |         1 |         0 |      1
           6 |     2 |         0 |         0 |      1
           7 |     1 |         0 |         0 |      0
           8 |     1 |         1 |         0 |      1
(8 rows)
```

The portable `CASE` versions give identical numbers (`code/04_conditional_aggregation.sql`):

- `SUM(CASE WHEN cond THEN 1 ELSE 0 END)` — adds 1 per matching row.
- `COUNT(CASE WHEN cond THEN 1 END)` — non-matching rows give `NULL`, which `COUNT` skips.

Ratios and percentages

"What share of X is Y": divide a filtered aggregate by the unfiltered one, multiplying by `100.0` (not `100`) to avoid integer division.

```
SELECT dept_id,
       sum(salary) AS payroll,
       round(100.0 * sum(salary) FILTER (WHERE hire_date < '2020-01-01') / sum(salary), 1) AS pct_veterans
FROM employees
WHERE dept_id IS NOT NULL
GROUP BY dept_id
ORDER BY dept_id;
```

Output:

```
 dept_id |  payroll  | pct_veterans
---------+-----------+--------------
       1 | 523000.00 |         62.5
       2 | 358000.00 |         41.9
       3 | 181000.00 |         66.3
       4 | 211000.00 |         64.0
       5 |  64000.00 |       (null)
(5 rows)
```

Support shows `NULL`, not 0: its only employee was hired in 2023, so the filtered `SUM` saw no rows. Use `COALESCE(…, 0)` to show 0.

Pivot-style reports

A **pivot** turns rows into columns (one column per category). Conditional aggregation is the standard SQL pivot when the columns are known in advance:

```
SELECT p.category,
       sum(oi.quantity) FILTER (WHERE o.status = 'delivered') AS delivered_units,
       sum(oi.quantity) FILTER (WHERE o.status = 'shipped')   AS shipped_units,
       sum(oi.quantity) FILTER (WHERE o.status = 'pending')   AS pending_units
FROM order_items oi
JOIN orders o   ON o.order_id = oi.order_id
JOIN products p ON p.product_id = oi.product_id
GROUP BY p.category
ORDER BY p.category NULLS LAST;
```

Output:

```
  category   | delivered_units | shipped_units | pending_units
-------------+-----------------+---------------+---------------
 Accessories |               6 |        (null) |             5
 Electronics |               9 |             3 |             1
 Furniture   |               2 |             3 |        (null)
 (null)      |               2 |        (null) |        (null)
(4 rows)
```

Dynamic pivots (columns unknown until you see the data) and `crosstab` (a PostgreSQL extension function for pivots): Chapter 14.

Trade-offs

Approach	Portable?	Readability	Notes
`agg(x) FILTER (WHERE c)`	PostgreSQL, SQLite 3.30+ (not MySQL, SQL Server)	Best — the condition is separate from the value	Works with every aggregate, including `string_agg`, `array_agg`.
`SUM(CASE WHEN c THEN 1 ELSE 0 END)`	Everywhere	Good	Returns 0, not `NULL`, when nothing matches.
`COUNT(CASE WHEN c THEN 1 END)`	Everywhere	Good	Relies on `COUNT` skipping `NULL`.
Separate queries / subqueries	Everywhere	Verbose	Several scans of the table.

MySQL booleans are 0/1, so `SUM(status = 'delivered')` works there; PostgreSQL can't sum `boolean` — write `SUM((status = 'delivered')::int)`.

Common mistakes

- `COUNT(CASE WHEN c THEN 1 ELSE 0 END)` counts *every* row (0 is not `NULL`). Use `SUM` with `ELSE 0`, or `COUNT` with no `ELSE`.
- Integer division in ratios (`100 * a / b`).
- Putting the condition in `WHERE` when other columns need unfiltered rows — it filters *all* aggregates.

In the interview

> **30-second answer.** "Conditional aggregation computes several filtered aggregates in one pass: `COUNT(*) FILTER (WHERE status='x')` in PostgreSQL, or `SUM(CASE WHEN status='x' THEN 1 ELSE 0 END)` anywhere. It's how you do per-group breakdowns, ratios and fixed pivots."

- *Q: Share of cancelled orders per month?* — `GROUP BY` month, `ROUND(100.0 * COUNT(*) FILTER (WHERE status='cancelled') / COUNT(*), 1)`.
- *Q: Why does `COUNT(CASE … ELSE 0 END)` give the wrong number?* — Because `COUNT` counts non-NULL values and 0 is non-NULL.

Self-check

1. "Employees with no email per department", two ways? → `COUNT(*) FILTER (WHERE email IS NULL)`; `SUM(CASE WHEN email IS NULL THEN 1 ELSE 0 END)`.
2. A `FILTER`ed `SUM` where no row matches? → `NULL`.

4.5 The logical order of query processing

What it is

You **write** `SELECT … FROM … WHERE … GROUP BY … HAVING … ORDER BY … LIMIT`; the database **conceptually evaluates** it starting with `FROM`. This *logical query processing order* is defined by the SQL standard's semantics: each step takes the previous step's table and hands a new table to the next, so each step only sees what earlier steps produced.

Step	Clause	Input → output	Can refer to
1	`FROM`, `JOIN … ON`	tables → one combined row set	table columns
2	`WHERE`	rows → fewer rows	table columns (no aggregates, no `SELECT` aliases)
3–4	`GROUP BY` + aggregates	rows → one row per group	table columns / expressions
5	`HAVING`	groups → fewer groups	grouping columns, aggregates
6	`SELECT`	groups/rows → output columns	grouping columns, aggregates; **window functions run here**
7	`DISTINCT`	output rows → unique output rows	output columns
8	set operators (`UNION`, `INTERSECT`, `EXCEPT`; Chapter 6)	two results → one	output columns
9	`ORDER BY`	rows → sorted rows	output aliases, output positions, and (without `DISTINCT`/set ops) input columns
10	`LIMIT` / `OFFSET`	sorted rows → a slice	—

Why it matters

It explains almost every "column does not exist" or "aggregate not allowed" error:

- **`WHERE` can't use a `SELECT` alias** — the alias is created in step 6, `WHERE` is step 2.
- **`WHERE` can't use aggregates** — groups don't exist until step 3.
- **`ORDER BY` *can* use aliases** — it runs after `SELECT`.
- **`LIMIT` applies to the sorted, final rows** — so `ORDER BY … LIMIT 3` really means "top 3".
- **Window functions can't be used in `WHERE`** — they are computed in step 6 (Chapter 8).

It also guides efficiency and debugging: filter early (`WHERE`, step 2) so fewer rows are grouped and sorted; and trace a surprising result step by step — what `FROM` produces, what survives `WHERE`, what groups form.

```
SELECT emp_id, salary * 12 AS yearly FROM employees WHERE yearly > 2000000;
```

Output:

```
ERROR:  column "yearly" does not exist
LINE 1: ...mp_id, salary * 12 AS yearly FROM employees WHERE yearly > 2...
                                                             ^
```

```
SELECT emp_id, salary * 12 AS yearly FROM employees ORDER BY yearly DESC LIMIT 3;
```

Output:

```
 emp_id |   yearly
--------+------------
      1 | 3000000.00
      2 | 2220000.00
      6 | 1800000.00
(3 rows)
```

Fix the `WHERE` case by repeating the expression (`WHERE salary * 12 > 2000000`) or computing it in a **derived table** (a subquery in `FROM`, Chapter 6) so it becomes an input column:

```
SELECT * FROM (SELECT emp_id, salary * 12 AS yearly FROM employees) t
WHERE yearly > 2000000;
```

Output:

```
 emp_id |   yearly
--------+------------
      1 | 3000000.00
      2 | 2220000.00
(2 rows)
```

A query that uses every clause

```
SELECT   dept_id, count(*) AS headcount, round(avg(salary)) AS avg_salary
FROM     employees
WHERE    hire_date >= '2018-01-01'
GROUP BY dept_id
HAVING   count(*) >= 2
ORDER BY avg_salary DESC
LIMIT    3;
```

Output:

```
 dept_id | headcount | avg_salary
---------+-----------+------------
       1 |         4 |     130750
       3 |         2 |      90500
       2 |         3 |      69333
(3 rows)
```

Trace: **FROM** 15 rows → **WHERE** drops Asha (2015), Kiran (2016), Daniel (2017) → 12 rows → **GROUP BY** groups 1 (4), 2 (3), 3 (2), 4 (1: Omar), 5 (1: Li), `NULL` (1: Rohan) → aggregates → **HAVING** keeps 1, 2, 3 → **SELECT** names `headcount`, `avg_salary` → **ORDER BY** the alias → **LIMIT 3** keeps all three.

`DISTINCT` happens after `SELECT` and before `ORDER BY` / `LIMIT`, and `ORDER BY` may sort by an aggregate that is not selected:

```
SELECT DISTINCT job_title FROM employees ORDER BY job_title LIMIT 4;

SELECT dept_id FROM employees WHERE dept_id IS NOT NULL
GROUP BY dept_id ORDER BY sum(salary) DESC;
```

Output:

```
   job_title
----------------
 Accountant
 CEO
 Content Writer
 Engineer
(4 rows)

 dept_id
---------
       1
       2
       4
       3
       5
(5 rows)
```

Logical order is not physical order

The logical order defines **what the answer is**, not **how it is computed**. The planner may push a `WHERE` into a join, read rows pre-sorted from an index and skip the sort, stop once `LIMIT` rows are found, or move a `HAVING` on a grouping column into the scan. `EXPLAIN` shows the physical plan (Chapter 12).

Dialect notes

Behaviour	PostgreSQL	MySQL	SQLite	SQL Server
`SELECT` alias in `GROUP BY`	allowed (extension)	allowed	allowed	**not** allowed
`SELECT` alias in `HAVING`	not allowed	allowed	allowed	not allowed
`SELECT` alias in `WHERE`	not allowed	not allowed	allowed (quirk)	not allowed
Row limit	`LIMIT n OFFSET m` / `FETCH FIRST n ROWS ONLY`	`LIMIT`	`LIMIT`	`TOP n` / `OFFSET … FETCH`

The extensions work because those databases look the name up among the output columns; they do not change the logical order.

In the interview

30-second answer. "Logically, SQL runs `FROM` /joins, then `WHERE`, `GROUP BY`, aggregates, `HAVING`, `SELECT` (including window functions), `DISTINCT`, set operators, `ORDER BY`, and finally `LIMIT` / `OFFSET`. That's why `WHERE` can't see aliases or aggregates but `ORDER BY` can. The optimizer may physically reorder work as long as the result is the same."

- *Q: Why can't I filter on `ROW_NUMBER()` in `WHERE`?* — Window functions are computed in the `SELECT` step, after `WHERE`. Wrap the query in a subquery or CTE (Common Table Expression — a named subquery written with `WITH`, Chapter 7) and filter outside.
- *Q: Is `LIMIT` applied before or after `ORDER BY`?* — After. Without `ORDER BY`, which rows you get is undefined.
- *Q: Does `WHERE` run before the join?* — Logically after `FROM` builds the joined rows; physically the planner often filters each table before joining.

Self-check

1. Order: `HAVING`, `FROM`, `ORDER BY`, `SELECT`, `WHERE`, `GROUP BY`, `LIMIT`? → `FROM`, `WHERE`, `GROUP BY`, `HAVING`, `SELECT`, `ORDER BY`, `LIMIT` (mnemonic F-W-G-H-S-D-O-L, with `DISTINCT` after `SELECT`).
2. Where do window functions run? → In the `SELECT` step — after `HAVING`, before `DISTINCT` / `ORDER BY`.

4.6 Subtotals: GROUPING SETS, ROLLUP and CUBE

What it is

Reports often need **several levels at once**: revenue per country *and* category, per-country subtotals, a grand total. `GROUPING SETS`, `ROLLUP` and `CUBE` (SQL:1999; PostgreSQL 9.5+) produce them in one query, as if several `GROUP BY` queries were stacked with `UNION ALL` (which appends the rows of several queries).

Syntax	Groupings produced	Example with (country, category)
`GROUPING SETS ((a, b), (a), ())`	exactly the ones you list	whatever you list
`ROLLUP (a, b)`	`(a, b)`, `(a)`, `()` — a hierarchy, dropping columns from the right	detail, per-country subtotal, grand total
`CUBE (a, b)`	every subset: `(a, b)`, `(a)`, `(b)`, `()` — 2^n groupings	detail, per-country, per-category, grand total

A "rolled-up" column shows as `NULL` in subtotal rows. `GROUPING(col)` returns 1 when `col` is `NULL` *because it was rolled up* and 0 otherwise — telling a subtotal `NULL` from a genuine data `NULL`.

How it works

A view (a saved query, Chapter 7) with one row per non-cancelled order line:

```
CREATE VIEW sales AS
SELECT c.country, coalesce(p.category, 'Other') AS category,
       oi.quantity * oi.unit_price * (1 - oi.discount) AS revenue
FROM order_items oi
JOIN orders o    ON o.order_id = oi.order_id
JOIN customers c ON c.customer_id = o.customer_id
JOIN products p  ON p.product_id = oi.product_id
WHERE o.status <> 'cancelled';
```

ROLLUP — detail, then a subtotal per country, then the grand total:

```
SELECT country, category, round(sum(revenue), 2) AS revenue
FROM sales
GROUP BY ROLLUP (country, category)
ORDER BY country NULLS LAST, category NULLS LAST;
```

Output:

```
 country |  category   | revenue
---------+-------------+---------
 India   | Accessories |  621.85
 India   | Electronics | 2558.04
 India   | Furniture   | 1441.47
 India   | (null)      | 4621.36
 UK      | Accessories |   49.50
 UK      | Electronics | 1808.86
 UK      | Furniture   |  499.00
 UK      | (null)      | 2357.36
 USA     | Electronics |  324.90
 USA     | Other       |  100.00
 USA     | (null)      |  424.90
 (null)  | (null)      | 7403.62
(12 rows)
```

`category = (null)` rows are per-country subtotals; the last row is the grand total. Labelling them with `GROUPING()`:

```
SELECT CASE WHEN grouping(country) = 1 THEN 'ALL countries' ELSE country END AS country,
       CASE WHEN grouping(category) = 1 THEN 'ALL categories' ELSE category END AS category,
       round(sum(revenue), 2) AS revenue
FROM sales
GROUP BY ROLLUP (country, category)
ORDER BY grouping(country), country, grouping(category), category;
```

Output:

```
    country    |    category    | revenue
---------------+----------------+---------
 India         | Accessories    |  621.85
 ...
 India         | ALL categories | 4621.36
 ...
 USA           | ALL categories |  424.90
 ALL countries | ALL categories | 7403.62
(12 rows)
```

CUBE adds per-category totals across countries. `grouping(country, category)` returns a bit mask (each binary digit flags one column: left bit `country`, right bit `category`) — 0 detail, 1 `category` rolled up, 2 `country` rolled up, 3 grand total:

```sql
SELECT country, category, round(sum(revenue), 2) AS revenue,
       grouping(country, category) AS level_bits
FROM sales
GROUP BY CUBE (country, category)
ORDER BY level_bits, country, category;
```

Output:

```
 country |  category   | revenue | level_bits
---------+-------------+---------+------------
 India   | Accessories |  621.85 |          0
 ...                               (8 detail rows)
 India   | (null)      | 4621.36 |          1
 UK      | (null)      | 2357.36 |          1
 USA     | (null)      |  424.90 |          1
 (null)  | Accessories |  671.35 |          2
 (null)  | Electronics | 4691.80 |          2
 (null)  | Furniture   | 1940.47 |          2
 (null)  | Other       |  100.00 |          2
 (null)  | (null)      | 7403.62 |          3
(16 rows)
```

GROUPING SETS picks exactly the levels — per country, per category, grand total, no detail:

```sql
SELECT country, category, count(*) AS lines
FROM sales
GROUP BY GROUPING SETS ((country), (category), ())
ORDER BY country NULLS LAST, category NULLS LAST;
```

Output:

```
 country |  category   | lines
---------+-------------+-------
 India   | (null)      |    11
 UK      | (null)      |     5
 USA     | (null)      |     4
 (null)  | Accessories |     6
 (null)  | Electronics |     9
 (null)  | Furniture   |     4
 (null)  | Other       |     1
 (null)  | (null)      |    20
(8 rows)
```

Trade-offs

Option	Pros	Cons
`ROLLUP` / `CUBE` / `GROUPING SETS`	One statement, one scan of the input, totals always consistent	Output mixes levels — readers must understand the `NULL`s; `CUBE` grows as 2^n
Several `GROUP BY` queries + `UNION ALL`	Works in every database	Repeats the query; scans the input once per level
Totals in the application / BI tool (business-intelligence reporting software)	Flexible formatting	Moves more data out of the database

Database	Support
PostgreSQL 9.5+	`GROUPING SETS`, `ROLLUP`, `CUBE`, `GROUPING()`
SQL Server, Oracle	all of the above
MySQL 8	`GROUP BY a, b WITH ROLLUP` and `GROUPING()`; no `CUBE` or `GROUPING SETS`
SQLite	none — use `UNION ALL`

Common mistakes

- Confusing a subtotal `NULL` with a real one (use `GROUPING()`, or `COALESCE` real `NULL`s to a label first, as the view does).
- Expecting subtotals in some order — `ORDER BY` (often on `GROUPING(...)`).
- `CUBE` on many columns: 5 columns = 32 groupings.

In the interview

> **30-second answer.** "`ROLLUP(a, b)` gives `(a, b)`, `(a)` and the grand total — hierarchical subtotals. `CUBE(a, b)` gives every combination including `(b)` alone. `GROUPING SETS` lets you list the exact groupings. Rolled-up columns show as `NULL`, and `GROUPING(col)` tells those apart from real `NULL`s."

- *Q: How many groupings does `CUBE(a, b, c)` produce?* — $2^3 = 8$.
- *Q: How would you add a grand-total row without `ROLLUP`?* — `UNION ALL` a second query with no `GROUP BY`.

Self-check

1. Groupings of `ROLLUP (year, month, day)`? → `(year, month, day)`, `(year, month)`, `(year)`, `()`.
2. `GROUP BY ROLLUP (a)` as `GROUPING SETS`? → `GROUP BY GROUPING SETS ((a), ())`.

Chapter 4 TL;DR

Idea	Remember
Aggregates	`COUNT(*)` counts rows; every other aggregate (and `COUNT(col)`) skips `NULL`s. Empty input → 0 for `COUNT`, `NULL` for the rest.
`GROUP BY`	One output row per distinct key; select only keys and aggregates (or columns of a grouped primary key). `NULL`s form one group. No automatic ordering.
`HAVING`	Filters groups after aggregation; `WHERE` filters rows before. Aggregates only in `HAVING`.
Conditional aggregation	`agg(...) FILTER (WHERE …)` or `SUM(CASE …)` — many filtered numbers in one scan; ratios with `100.0 *`.
Logical order	FROM → WHERE → GROUP BY → aggregates → HAVING → SELECT (windows) → DISTINCT → set ops → ORDER BY → LIMIT.
Subtotals	`ROLLUP`, `CUBE`, `GROUPING SETS`; label rolled-up rows with `GROUPING()`.

Next: Chapter 5 — Joins combines rows from several tables, which is what most real aggregations are built on.

Chapter 5 — Joins: Combining Tables

A well-designed database splits information into narrow tables to avoid duplication (chapter 11), so almost any real question — "which department does Neha work in?", "what did Anita buy?" — spans tables. A **join** puts the pieces back together at query time. Joins are SQL's most important skill and its most common interview topic.

Sections: 5.1 Cartesian product and `CROSS JOIN` · 5.2 the four join types side by side (Venn-style pictures — overlapping circles — plus row-by-row results) · 5.3–5.6 `INNER`, `LEFT`, `RIGHT`, `FULL` · 5.7 self joins · 5.8 semi- and anti-joins · 5.9 pitfalls (fan-out, `ON` vs `WHERE`, `NULL` keys, `COUNT(*)`) · 5.10 `USING`, `NATURAL`, multi-table joins and join algorithms.

Running the examples. All queries are in `code/` (`bash tools_run_sql.sh sql '05-*'`), outputs real psql output from PostgreSQL 14 with `NULL` shown as `(null)`. Data: the `company` database (`code/00_schema.sql`, Chapter 1); its foreign keys — every one a join path — are drawn in the chapter 1 ER diagram.

A **foreign key** is a column whose values must match another table's **primary key** (the never-`NULL` column uniquely identifying each row) — `employees.dept_id` must be an existing `departments.dept_id`. Foreign keys are the natural join paths. Quirks: **Asha (CEO) and Rohan (intern) have no department**, **Research** has **no employees**, Hana and Ivan **never ordered**, Desk Lamp was **never sold**, online orders have **no sales rep**.

5.1 Join theory: the Cartesian product and CROSS JOIN

What it is

The **Cartesian product** of two tables (after René Descartes' coordinate pairs) pairs **every row of the first with every row of the second**: $m \times n$ rows. SQL writes it `CROSS JOIN` (or, old style, a comma between tables) — 3 sizes × 2 colours = 6 labels.

```
CREATE TABLE sizes  (size text);
CREATE TABLE colors (color text);
INSERT INTO sizes  VALUES ('S'), ('M'), ('L');
INSERT INTO colors VALUES ('red'), ('blue');

SELECT size, color FROM sizes CROSS JOIN colors ORDER BY size, color;
SELECT count(*) AS pairs FROM sizes, colors;
```

Output:

```
 size | color
------+-------
 L    | blue
 L    | red
 M    | blue
 M    | red
 S    | blue
 S    | red
(6 rows)

 pairs
-------
     6
(1 row)
```

Every join is a filtered Cartesian product

The key idea of the chapter. Conceptually, `A JOIN B ON condition`:

1. Forms every pair (a, b) — the Cartesian product.
2. Keeps only the pairs for which the **join condition** (the `ON` clause) is `TRUE`.
3. For outer joins only: adds back the rows that found **no partner**, padding the missing side with `NULL`s.

(Real databases never build the full product when they can avoid it — §5.10 — but the *result* is defined this way.)

```
SELECT count(*) AS all_pairs FROM employees, departments;
SELECT count(*) AS matching_pairs FROM employees e, departments d WHERE e.dept_id = d.dept_id;
```

Output:

```
 all_pairs
-----------
        90
(1 row)

 matching_pairs
----------------
             13
(1 row)
```

15 × 6 = 90 pairs, only 13 meaningful. Forgetting the join condition — an **accidental Cartesian product** — is a classic bug: the query "works" but returns far too many rows and wildly wrong aggregates.

When CROSS JOIN is what you want

A deliberate cross join builds **every combination**: a report grid (every department × every month, so empty months still appear), test data, a calendar.

```
SELECT d.name, m.month
FROM departments d
CROSS JOIN (VALUES ('2025-01'), ('2025-02')) AS m(month)
WHERE d.dept_id <= 2
ORDER BY d.name, m.month;
```

Output:

```
    name     |  month
-------------+---------
 Engineering | 2025-01
 Engineering | 2025-02
 Sales       | 2025-01
 Sales       | 2025-02
(4 rows)
```

Join syntax: explicit vs implicit

Style	Example	Notes
Explicit (ANSI SQL-92, the 1992 SQL standard)	`FROM a JOIN b ON a.k = b.k`	Join condition sits next to the join; required for outer joins. **Use this.**
Implicit (comma)	`FROM a, b WHERE a.k = b.k`	Older style; easy to forget a condition in a long `WHERE`; can't express outer joins in standard SQL.

Vocabulary: an **equi-join** uses only `=`; a **non-equi join** uses other operators (`<`, `BETWEEN` …); the **join key** is the column(s) compared; **left/right** table mean written before/after `JOIN`.

5.2 The four main join types at a glance

All four on two tiny tables, showing **every row** returned (no foreign key on purpose, so a student can be in a non-existent club).

```sql
CREATE TABLE clubs    (club_id int PRIMARY KEY, club text);
CREATE TABLE students (id int PRIMARY KEY, name text, club_id int);   -- no FK on purpose
INSERT INTO clubs    VALUES (10, 'Chess'), (20, 'Drama'), (30, 'Robotics');
INSERT INTO students VALUES (1, 'Ana', 10), (2, 'Ben', 20), (3, 'Cai', 20),
                            (4, 'Dev', NULL), (5, 'Eli', 40);
```

Which pairs match on `c.club_id = s.club_id`?

Student	s.club_id	Matching club	Why
Ana	10	Chess	10 = 10
Ben	20	Drama	20 = 20
Cai	20	Drama	20 = 20 (Drama matches **two** students)
Dev	`NULL`	none	`NULL = anything` is UNKNOWN, never TRUE
Eli	40	none	no club 40 (an "orphan" row)
—	—	Robotics (30)	no student has 30

The Venn-style picture

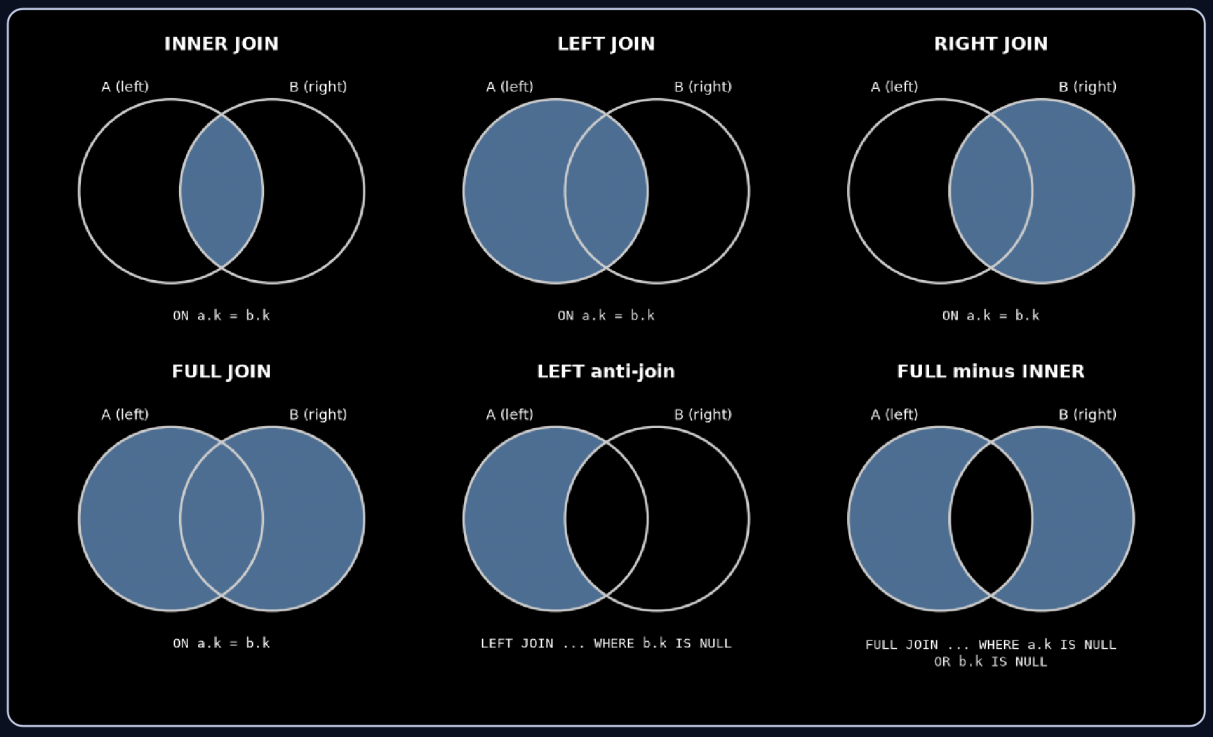


Figure 5.2 — Join "Venn" cheat sheet (drawn by `code/make_join_venn.py`). Left circle = rows of the left table, right circle = rows of the right table, overlap = pairs that satisfy the `ON` condition. Shaded = what the query returns.

Caution: a Venn diagram shows **sets of rows**, but a join produces **pairs** — Drama pairs with Ben *and* Cai, so the "overlap" can exceed either table. Use it to remember *which unmatched rows are kept*; the tables below show *how many rows*.

The result of each join, row by row

`INNER JOIN` — only the matched pairs (3 rows):

```sql
SELECT s.id, s.name, s.club_id AS s_club, c.club_id AS c_club, c.club
FROM students s INNER JOIN clubs c ON c.club_id = s.club_id
ORDER BY s.id;
```

```
 id | name | s_club | c_club | club
----+------+--------+--------+-------
  1 | Ana  |     10 |     10 | Chess
  2 | Ben  |     20 |     20 | Drama
  3 | Cai  |     20 |     20 | Drama
(3 rows)
```

`LEFT JOIN` — the matched pairs **plus** every unmatched *left* row, padded with `NULL`s on the right (5 rows):

```
SELECT s.id, s.name, s.club_id AS s_club, c.club_id AS c_club, c.club
FROM students s LEFT JOIN clubs c ON c.club_id = s.club_id
ORDER BY s.id;
```

```
 id | name | s_club | c_club |  club
----+------+--------+--------+--------
  1 | Ana  |     10 |     10 | Chess
  2 | Ben  |     20 |     20 | Drama
  3 | Cai  |     20 |     20 | Drama
  4 | Dev  | (null) | (null) | (null)
  5 | Eli  |     40 | (null) | (null)
(5 rows)
```

`RIGHT JOIN` — the matched pairs **plus** every unmatched *right* row, padded with `NULL`s on the left (4 rows):

```
SELECT s.id, s.name, s.club_id AS s_club, c.club_id AS c_club, c.club
FROM students s RIGHT JOIN clubs c ON c.club_id = s.club_id
ORDER BY c.club_id, s.id;
```

```
   id   |  name  | s_club | c_club |   club
--------+--------+--------+--------+----------
      1 | Ana    |     10 |     10 | Chess
      2 | Ben    |     20 |     20 | Drama
      3 | Cai    |     20 |     20 | Drama
 (null) | (null) | (null) |     30 | Robotics
(4 rows)
```

`FULL JOIN` — matched pairs, plus unmatched rows from **both** sides (6 rows):

```
SELECT s.id, s.name, s.club_id AS s_club, c.club_id AS c_club, c.club
FROM students s FULL JOIN clubs c ON c.club_id = s.club_id
ORDER BY s.id NULLS LAST, c.club_id;
```

```
   id   |  name  | s_club | c_club |   club
--------+--------+--------+--------+----------
      1 | Ana    |     10 |     10 | Chess
      2 | Ben    |     20 |     20 | Drama
      3 | Cai    |     20 |     20 | Drama
      4 | Dev    | (null) | (null) | (null)
      5 | Eli    |     40 | (null) | (null)
 (null) | (null) | (null) |     30 | Robotics
(6 rows)
```

Summary of the same example:

Join	Matched pairs	+ unmatched left (Dev, Eli)	+ unmatched right (Robotics)	Rows
`INNER`	✓			3
`LEFT`	✓	✓		5
`RIGHT`	✓		✓	4
`FULL`	✓	✓	✓	6
`CROSS`	every pair, no condition			5 × 3 = 15

In the `LEFT` result, Dev's `s_club` is `NULL` from his own data; Eli's `c_club` is `NULL` **added by the join**. A right-side `NULL` after a `LEFT JOIN` means "no partner" — unless that column can be `NULL` in the data, so test "no match" on a never-`NULL` right column, typically its primary key.

The edges of the Venn diagram: "only in one side"

`WHERE <right key> IS NULL` after a `LEFT JOIN` keeps only rows with no partner — the left-only crescent (an **anti-join**, §5.8). Likewise for the right side and "only in one of them":

```
SELECT s.name FROM students s LEFT JOIN clubs c ON c.club_id = s.club_id
WHERE c.club_id IS NULL ORDER BY s.id;

SELECT c.club FROM students s RIGHT JOIN clubs c ON c.club_id = s.club_id
WHERE s.id IS NULL;

SELECT s.name, c.club FROM students s FULL JOIN clubs c ON c.club_id = s.club_id
WHERE s.id IS NULL OR c.club_id IS NULL ORDER BY s.id NULLS LAST;
```

Output:

```
 name
------
 Dev
 Eli
(2 rows)

   club
----------
 Robotics
(1 row)

  name  |   club
--------+----------
 Dev    | (null)
 Eli    | (null)
 (null) | Robotics
(3 rows)
```

5.3 INNER JOIN

What it is

An **inner join** returns only the pairs for which the join condition is true; rows with no partner disappear. `JOIN` alone means `INNER JOIN`.

How it works

Per left row, find right rows where `ON` is `TRUE` and output one combined row per pair — 3 matches → 3 rows, 0 matches → none. `NULL` keys never match (the comparison is UNKNOWN).

```
SELECT e.emp_id, e.first_name, d.name AS department
FROM employees e
INNER JOIN departments d ON d.dept_id = e.dept_id
ORDER BY e.emp_id;
```

Output:

```
 emp_id | first_name | department
--------+------------+-------------
      2 | Vikram     | Engineering
      3 | Neha       | Engineering
 ...
     14 | Li         | Support
(13 rows)
```

Asha (1) and Rohan (15) are missing (`dept_id` `NULL`), and so is Research (no employee points to it).

Table aliases (`e` , `d`) are needed when a column exists in both tables (`dept_id`). Always qualify columns — `e.dept_id` — or you get `column reference "dept_id" is ambiguous` .

Filters with joins

For an inner join a condition gives the same result in `ON` or `WHERE` ; convention: relationship in `ON` , other filters in `WHERE` . (For outer joins placement **changes the result** — §5.9.)

```
SELECT e.first_name, e.salary, d.name AS department, d.location
FROM employees e
JOIN departments d ON d.dept_id = e.dept_id
WHERE d.location = 'Mumbai' AND e.salary > 70000
ORDER BY e.salary DESC;
```

Output:

```
 first_name |  salary   | department | location
------------+-----------+------------+----------
 Daniel     | 150000.00 | Sales      | Mumbai
 Lucas      | 120000.00 | Marketing  | Mumbai
 Priya      |  72000.00 | Sales      | Mumbai
(3 rows)
```

Joining three or more tables

Each `JOIN` adds one table to the result so far. Orders with customer and sales rep:

```
SELECT o.order_id, c.first_name AS customer, e.first_name AS rep, o.status
FROM orders o
JOIN customers c ON c.customer_id = o.customer_id
JOIN employees e ON e.emp_id = o.sales_rep_id
ORDER BY o.order_id;
```

Output:

```
 order_id | customer |  rep  |  status
----------+----------+-------+-----------
      101 | Anita    | Priya | delivered
      103 | Chen     | Arjun | delivered
      105 | Divya    | Priya | cancelled
      107 | Fatima   | Sara  | shipped
      108 | Ben      | Priya | delivered
      110 | George   | Arjun | shipped
      112 | Ethan    | Sara  | shipped
      113 | Anita    | Priya | pending
(8 rows)
```

Only 8 of 14 orders: the 6 online orders (`sales_rep_id = NULL`) were dropped by the inner join to `employees` (§5.10 keeps them with a `LEFT JOIN`).

A **many-to-many** relationship (one employee on many projects, one project with many employees) goes through a **junction table** (`employee_projects` , each row linking one employee to one project):

```
SELECT p.name AS project, d.name AS dept, e.first_name, ep.role, ep.hours
FROM employee_projects ep
JOIN employees   e ON e.emp_id     = ep.emp_id
JOIN projects    p ON p.project_id = ep.project_id
JOIN departments d ON d.dept_id    = p.dept_id
WHERE p.project_id IN (1, 2)
ORDER BY p.project_id, ep.hours DESC;
```

Output:

```
    project     |    dept     | first_name |   role    | hours
----------------+-------------+------------+-----------+-------
 Checkout Revamp | Engineering | Neha       | Developer |   400
 Checkout Revamp | Engineering | Rahul      | Developer |   350
 Checkout Revamp | Engineering | Vikram     | Lead      |   120
 Mobile App      | Engineering | Neha       | Lead      |   300
 Mobile App      | Engineering | Meera      | Developer |   280
 Mobile App      | Engineering | Rohan      | Tester    |    90
(6 rows)
```

Join first, then group (Chapter 4):

```sql
SELECT e.first_name AS rep, count(DISTINCT o.order_id) AS orders,
       round(sum(oi.quantity * oi.unit_price * (1 - oi.discount)), 2) AS revenue
FROM orders o
JOIN employees e    ON e.emp_id = o.sales_rep_id
JOIN order_items oi ON oi.order_id = o.order_id
WHERE o.status <> 'cancelled'
GROUP BY e.emp_id, e.first_name
ORDER BY revenue DESC;
```

Output:

```
  rep  | orders | revenue
-------+--------+---------
 Priya |      3 | 4141.04
 Sara  |      2 |  879.91
 Arjun |      2 |  687.00
(3 rows)
```

`count(DISTINCT o.order_id)`, not `count(*)`: after joining `order_items` each order appears once *per line* (fan-out, §5.9).

Non-equi joins

The condition can be any boolean expression — order lines paid below the current list price:

```sql
SELECT oi.order_id, p.name, p.unit_price AS list_price, oi.unit_price AS paid
FROM order_items oi
JOIN products p ON p.product_id = oi.product_id
               AND oi.unit_price < p.unit_price
ORDER BY oi.order_id;
```

Output:

```
 order_id |     name      | list_price |  paid
----------+---------------+------------+--------
      104 | Standing Desk |     499.00 |  479.00
      108 | Laptop Pro 14 |    1299.00 | 1249.00
(2 rows)
```

Also common: value-to-range (`ON s.amount BETWEEN b.low AND b.high` for tax brackets, grade bands) and time windows (`ON b.t > a.t AND b.t <= a.t + interval '1 day'`).

In the interview

30-second answer. "An inner join returns the combinations of rows from both tables that satisfy the join condition; unmatched rows on either side are dropped, and `NULL` keys never match. If a row has several matches it appears several times."

- *Q: Is `JOIN` the same as `INNER JOIN`?* — Yes.
- *Q: `ON` vs `WHERE` for an inner join?* — Same result; put the relationship in `ON` for readability. For outer joins they differ.

- *Q: Why did my row count go up after a join?* — One-to-many (one left row relates to many right rows): each left row repeats once per matching right row.

Self-check

1. Can `employees JOIN departments` return more than 15 rows? → No: each employee has at most one department (the key is the department's primary key). Joining to a many-side (`orders JOIN order_items`) can.

5.4 LEFT OUTER JOIN

What it is

A **left outer join** (`LEFT JOIN`; `OUTER` is optional) returns **every row of the left table**: matched rows as in an inner join, unmatched rows once with every right column `NULL` (**NULL padding** / **NULL extension**) — like a class register listing every student, with a blank mark for those who missed the exam.

Why it matters

It answers what inner joins silently get wrong: "**all** customers with order count — **including zero**", "all employees, even unassigned", "every product and its sales".

How it works (rows of employees LEFT JOIN departments)

```
SELECT e.first_name, e.dept_id AS e_dept, d.dept_id AS d_dept, d.name
FROM employees e LEFT JOIN departments d ON d.dept_id = e.dept_id
ORDER BY d.dept_id NULLS LAST, e.emp_id;
```

Output:

```
 first_name | e_dept | d_dept |    name
------------+--------+--------+-------------
 Vikram     |      1 |      1 | Engineering
 ...                                (13 matched rows)
 Li         |      5 |      5 | Support
 Asha       | (null) | (null) | (null)
 Rohan      | (null) | (null) | (null)
(15 rows)
```

The 13 matched employees appear as in the inner join; Asha and Rohan appear **once** each, `NULL`-padded; Research is absent (only unmatched **left** rows are kept).

Rule of thumb: `LEFT JOIN` **never returns fewer rows than the left table has.** It returns more if some left rows match several right rows.

The everyday use: counts that include zero

```
SELECT c.customer_id, c.first_name, count(o.order_id) AS orders
FROM customers c
LEFT JOIN orders o ON o.customer_id = c.customer_id
GROUP BY c.customer_id, c.first_name
ORDER BY orders, c.customer_id;
```

Output:

```
 customer_id | first_name | orders
-------------+------------+--------
           9 | Hana       |      0
          10 | Ivan       |      0
           4 | Divya      |      1
 ...
           1 | Anita      |      3
(10 rows)
```

It must be `count(o.order_id)` (a right-table column), not `count(*)`, which counts the padded row as 1 (§5.9).

When not to use it

- Only **left rows that have a match**, no right columns? A semi-join with `EXISTS` is clearer and never duplicates rows (§5.8).
- Match required? Use `INNER JOIN` — a `LEFT JOIN` plus `WHERE` on a right column silently becomes one anyway (§5.9).

Common mistakes

- Filtering the right table in `WHERE` (back to an inner join) — put it in `ON`.
- `a LEFT JOIN b … JOIN c ON c.x = b.x`: the inner join drops the padded rows again (`b.x` is `NULL`). Make it a `LEFT JOIN` too.
- `count(*)` for "how many matches".

In the interview

30-second answer. "A left join returns every row of the left table; where there is a matching right row the columns are filled in, otherwise they're `NULL`. It's the standard way to keep rows with no related data — like customers with zero orders — and, with `WHERE right.pk IS NULL`, to find rows with no match."

- *Q: How many rows can a left join return?* — At least as many as the left table; more if there are one-to-many matches.
- *Q: Find employees without a department.* — `LEFT JOIN departments … WHERE d.dept_id IS NULL`, or simply `WHERE dept_id IS NULL` on employees — plus rows whose `dept_id` points to a missing department, if there is no foreign key.

Self-check

1. Would swapping the tables change the result? → Yes, it becomes "all departments" (§5.5).

5.5 RIGHT OUTER JOIN

What it is

A **right outer join** mirrors it: **every row of the right table**, left side `NULL`-padded. `A RIGHT JOIN B` returns the same rows as `B LEFT JOIN A` (only column order differs).

```
SELECT e.first_name, e.dept_id AS e_dept, d.dept_id AS d_dept, d.name
FROM employees e RIGHT JOIN departments d ON d.dept_id = e.dept_id
ORDER BY d.dept_id, e.emp_id;
```

Output:

```
 first_name | e_dept | d_dept |    name
------------+--------+--------+-------------
 Vikram     |      1 |      1 | Engineering
 ...                                (13 matched rows)
 Li         |      5 |      5 | Support
 (null)     | (null) |      6 | Research
(14 rows)
```

Research appears with `NULL` employee columns; Asha and Rohan (unmatched *left* rows) are gone. `departments d LEFT JOIN employees e` gives the same 14 rows.

When to use it

Rarely. Most teams write every outer join as `LEFT JOIN` with the "keep everything" table first — easier to read. `RIGHT JOIN` helps when appending a table to a long join chain and keeping all *its* rows without restructuring. SQLite supports `RIGHT` and `FULL` joins only from 3.39.0 (2022).

Self-check

1. Rewrite `orders o RIGHT JOIN customers c ON …` as a left join. → `customers c LEFT JOIN orders o ON …`.

5.6 FULL OUTER JOIN

What it is

A **full outer join** keeps **all rows from both tables**, padding whichever side is unmatched — the union of the left- and right-join results (matched pairs once).

```
SELECT e.first_name, e.dept_id AS e_dept, d.dept_id AS d_dept, d.name
FROM employees e FULL JOIN departments d ON d.dept_id = e.dept_id
ORDER BY d.dept_id NULLS LAST, e.emp_id;
```

Output:

```
 first_name | e_dept | d_dept |    name
------------+--------+--------+-------------
 Vikram     |      1 |      1 | Engineering
 ...                                (13 matched rows)
 (null)     | (null) |      6 | Research
 Asha       | (null) | (null) | (null)
 Rohan      | (null) | (null) | (null)
(16 rows)
```

Row counts line up with the edge cases — 13 matched pairs, 2 unmatched employees, 1 unmatched department:

```
SELECT (SELECT count(*) FROM employees e JOIN departments d ON d.dept_id = e.dept_id)       AS inner_rows,
       (SELECT count(*) FROM employees e LEFT JOIN departments d ON d.dept_id = e.dept_id)  AS left_rows,
       (SELECT count(*) FROM employees e RIGHT JOIN departments d ON d.dept_id = e.dept_id) AS right_rows,
       (SELECT count(*) FROM employees e FULL JOIN departments d ON d.dept_id = e.dept_id)  AS full_rows;
```

```
 inner_rows | left_rows | right_rows | full_rows
------------+-----------+------------+-----------
         13 |        15 |         14 |        16
(1 row)
```

In general: `full = inner + unmatched left + unmatched right`, `left = inner + unmatched left`, `right = inner + unmatched right`.

Real-world use: reconciliation

`FULL JOIN` shines in **reconciliation** — checking two records of the same facts agree, where each side may have entries the other lacks: planned vs actual, yesterday's vs today's snapshot, bank statement vs ledger.

```sql
CREATE TABLE planned (emp_id int, hours int);
CREATE TABLE actual  (emp_id int, hours int);
INSERT INTO planned VALUES (3, 300), (4, 200), (5, 250);
INSERT INTO actual  VALUES (3, 310), (5, 250), (7, 40);
SELECT coalesce(p.emp_id, a.emp_id) AS emp_id, p.hours AS planned, a.hours AS actual,
       CASE WHEN p.emp_id IS NULL THEN 'unplanned work'
            WHEN a.emp_id IS NULL THEN 'no work logged'
            WHEN p.hours <> a.hours THEN 'different'
            ELSE 'ok' END AS verdict
FROM planned p FULL JOIN actual a ON a.emp_id = p.emp_id
ORDER BY 1;
```

Output:

```
 emp_id | planned | actual |    verdict
--------+---------+--------+---------------
      3 |     300 |    310 | different
      4 |     200 | (null) | no work logged
      5 |     250 |    250 | ok
      7 |  (null) |     40 | unplanned work
(4 rows)
```

`COALESCE(p.emp_id, a.emp_id)` picks whichever key is present (one is `NULL` on unmatched rows).

Dialects

Database	FULL OUTER JOIN
PostgreSQL, SQL Server, Oracle	supported
SQLite	supported since 3.39.0
MySQL / MariaDB	**not supported** — emulate: `SELECT … FROM a LEFT JOIN b ON … UNION SELECT … FROM a RIGHT JOIN b ON …` (or `UNION ALL` with the right-join half filtered to `WHERE a.key IS NULL` to avoid removing genuine duplicates)

PostgreSQL restricts `FULL JOIN` conditions to ones runnable as a merge or hash join (§5.10) — in practice at least one equality between the sides; `ON a.x < b.y` alone is an error.

In the interview

> **30-second answer.** "A full outer join returns every row from both tables, matching where it can and padding the other side with `NULL`s where it can't. It's used to reconcile two datasets. MySQL lacks it, so you emulate it with a `LEFT JOIN` `UNION` a `RIGHT JOIN`."

- *Q: How do you find rows that are in exactly one of two tables?* — `FULL JOIN … WHERE a.pk IS NULL OR b.pk IS NULL` (or two anti-joins, or `EXCEPT` — "rows in the first result but not the second" — both ways; Chapter 6).

5.7 SELF JOIN

What it is

A **self join** joins a table **to itself** — an ordinary join with the same table twice under two aliases ("employee" row vs "manager" row). Use it when rows refer to rows of the **same** table (hierarchy via `manager_id`) or to compare rows with each other (pairs, sequences).

How it works

```
SELECT e.emp_id, e.first_name AS employee, m.first_name AS manager
FROM employees e
LEFT JOIN employees m ON m.emp_id = e.manager_id
ORDER BY e.emp_id;
```

Output:

```
 emp_id | employee | manager
--------+----------+---------
      1 | Asha     | (null)
      2 | Vikram   | Asha
      3 | Neha     | Vikram
 ...
     15 | Rohan    | Vikram
(15 rows)
```

`e` is the employee copy, `m` the manager copy; `m.emp_id = e.manager_id` pairs each employee with their manager's row. `LEFT JOIN` keeps Asha (`manager_id` `NULL`); an inner join would drop her.

More self-join patterns

Compare each employee with their manager:

```
SELECT e.first_name, e.salary, m.first_name AS manager, m.salary AS mgr_salary
FROM employees e JOIN employees m ON m.emp_id = e.manager_id
WHERE e.salary >= 0.7 * m.salary
ORDER BY e.emp_id;
```

```
 first_name |  salary   | manager | mgr_salary
------------+-----------+---------+------------
 Vikram     | 185000.00 | Asha    |  250000.00
 Neha       | 142000.00 | Vikram  |  185000.00
(2 rows)
```

Count direct reports (join, then group by the manager):

```
SELECT m.first_name AS manager, count(*) AS direct_reports
FROM employees e JOIN employees m ON m.emp_id = e.manager_id
GROUP BY m.emp_id, m.first_name
ORDER BY direct_reports DESC, manager;
```

```
 manager | direct_reports
---------+----------------
 Vikram  |              5
 Asha    |              4
 Daniel  |              3
 Kiran   |              1
 Lucas   |              1
(5 rows)
```

Pairs sharing a value — `a.emp_id < b.emp_id` removes self-pairs (Arjun–Arjun) and mirror duplicates (Arjun–Sara *and* Sara–Arjun):

```
SELECT a.first_name AS emp_a, b.first_name AS emp_b, a.salary
FROM employees a JOIN employees b ON a.salary = b.salary AND a.emp_id < b.emp_id
ORDER BY a.salary;
```

```
 emp_a | emp_b |  salary
-------+-------+----------
 Arjun | Sara  | 68000.00
 Rahul | Meera | 98000.00
(2 rows)
```

Two levels up with a second self join (arbitrary depth needs a recursive CTE — a Common Table Expression is a named subquery written with `WITH`; a *recursive* one keeps joining to its own previous output until nothing new appears; Chapter 7):

```
SELECT e.first_name AS employee, m.first_name AS manager, g.first_name AS grand_manager
FROM employees e
JOIN employees m ON m.emp_id = e.manager_id
LEFT JOIN employees g ON g.emp_id = m.manager_id
WHERE e.dept_id = 1
ORDER BY e.emp_id;
```

```
 employee | manager | grand_manager
----------+---------+--------------
 Vikram   | Asha    | (null)
 Neha     | Vikram  | Asha
 Rahul    | Vikram  | Asha
 Meera    | Vikram  | Asha
(4 rows)
```

Rows **in time** — customers reordering within 60 days:

```
SELECT a.customer_id, a.order_id AS first_order, b.order_id AS next_order,
       b.placed_at::date - a.placed_at::date AS days_between
FROM orders a JOIN orders b
  ON b.customer_id = a.customer_id AND b.placed_at > a.placed_at
 AND b.placed_at <= a.placed_at + interval '60 days'
ORDER BY a.customer_id, a.order_id;
```

```
 customer_id | first_order | next_order | days_between
-------------+-------------+------------+--------------
           1 |         101 |        104 |           39
(1 row)
```

"Compare with the previous row" problems (consecutive days, streaks) are solved like this or, more efficiently, with `LAG` / `LEAD` window functions (read the previous/next row's value without collapsing rows; Chapter 8).

Common mistakes

- Forgetting aliases (`employees JOIN employees` is an error: `table name "employees" specified more than once`).
- Using an inner join for hierarchies and losing the root (the CEO).
- Getting every pair twice (and each row paired with itself) — use `a.id < b.id`.

In the interview

30-second answer. "A self join joins a table to itself using two aliases — for hierarchies like employee → manager, for finding pairs of rows with something in common, and for comparing a row with a related row of the same table. Use a left join to keep rows without a parent, and `a.id < b.id` to avoid duplicate pairs."

- *Q: Employees who earn more than their manager?* — `FROM employees e JOIN employees m ON m.emp_id = e.manager_id WHERE e.salary > m.salary`. (In our data: nobody.)
- *Q: How do you get the whole chain of managers?* — A recursive CTE; self joins only reach a fixed depth.

5.8 Semi-joins and anti-joins

What they are

- **Semi-join** — "rows of A that **have at least one** match in B" (customers who ordered); each A row **at most once**, no B columns needed.
- **Anti-join** — "rows of A with **no** match in B" (customers who never ordered).

There is no `SEMI JOIN` keyword: write `EXISTS` / `IN` (semi) and `NOT EXISTS` / `LEFT JOIN … IS NULL` / `NOT IN` (anti). The planner (the component deciding how to execute a query) recognises these shapes and uses dedicated *semi-join* / *anti-join* algorithms.

Semi-join: EXISTS vs a plain JOIN

```
SELECT c.customer_id, c.first_name
FROM customers c
WHERE EXISTS (SELECT 1 FROM orders o WHERE o.customer_id = c.customer_id)
ORDER BY c.customer_id;
```

```
 customer_id | first_name
-------------+------------
           1 | Anita
           2 | Ben
           3 | Chen
           4 | Divya
           5 | Ethan
           6 | Fatima
           7 | George
           8 | Anita
(8 rows)
```

A plain `customers JOIN orders` answers a different question — one row **per order** (14 rows, Anita three times). `SELECT DISTINCT` fixes it but builds all pairs then de-duplicates and hides the intent; `EXISTS` stops at each customer's first match.

Anti-join: three ways

```
-- 1. NOT EXISTS
SELECT c.customer_id, c.first_name FROM customers c
WHERE NOT EXISTS (SELECT 1 FROM orders o WHERE o.customer_id = c.customer_id)
ORDER BY 1;

-- 2. LEFT JOIN ... IS NULL
SELECT c.customer_id, c.first_name FROM customers c
LEFT JOIN orders o ON o.customer_id = c.customer_id
WHERE o.order_id IS NULL
ORDER BY 1;

-- 3. NOT IN
SELECT c.customer_id, c.first_name FROM customers c
WHERE c.customer_id NOT IN (SELECT customer_id FROM orders)
ORDER BY 1;
```

Output (identical for all three here):

```
 customer_id | first_name
-------------+------------
           9 | Hana
          10 | Ivan
(2 rows)
```

Form	NULL-safe?	Notes
`NOT EXISTS (correlated subquery)` (a subquery that refers to the outer row)	**Yes**	Clearest intent; PostgreSQL plans it as an anti join. **Recommended.**
`LEFT JOIN … WHERE b.pk IS NULL`	Yes, if you test a never-NULL column (the PK)	Also planned as an anti join; handy when you want other columns from the join.
`NOT IN (subquery)`	**No**	If the subquery returns even one `NULL`, the result is **empty** — see Chapter 6. Also often planned less efficiently.

`EXPLAIN` (shows the chosen strategy; `Seq Scan` = read the whole table) runs `NOT EXISTS` as a **Hash Anti Join**:

```
EXPLAIN (COSTS OFF)
SELECT c.customer_id FROM customers c
WHERE NOT EXISTS (SELECT 1 FROM orders o WHERE o.customer_id = c.customer_id);
```

```
                QUERY PLAN
------------------------------------------
 Hash Anti Join
   Hash Cond: (c.customer_id = o.customer_id)
   ->  Seq Scan on customers c
   ->  Hash
         ->  Seq Scan on orders o
(5 rows)
```

More anti-joins — the product never sold and the department with no staff:

```
SELECT p.name FROM products p
WHERE NOT EXISTS (SELECT 1 FROM order_items oi WHERE oi.product_id = p.product_id);

SELECT d.name FROM departments d
LEFT JOIN employees e ON e.dept_id = d.dept_id
WHERE e.emp_id IS NULL;
```

```
   name
-----------
 Desk Lamp
(1 row)

   name
----------
 Research
(1 row)
```

In the interview

30-second answer. "A semi-join returns rows of A that have a match in B, once each — write it with `EXISTS` or `IN`. An anti-join returns rows of A with no match — write it with `NOT EXISTS` or `LEFT JOIN … WHERE b.pk IS NULL`. Avoid `NOT IN` with a subquery on a nullable column: one `NULL` makes it return nothing."

- *Q: Customers who never placed an order?* — Any of the three forms; say why you prefer `NOT EXISTS`.
- *Q: Why not `JOIN` + `DISTINCT` for a semi-join?* — It multiplies then de-duplicates; `EXISTS` stops at the first match and states the intent.

Self-check

1. In `LEFT JOIN … IS NULL`, why test `o.order_id`, not `o.sales_rep_id`? → `sales_rep_id` is `NULL` on real rows (online orders); a `NULL` primary key can only be padding.

5.9 Join pitfalls

Pitfall 1 — fan-out (row multiplication)

Joining a parent to **two child tables** at once pairs every child of one kind with every child of the other: Engineering's 4 employees × 2 projects = 8 rows, every salary counted twice:

```
SELECT d.name, count(*) AS joined_rows, sum(e.salary) AS wrong_payroll
FROM departments d
JOIN employees e ON e.dept_id = d.dept_id
JOIN projects  p ON p.dept_id = d.dept_id
WHERE d.dept_id = 1
GROUP BY d.name;
```

```
    name     | joined_rows | wrong_payroll
-------------+-------------+---------------
 Engineering |           8 |    1046000.00
(1 row)
```

The real payroll is 523,000. **Fix:** aggregate each child separately (derived table — a subquery in `FROM` — or CTE) to one row per department, *then* join:

```
SELECT d.name, e.payroll, p.projects
FROM departments d
JOIN (SELECT dept_id, sum(salary) AS payroll FROM employees GROUP BY dept_id) e ON e.dept_id = d.dept_id
JOIN (SELECT dept_id, count(*) AS projects FROM projects GROUP BY dept_id)     p ON p.dept_id = d.dept_id
ORDER BY d.dept_id;
```

```
    name     |  payroll  | projects
-------------+-----------+----------
 Engineering | 523000.00 |        2
 Marketing   | 181000.00 |        1
 Finance     | 211000.00 |        1
 Support     |  64000.00 |        1
(4 rows)
```

Signs: totals that are exact multiples of the right answer; `count(*)` above the number of parents. `count(DISTINCT …)` hides it for counts but **not** sums.

Pitfall 2 — a WHERE on the right table undoes a LEFT JOIN

Goal: every customer with their number of **delivered** orders, 0 if none.

```
SELECT c.customer_id, count(o.order_id) AS delivered
FROM customers c LEFT JOIN orders o ON o.customer_id = c.customer_id
WHERE o.status = 'delivered'
GROUP BY c.customer_id ORDER BY c.customer_id;          -- wrong: customers with 0 vanish
```

```
 customer_id | delivered
-------------+-----------
           1 |         2
           2 |         2
           3 |         1
           5 |         1
           8 |         1
(5 rows)
```

Only 5: `WHERE o.status = 'delivered'` ran **after** the join (logical order, Chapter 4) and discarded the padded rows (Hana, Ivan: `o.status = NULL`) and customers with no delivered order. Put the right-table condition **in** `ON`, so it limits which rows *match*, not which *survive*:

```
SELECT c.customer_id, count(o.order_id) AS delivered
FROM customers c LEFT JOIN orders o ON o.customer_id = c.customer_id
                                    AND o.status = 'delivered'
GROUP BY c.customer_id ORDER BY c.customer_id;          -- right: condition in ON
```

```
 customer_id | delivered
-------------+-----------
           1 |         2
           2 |         2
           3 |         1
           4 |         0
 ...
          10 |         0
(10 rows)
```

Condition on the right table of a LEFT JOIN placed in…	Effect
`ON`	Decides which right rows count as a match; unmatched left rows are still kept (padded).
`WHERE`	Filters the joined result; padded rows have `NULL` there and are removed → behaves like an `INNER JOIN`.

(A `WHERE right.pk IS NULL` test is the deliberate exception — that's the anti-join.) Conditions on the **left** table belong in `WHERE`; put in `ON` they don't remove left rows, they just stop them matching.

Pitfall 3 — NULL keys never match

```
CREATE TABLE a (k int, label text);
CREATE TABLE b (k int, label text);
INSERT INTO a VALUES (1, 'a1'), (2, 'a2'), (NULL, 'a-null');
INSERT INTO b VALUES (1, 'b1'), (3, 'b3'), (NULL, 'b-null');
SELECT a.label, b.label FROM a JOIN b ON a.k = b.k;
SELECT a.label, b.label FROM a JOIN b ON a.k IS NOT DISTINCT FROM b.k ORDER BY 1;
```

```
 label | label
-------+-------
 a1    | b1
(1 row)

 label  | label
--------+--------
 a-null | b-null
 a1     | b1
(2 rows)
```

`NULL = NULL` is UNKNOWN, so `NULL` keys don't pair. To match `NULL` with `NULL` (e.g. comparing snapshots of a nullable column) use `IS NOT DISTINCT FROM` — but PostgreSQL can't run it as a hash or merge join, so it can be slow on large tables.

Pitfall 4 — COUNT(*) after an outer join

```
SELECT d.name, count(*) AS count_star, count(e.emp_id) AS count_col
FROM departments d LEFT JOIN employees e ON e.dept_id = d.dept_id
WHERE d.dept_id IN (5, 6)
GROUP BY d.name ORDER BY d.name;
```

```
   name   | count_star | count_col
----------+------------+-----------
 Research |          1 |         0
 Support  |          1 |         1
(2 rows)
```

Research has no employees but `count(*)` says 1 (the padded row). Count a right-side, never-NULL column.

Other pitfalls, briefly

- **Wrong columns** — `ON e.emp_id = d.dept_id` "works" (both integers) and returns nonsense; foreign keys document the right paths.
- **Type mismatches** — `text` to `integer` fails in PostgreSQL; casting in the condition (`ON a.code::int = b.id`) prevents index use on `a.code` (an **index** is a sorted lookup structure for finding rows by value without reading everything).
- **Duplicated keys in a lookup table** — if the "one" side isn't unique, every match fans out. Check with `GROUP BY key HAVING count(*) > 1`.

Self-check

1. A query sums order amounts and joins both `order_items` and `payments` to `orders`. What goes wrong? → Fan-out: each item pairs with each payment, inflating the sums. Aggregate each child first.
2. `a LEFT JOIN b ON a.id = b.a_id WHERE b.active` — what kind of join is this really? → An inner join (padded rows have `b.active = NULL`). Move `b.active` to `ON`.

5.10 USING, NATURAL JOIN, multi-table joins, and how joins are executed

USING

When join columns share a **name**, `USING (col)` is shorthand for `ON a.col = b.col` ; the column appears **once** in the output (and `SELECT *`) and is referenced without a prefix:

```
SELECT order_id, customer_id, first_name, status
FROM orders JOIN customers USING (customer_id)
WHERE order_id <= 103
ORDER BY order_id;
```

```
 order_id | customer_id | first_name |  status
----------+-------------+------------+-----------
      101 |           1 | Anita      | delivered
      102 |           2 | Ben        | delivered
      103 |           3 | Chen       | delivered
(3 rows)
```

In `FULL JOIN … USING` the merged column is `COALESCE(left.col, right.col)`, filled for unmatched rows on either side:

```
SELECT dept_id, first_name, name
FROM employees FULL JOIN departments USING (dept_id)
WHERE first_name IS NULL OR name IS NULL
ORDER BY dept_id NULLS LAST;
```

```
 dept_id | first_name |   name
---------+------------+----------
       6 | (null)     | Research
  (null) | Asha       | (null)
  (null) | Rohan      | (null)
(3 rows)
```

NATURAL JOIN — and why to avoid it

`NATURAL JOIN` joins on **every** same-named column pair without listing them — convenient until a table gains a column:

```
SELECT count(*) AS natural_rows FROM projects NATURAL JOIN departments;
SELECT count(*) AS using_rows   FROM projects JOIN departments USING (dept_id);
```

```
 natural_rows
--------------
            0
(1 row)

 using_rows
------------
          5
(1 row)
```

`projects` and `departments` share `dept_id` — and `name` and `budget`; all three had to match, so nothing did. Schemas (table definitions) change (someone adds `created_at` to both) and `NATURAL JOIN` silently changes meaning. **Name your join columns.**

	ON	USING	NATURAL
Columns compared	any expression	listed same-named columns	*all* same-named columns
Duplicated key column in `SELECT *`	yes (both copies)	no (one merged column)	no
Safe against schema changes	yes	yes	**no**
SQL Server support	yes	no	no

Multi-table joins

Every order line with customer, product and sales rep, keeping online orders:

```
SELECT o.order_id, c.first_name AS customer, p.name AS product, oi.quantity,
       coalesce(e.first_name, 'online') AS rep
FROM orders o
JOIN customers c    ON c.customer_id = o.customer_id
JOIN order_items oi ON oi.order_id   = o.order_id
JOIN products p     ON p.product_id  = oi.product_id
LEFT JOIN employees e ON e.emp_id    = o.sales_rep_id
WHERE o.order_id BETWEEN 101 AND 104
ORDER BY o.order_id, p.name;
```

```
 order_id | customer |           product           | quantity |  rep
----------+----------+-----------------------------+----------+--------
      101 | Anita    | Laptop Pro 14               |        1 | Priya
      101 | Anita    | Wireless Mouse              |        1 | Priya
      102 | Ben      | Noise-Cancelling Headphones |        2 | online
      103 | Chen     | Mechanical Keyboard         |        1 | Arjun
      103 | Chen     | USB-C Hub                   |        2 | Arjun
      104 | Anita    | Ergonomic Chair             |        1 | online
      104 | Anita    | Standing Desk               |        1 | online
(7 rows)
```

A department overview from outer joins, so Sales (no projects) and Research (nothing) still appear:

```sql
SELECT d.name AS department,
       count(DISTINCT e.emp_id)     AS employees,
       count(DISTINCT p.project_id) AS projects,
       coalesce(sum(ep.hours), 0)   AS hours_on_own_projects
FROM departments d
LEFT JOIN employees e          ON e.dept_id = d.dept_id
LEFT JOIN projects p           ON p.dept_id = d.dept_id
LEFT JOIN employee_projects ep ON ep.project_id = p.project_id AND ep.emp_id = e.emp_id
GROUP BY d.dept_id, d.name
ORDER BY d.dept_id;
```

```
 department  | employees | projects | hours_on_own_projects
-------------+-----------+----------+----------------------
 Engineering |         4 |        2 |                  1450
 Sales       |         4 |        0 |                     0
 Marketing   |         2 |        1 |                   350
 Finance     |         2 |        1 |                   240
 Support     |         1 |        1 |                    60
 Research    |         0 |        0 |                     0
(6 rows)
```

It deliberately fans out (employees × projects) and is correct only because counts use `DISTINCT` and hours come from `employee_projects` joined on **both** keys (each assignment once). When in doubt, pre-aggregate (§5.9).

Rules: (1) inner joins can be written in any order — same result, the planner picks the physical order; (2) **outer joins are not freely reorderable** — `(a LEFT JOIN b) JOIN c` can differ from `a LEFT JOIN (b JOIN c)`, so write the order you mean with the "keep everything" table first; (3) every table after the first needs its own join condition, or you get a Cartesian product.

How databases execute joins

Joins don't run as filtered Cartesian products. The planner picks one of three algorithms per join from table sizes, indexes and statistics (stored summaries of row counts and value spread) — visible in `EXPLAIN` (Chapter 12):

Algorithm	How it works	Good when	Cost (roughly)
Nested loop	For each outer row, look up matching inner rows (ideally through an index)	Outer side small, index on the inner join key; also the only option for non-equi joins	*outer rows × cost of one lookup*
Hash join	Build a hash table (a lookup structure that finds a key in about one step) on the smaller input's key, then probe it with each row of the other	Equi-joins of large unsorted inputs	*O(n + m)* (work grows with the total size of both inputs), needs memory for the hash table
Merge join	Sort both inputs by the key (or read them in index order), then walk them together	Inputs already sorted / indexed; very large inputs	*O(n + m)* after sorting

Index your foreign-key columns (`orders.customer_id`, `order_items.product_id` …): PostgreSQL indexes primary keys automatically but **not** foreign keys, and parent-to-child nested loops depend on them.

Dialect summary for joins

Feature	PostgreSQL	MySQL 8	SQLite	SQL Server
`INNER` , `LEFT` , `CROSS`	yes	yes	yes	yes
`RIGHT JOIN`	yes	yes	3.39+	yes
`FULL JOIN`	yes	**no** (emulate with `UNION`)	3.39+	yes
`USING` , `NATURAL`	yes	yes	yes	**no**
`LATERAL` (Chapter 6)	yes	8.0.14+	no	`CROSS APPLY` / `OUTER APPLY`
`IS NOT DISTINCT FROM`	yes	`<=>`	`IS`	2022+

In the interview

30-second answer (whole chapter). "A join pairs rows from two tables that satisfy a condition — conceptually a filtered Cartesian product. Inner join keeps matches only; left/right keep all rows of one side with `NULL` padding; full keeps both; cross gives every combination. Self joins use two aliases on one table. For 'has a match / has no match' use `EXISTS` / `NOT EXISTS` . Watch for fan-out, filters on the outer side in `WHERE` , and `NULL` keys."

- *Q: What's the difference between `WHERE` and `ON` in a left join?* — `ON` decides matches; `WHERE` filters the result and removes padded rows.
- *Q: What join algorithms do you know?* — Nested loop, hash, merge; the optimizer chooses based on sizes, indexes and sort order.
- *Q: How do you do a full outer join in MySQL?* — `LEFT JOIN` `UNION` `RIGHT JOIN` .

Self-check

1. What columns does `SELECT *` return for `orders JOIN customers USING (customer_id)` ? → `customer_id` once, then the other columns of `orders` and `customers` .
2. Why is `NATURAL JOIN` risky? → It joins on every same-named column, which changes when schemas change.
3. Which physical join works for `ON a.x < b.y` ? → Nested loop (hash and merge need equality).

Chapter 5 TL;DR

Join	Returns	Typical use
`CROSS JOIN`	every pair (m × n)	grids, combinations
`INNER JOIN`	matched pairs only	"orders with their customers"
`LEFT JOIN`	all left rows + matches, `NULL` -padded	"all customers, with orders if any"
`RIGHT JOIN`	all right rows + matches	same, mirrored — write as `LEFT`
`FULL JOIN`	all rows of both sides	reconciliation
Self join	table joined to itself (two aliases)	hierarchies, pairs, previous/next
Semi-join (`EXISTS`)	left rows with ≥ 1 match, once each	"customers who ordered"
Anti-join (`NOT EXISTS` , `LEFT … IS NULL`)	left rows with no match	"customers who never ordered"

Pitfalls: fan-out, right-table filters in `WHERE` after a `LEFT JOIN`, `NULL` keys, `count(*)` on padded rows, `NOT IN` with `NULL`s, `NATURAL JOIN`.

Next: Chapter 6 — Subqueries and Set Operations.

Chapter 6 — Subqueries and Set Operations

Some questions need **another answer first**: "who earns more than average?" needs the average; "which customers never ordered?" needs those who did. A **subquery** is a query **inside** another, whose result serves as a value, a list or a table. The second half covers stacking or comparing whole results with the **set operators** `UNION`, `INTERSECT` and `EXCEPT`.

Sections: 6.1 subqueries (scalar, row, table; non-correlated vs correlated) · 6.2 `IN` / `NOT IN` and the `NULL` trap · 6.3 correlated subqueries · 6.4 `EXISTS` · 6.5 `ANY` / `SOME` / `ALL` · 6.6 derived tables and row subqueries · 6.7 `LATERAL` for top-N per group (the first *N* rows inside each group) · 6.8–6.10 `UNION`, `INTERSECT`, `EXCEPT` (and `ALL` forms) · 6.11 precedence, sorting, choosing a tool.

Running the examples. All queries are in `code/` (`bash tools_run_sql.sh sql '06-*'`), real psql output from PostgreSQL 14, `NULL` shown as `(null)`, on the `company` database (`code/00_schema.sql` ; Chapter 1).

6.1 Subqueries: queries inside queries

What it is

A **subquery** (**nested query**, **inner query**) is a complete `SELECT` in parentheses inside another statement (the **outer query**), whose result is used as a single value, a list to compare against, or a temporary table.

Where a subquery can appear

Position	Example	What the subquery must return
`WHERE` / `HAVING` with `=`, `<`, `>` …	`WHERE salary > (SELECT avg(salary) …)`	**one value** (scalar)
`WHERE` with `IN`, `ANY`, `ALL`	`WHERE dept_id IN (SELECT dept_id …)`	**one column, many rows**
`WHERE` with `EXISTS`	`WHERE EXISTS (SELECT 1 …)`	anything — only "any rows?" matters
`SELECT` list	`SELECT name, (SELECT count(*) …) AS n`	one value per outer row
`FROM` / `JOIN`	`FROM (SELECT … ) AS t`	a whole table (**derived table**)
Row comparison	`WHERE (a, b) = (SELECT x, y …)`	one row with several columns
`INSERT … SELECT`, `UPDATE … SET col = (…)`, `DELETE … WHERE … IN (…)`	see §6.3	depends on position

Types of subqueries

Two independent classifications. **By result shape:**

Type	Returns	Typical operators
Scalar subquery	exactly one row, one column (or no row → `NULL`)	`=`, `<>`, `<`, `>`, arithmetic, `SELECT` list
Row subquery	one row, several columns	`(a, b) = (…)`
Column / multi-row subquery	one column, many rows	`IN`, `NOT IN`, `ANY`, `ALL`
Table subquery	many rows and columns	`FROM`, `EXISTS`, `IN` with row values

By dependence on the outer query:

Type	Meaning	Evaluated
Non-correlated (simple, nested)	The inner query does not mention the outer query's tables. It can run on its own.	Conceptually **once**; the result is reused.
Correlated	The inner query refers to a column of the current outer row (e.g. `WHERE x.dept_id = e.dept_id`). It can't run on its own.	Conceptually **once per outer row** (§6.3).

Other names you may meet: *nested scalar subquery* (one value in a comparison, below); *single-row subquery* (at most one row, used with `=`, `<`, `>` or a row constructor, §6.6); *multiple-row subquery* (several rows, needs `IN`, `ANY`, `ALL` or `EXISTS`, §6.2); *single-column subquery* (one column, one or many rows — every `IN (SELECT one_column …)`).

How a non-correlated scalar subquery works

```
SELECT first_name, salary
FROM employees
WHERE salary > (SELECT avg(salary) FROM employees)
ORDER BY salary DESC;
```

Output:

```
 first_name |  salary
------------+-----------
 Asha       | 250000.00
 Vikram     | 185000.00
 Daniel     | 150000.00
 Neha       | 142000.00
 Kiran      | 135000.00
 Lucas      | 120000.00
(6 rows)
```

The inner query runs once (average 107,800), the value is substituted (`WHERE salary > 107800`), then the outer query runs. Without a subquery the application needs two round trips (request plus reply) — and the value could change between them.

A scalar subquery works anywhere a single value can, including the `SELECT` list and arithmetic:

```
SELECT first_name, salary,
       round((SELECT avg(salary) FROM employees)) AS company_avg,
       round(salary - (SELECT avg(salary) FROM employees)) AS diff
FROM employees
WHERE dept_id = 1
ORDER BY emp_id;
```

Output:

```
 first_name |  salary   | company_avg | diff
------------+-----------+-------------+-------
 Vikram     | 185000.00 |      107800 | 77200
 Neha       | 142000.00 |      107800 | 34200
 Rahul      |  98000.00 |      107800 | -9800
 Meera      |  98000.00 |      107800 | -9800
(4 rows)
```

Classic uses — the row with the extreme value, and the second-highest value:

```sql
SELECT first_name, hire_date FROM employees
WHERE hire_date = (SELECT max(hire_date) FROM employees);

SELECT max(salary) AS second_highest
FROM employees
WHERE salary < (SELECT max(salary) FROM employees);
```

Output:

```
 first_name | hire_date
------------+------------
 Rohan      | 2025-06-02
(1 row)

 second_highest
----------------
      185000.00
(1 row)
```

"The largest salary smaller than the largest salary." For the *N*-th highest, window functions (compute over related rows while keeping every row) such as `DENSE_RANK()` are cleaner (Chapter 8; interview versions in Chapter 19).

Scalar subqueries: the two edge cases

No rows → `NULL`:

```sql
SELECT (SELECT salary FROM employees WHERE emp_id = 999) AS missing;
```

```
 missing
---------
  (null)
(1 row)
```

More than one row → run-time failure:

```sql
SELECT first_name FROM employees
WHERE salary = (SELECT salary FROM employees WHERE dept_id = 1);
```

```
ERROR:  more than one row returned by a subquery used as an expression
```

The error depends on the **data** — it can pass testing and fail in production once a second matching row appears. If several rows are possible, use `IN` (§6.2) or make the subquery provably single-row (aggregate, or filter on a unique key).

Common mistakes

- `=` with a subquery that can return several rows (use `IN`).
- `ORDER BY` inside a subquery expecting an ordered outer result — only the outermost `ORDER BY` counts.
- A subquery where a join is simpler, e.g. fetching a related column for every row (§6.3).

Self-check

1. Is `(SELECT avg(salary) FROM employees)` correlated? → No — it refers to no outer table, so it runs once.
2. Four places a subquery can appear in a `SELECT`? → `SELECT` list, `FROM` / `JOIN`, `WHERE`, `HAVING`.

6.2 Multi-row subqueries: IN and NOT IN

What it is

For **one column, many rows**: `x IN (subquery)` is true if `x` equals **any** value; `x NOT IN (subquery)` if it equals **none**.

```
SELECT first_name, dept_id FROM employees
WHERE dept_id IN (SELECT dept_id FROM departments WHERE location = 'Mumbai')
ORDER BY emp_id;
```

Output:

```
 first_name | dept_id
------------+---------
 Daniel     |       2
 Priya      |       2
 Arjun      |       2
 Sara       |       2
 Lucas      |       3
 Ananya     |       3
(6 rows)
```

The subquery returns `{2, 3}` (Mumbai departments). `IN` never duplicates outer rows even if the list repeats values — it is a semi-join (each outer row at most once; Chapter 5). The subquery can have its own joins and filters:

```
SELECT name FROM products
WHERE product_id IN (SELECT oi.product_id
                     FROM order_items oi JOIN orders o ON o.order_id = oi.order_id
                     WHERE o.status = 'delivered')
ORDER BY name;
```

Output:

```
           name
----------------------------
 Ergonomic Chair
 Gift Card
 ...
 Wireless Mouse
(8 rows)
```

Nesting several levels

Read nested subqueries **from the innermost outwards**: Furniture products → orders containing them → their customers.

```
SELECT first_name, last_name FROM customers
WHERE customer_id IN (
      SELECT customer_id FROM orders
      WHERE order_id IN (
            SELECT order_id FROM order_items
            WHERE product_id IN (SELECT product_id FROM products WHERE category = 'Furniture')))
ORDER BY customer_id;
```

Output:

```
 first_name | last_name
------------+-----------
 Anita      | Shah
 Fatima     | Qureshi
 George     | Hill
(3 rows)
```

Deep nesting is legal but hard to read; joins plus `DISTINCT` / `EXISTS`, or a chain of CTEs (Common Table Expressions — named subqueries written with `WITH`; Chapter 7), are usually clearer.

The NOT IN trap

Question: which employees are **not** anybody's manager?

```
SELECT count(*) AS not_in_rows FROM employees
WHERE emp_id NOT IN (SELECT manager_id FROM employees);
```

Output:

```
 not_in_rows
-------------
           0
(1 row)
```

Zero — wrong. The list contains a `NULL` (the CEO has no manager). `x NOT IN (a, b, NULL)` means `x <> a AND x <> b AND x <> NULL`; the last part is UNKNOWN for every `x`, so the condition is never TRUE and `WHERE` keeps nothing (three-valued logic, Chapter 2).

Condition	x = 3 , list (1, 2)	x = 3 , list (1, 2, NULL)	x = 1 , list (1, 2, NULL)
`x IN (list)`	FALSE	**UNKNOWN**	TRUE
`x NOT IN (list)`	TRUE	**UNKNOWN** (row dropped)	FALSE

Two fixes:

```
-- Fix 1: filter the NULLs out of the subquery
SELECT first_name FROM employees
WHERE emp_id NOT IN (SELECT manager_id FROM employees WHERE manager_id IS NOT NULL)
ORDER BY emp_id;

-- Fix 2 (preferred): NOT EXISTS is NULL-safe
SELECT first_name FROM employees e
WHERE NOT EXISTS (SELECT 1 FROM employees r WHERE r.manager_id = e.emp_id)
ORDER BY emp_id;
```

Output (both):

```
 first_name
------------
 Neha
 Rahul
 ...
 Rohan
(10 rows)
```

Prefer `NOT EXISTS`: correct whatever the data, states the intent, and PostgreSQL plans it as an anti join (returns outer rows with *no* match), whereas `NOT IN (subquery)` stays a filter checking the whole list (hashed into an in-memory hash table if it fits, otherwise re-scanned per row).

In the interview

30-second answer. " `IN (subquery)` checks membership in the list the subquery returns — a semi-join, no duplicates. `NOT IN` is dangerous: if the list contains a `NULL`, `NOT IN` is never true and returns

nothing. Use `NOT EXISTS` (or filter out `NULL`s) for anti-joins."

- *Q: Why does `NOT IN` return no rows?* — The list has a `NULL`; `x <> NULL` is UNKNOWN, so the `AND` chain is never TRUE.
- *Q: Does `IN` have the same problem?* — No: a `NULL` in the list can't make a match TRUE into something else; it can only turn FALSE into UNKNOWN, and both are dropped by `WHERE`.

Self-check

1. `SELECT 5 NOT IN (1, 2, NULL)`? → `NULL` (UNKNOWN).
2. Columns in our schema that make `NOT IN (SELECT col …)` risky? → Any nullable one: `employees.manager_id`, `employees.dept_id`, `orders.sales_rep_id`.

6.3 Correlated subqueries

What it is

A **correlated subquery** refers to a column of the **current outer row**, so it can't run once up front: conceptually it re-runs for every outer row with that row's values plugged in.

How it works

"Employees who earn more than the average **of their own department**":

```
SELECT e.first_name, e.dept_id, e.salary
FROM employees e
WHERE e.salary > (SELECT avg(x.salary) FROM employees x WHERE x.dept_id = e.dept_id)
ORDER BY e.dept_id, e.salary DESC;
```

Output:

```
 first_name | dept_id |  salary
------------+---------+-----------
 Vikram     |       1 | 185000.00
 Neha       |       1 | 142000.00
 Daniel     |       2 | 150000.00
 Lucas      |       3 | 120000.00
 Kiran      |       4 | 135000.00
(5 rows)
```

For Vikram (`e.dept_id = 1`) the inner query gives Engineering's average 130,750; 185,000 > 130,750, so he is kept; repeat per row. Asha and Rohan (`dept_id` `NULL`) are never kept: `x.dept_id = NULL` matches nothing, the average is `NULL`, the comparison UNKNOWN.

The aliases `e` (outer) and `x` (inner) are essential: an unqualified column resolves to the **nearest** table that has it (inner first), which can silently turn a correlated subquery into a non-correlated one or vice versa.

Top earner per department (ties all returned):

```
SELECT e.dept_id, e.first_name, e.salary
FROM employees e
WHERE e.salary = (SELECT max(x.salary) FROM employees x WHERE x.dept_id = e.dept_id)
ORDER BY e.dept_id;
```

Output:

```
 dept_id | first_name |  salary
---------+------------+-----------
       1 | Vikram     | 185000.00
       2 | Daniel     | 150000.00
       3 | Lucas      | 120000.00
       4 | Kiran      | 135000.00
       5 | Li         |  64000.00
(5 rows)
```

Correlated subqueries in SELECT, and the join alternative

In the `SELECT` list it works like a per-row lookup:

```
SELECT c.customer_id, c.first_name,
       (SELECT count(*)       FROM orders o WHERE o.customer_id = c.customer_id) AS orders,
       (SELECT max(placed_at)::date FROM orders o WHERE o.customer_id = c.customer_id) AS last_order
FROM customers c
ORDER BY c.customer_id;
```

Output:

```
 customer_id | first_name | orders | last_order
-------------+------------+--------+------------
           1 | Anita      |      3 | 2025-06-02
           2 | Ben        |      2 | 2025-04-01
 ...
           9 | Hana       |      0 | (null)
          10 | Ivan       |      0 | (null)
(10 rows)
```

The same result with a `LEFT JOIN` and `GROUP BY`:

```
SELECT c.customer_id, c.first_name, count(o.order_id) AS orders, max(o.placed_at)::date AS last_order
FROM customers c LEFT JOIN orders o ON o.customer_id = c.customer_id
GROUP BY c.customer_id, c.first_name
ORDER BY c.customer_id;
```

(Identical output.) How the optimizer runs the subquery version, per `EXPLAIN` (prints the chosen plan; `Seq Scan` = read the whole table):

```
EXPLAIN (COSTS OFF)
SELECT c.customer_id, (SELECT count(*) FROM orders o WHERE o.customer_id = c.customer_id) AS orders
FROM customers c;
```

```
                     QUERY PLAN
-----------------------------------------------------
 Seq Scan on customers c
   SubPlan 1
     ->  Aggregate
           ->  Seq Scan on orders o
                 Filter: (customer_id = c.customer_id)
(5 rows)
```

`SubPlan 1` means the inner aggregate really **runs once per customer**. With an index on `orders.customer_id` (an **index** is a sorted lookup structure finding rows by value without reading the whole table) each run is a quick lookup — fine for a page of 20 rows. Over a million customers it's a million lookups, while the join version reads `orders` once with a single hash join (build a hash table from one input, probe it with the other).

	Correlated subquery	Join + GROUP BY (or derived table)
Readability	Very clear for "per row, look up X"	Clear once you know joins
Execution (PostgreSQL)	`EXISTS` / `IN` forms become semi/anti joins; scalar forms in `SELECT` / `WHERE` run per outer row	Set-based (whole tables at once): one pass, hash/merge join
Multiple values from the same lookup	One subquery **per value** (repeated work)	One join gives all columns
Risk	Slow on large outer tables without an index	Fan-out if you join several children (Chapter 5 §5.9)

Rule: correlated `EXISTS` / `NOT EXISTS` are excellent and idiomatic; for scalar lookups over **many** rows prefer a join, derived table or `LATERAL` (§6.7).

Correlated subqueries in UPDATE and DELETE

The standard way to update a table from related data:

```
ALTER TABLE projects ADD COLUMN total_hours int;
UPDATE projects p
SET total_hours = (SELECT coalesce(sum(hours), 0) FROM employee_projects ep WHERE ep.project_id = p.project_id);
SELECT project_id, name, total_hours FROM projects ORDER BY project_id;
```

Output:

```
 project_id |       name       | total_hours
------------+------------------+-------------
          1 | Checkout Revamp  |         870
          2 | Mobile App       |         670
          3 | Spring Campaign  |         370
          4 | Year-End Audit   |         240
          5 | Helpdesk Chatbot |          75
(5 rows)
```

(PostgreSQL also offers `UPDATE … FROM` for join-style updates — Chapter 9.)

Common mistakes

- Forgetting to correlate: `WHERE x.dept_id = dept_id` resolves to `x.dept_id = x.dept_id`, always true for non-NULL values.
- A correlated scalar subquery that can return two rows (run-time error).
- Per-row subqueries in large reports without an index on the correlated column.

In the interview

30-second answer. "A correlated subquery references the outer row, so logically it's evaluated once per outer row — like 'salary above *their department's* average'. The optimizer turns `EXISTS` / `IN` forms into semi-joins, but scalar ones often really run per row, so for large sets I'd rewrite with a join or window function."

- *Q: Rewrite 'above department average' without a correlated subquery.* — Join to a derived table of department averages, or use `avg(salary) OVER (PARTITION BY dept_id)` in a subquery and filter outside.
- *Q: Can a correlated subquery appear in `SELECT`?* — Yes, as a scalar subquery; it must return at most one row.

Self-check

1. What does `SubPlan` in an `EXPLAIN` tell you? → A subquery evaluated (potentially) once per outer row.

6.4 EXISTS and NOT EXISTS

What it is

`EXISTS (subquery)` is TRUE if the subquery returns **at least one row**, else FALSE — never UNKNOWN, and the subquery's `SELECT` list is irrelevant. `NOT EXISTS` is the opposite. Both are almost always correlated.

```
SELECT d.name FROM departments d
WHERE EXISTS (SELECT 1 FROM projects p WHERE p.dept_id = d.dept_id AND p.end_date IS NULL)
ORDER BY d.dept_id;
```

Output:

```
    name
------------
 Engineering
 Support
(2 rows)
```

Departments with a project still running; the search stops at the first match per department. `SELECT 1`, `SELECT *` or even `SELECT NULL` inside `EXISTS` behave identically (`EXISTS (SELECT NULL FROM employees e WHERE e.dept_id = d.dept_id)` still counts 5 departments); `SELECT 1` signals "no columns needed".

NOT EXISTS — the anti-join

"Employees who are on no project":

```
SELECT e.first_name, e.job_title FROM employees e
WHERE NOT EXISTS (SELECT 1 FROM employee_projects ep WHERE ep.emp_id = e.emp_id)
ORDER BY e.emp_id;
```

Output:

```
 first_name |   job_title
------------+--------------
 Asha       | CEO
 Daniel     | Sales Manager
 Arjun      | Sales Rep
 Sara       | Sales Rep
(4 rows)
```

"For all" questions: double NOT EXISTS

No "for all" operator exists, but "X for **all** Y" = "**no** Y for which X is missing". Customers who bought **every** Accessories product:

```
SELECT c.customer_id, c.first_name FROM customers c
WHERE NOT EXISTS (
    SELECT 1 FROM products p
    WHERE p.category = 'Accessories'
      AND NOT EXISTS (SELECT 1 FROM orders o JOIN order_items oi ON oi.order_id = o.order_id
                      WHERE o.customer_id = c.customer_id AND oi.product_id = p.product_id));
```

Output:

```
 customer_id | first_name
-------------+------------
           3 | Chen
(1 row)
```

"Customers for whom no Accessories product is unbought" — **relational division**. Often simpler: count each customer's distinct Accessories purchases and keep those matching the total (`HAVING count(DISTINCT product_id) = (SELECT count(*) FROM products WHERE category = 'Accessories')`).

IN vs EXISTS vs NOT IN vs NOT EXISTS with a NULL

A block list containing a `NULL`:

```
CREATE TABLE blocked (customer_id int);
INSERT INTO blocked VALUES (4), (NULL);
SELECT (SELECT count(*) FROM customers WHERE customer_id IN (SELECT customer_id FROM blocked))           AS
in_rows,
       (SELECT count(*) FROM customers c WHERE EXISTS (SELECT 1 FROM blocked b WHERE b.customer_id =
c.customer_id)) AS exists_rows,
       (SELECT count(*) FROM customers WHERE customer_id NOT IN (SELECT customer_id FROM blocked))       AS
not_in_rows,
       (SELECT count(*) FROM customers c WHERE NOT EXISTS (SELECT 1 FROM blocked b WHERE b.customer_id =
c.customer_id)) AS not_exists_rows;
```

Output:

```
 in_rows | exists_rows | not_in_rows | not_exists_rows
---------+-------------+-------------+-----------------
       1 |           1 |           0 |               9
(1 row)
```

`IN` and `EXISTS` agree; `NOT IN` returns 0 because of the `NULL`; `NOT EXISTS` correctly returns the 9 others.

	IN (subquery)	EXISTS (subquery)
Question asked	"is this value in the list?"	"is there any matching row?"
Correlated?	usually not	usually yes
Multiple columns	`(a, b) IN (SELECT x, y …)`	any condition in the subquery's `WHERE`
Negation with `NULL`s	`NOT IN` breaks	`NOT EXISTS` is safe
Performance in PostgreSQL	both become semi joins	both become semi joins

"`EXISTS` is always faster than `IN`" is advice from older optimizers; modern ones (PostgreSQL included) turn both into the same semi-join. Choose by clarity — but always `NOT EXISTS` over `NOT IN`.

Self-check

1. `EXISTS` over a single all-`NULL` row? → TRUE — a row exists.
2. `WHERE dept_id IN (SELECT dept_id FROM departments WHERE location = 'Pune')` with `EXISTS`? → `WHERE EXISTS (SELECT 1 FROM departments d WHERE d.dept_id = e.dept_id AND d.location = 'Pune')`.

6.5 ANY, SOME and ALL

What it is

`ANY` and `ALL` apply any comparison (`=`, `<>`, `<`, `>`, `<=`, `>=`) against a list: `x > ANY (subquery)` — greater than **at least one** value (the minimum; `SOME` is an exact synonym); `x > ALL (subquery)` — greater than **every** value (the maximum).

```
SELECT first_name, salary FROM employees
WHERE salary > ANY (SELECT salary FROM employees WHERE dept_id = 2)
  AND dept_id <> 2
ORDER BY salary;

SELECT first_name, salary FROM employees
WHERE salary > ALL (SELECT salary FROM employees WHERE dept_id = 2)
ORDER BY salary;
```

Output:

```
 first_name |  salary
------------+-----------
 Omar       |  76000.00
 Rahul      |  98000.00
 ...
 Vikram     | 185000.00
(7 rows)

 first_name |  salary
------------+-----------
 Vikram     | 185000.00
 Asha       | 250000.00
(2 rows)
```

Sales salaries run 68,000–150,000. `> ANY` = "above 68,000": Ananya (61,000) and Li (64,000) fail; Asha and Rohan drop out because `dept_id <> 2` is UNKNOWN for their `NULL` department. `> ALL` = "above 150,000" (Vikram, Asha).

Expression	Same as
`x = ANY (list)`	`x IN (list)`
`x <> ALL (list)`	`x NOT IN (list)` — with the same `NULL` trap
`x > ANY (list)`	`x > (SELECT min(…))` (if the list is non-empty and has no `NULL` s)
`x > ALL (list)`	`x > (SELECT max(…))` (same caveats)
`x <> ANY (list)`	"differs from at least one value" — rarely what people mean

```
SELECT count(*) AS eq_any FROM products WHERE category = ANY (SELECT DISTINCT category FROM products WHERE
unit_price > 300);
SELECT count(*) AS ne_all_with_null FROM products
WHERE unit_price <> ALL (SELECT unit_price FROM order_items UNION ALL SELECT NULL);
```

Output:

```
 eq_any
--------
      6
(1 row)

 ne_all_with_null
------------------
                0
(1 row)
```

Empty lists

Over an **empty** subquery `ALL` is TRUE (vacuously — no value fails) and `ANY` is FALSE (none passes), unlike the `min` / `max` rewrite where `max` of nothing is `NULL`:

```
SELECT (5 > ALL (SELECT salary FROM employees WHERE dept_id = 6)) AS all_empty,
       (5 > ANY (SELECT salary FROM employees WHERE dept_id = 6)) AS any_empty;
```

```
 all_empty | any_empty
-----------+-----------
 t         | f
(1 row)
```

PostgreSQL also accepts an **array** (an ordered list held in one value) on the right — how application code passes a list as one parameter (a value sent separately from the SQL text into a placeholder):

```
SELECT name FROM products WHERE product_id = ANY (ARRAY[1, 3, 5]) ORDER BY product_id;
```

```
     name
--------------
 Laptop Pro 14
 USB-C Hub
 Standing Desk
(3 rows)
```

Database	ANY / SOME / ALL with subquery	= ANY(array)
PostgreSQL	yes	yes
MySQL, SQL Server, Oracle	yes	no arrays
SQLite	**no** — use `IN`, `EXISTS`, `min` / `max`	no

Self-check

1. `salary > ALL (SELECT salary FROM employees WHERE dept_id = 99)` with no department 99? → Every row passes (ALL over empty is TRUE).

6.6 Table subqueries: derived tables and row subqueries

What it is

A subquery in `FROM` is a **derived table** (PostgreSQL docs: *subquery in FROM*; also *inline view*) — a temporary, unnamed table for this query only. It needs an alias (optional from PostgreSQL 16; our server is 14):

```sql
SELECT * FROM (SELECT 1 AS x);
```

```
ERROR:  subquery in FROM must have an alias
LINE 1: SELECT * FROM (SELECT 1 AS x);
                      ^
HINT:  For example, FROM (SELECT ...) [AS] foo.
```

Why it matters

Derived tables **build a query in stages** — compute, then filter, join or aggregate — solving "can't use an alias in `WHERE`" and "can't aggregate an aggregate" (Chapter 4).

```sql
SELECT dept_id, headcount
FROM (SELECT dept_id, count(*) AS headcount FROM employees GROUP BY dept_id) AS t
WHERE headcount >= 2
ORDER BY dept_id;
```

Output:

```
 dept_id | headcount
---------+-----------
       1 |         4
       2 |         4
       3 |         2
       4 |         2
  (null) |         2
(5 rows)
```

(`HAVING` could do this filter; the derived table is needed when the next step is beyond `HAVING`.) An **aggregate of an aggregate** — average department payroll — needs two levels, since `avg(sum(salary))` is not allowed:

```
SELECT round(avg(payroll)) AS avg_department_payroll
FROM (SELECT dept_id, sum(salary) AS payroll
      FROM employees WHERE dept_id IS NOT NULL GROUP BY dept_id) AS per_dept;
```

Output:

```
 avg_department_payroll
------------------------
                 267400
(1 row)
```

Derived tables can be **joined** — the standard cure for fan-out (Chapter 5 §5.9). Each order's total next to the customer's average:

```
SELECT o.order_id, o.customer_id, round(t.order_total, 2) AS order_total, round(ca.avg_total, 2) AS customer_avg
FROM orders o
JOIN (SELECT order_id, sum(quantity * unit_price * (1 - discount)) AS order_total
      FROM order_items GROUP BY order_id) t ON t.order_id = o.order_id
JOIN (SELECT o2.customer_id, avg(t2.total) AS avg_total
      FROM orders o2
      JOIN (SELECT order_id, sum(quantity * unit_price * (1 - discount)) AS total
            FROM order_items GROUP BY order_id) t2 ON t2.order_id = o2.order_id
      GROUP BY o2.customer_id) ca ON ca.customer_id = o.customer_id
WHERE o.customer_id IN (1, 5)
ORDER BY o.order_id;
```

Output:

```
 order_id | customer_id | order_total | customer_avg
----------+-------------+-------------+--------------
      101 |           1 |     1323.99 |      1151.36
      104 |           1 |      811.49 |      1151.36
      106 |           5 |      174.97 |       212.45
      112 |           5 |      249.93 |       212.45
      113 |           1 |     1318.60 |      1151.36
(5 rows)
```

The order-total subquery is written **twice** — the repetition a CTE (`WITH order_totals AS (…)`) removes (Chapter 7). Since PostgreSQL 12 a simple CTE referenced once is inlined (pasted into the main query), so it runs like the derived table; CTEs win on readability and reuse.

Row subqueries

A **row subquery** returns one row of several columns, compared with a **row constructor** `(a, b)` — same department *and* job title as employee 4:

```
SELECT first_name, job_title, dept_id FROM employees
WHERE (dept_id, job_title) = (SELECT dept_id, job_title FROM employees WHERE emp_id = 4)
ORDER BY emp_id;
```

Output:

```
 first_name | job_title | dept_id
------------+-----------+---------
 Rahul      | Engineer  |       1
 Meera      | Engineer  |       1
(2 rows)
```

With `IN`, a multi-column subquery can return many rows — each customer's latest order:

```
SELECT customer_id, order_id, placed_at::date
FROM orders
WHERE (customer_id, placed_at) IN (SELECT customer_id, max(placed_at) FROM orders GROUP BY customer_id)
ORDER BY customer_id;
```

Output:

```
 customer_id | order_id | placed_at
-------------+----------+------------
           1 |      113 | 2025-06-02
           2 |      108 | 2025-04-01
 ...
           8 |      109 | 2025-04-09
(8 rows)
```

(Row constructors with `IN` work in PostgreSQL, MySQL and Oracle, but **not** in SQL Server — use `EXISTS` there.)

Self-check

1. Why not `SELECT avg(sum(salary)) FROM employees GROUP BY dept_id`? → Aggregates can't nest; sum in a derived table, average outside.

6.7 LATERAL subqueries

What it is

A normal derived table can't refer to other `FROM` items. A `LATERAL` subquery **can** refer to tables listed *before* it and is re-evaluated per row of them — a correlated subquery in `FROM` that may return **many rows and columns**. Without `LATERAL`:

```
SELECT d.name, t.n FROM departments d,
     (SELECT count(*) AS n FROM employees e WHERE e.dept_id = d.dept_id) t;
```

```
ERROR:  invalid reference to FROM-clause entry for table "d"
LINE 2: ... count(*) AS n FROM employees e WHERE e.dept_id = d.dept_id)...
                                                             ^
HINT:  There is an entry for table "d", but it cannot be referenced from this part of the query.
```

The problem it solves: top-N per group

"Two highest-paid per department" is hard with plain subqueries (no per-group `LIMIT`). With `LATERAL`, per department sort its employees and take 2:

```
SELECT d.name, t.first_name, t.salary
FROM departments d
CROSS JOIN LATERAL (
    SELECT e.first_name, e.salary FROM employees e
    WHERE e.dept_id = d.dept_id
    ORDER BY e.salary DESC, e.emp_id
    LIMIT 2) AS t
ORDER BY d.dept_id, t.salary DESC;
```

Output:

```
    name     | first_name |  salary
-------------+------------+-----------
 Engineering | Vikram     | 185000.00
 Engineering | Neha       | 142000.00
 Sales       | Daniel     | 150000.00
 ...
 Support     | Li         |  64000.00
(9 rows)
```

`CROSS JOIN LATERAL` drops outer rows with an empty subquery (Research). Keep them with `LEFT JOIN LATERAL … ON true` (`ON true` because the correlation is inside):

```sql
SELECT d.name, t.first_name, t.salary
FROM departments d
LEFT JOIN LATERAL (
    SELECT e.first_name, e.salary FROM employees e
    WHERE e.dept_id = d.dept_id
    ORDER BY e.salary DESC LIMIT 1) AS t ON true
ORDER BY d.dept_id;
```

Output:

```
    name     | first_name |  salary
-------------+------------+-----------
 Engineering | Vikram     | 185000.00
 ...
 Support     | Li         |  64000.00
 Research    | (null)     |    (null)
(6 rows)
```

Each customer's latest order with several columns (a scalar subquery gives one column at a time):

```sql
SELECT c.first_name, last.order_id, last.status, last.placed_at::date
FROM customers c
LEFT JOIN LATERAL (
    SELECT o.order_id, o.status, o.placed_at FROM orders o
    WHERE o.customer_id = c.customer_id
    ORDER BY o.placed_at DESC LIMIT 1) AS last ON true
ORDER BY c.customer_id;
```

Output:

```
 first_name | order_id |  status   | placed_at
------------+----------+-----------+------------
 Anita      |      113 | pending   | 2025-06-02
 Ben        |      108 | delivered | 2025-04-01
 ...
 Hana       |   (null) | (null)    | (null)
 Ivan       |   (null) | (null)    | (null)
(10 rows)
```

With an index on `orders (customer_id, placed_at)` each lateral lookup reads one index entry — often fastest for "latest row per group" with few outer rows. Window functions (`ROW_NUMBER() OVER (PARTITION BY …)` , Chapter 8) are usually better for *every* group of a large table.

Database	Syntax
PostgreSQL 9.3+	`CROSS JOIN LATERAL (…)` , `LEFT JOIN LATERAL (…) ON true`
MySQL 8.0.14+	`LATERAL`
SQL Server, Oracle 12c+	`CROSS APPLY` / `OUTER APPLY` (Oracle also has `LATERAL`)
SQLite	not supported

6.8 Set operations and UNION

What it is

Set operations combine the **results of two queries** by rows, like mathematical set operations (a *set* is a collection of distinct items; a *Venn picture* draws two sets as overlapping circles):

Operator	Result	Venn picture
`UNION`	rows in the first **or** the second result (duplicates removed)	both circles
`UNION ALL`	all rows of both results, duplicates kept	both circles, overlap counted twice
`INTERSECT`	rows in **both** results	only the overlap
`EXCEPT`	rows in the first result that are **not** in the second	left circle minus overlap

Joins combine tables **side by side** (more columns); set operations stack results **on top of each other** (same columns, more or fewer rows).

The rules

1. Both queries must return the **same number of columns**.
2. Corresponding columns must have **compatible types** (PostgreSQL resolves them to a common type, like `CASE` does).
3. Column **names** of the result come from the **first** query.
4. Rows are compared **as whole rows**, and — unlike `=` — two `NULL`s are treated as **equal** (not distinct).
5. `ORDER BY` / `LIMIT` at the end apply to the **combined** result (§6.11).

Breaking rule 1 or 2:

```
SELECT first_name, last_name FROM employees UNION SELECT first_name FROM customers;
SELECT emp_id FROM employees UNION SELECT first_name FROM customers;
```

```
ERROR:  each UNION query must have the same number of columns
LINE 1: ...first_name, last_name FROM employees UNION SELECT first_name...
                                                             ^

ERROR:  UNION types integer and text cannot be matched
LINE 1: SELECT emp_id FROM employees UNION SELECT first_name FROM cu...
                                                 ^
```

UNION

One contact list from two tables, a literal column (`'employee'`) marking the source:

```
SELECT first_name, last_name, 'employee' AS kind FROM employees WHERE dept_id = 3
UNION
SELECT first_name, last_name, 'customer' FROM customers WHERE country = 'UK'
ORDER BY kind, last_name;
```

Output:

```
 first_name | last_name |   kind
------------+-----------+----------
 Ben        | Carter    | customer
 George     | Hill      | customer
 Ananya     | Das       | employee
 Lucas      | Martin    | employee
(4 rows)
```

Every city with customers or offices — `UNION` removes duplicates:

```
SELECT city FROM customers WHERE city IS NOT NULL
UNION
SELECT location FROM departments WHERE location IS NOT NULL
ORDER BY city;
```

Output:

```
   city
-----------
 Bengaluru
 Delhi
 ...
 Pune
 mumbai
(7 rows)
```

`mumbai` and `Mumbai` differ, so both survive — clean first (`initcap(city)` , Chapter 3).

UNION vs UNION ALL

```
SELECT count(*) AS union_rows     FROM (SELECT country FROM customers UNION     SELECT 'India') t;
SELECT count(*) AS union_all_rows FROM (SELECT country FROM customers UNION ALL SELECT 'India') t;
```

Output:

```
 union_rows
------------
          3
(1 row)

 union_all_rows
----------------
             11
(1 row)
```

`UNION` : 3 (duplicates inside **each** input are removed too); `UNION ALL` : 10 + 1 = 11.

	UNION	UNION ALL
Duplicates	removed (whole-row comparison)	kept
Extra work	a sort or hash over the whole result	none — rows are just appended
Use when	you need distinct rows	the inputs can't overlap, or duplicates are meaningful (counting, event logs)

Default to `UNION ALL` ; `UNION` on large results costs a sort or hash and hides data problems. Typical use — two event kinds in one timeline:

```
SELECT placed_at AS at, 'order placed'  AS event, order_id FROM orders   WHERE customer_id = 1
UNION ALL
SELECT shipped_at,      'order shipped',          order_id FROM orders   WHERE customer_id = 1 AND shipped_at IS
NOT NULL
ORDER BY at;
```

Output:

```
         at          |     event     | order_id
---------------------+---------------+----------
 2025-01-03 10:15:00 | order placed  |      101
 2025-01-04 09:00:00 | order shipped |      101
 2025-02-11 09:05:00 | order placed  |      104
 2025-02-12 16:45:00 | order shipped |      104
 2025-06-02 10:30:00 | order placed  |      113
(5 rows)
```

And rule 4 — set operations treat `NULL` s as duplicates of each other:

```
SELECT NULL::text AS v UNION SELECT NULL::text;
```

```
   v
--------
 (null)
(1 row)
```

Self-check

1. `SELECT 1 UNION SELECT 1 UNION ALL SELECT 1`? → 2 rows (left to right: `(1 UNION 1)` = 1 row, then `UNION ALL 1`).

6.9 INTERSECT

What it is

`INTERSECT` returns rows in **both** results (the overlap), without duplicates. Cities with both customers and an office:

```
SELECT city FROM customers
INTERSECT
SELECT location FROM departments
ORDER BY city;
```

Output:

```
   city
-----------
 Bengaluru
 Mumbai
 (null)
(3 rows)
```

The `(null)` row is rule 4: a city-less customer and a location-less department match because set operations treat `NULL`s as equal (an inner join on `c.city = d.location` wouldn't). Add `WHERE city IS NOT NULL` if unwanted.

Employees on a project **and** managing someone:

```
SELECT emp_id FROM employee_projects
INTERSECT
SELECT manager_id FROM employees
ORDER BY emp_id;
```

Output:

```
 emp_id
--------
      2
     10
     12
(3 rows)
```

`INTERSECT` = `EXISTS` semi-join on all columns (`IS NOT DISTINCT FROM` for `NULL`s) plus `DISTINCT`. Use it to compare whole rows of similar results; `EXISTS` / `JOIN` when you need other columns.

6.10 EXCEPT

What it is

`EXCEPT` returns rows of the first result **not** in the second (the left crescent), without duplicates. Oracle calls it `MINUS` (21c also accepts `EXCEPT`). Products never ordered:

```sql
SELECT product_id FROM products
EXCEPT
SELECT product_id FROM order_items;
```

Output:

```
 product_id
------------
          8
(1 row)
```

Unlike `UNION` and `INTERSECT`, **order matters**: `A EXCEPT B` ≠ `B EXCEPT A`.

```sql
SELECT city FROM customers EXCEPT SELECT location FROM departments ORDER BY 1;
SELECT location FROM departments EXCEPT SELECT city FROM customers ORDER BY 1;
```

Output:

```
   city
----------
 Delhi
 London
 New York
 mumbai
(4 rows)

 location
----------
 Pune
(1 row)
```

Customer cities with no office; office cities with no customers (Pune).

Because `NULL`s compare equal, `EXCEPT` **diffs queries containing** `NULL`**s** — e.g. a refactored query (rewritten, meant to behave identically) vs the old: `(old EXCEPT new)` and `(new EXCEPT old)` must both be empty. Meera's row is removed despite her `NULL` email:

```sql
SELECT first_name, email FROM employees WHERE dept_id = 1
EXCEPT
SELECT first_name, email FROM employees WHERE emp_id IN (2, 5)
ORDER BY 1;
```

Output:

```
 first_name |            email
------------+-----------------------------
 Neha       | Neha.Gupta@Company.example
 Rahul      | rahul.verma@company.example
(2 rows)
```

Without INTERSECT and EXCEPT

MySQL only gained `INTERSECT` and `EXCEPT` in 8.0.31 (2022), so you often see semi-/anti-join equivalents (Chapter 5 §5.8):

```sql
-- INTERSECT-like: customer cities that also have an office
SELECT DISTINCT c.city FROM customers c
WHERE EXISTS (SELECT 1 FROM departments d WHERE d.location IS NOT DISTINCT FROM c.city)
ORDER BY c.city;

-- EXCEPT-like: customers whose city has no office (whole rows)
SELECT c.customer_id, c.first_name, c.city FROM customers c
WHERE NOT EXISTS (SELECT 1 FROM departments d WHERE d.location = c.city)
ORDER BY c.customer_id;
```

Output:

```
   city
-----------
 Bengaluru
 Mumbai
 (null)
(3 rows)

 customer_id | first_name |   city
-------------+------------+----------
           2 | Ben        | London
           4 | Divya      | Delhi
           5 | Ethan      | New York
           6 | Fatima     | (null)
 ...
          10 | Ivan       | Delhi
(8 rows)
```

(A `LEFT JOIN (SELECT DISTINCT location FROM departments) d ON d.location = c.city WHERE d.location IS NULL` returns the same 8 rows.) Differences from the real operators: add `DISTINCT` yourself for set semantics, and `=` does **not** treat `NULL`s as equal — hence `IS NOT DISTINCT FROM` to reproduce `INTERSECT`'s `(null)` row, and Fatima (no city) appearing although `EXCEPT` would remove her. In return, `EXISTS` forms can return any outer columns.

The ALL variants: bag semantics

SQL tables are **bags** (multisets): a row can appear several times. `ALL` forms track **how many times**:

Operator	Row appearing *m* times in A and *n* times in B appears …
`UNION ALL`	*m* + *n* times
`UNION`	once
`INTERSECT ALL`	*min(m, n)* times
`INTERSECT`	once, if *m* > 0 and *n* > 0
`EXCEPT ALL`	*max(m − n, 0)* times
`EXCEPT`	once, if *m* > 0 and *n* = 0

```sql
CREATE TABLE a (x int);
CREATE TABLE b (x int);
INSERT INTO a VALUES (1), (1), (1), (2), (3);
INSERT INTO b VALUES (1), (1), (2), (2), (4);
SELECT x FROM a INTERSECT     SELECT x FROM b ORDER BY x;
SELECT x FROM a INTERSECT ALL SELECT x FROM b ORDER BY x;
SELECT x FROM a EXCEPT        SELECT x FROM b ORDER BY x;
SELECT x FROM a EXCEPT ALL    SELECT x FROM b ORDER BY x;
```

Output (in order):

```
 x
---
 1
 2
(2 rows)

 x
---
 1
 1
 2
(3 rows)

 x
---
 3
(1 row)

 x
---
 1
 3
(2 rows)
```

Value 1 (3× in `a`, 2× in `b`) → `INTERSECT ALL` min(3, 2) = 2 copies, `EXCEPT ALL` 3 − 2 = 1. Value 2 (1×, 2×) → 1 copy, 0 copies. Value 3 (only in `a`) → only in the `EXCEPT` results. `UNION` gives `1, 2, 3, 4`; `UNION ALL` all 10 values.

Dialect support

Operator	PostgreSQL	MySQL	SQLite	SQL Server	Oracle
`UNION`, `UNION ALL`	yes	yes	yes	yes	yes
`INTERSECT`, `EXCEPT`	yes	8.0.31+	yes	yes	`INTERSECT`, `MINUS` (+ `EXCEPT` in 21c)
`INTERSECT ALL`, `EXCEPT ALL`	yes	8.0.31+	no	no	21c+

In the interview

30-second answer. "`UNION` combines two results and removes duplicates; `UNION ALL` keeps them and is cheaper. `INTERSECT` keeps rows in both; `EXCEPT` keeps rows in the first but not the second. Both inputs need the same number of columns with compatible types, and set operators treat `NULL`s as equal."

- *Q: `UNION` vs `UNION ALL`?* — De-duplication vs none; use `UNION ALL` unless you need distinct rows.
- *Q: Find IDs present in table A but not in B.* — `SELECT id FROM a EXCEPT SELECT id FROM b`, or `NOT EXISTS`. `EXCEPT` compares only the selected columns and returns distinct rows; `NOT EXISTS` returns full rows of `a` (with duplicates if `a` has them).
- *Q: How do you check two queries return identical results?* — `(q1 EXCEPT ALL q2) UNION ALL (q2 EXCEPT ALL q1)` must be empty.

Self-check

1. `A = {1,1,2}`, `B = {1}`: `A EXCEPT B`, `A EXCEPT ALL B`? → `{2}`, `{1, 2}`.
2. Which set operator is not commutative (swapping sides changes the answer)? → `EXCEPT`.

6.11 Precedence, sorting, and choosing the right tool

Precedence

`INTERSECT` binds **tighter** than `UNION` and `EXCEPT` (like × before +); `UNION` / `EXCEPT` go left to right. Parentheses override:

```
SELECT 1 AS v UNION SELECT 2 INTERSECT SELECT 3;          -- = 1 UNION (2 INTERSECT 3)
(SELECT 1 AS v UNION SELECT 2) INTERSECT SELECT 3;        -- parentheses change it
```

Output:

```
 v
---
 1
(1 row)

 v
---
(0 rows)
```

When mixing operators, **always parenthesise**.

ORDER BY and LIMIT

A single `ORDER BY` / `LIMIT` at the end applies to the **whole combined result**:

```
SELECT first_name, salary FROM employees WHERE dept_id = 1
UNION ALL
SELECT first_name, salary FROM employees WHERE dept_id = 2
ORDER BY salary DESC
LIMIT 3;
```

Output:

```
 first_name |  salary
------------+----------
 Vikram     | 185000.00
 Daniel     | 150000.00
 Neha       | 142000.00
(3 rows)
```

To sort or limit **each branch**, wrap the branches in parentheses:

```
(SELECT first_name, salary FROM employees WHERE dept_id = 1 ORDER BY salary DESC LIMIT 1)
UNION ALL
(SELECT first_name, salary FROM employees WHERE dept_id = 2 ORDER BY salary DESC LIMIT 1);
```

Output:

```
 first_name |  salary
------------+----------
 Vikram     | 185000.00
 Daniel     | 150000.00
(2 rows)
```

The final `ORDER BY` accepts only **output column names or positions**, not expressions — move the set operation into a derived table:

```
SELECT first_name FROM employees UNION SELECT first_name FROM customers ORDER BY upper(first_name);
```

```
ERROR:  invalid UNION/INTERSECT/EXCEPT ORDER BY clause
LINE 1: ...s UNION SELECT first_name FROM customers ORDER BY upper(firs...
                                                             ^
DETAIL:  Only result column names can be used, not expressions or functions.
HINT:  Add the expression/function to every SELECT, or move the UNION into a FROM clause.
```

```
SELECT * FROM (SELECT first_name FROM employees UNION SELECT first_name FROM customers) t
ORDER BY upper(first_name) LIMIT 3;
```

```
 first_name
------------
 Ananya
 Anita
 Arjun
(3 rows)
```

(Without any `ORDER BY`, the row order of a set operation — like any query — is not guaranteed.)

EXCEPT vs NOT EXISTS

`SELECT customer_id FROM customers EXCEPT SELECT customer_id FROM orders` and the `NOT EXISTS` anti-join both find customers 9 and 10, but `EXCEPT` returns only the compared columns, while `NOT EXISTS` returns any columns of the outer table (Hana, New York; Ivan, Delhi).

Choosing between joins, subqueries and set operations

You want…	Best tool
Columns from several related tables in one row	`JOIN` (Chapter 5)
Rows of A that have / don't have a match in B	`EXISTS` / `NOT EXISTS` (semi / anti-join)
Compare with one computed value (average, max)	scalar subquery
Compare with a list	`IN` (never `NOT IN` on nullable columns)
Build in stages; aggregate an aggregate; pre-aggregate before a join	derived table or CTE (Chapter 7)
Top-N / latest row per group	`LATERAL` or a window function (Chapter 8)
Stack similar results from different sources	`UNION ALL` (`UNION` if de-duplication is needed)
Rows common to / missing from two similar results, whole-row comparison, `NULL`-safe	`INTERSECT` / `EXCEPT`

In the interview

30-second answer (whole chapter). "A subquery is a query nested in another; it can return a scalar, a row, a column or a table. Non-correlated subqueries run once; correlated ones depend on the outer row. I use `EXISTS` / `NOT EXISTS` for semi- and anti-joins and avoid `NOT IN` because of `NULL`s. Derived tables and `LATERAL` let me build queries in stages. Set operators combine whole results: `UNION ALL` by default, `UNION` to de-duplicate, `INTERSECT` for common rows, `EXCEPT` for differences — same column count, compatible types, `NULL`s treated as equal."

- *Q: When is a subquery better than a join?* — When you only test existence (`EXISTS`), compare with an aggregate, or need to aggregate before joining. A join is better when you need columns from both sides.
- *Q: Can you `ORDER BY` inside a `UNION` branch?* — Only inside parentheses and normally with `LIMIT`; otherwise the final `ORDER BY` sorts the whole result.

Self-check

1. `SELECT 1 UNION SELECT 2 INTERSECT SELECT 2`? → `{1, 2}` — `INTERSECT` first gives `{2}`, then `UNION` with `{1}`.
2. Sort a `UNION` by `lower(name)`? → Wrap it in a derived table, or add `lower(name)` as an output column of every branch.

Chapter 6 TL;DR

Tool	Returns / does	Watch out for
Scalar subquery	one value	0 rows → `NULL`; 2+ rows → run-time error
`IN (subquery)`	membership (semi-join)	—
`NOT IN (subquery)`	non-membership	any `NULL` in the list → no rows
Correlated subquery	per-outer-row evaluation	slow for big scalar lookups; qualify columns
`EXISTS` / `NOT EXISTS`	"is there a row?"	the safe semi/anti-join
`ANY` / `ALL`	compare with at least one / every value	`ALL` over empty = TRUE; `<> ALL` = `NOT IN`
Derived table	subquery in `FROM`	needs an alias (before PostgreSQL 16)
`LATERAL`	per-row subquery in `FROM`	`LEFT JOIN LATERAL … ON true` to keep all rows
`UNION` / `UNION ALL`	stack results	`UNION` costs a de-duplication
`INTERSECT` / `EXCEPT`	common / missing rows	`EXCEPT` isn't commutative; `NULL`s compare equal

Next: Chapter 7 — CTEs, Views and Recursion gives names to subqueries so that long queries read top to bottom.

Chapter 7 — CTEs, Views and Recursion: Naming Queries and Walking Trees

A report needing "order totals, then customer totals, then customers above average" becomes three unreadable levels of nested subqueries. This chapter covers SQL's tools for **naming** a piece of a query so big queries are built from small, readable steps:

- a **common table expression (CTE)** names a query for the length of *one* statement;
- a **recursive CTE** calls itself, which lets plain SQL walk hierarchies (org charts, category trees, bills of materials — parts lists where each part can itself be made of parts) and graphs (networks of connected items: flight routes, friend networks) and produce sequences;
- a **view** names a query *permanently* in the database, so everyone can reuse it like a table;
- a **materialized view** also stores the query's *result*, trading freshness for speed.

Plus `generate_series`, PostgreSQL's way to produce rows of numbers or dates for "fill in the missing days" reports.

Sections: 7.1 CTE basics · 7.2 Data-modifying CTEs · 7.3 Recursive CTE mechanics · 7.4 Hierarchies · 7.5 Graphs and cycles · 7.6 `generate_series` · 7.7 Views · 7.8 Materialized views · 7.9 Choosing.

Setup. The `company` database (chapter 1, 1.11) plus a `routes` table (a small flight network, 7.5); `code/` `00_schema.sql` builds everything. Subqueries and derived tables, which CTEs replace: chapter 6.

7.1 Common table expressions (WITH)

What it is

A **common table expression** is a named, temporary result set defined at the start of a statement with `WITH name AS (query)` and used in the rest of that statement like a table — named scrap paper ("let *x* = total per customer") that disappears when the statement ends.

```
WITH cte_name [(col1, col2, ...)] AS (
    SELECT ...                -- any query
)
[, another_cte AS (SELECT ... FROM cte_name ...)]   -- later CTEs may use earlier ones
SELECT ... FROM cte_name ...;  -- the main statement
```

The problem it solves

Nested derived tables (subqueries in `FROM`) read *inside-out* — the first step is buried deepest. CTEs write the logic *top-down* with a name per step, and let one intermediate result be used **several times** without copying it.

How it works

1. Each `name AS (query)` defines one CTE; separate several with commas, write `WITH` once.
2. A CTE can reference CTEs defined **before** it (and, with `RECURSIVE`, itself — 7.3).
3. The final `SELECT` / `INSERT` / `UPDATE` / `DELETE` uses them by name.
4. Nothing is stored in the catalog (the database's internal list of objects); the CTE lives for this one statement.

One CTE — employees paid above their department average:

```sql
-- code/01_cte_basics.sql
WITH dept_avg AS (
    SELECT dept_id, round(avg(salary), 0) AS avg_salary
    FROM employees
    WHERE dept_id IS NOT NULL
    GROUP BY dept_id
)
SELECT e.first_name, e.dept_id, e.salary, d.avg_salary
FROM employees e
JOIN dept_avg d ON d.dept_id = e.dept_id
WHERE e.salary > d.avg_salary
ORDER BY e.dept_id, e.salary DESC;
```

Output:

```
 first_name | dept_id |  salary   | avg_salary
------------+---------+-----------+------------
 Vikram     |       1 | 185000.00 |     130750
 Neha       |       1 | 142000.00 |     130750
 Daniel     |       2 | 150000.00 |      89500
 Lucas      |       3 | 120000.00 |      90500
 Kiran      |       4 | 135000.00 |     105500
(5 rows)
```

`dept_avg` behaves like a table with one row per staffed department.

Several CTEs as a pipeline — order value → customer lifetime value → overall average:

```sql
WITH order_totals AS (          -- step 1: value of each non-cancelled order
    SELECT o.order_id, o.customer_id,
           sum(oi.quantity * oi.unit_price * (1 - oi.discount)) AS order_value
    FROM orders o
    JOIN order_items oi USING (order_id)
    WHERE o.status <> 'cancelled'
    GROUP BY o.order_id, o.customer_id
),
customer_totals AS (            -- step 2: lifetime value per customer
    SELECT customer_id, count(*) AS orders, round(sum(order_value), 2) AS lifetime_value
    FROM order_totals
    GROUP BY customer_id
),
overall AS (                    -- step 3: one number to compare against
    SELECT round(avg(lifetime_value), 2) AS avg_ltv FROM customer_totals
)
SELECT c.first_name, c.last_name, ct.orders, ct.lifetime_value, o.avg_ltv
FROM customer_totals ct
JOIN customers c USING (customer_id)
CROSS JOIN overall o
WHERE ct.lifetime_value > o.avg_ltv
ORDER BY ct.lifetime_value DESC;
```

Output:

```
 first_name | last_name | orders | lifetime_value | avg_ltv
------------+-----------+--------+----------------+---------
 Anita      | Shah      |      3 |        3454.08 | 1057.66
 Ben        | Carter    |      2 |        1858.36 | 1057.66
(2 rows)
```

`overall` has one row, so `CROSS JOIN` attaches the average to every row. Nested subqueries would be three levels deep.

Using a CTE twice — `monthly` is read by the main query *and* a scalar subquery:

```sql
WITH monthly AS (
    SELECT date_trunc('month', placed_at)::date AS month, count(*) AS orders
    FROM orders
    GROUP BY 1
)
SELECT month, orders,
       orders = (SELECT max(orders) FROM monthly) AS is_best_month
FROM monthly
ORDER BY month;
```

Output:

```
   month    | orders | is_best_month
------------+--------+--------------
 2025-01-01 |      3 | t
 2025-02-01 |      2 | f
 ...
 2025-04-01 |      3 | t
 ...
(6 rows)
```

Naming the columns in the header. `WITH dept_size(dept, headcount) AS (SELECT d.name, count(e.emp_id) …)` renames the output columns — handy for unnamed expressions like `count(e.emp_id)` (`code/01_cte_basics.sql`).

How PostgreSQL executes a CTE: inlined or materialized

- **Inlined** — the optimizer (the component choosing how to run a query) pastes the CTE into the main query like a subquery and optimizes everything together (e.g. pushing a `WHERE` down so an index — a sorted lookup structure — can be used).
- **Materialized** — the CTE runs once into a temporary work area that the main query scans; outside filters can't reach inside.

Since **PostgreSQL 12**, a non-recursive, side-effect-free (read-only) CTE referenced **once** is inlined automatically; one referenced two or more times is materialized. Override with `AS MATERIALIZED` / `AS NOT MATERIALIZED`. (Before 12 every CTE was materialized — the "optimization fence" old blog posts warn about.)

```sql
EXPLAIN (COSTS OFF)
WITH big AS (SELECT * FROM employees)
SELECT first_name FROM big WHERE emp_id = 3;

EXPLAIN (COSTS OFF)
WITH big AS MATERIALIZED (SELECT * FROM employees)
SELECT first_name FROM big WHERE emp_id = 3;
```

Output:

```
                  QUERY PLAN
-----------------------------------------------
 Index Scan using employees_pkey on employees
   Index Cond: (emp_id = 3)
(2 rows)

          QUERY PLAN
------------------------------
 CTE Scan on big
   Filter: (emp_id = 3)
   CTE big
     ->  Seq Scan on employees
(4 rows)
```

The first plan never mentions the CTE — the filter reached the primary-key index; the second copies the whole table, then filters. (`EXPLAIN` prints the chosen plan; `Seq Scan` = read the whole table; chapter 12.)

`MATERIALIZED` suits an expensive CTE referenced several times, or something to compute exactly once (a slow function call). `NOT MATERIALIZED` suits a cheap multiply-referenced CTE that each use filters differently.

Dialect notes

Feature	PostgreSQL	MySQL	SQLite	SQL Server
`WITH` (non-recursive)	yes	8.0+	3.8.3+	2005+
`WITH RECURSIVE` keyword	required for recursion	required	required	*not used* — plain `WITH` can recurse
`MATERIALIZED` hint	12+	no	3.35+	no
CTE with `INSERT/UPDATE/DELETE` inside	yes (7.2)	no	no	no (but a CTE can be the *target* of an `UPDATE` / `DELETE`)

In SQL Server the statement *before* a `WITH` must end with a semicolon — hence `;WITH` in T-SQL code (not run).

Trade-offs

	Derived table (subquery in `FROM`)	CTE
Readability	inside-out	top-down, named steps
Reuse in the same statement	copy-paste	reference by name
Recursion	no	yes
Performance (PostgreSQL 12+)	same when inlined	same when inlined; can force materialization
Scope	the one place it's written	the whole statement

When not to use it

- The query is readable without it — one level of subquery is fine.
- Many statements need the same logic: a **view** (7.7).
- The intermediate result is needed across statements in a session (one connection's lifetime), perhaps indexed: a **temporary table** (visible only to your session, dropped when it ends; 7.9).

Common mistakes

- `WITH` **twice** (`WITH a AS (...) WITH b AS (...)`), a missing comma between CTEs, or an extra one after the last.
- **Referencing a CTE defined later** — a CTE sees only those above it (or itself, if recursive).
- **Assuming a CTE "caches" work** — a single-use CTE is inlined (12+); use `MATERIALIZED` for one evaluation.

Real-world examples

Analytics teams chain CTEs (`raw → cleaned → joined → aggregated → final`); dbt (a tool for building data-transformation pipelines from SQL files) encourages this style.

In the interview

30-second answer: "A CTE is a named subquery declared with `WITH` that exists for one statement. It makes complex queries readable as a sequence of steps, lets you reference the same intermediate result several times, and — with `WITH RECURSIVE` — lets a query refer to itself to walk hierarchies. In PostgreSQL 12 and later a CTE referenced once is inlined, so it costs the same as a subquery; you can force materialization with `AS MATERIALIZED`."

- *Is a CTE faster than a subquery?* → Usually identical (inlined). Slower if materialized and a selective outer filter can't be pushed in; faster if an expensive CTE is materialized once and reused.
- *CTE vs temporary table?* → A CTE lasts one statement and isn't indexed; a temp table lasts the session (or transaction), can be indexed and analyzed (statistics gathered for the optimizer), and serves many statements.
- *Can you update through a CTE?* → In PostgreSQL the CTE itself can contain `UPDATE/DELETE/INSERT ... RETURNING` (next section).

Self-check

1. *In PostgreSQL 14, is a CTE referenced once materialized?* → No, inlined — unless `AS MATERIALIZED`, recursive, or with side effects.

7.2 Data-modifying CTEs (PostgreSQL)

What it is

In PostgreSQL a CTE may contain `INSERT`, `UPDATE` or `DELETE` with `RETURNING`, whose rows feed the rest of the statement. Several changes ("delete these rows and copy them to an archive") become **one statement**, automatically atomic — all or nothing.

How it works

1. `DELETE/UPDATE/INSERT ... RETURNING` inside a CTE produces the affected rows; later CTEs or the main statement read them by name.
2. Every data-modifying CTE runs **to completion exactly once**, even if nothing reads its output.
3. All parts see the **same snapshot** (a frozen picture of the data) from *before* the statement — not each other's changes, except through `RETURNING`.

Example — archive cancelled orders and their items:

```sql
-- code/02_data_modifying_cte.sql
CREATE TABLE orders_archive (LIKE orders INCLUDING ALL);
CREATE TABLE order_items_archive (LIKE order_items INCLUDING ALL);

WITH gone_items AS (
    DELETE FROM order_items
    WHERE order_id IN (SELECT order_id FROM orders WHERE status = 'cancelled')
    RETURNING *
),
saved_items AS (
    INSERT INTO order_items_archive SELECT * FROM gone_items RETURNING order_id
),
gone_orders AS (
    DELETE FROM orders
    WHERE status = 'cancelled'
    RETURNING *
)
INSERT INTO orders_archive
SELECT * FROM gone_orders
RETURNING order_id, status;
```

Output:

```
 order_id |  status
----------+-----------
      105 | cancelled
      111 | cancelled
(2 rows)
```

```
SELECT (SELECT count(*) FROM orders)              AS live_orders,
       (SELECT count(*) FROM orders_archive)      AS archived_orders,
       (SELECT count(*) FROM order_items_archive) AS archived_items;
```

Output:

```
 live_orders | archived_orders | archived_items
-------------+-----------------+---------------
          12 |               2 |              2
(1 row)
```

`CREATE TABLE ... (LIKE t INCLUDING ALL)` copies the column definitions, defaults and constraints of `t` (see chapter 9).

The snapshot rule, shown:

```
WITH bump AS (
    UPDATE products SET unit_price = unit_price + 1 WHERE product_id = 2
    RETURNING unit_price
)
SELECT (SELECT unit_price FROM products WHERE product_id = 2) AS seen_by_select,
       (SELECT unit_price FROM bump)                          AS new_value;
```

Output:

```
 seen_by_select | new_value
----------------+-----------
          24.99 |     25.99
(1 row)
```

The outer `SELECT` reads the old price; only `RETURNING` shows the new one. Afterwards the table holds 25.99.

Common mistakes

- **Modifying the same row twice in one statement** (two CTEs) — unpredictable result.
- **Expecting a later CTE to "see" an earlier change via the table** — read the earlier CTE's `RETURNING`.
- **Porting to MySQL/SQL Server** — unsupported; use a transaction (statements committed or undone together) of separate statements (chapter 13).

7.3 Recursive CTEs: the mechanics

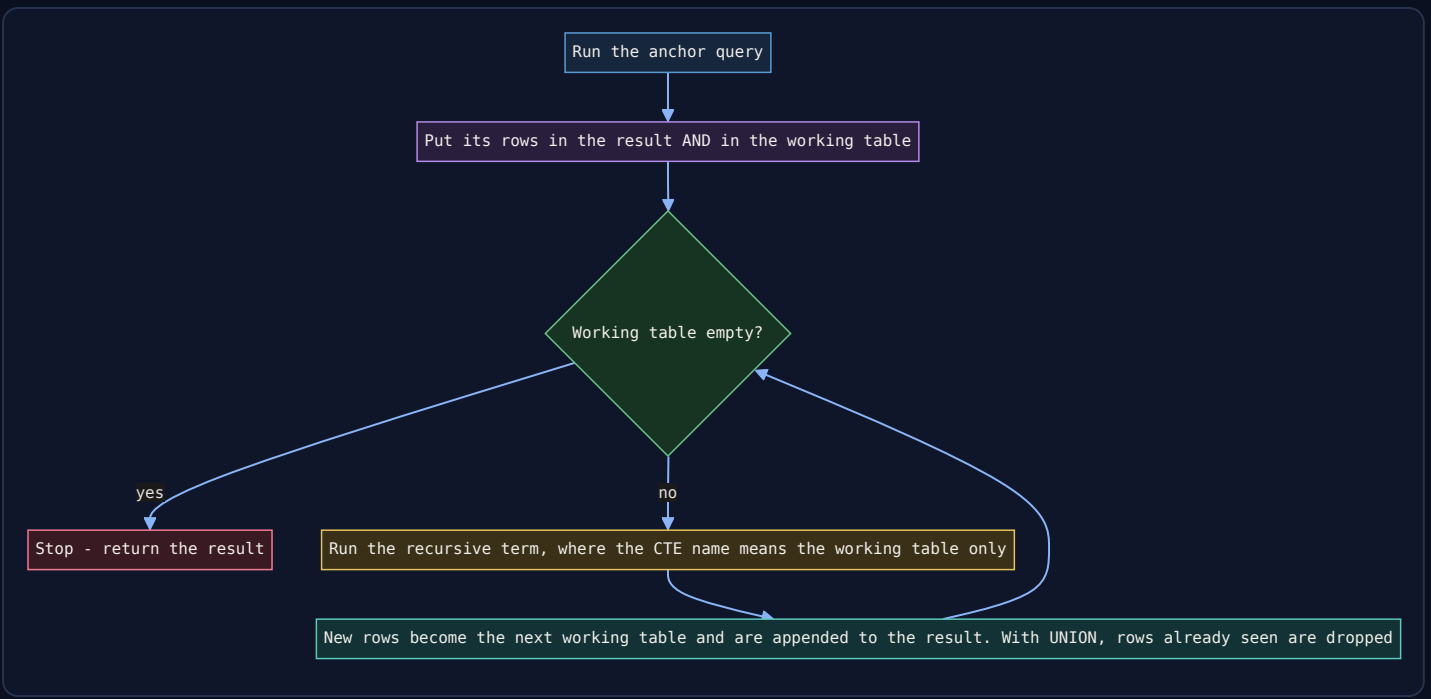
What it is

A **recursive CTE** refers to itself: `WITH RECURSIVE name AS (anchor UNION ALL recursive_term)`. It is standard SQL's *repetition*: "start with these rows, keep deriving new rows from the last batch until none appear" — like listing a grandparent's descendants generation by generation until one has no children.

The problem it solves

Questions with no fixed depth — "all reports under this manager at every level", "every category inside Electronics", "every city reachable from Mumbai". A join handles one level (chapter 5 self-join); a recursive CTE handles any depth.

How it works



1. **Anchor** (before `UNION [ALL]`) runs once, producing the starting rows.
2. **Recursive term** (after it) references the CTE name, which on each pass means *only the previous pass's rows* (the "working table"); its output is appended to the result and feeds the next pass.
3. **Termination:** stop when a pass produces no rows — your `WHERE` or join must make that happen.
4. `UNION ALL` keeps every row; `UNION` discards rows already produced, which also stops loops whose state repeats.

PostgreSQL rules: the recursive term references the CTE only **once**, not inside an aggregate, a subquery, or the nullable side of an outer join. Anchor and recursive term need the same column count and compatible types — cast in the anchor (`0::bigint`, `name::text`) if the recursive term produces a wider type.

Counting 1 to 5:

```sql
-- code/03_recursive_series.sql
WITH RECURSIVE counter(n) AS (
    SELECT 1                -- anchor: the starting row(s)
    UNION ALL
    SELECT n + 1            -- recursive term: runs on the rows produced last time
    FROM counter
    WHERE n < 5             -- stop condition
)
SELECT n FROM counter;
```

Output:

```
 n
---
 1
 2
 3
 4
 5
(5 rows)
```

Trace: anchor → {1}. Pass 1 reads {1} → {2}. Pass 2 reads {2} → {3} … pass 4 reads {4} → {5}. Pass 5 reads {5}; `5 < 5` is false → no rows → stop.

Carrying state in several columns — the next Fibonacci pair (each number the sum of the two before) and a running factorial ($n!$ = 1 × 2 × … × n):

```
WITH RECURSIVE fib(i, a, b, fact) AS (
    SELECT 1, 0::bigint, 1::bigint, 1::numeric
    UNION ALL
    SELECT i + 1, b, a + b, fact * (i + 1)
    FROM fib
    WHERE i < 10
)
SELECT i, a AS fibonacci, fact AS factorial FROM fib;
```

Output:

```
 i  | fibonacci | factorial
----+-----------+-----------
  1 |         0 |         1
  2 |         1 |         2
  3 |         1 |         6
  4 |         2 |        24
 ...
 10 |        34 |   3628800
(10 rows)
```

Walking dates works the same way: anchor `SELECT DATE '2025-01-06'`, recursive term `SELECT (week_start + 7)::date FROM weeks WHERE week_start < DATE '2025-02-03'` → five Mondays (`code/03_recursive_series.sql`). PostgreSQL's `generate_series` (7.6) is simpler; the recursive form is for MySQL, which lacks it.

A runaway query and two safety nets. Without a stop condition recursion never ends. While experimenting, an outer `LIMIT` stops it (PostgreSQL evaluates recursion lazily, only as far as the outer query asks):

```
WITH RECURSIVE forever(n) AS (
    SELECT 1 UNION ALL SELECT n + 1 FROM forever      -- no stop condition!
)
SELECT n FROM forever LIMIT 3;
```

Output:

```
 n
---
 1
 2
 3
(3 rows)
```

And `UNION` stops a computation whose state repeats: doubling modulo 7 (remainder after dividing by 7) goes 1 → 2 → 4 → 1; the second 1 is discarded and the working table empties:

```
WITH RECURSIVE mod_cycle(n) AS (
    SELECT 1
    UNION
    SELECT (n * 2) % 7 FROM mod_cycle
)
SELECT n FROM mod_cycle;
```

Output:

```
 n
---
 1
 2
 4
(3 rows)
```

`UNION` only helps when the *whole row* repeats; a growing column (depth, path, running total) never repeats — use explicit cycle detection (7.5).

Dialect notes

	PostgreSQL	MySQL 8	SQLite	SQL Server
Syntax	`WITH RECURSIVE`	`WITH RECURSIVE`	`WITH RECURSIVE`	`WITH` (no keyword)
Built-in depth limit	none	`cte_max_recursion_depth` = 1000 by default	none	100 by default; `OPTION (MAXRECURSION n)`, 0 = unlimited
`UNION` (distinct) in recursion	yes	yes	yes	no, `UNION ALL` only
`SEARCH` / `CYCLE` clauses	14+	no	no	no

Oracle also supports recursive `WITH` (since 11g R2) and its older proprietary `CONNECT BY` syntax.

Common mistakes

- **No stop condition** → runs until memory or disk runs out (MySQL/SQL Server: depth-limit error).
- **Anchor/recursive type mismatch.** The anchor fixes column types: a `varchar(40)` anchor column extended by `path || ' > ' || name` (`text`) fails with *"column has type character varying(40) in non-recursive term but type text overall"*; `SELECT 1` makes an `integer` that a growing factorial overflows. Cast in the anchor (`first_name::text`, `1::numeric`).
- **Thinking the recursive term sees the whole result** — only the previous pass.
- **Relying on output order** — build a sort key (depth, path, or `SEARCH`, 7.5) and `ORDER BY` it.

Self-check

1. *When does recursion stop?* → When a pass produces no new rows.
2. *What does SQL Server need instead of `WITH RECURSIVE`?* → Just `WITH`; recursion is limited to 100 levels unless `OPTION (MAXRECURSION n)`.

7.4 Hierarchies: trees stored as parent pointers

What it is

The usual relational tree is the **adjacency list**: each row stores its parent's key (`employees.manager_id` → another `emp_id`; `NULL` marks the root), queried with recursive CTEs.

How it works — three classic queries

Question	Anchor	Recursive step
Top-down: the whole tree (or a subtree)	the root(s): `manager_id IS NULL` (or `emp_id = X`)	children: `e.manager_id = parent.emp_id`
Bottom-up: the chain of ancestors	the starting node	parent: `m.emp_id = child.manager_id`
Subtree aggregates	every (boss, direct report) pair	extend each pair one level down, keeping the boss

Top-down with depth and path (the path doubles as a sort key keeping each team under its manager):

```sql
-- code/04_hierarchy.sql
WITH RECURSIVE org AS (
    SELECT emp_id, first_name, manager_id, 1 AS depth,
           first_name::text AS path
    FROM employees
    WHERE manager_id IS NULL                        -- anchor: the root
    UNION ALL
    SELECT e.emp_id, e.first_name, e.manager_id, o.depth + 1,
           o.path || ' > ' || e.first_name
    FROM employees e
    JOIN org o ON e.manager_id = o.emp_id           -- children of rows found so far
)
SELECT repeat('    ', depth - 1) || first_name AS org_chart, depth, path
FROM org
ORDER BY path;
```

Output:

```
   org_chart    | depth |         path
----------------+-------+----------------------
 Asha           |     1 | Asha
     Daniel     |     2 | Asha > Daniel
         Arjun  |     3 | Asha > Daniel > Arjun
         Priya  |     3 | Asha > Daniel > Priya
         Sara   |     3 | Asha > Daniel > Sara
     Kiran      |     2 | Asha > Kiran
         Omar   |     3 | Asha > Kiran > Omar
 ...
     Vikram     |     2 | Asha > Vikram
         Li     |     3 | Asha > Vikram > Li
 ...
         Rohan  |     3 | Asha > Vikram > Rohan
(15 rows)
```

Pass 1 found Asha's four reports (depth 2), pass 2 theirs (depth 3), pass 3 nobody — stop. A text path sorts correctly only because names lack `>`; a robust key is an id array (`ARRAY[emp_id]`, then `path || e.emp_id`), compared element by element. For one subtree, anchor on `WHERE emp_id = 2`.

Bottom-up: who is above the intern?

```sql
WITH RECURSIVE chain AS (
    SELECT emp_id, first_name, job_title, manager_id, 0 AS steps_up
    FROM employees WHERE emp_id = 15
    UNION ALL
    SELECT m.emp_id, m.first_name, m.job_title, m.manager_id, c.steps_up + 1
    FROM employees m
    JOIN chain c ON m.emp_id = c.manager_id
)
SELECT steps_up, first_name, job_title FROM chain ORDER BY steps_up;
```

Output:

```
 steps_up | first_name |      job_title
----------+------------+--------------------
        0 | Rohan      | Intern
        1 | Vikram     | Engineering Manager
        2 | Asha       | CEO
(3 rows)
```

Only the join direction differs: `m.emp_id = c.manager_id` ("find my parent") vs `e.manager_id = o.emp_id` ("find my children"). Uses: breadcrumbs (`Home > Electronics > Laptops`), approval chains, permission inheritance.

Subtree totals — carry the *boss* unchanged while walking down, one row per (boss, anyone below):

```sql
WITH RECURSIVE reports AS (
    SELECT manager_id AS boss, emp_id, salary        -- direct reports
    FROM employees WHERE manager_id IS NOT NULL
    UNION ALL
    SELECT r.boss, e.emp_id, e.salary                -- their reports, and so on
    FROM reports r
    JOIN employees e ON e.manager_id = r.emp_id
)
SELECT b.first_name AS manager, count(*) AS team_size, sum(r.salary) AS team_salary
FROM reports r JOIN employees b ON b.emp_id = r.boss
GROUP BY b.first_name
ORDER BY team_size DESC, manager;
```

Output:

```
 manager | team_size | team_salary
---------+-----------+-------------
 Asha    |        14 |  1367000.00
 Vikram  |         5 |   432000.00
 Daniel  |         3 |   208000.00
 Kiran   |         1 |    76000.00
 Lucas   |         1 |    61000.00
(5 rows)
```

Asha has all 14 others below her; Vikram has Neha, Rahul, Meera, Li, Rohan. The aggregate runs *outside* — aggregates aren't allowed in the recursive term.

Other ways to store trees (for comparison)

Model	How	Reads	Writes	Notes
Adjacency list (this chapter)	`parent_id` column	recursive CTE	trivial (change one pointer)	the default choice
Materialized path	store `'1/2/15'` (or PostgreSQL `ltree`, an extension type for dotted label paths)	`LIKE '1/2/%'`, indexable	moving a subtree rewrites all descendants' paths	great for breadcrumbs
Nested sets	store `lft` / `rgt` numbers	subtree = range query	inserts renumber many rows	read-heavy, rarely-changing trees
Closure table	separate table of every (ancestor, descendant, depth) pair	simple joins	many rows per insert	*SQL Antipatterns* recommends it for heavy tree querying

Common mistakes

- **Anchoring on** `manager_id = X` lists X's reports but *not X*; use `emp_id = X` to include X.
- **Cyclic bad data** (A manages B, B manages A): a walk from a real root never reaches it, but one starting inside loops forever. Protect with a path check (7.5) or a `CHECK`/trigger (function run automatically on row changes) preventing cycles.
- **Sorting by depth only** loses the "team under manager" layout; sort by path.

In the interview

30-second answer: "Store the tree as an adjacency list — each row points to its parent. To read a subtree, anchor on the root and recursively join children whose parent is in the previous level, carrying depth and a path for ordering. For ancestors, reverse the join. For totals per node, carry the top node through the recursion and aggregate outside."

- *How would you stop infinite loops if the data could contain a cycle?* → Carry an array of visited ids and add `WHERE NOT e.emp_id = ANY(path)`, or use PostgreSQL 14's `CYCLE` clause.

- *Huge tree queried constantly?* → Index `manager_id`; consider a closure table or materialized path (`ltree` with a GiST index — a PostgreSQL index type for not-simply-sortable data like paths or shapes).

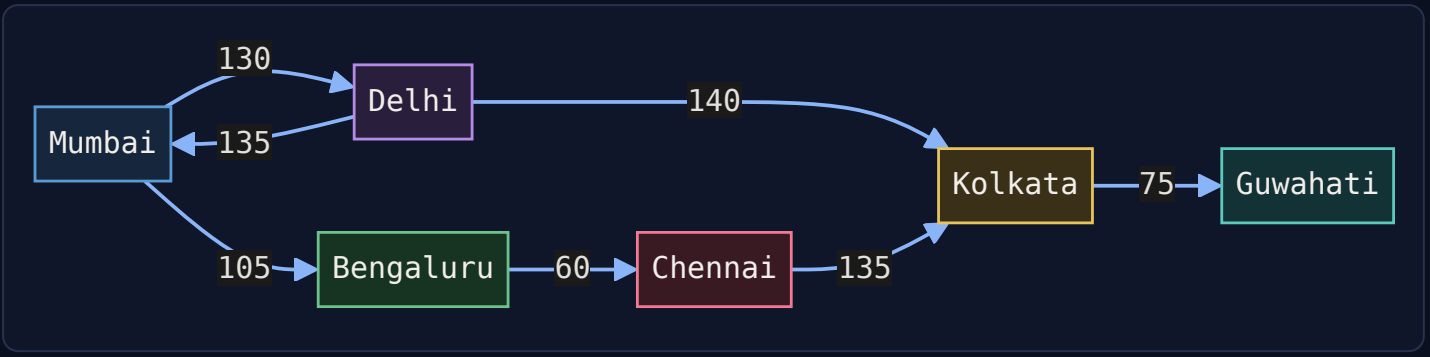
Self-check

1. *Which join walks down, which up?* → Down: `child.manager_id = found.emp_id`; up: `parent.emp_id = found.manager_id`.

7.5 Graphs and cycles

What it is

A **graph** is a tree whose nodes can have several parents and whose paths can loop back (a **cycle**): flight routes, roads, "follows", package dependencies. `routes` stores one-way flights; Mumbai → Delhi → Mumbai is a cycle.



The problem it solves

A naive walk loops forever (Mumbai → Delhi → Mumbai …), and each row differs (total time grows), so `UNION` won't stop it. Remember where you have been:

How it works (manual cycle detection)

carry an **array of visited nodes** (`path`) and follow an edge only if its destination isn't in it (`WHERE r.to_city <> ALL (t.path)`). Every row is then a *simple path* (no repeated city); there are finitely many, so recursion ends.

```sql
-- code/05_graph_cycles.sql
WITH RECURSIVE trips AS (
    SELECT to_city AS city, ARRAY['Mumbai', to_city] AS path, minutes AS total_min
    FROM routes WHERE from_city = 'Mumbai'
    UNION ALL
    SELECT r.to_city, t.path || r.to_city, t.total_min + r.minutes
    FROM trips t
    JOIN routes r ON r.from_city = t.city
    WHERE r.to_city <> ALL (t.path)              -- never revisit a city
)
SELECT city, array_to_string(path, ' -> ') AS route, total_min
FROM trips
ORDER BY city, total_min;
```

Output:

```
   city    |                         route                          | total_min
-----------+--------------------------------------------------------+-----------
 Bengaluru | Mumbai -> Bengaluru                                    |       105
 Chennai   | Mumbai -> Bengaluru -> Chennai                         |       165
 Delhi     | Mumbai -> Delhi                                        |       130
 Guwahati  | Mumbai -> Delhi -> Kolkata -> Guwahati                 |       345
 Guwahati  | Mumbai -> Bengaluru -> Chennai -> Kolkata -> Guwahati  |       375
 Kolkata   | Mumbai -> Delhi -> Kolkata                             |       270
 Kolkata   | Mumbai -> Bengaluru -> Chennai -> Kolkata              |       300
(7 rows)
```

`ARRAY[...]` builds an array (chapter 14), `||` appends, `x <> ALL (array)` is true when `x` differs from every element (chapter 6). Delhi → Mumbai is never followed: Mumbai is in every path.

Shortest (fastest) route to each city — keep the best row per city with `DISTINCT ON` (chapter 2):

```sql
-- same trips CTE as above, then:
SELECT DISTINCT ON (city) city, total_min, array_to_string(path, ' -> ') AS route
FROM trips
WHERE city <> 'Mumbai'
ORDER BY city, total_min;
```

Output:

```
   city    | total_min |                route
-----------+-----------+--------------------------------------
 Bengaluru |       105 | Mumbai -> Bengaluru
 Chennai   |       165 | Mumbai -> Bengaluru -> Chennai
 Delhi     |       130 | Mumbai -> Delhi
 Guwahati  |       345 | Mumbai -> Delhi -> Kolkata -> Guwahati
 Kolkata   |       270 | Mumbai -> Delhi -> Kolkata
(5 rows)
```

This enumerates *every* simple path — exponential work (can double per extra hop) on big, dense graphs. Fine for small graphs and interviews; real route planners use shortest-path algorithms (Dijkstra, A* — never enumerate every path) in code or extensions such as pgRouting, or graph databases (store nodes and edges directly; chapter 16).

The CYCLE and SEARCH clauses (PostgreSQL 14+)

SQL:1999 helper clauses, implemented in PostgreSQL 14 (this book's version):

- `CYCLE col SET is_cycle USING path_col` — tracks visited `col` values, flags a row whose value was already on its path with `is_cycle = true`, and stops recursing from it.
- `SEARCH DEPTH FIRST BY col SET ord` / `SEARCH BREADTH FIRST BY col SET ord` — adds `ord` to `ORDER BY` for depth-first (one branch fully down before the next) or breadth-first (level by level) order.

```sql
WITH RECURSIVE trips(city, total_min) AS (
    SELECT to_city, minutes FROM routes WHERE from_city = 'Mumbai'
    UNION ALL
    SELECT r.to_city, t.total_min + r.minutes
    FROM trips t JOIN routes r ON r.from_city = t.city
) CYCLE city SET is_cycle USING visited
SELECT city, total_min, is_cycle FROM trips
WHERE city IN ('Mumbai', 'Delhi')
ORDER BY total_min;
```

Output:

```
  city  | total_min | is_cycle
--------+-----------+----------
 Delhi  |       130 | f
 Mumbai |       265 | f
 Delhi  |       395 | t
(3 rows)
```

Mumbai (265) is *not* flagged: the anchor starts from *destinations*, so Mumbai wasn't yet on the tracked path; the hop back to Delhi repeats Delhi and is flagged. To count the start city as visited, include it in the anchor (as `ARRAY['Mumbai', to_city]` did).

```
WITH RECURSIVE org(emp_id, first_name, manager_id) AS (
    SELECT emp_id, first_name, manager_id FROM employees WHERE manager_id IS NULL
    UNION ALL
    SELECT e.emp_id, e.first_name, e.manager_id
    FROM employees e JOIN org o ON e.manager_id = o.emp_id
) SEARCH BREADTH FIRST BY emp_id SET ord
SELECT first_name, ord FROM org ORDER BY ord LIMIT 6;
```

Output:

```
 first_name |  ord
------------+--------
 Asha       | (0,1)
 Vikram     | (1,2)
 Daniel     | (1,6)
 Lucas      | (1,10)
 Kiran      | (1,12)
 Neha       | (2,3)
(6 rows)
```

Breadth-first `ord` is (level, id); `DEPTH FIRST` produces an array path like our manual `path`.

Self-check

1. *Why doesn't `UNION` stop the Mumbai–Delhi loop?* → Each lap has a different `total_min`/path, so rows never repeat exactly.

7.6 generate_series: making rows (PostgreSQL)

What it is

`generate_series(start, stop [, step])` is a PostgreSQL **set-returning function**: in `FROM` it returns one row per value from `start` to `stop`, for integers, numerics and timestamps/dates (step as an `interval`).

```
-- code/06_generate_series.sql
SELECT * FROM generate_series(1, 5) AS n;
SELECT * FROM generate_series(10, 0, -5) AS n;
SELECT d::date AS day FROM generate_series(DATE '2025-03-01', DATE '2025-03-04', INTERVAL '1 day') AS d;
```

Output (the three results, one after another):

```
 n
---
 1
 2
 ...
 5
(5 rows)

 n
----
 10
  5
  0
(3 rows)

    day
------------
 2025-03-01
 2025-03-02
 2025-03-03
 2025-03-04
(4 rows)
```

With `date` arguments the `timestamp with time zone` version is used, so cast back to `date`.

The problem it solves: missing rows in reports

`GROUP BY` only makes groups for values that **exist**: a month with no orders vanishes and a chart silently skips it. Generate the full calendar, `LEFT JOIN` the data, count:

```sql
WITH months AS (
    SELECT m::date AS month
    FROM generate_series(DATE '2025-01-01', DATE '2025-07-01', INTERVAL '1 month') AS m
)
SELECT m.month, count(o.order_id) AS orders
FROM months m
LEFT JOIN orders o ON date_trunc('month', o.placed_at) = m.month
GROUP BY m.month
ORDER BY m.month;
```

Output:

```
   month    | orders
------------+--------
 2025-01-01 |      3
 2025-02-01 |      2
 ...
 2025-06-01 |      2
 2025-07-01 |      0
(7 rows)
```

July shows `0` — via `count(o.order_id)`; `count(*)` would count July's `NULL`-padded row as 1 (chapter 4).

Test data in one statement:

```sql
CREATE TABLE fake_events AS
SELECT g AS id, DATE '2025-01-01' + (g % 90) AS day, (g * 37) % 100 AS score
FROM generate_series(1, 1000) AS g;
SELECT count(*), min(day), max(day), round(avg(score), 1) AS avg_score FROM fake_events;
```

Output:

```
 count |    min     |    max     | avg_score
-------+------------+------------+-----------
  1000 | 2025-01-01 | 2025-03-31 |      49.5
(1 row)
```

Arithmetic instead of `random()` keeps data deterministic (identical every run) for reproducible performance tests (chapter 12 scales this to millions of rows).

Dialect notes

	Numbers / dates series
PostgreSQL	`generate_series(...)`
SQL Server 2022+	`GENERATE_SERIES(start, stop [, step])` — numbers only (SQL Server — not run)
SQLite	`generate_series` table-valued function from the `series` extension (built into the `sqlite3` shell)
MySQL 8	none — use a recursive CTE (7.3) or a permanent "numbers"/"calendar" table

Data warehouses (databases for analysing large historical data) often keep a permanent **calendar table** (one row per date with flags like `is_weekend`, `fiscal_quarter`) for this.

7.7 Views

What it is

A **view** is a named query stored in the database, queried like a table (`SELECT * FROM staff_directory`) but storing **no data**: each use substitutes its query into yours and runs it against the current **base** tables — like a saved email search, always up to date.

Why it matters

Benefit	Example
Simplicity — hide a complex join once	analysts query `staff_directory` instead of re-writing the join
Security — expose only some columns/rows	grant access to a view without `salary`, not to `employees`
Stable interface — change tables, keep the view	rename/split a table and redefine the view so old queries still work
Consistency — one definition of a metric	"active customer" is defined once, not in 20 reports

How it works

`CREATE VIEW name AS query;` stores the parsed query in the catalog. When a query mentions the view, PostgreSQL's rewriter (a pre-planning stage that expands views) substitutes it — like an inlined CTE — and the optimizer plans the whole. So a view is always current and costs exactly its query.

```sql
-- code/07_views.sql
CREATE VIEW staff_directory AS
SELECT e.emp_id, e.first_name || ' ' || e.last_name AS full_name,
       e.job_title, d.name AS department, e.email
FROM employees e
LEFT JOIN departments d USING (dept_id);

SELECT full_name, job_title, department FROM staff_directory
WHERE department = 'Sales' ORDER BY full_name;
```

Output:

```
   full_name   |   job_title   | department
---------------+---------------+------------
 Arjun Singh   | Sales Rep     | Sales
 Daniel Brooks | Sales Manager | Sales
 Priya Nair    | Sales Rep     | Sales
 Sara Khan     | Sales Rep     | Sales
(4 rows)
```

Always current: after `UPDATE employees SET job_title = 'Senior Sales Rep' WHERE emp_id = 7`, the view shows Priya Nair as Senior Sales Rep immediately.

Changing and dropping views

- `CREATE OR REPLACE VIEW` swaps the definition in place (keeping permissions), but in PostgreSQL only if it returns the **same columns, names, types and order**; you may only **add** columns at the end.
- To remove or reorder columns: `DROP VIEW` + `CREATE VIEW` (re-granting privileges given with `GRANT`).
- Dependencies are tracked: a table (or view) used by a view can't be dropped without `CASCADE`, which drops the dependent views too.

```sql
-- adding hire_date at the end: allowed
CREATE OR REPLACE VIEW staff_directory AS
SELECT e.emp_id, e.first_name || ' ' || e.last_name AS full_name,
       e.job_title, d.name AS department, e.email, e.hire_date
FROM employees e
LEFT JOIN departments d USING (dept_id);

-- removing columns: refused
CREATE OR REPLACE VIEW staff_directory AS
SELECT e.emp_id, e.job_title FROM employees e;

-- dropping a table the view depends on: refused
DROP TABLE departments;
```

Output:

```
ERROR:  cannot drop columns from view
ERROR:  cannot drop table departments because other objects depend on it
DETAIL:  constraint employees_dept_id_fkey on table employees depends on table departments
constraint projects_dept_id_fkey on table projects depends on table departments
view staff_directory depends on table departments
HINT:  Use DROP ... CASCADE to drop the dependent objects too.
```

(`CASCADE` would also drop the two foreign-key constraints — rarely what you want.)

Updatable views

Writes through a view work if each view row maps to exactly one base row. Since PostgreSQL 9.3 a view is **automatically updatable** when it reads exactly **one** table or updatable view and has no `GROUP BY`, `HAVING`, `DISTINCT`, `LIMIT` / `OFFSET`, window functions, aggregates, `UNION` / `INTERSECT` / `EXCEPT`, or top-level `WITH`. Computed columns are read-only. Otherwise write an `INSTEAD OF` trigger (a function run automatically in place of the write; chapter 14).

```sql
CREATE VIEW furniture AS
SELECT product_id, name, unit_price, category
FROM products
WHERE category = 'Furniture';

UPDATE furniture SET unit_price = 329.99 WHERE name = 'Ergonomic Chair';
INSERT INTO furniture VALUES (10, 'Bookshelf', 120.00, 'Furniture');
-- Nothing stops a row that falls OUTSIDE the view's WHERE...
INSERT INTO furniture VALUES (11, 'Webcam', 59.00, 'Electronics');
SELECT product_id, name, category FROM products WHERE product_id >= 10;
SELECT count(*) AS visible_in_view FROM furniture WHERE product_id >= 10;
```

Output:

```
 product_id |   name    |  category
------------+-----------+-------------
         10 | Bookshelf | Furniture
         11 | Webcam    | Electronics
(2 rows)

 visible_in_view
-----------------
               1
(1 row)
```

The Webcam went into `products` through `furniture` but is invisible there — a "disappearing row". (`discontinued`, not in the view, got its default `false`.) `WITH CHECK OPTION` rejects writes whose row would not satisfy the view's `WHERE`:

```
CREATE OR REPLACE VIEW furniture AS
SELECT product_id, name, unit_price, category
FROM products
WHERE category = 'Furniture'
WITH CHECK OPTION;

INSERT INTO furniture VALUES (12, 'Monitor Arm', 79.00, 'Accessories');
```

Output:

```
ERROR:  new row violates check option for view "furniture"
DETAIL:  Failing row contains (12, Monitor Arm, Accessories, 79.00, f).
```

(`LOCAL` checks only this view's condition; `CASCADED`, the default, also those of underlying views.) A view with a join or `GROUP BY` is read-only:

```
CREATE VIEW dept_payroll AS
SELECT d.name AS department, count(e.emp_id) AS headcount, sum(e.salary) AS payroll
FROM departments d LEFT JOIN employees e USING (dept_id)
GROUP BY d.name;
SELECT * FROM dept_payroll ORDER BY payroll DESC NULLS LAST;
UPDATE dept_payroll SET headcount = 0 WHERE department = 'Sales';
```

Output:

```
 department  | headcount |  payroll
-------------+-----------+-----------
 Engineering |         4 | 523000.00
 Sales       |         4 | 358000.00
 ...
 Research    |         0 |    (null)
(6 rows)

ERROR:  cannot update view "dept_payroll"
DETAIL:  Views containing GROUP BY are not automatically updatable.
HINT:  To enable updating the view, provide an INSTEAD OF UPDATE trigger or an unconditional ON UPDATE DO INSTEAD rule.
```

The catalog knows which views are updatable:

```
SELECT table_name, is_updatable
FROM information_schema.views
WHERE table_schema = 'public'
ORDER BY table_name;
```

Output:

```
   table_name    | is_updatable
-----------------+--------------
 dept_payroll    | NO
 furniture       | YES
 staff_directory | NO
(3 rows)
```

Views and security

A PostgreSQL view reads base tables with the **view owner's** privileges, so `GRANT SELECT ON staff_directory TO analyst` exposes names and titles but never salaries, with no grant on `employees`.

- `WITH (security_barrier)` stops a crafted user function in the caller's `WHERE` from peeking at rows the view's `WHERE` hides (otherwise the optimizer may run the cheap function first).
- PostgreSQL 15 added `WITH (security_invoker = true)` to use the *caller's* privileges (not run here on 14).

Roles, `GRANT` and row-level security (per-user row limits): chapter 14.

Dialect notes

	PostgreSQL	MySQL	SQLite	SQL Server
`CREATE OR REPLACE VIEW`	yes (add columns only)	yes (any change)	no — drop & create	`CREATE OR ALTER VIEW` (2016 SP1+)
Auto-updatable simple views	yes	yes	no — read-only, use `INSTEAD OF` triggers	yes
`WITH CHECK OPTION`	yes	yes	no	yes
Materialized views	yes (7.8)	no	no	"indexed views" (kept up to date automatically)

When not to use it

- **Deep view stacks** — each looks simple, the combined query is huge and hard to tune.
- **As a performance tool** — a view is exactly as fast as its query; use a materialized view or indexes.
- **`SELECT *` in a definition** — `*` is expanded at *creation*, so later table columns do **not** appear. List columns.

Common mistakes

- Expecting a view to store data (that's a materialized view).
- Relying on `ORDER BY` inside a view — only the **outer** query's `ORDER BY` guarantees order.

Real-world examples

BI (business-intelligence) tools (Metabase, Tableau, Power BI) point at views, not raw tables; after splitting `users` into `accounts` and `profiles`, a `users` view keeps old code working; PostgreSQL's `information_schema` is views over the system catalogs.

In the interview

30-second answer: "A view is a stored query that acts like a virtual table. It stores no data, so it's always current and costs the same as running its query. Views simplify complex joins, provide a stable interface when tables change, and enforce security by exposing only certain columns or rows. Simple single-table views are updatable; `WITH CHECK OPTION` stops writes that would fall outside the view."

- *Does a view improve performance?* → Not by itself; it is expanded into the query. A materialized view (or indexed view in SQL Server) can.

- *Can you insert into a view?* → Yes if it's a simple single-table view (auto-updatable) or has `INSTEAD OF` triggers.
- *What happens to a view if you add a column to its table?* → Nothing; the view keeps the column list captured when it was created.

Self-check

1. *Why did the Webcam vanish from `furniture`, and what prevents it?* → It doesn't satisfy `category = 'Furniture'`; `WITH CHECK OPTION`.
2. *Why is `staff_directory` not updatable?* → It joins two tables.

7.8 Materialized views

What it is

A **materialized view** stores its query's *result* on disk like a table and recomputes only on `REFRESH MATERIALIZED VIEW` — trading **freshness** for **speed** (like a printed report: instant to read, stale until reprinted).

How it works

1. `CREATE MATERIALIZED VIEW name AS query;` runs the query and stores the rows (`WITH NO DATA` creates it empty, unqueryable until refreshed). Reads can use indexes you create on it.
2. `REFRESH MATERIALIZED VIEW name;` re-runs and replaces the contents under a strong lock (others wait): **reads are blocked**.
3. `REFRESH MATERIALIZED VIEW CONCURRENTLY name;` builds the new result separately and applies only differences, not blocking readers; needs a **unique index** (plain columns, no `WHERE`) and is slower overall.

```sql
-- code/08_materialized_views.sql
CREATE MATERIALIZED VIEW product_sales AS
SELECT p.product_id, p.name,
       coalesce(sum(oi.quantity), 0) AS units_sold,
       coalesce(round(sum(oi.quantity * oi.unit_price * (1 - oi.discount)), 2), 0) AS revenue
FROM products p
LEFT JOIN order_items oi USING (product_id)          -- every order line
GROUP BY p.product_id, p.name;

SELECT name, units_sold, revenue FROM product_sales ORDER BY revenue DESC LIMIT 3;
```

Output:

```
     name       | units_sold | revenue
----------------+------------+---------
 Laptop Pro 14  |          5 | 6380.05
 Standing Desk  |          2 |  978.00
 Ergonomic Chair |          3 |  962.47
(3 rows)
```

Stale until refreshed:

```sql
INSERT INTO orders VALUES (115, 9, NULL, '2025-06-20 09:00', 'pending', NULL);
INSERT INTO order_items VALUES (115, 5, 3, 499.00, 0.00);

SELECT name, units_sold, revenue FROM product_sales WHERE name = 'Standing Desk';
REFRESH MATERIALIZED VIEW product_sales;
SELECT name, units_sold, revenue FROM product_sales WHERE name = 'Standing Desk';
```

Output:

```
     name      | units_sold | revenue
---------------+------------+---------
 Standing Desk |          2 |  978.00
(1 row)

     name      | units_sold | revenue
---------------+------------+---------
 Standing Desk |          5 | 2475.00
(1 row)
```

Concurrent refresh needs a unique index:

```
REFRESH MATERIALIZED VIEW CONCURRENTLY product_sales;          -- fails
CREATE UNIQUE INDEX product_sales_pk ON product_sales (product_id);
REFRESH MATERIALIZED VIEW CONCURRENTLY product_sales;          -- works
CREATE INDEX product_sales_revenue ON product_sales (revenue DESC);
SELECT matviewname, ispopulated FROM pg_matviews;
```

Output:

```
ERROR:  cannot refresh materialized view "public.product_sales" concurrently
HINT:  Create a unique index with no WHERE clause on one or more columns of the materialized view.
  matviewname  | ispopulated
---------------+-------------
 product_sales | t
(1 row)
```

Trade-offs

	View	Materialized view	Summary table maintained by triggers/jobs
Stores data	no	yes	yes
Freshness	always current	as of last refresh	as current as your code keeps it
Read speed	cost of the query	fast (indexable)	fast
Write overhead	none	none at write time; refresh cost later	on every write
Effort	trivial	trivial + schedule refreshes	most code, most bugs

PostgreSQL has **no automatic or incremental refresh** — each refresh recomputes everything. Schedule it (cron, the Unix timetable scheduler; the `pg_cron` extension; a job scheduler) or run it after batch loads. Oracle has incremental "fast refresh"; SQL Server's indexed views update on every write (many restrictions); MySQL has none.

When to use / not use

- **Use** for dashboards/reports that may be minutes or hours old, expensive aggregations read far more than the data changes, precomputed lookup data.
- **Don't** when readers need up-to-the-second numbers, or refreshing costs more than it saves.

Common mistakes

- No scheduled refresh — the dashboard shows last month's numbers.
- Plain `REFRESH` on a busy materialized view, blocking every reader.

In the interview

30-second answer: "A view stores a query; a materialized view stores the query's result. Materialized views make expensive aggregations fast to read and can be indexed, but they're stale

until refreshed. In PostgreSQL refresh is manual and full; `CONCURRENTLY` avoids blocking readers but needs a unique index."

- *Keeping it fresh?* → Scheduled refresh (e.g. every 5 minutes with pg_cron) or after each ETL batch (Extract-Transform-Load job copying and reshaping data in); for real-time, maintain a summary table incrementally.
- *vs a cache like Redis?* → The materialized view is in the database, SQL-queryable, consistent as of a point in time; an external cache (hot data outside the database; Redis is an in-memory key-value store) is faster per key but needs invalidation (updating copies when data changes; System Design Fundamentals, chapter 15).

Self-check

1. *What does `CONCURRENTLY` need?* → A unique index on the materialized view with no `WHERE` clause.

7.9 Choosing the right tool

Need	Use	Lifetime	Stores rows?	Indexable?
One-off intermediate result inside one query, used once	subquery / derived table (ch. 6)	one statement	no	no
Readable multi-step query, reuse within the statement, recursion, move rows atomically	CTE	one statement	only if materialized, temporarily	no
Reusable definition for many queries and users, security, stable interface	view	permanent	no	no (index the base tables)
Fast reads of expensive results that may be slightly stale	materialized view	permanent	yes	yes
Intermediate data reused by several statements in a session, needs indexes/statistics	`CREATE TEMP TABLE` (ch. 9)	session or transaction	yes	yes

---

Chapter 7 TL;DR

- **CTE** (`WITH name AS (...)`) names a step for one statement; chain with commas; reuse by name. PostgreSQL 12+ inlines single-use CTEs; `MATERIALIZED` / `NOT MATERIALIZED` override.
- **Data-modifying CTEs** (PostgreSQL) run `INSERT/UPDATE/DELETE ... RETURNING` inside `WITH` — atomic multi-step changes; all parts see the pre-statement snapshot.
- **Recursive CTE** = anchor `UNION [ALL]` recursive term; the recursive term sees only the previous pass's rows; stops when a pass returns nothing. Cast types in the anchor, always have a stop condition, `ORDER BY` a path/depth.
- **Trees**: adjacency list + recursion — join children to go down, join the parent to go up, carry the top node and aggregate outside for subtree totals.
- **Graphs**: carry a visited array and refuse revisits, or use `CYCLE` (PostgreSQL 14+); `SEARCH BREADTH/DEPTH FIRST` gives an ordering column.
- **`generate_series`** makes number/date rows: full calendar + `LEFT JOIN` + `count(col)` fills gaps with zeros. MySQL uses a recursive CTE instead.
- **View** = stored query, no data, always current; for simplicity, security, stable interfaces. Simple single-table views are updatable; `WITH CHECK OPTION` blocks out-of-view writes; `CREATE OR REPLACE` can only append columns in PostgreSQL.

- **Materialized view** = stored result; fast, indexable, stale until `REFRESH` (`CONCURRENTLY` needs a unique index). No auto-refresh in PostgreSQL.

Next: chapter 8 — window functions, which compute running totals, rankings and comparisons with neighbouring rows without collapsing rows the way `GROUP BY` does.

Chapter 8 — Window Functions: Rankings, Running Totals and Row-to-Row Comparisons

`GROUP BY` gives "total per department" but discards the rows. Often you need both: *each* employee **and** the department average; *each* day **and** the running total; *each* order **and** the customer's previous order. **Window functions** compute across related rows — the *window* — while keeping every row. They replace awkward self-joins and correlated subqueries (re-run per outer row) and underlie many interview questions (top-N per group, running totals, second-highest salary, streaks, month-over-month growth).

Sections: 8.1 `OVER()` · 8.2 `PARTITION BY` · 8.3 `ORDER BY` in a window · 8.4 Frames: `ROWS` / `RANGE` / `GROUPS`, `EXCLUDE` · 8.5 Ranking · 8.6 Value functions · 8.7 Running & moving calculations · 8.8 Top-N per group & dedup (de-duplication: removing extra copies) · 8.9 Gaps and islands · 8.10 Named windows, `FILTER`, evaluation order, performance.

Setup. The `company` sample database from chapter 1 plus two small tables added in `code/00_schema.sql`:

Table	Rows	Why
`daily_sales(store, day, amount)`	Mumbai 1–7 March 2025; Pune the same week **except 4 March** (closed)	running totals, moving averages, frames, a gap
`logins(user_id, login_date)`	two users' login days in May 2025 with breaks	streaks (gaps and islands)

Deliberate ties: Mumbai's 3 and 4 March are both 1500, two Pune days are 650; Rahul and Meera earn 98 000, Arjun and Sara 68 000.

8.1 What a window function is

What it is

A **window function** looks, for each row, at a set of related rows (its **window**) and returns one value — **without merging the rows**. It is recognised by `OVER (...)`:

```
function_name(args) OVER (
    [PARTITION BY ...]      -- split rows into independent groups
    [ORDER BY ...]          -- order rows inside each group
    [frame clause]          -- which rows around the current one to use
)
```

The problem it solves

Compare `GROUP BY` with a window:

```
-- code/01_window_basics.sql
SELECT dept_id, round(avg(salary)) AS dept_avg
FROM employees WHERE dept_id IN (1, 2)
GROUP BY dept_id ORDER BY dept_id;
```

Output:

```
 dept_id | dept_avg
---------+----------
       1 |   130750
       2 |    89500
(2 rows)
```

Two rows; the employees are gone. With windows every employee stays, aggregates beside them:

```sql
SELECT first_name, dept_id, salary,
       round(avg(salary) OVER ())                    AS company_avg,
       round(avg(salary) OVER (PARTITION BY dept_id)) AS dept_avg,
       count(*)          OVER (PARTITION BY dept_id)  AS dept_size
FROM employees
WHERE dept_id IN (1, 2)
ORDER BY dept_id, salary DESC;
```

Output:

```
 first_name | dept_id |  salary   | company_avg | dept_avg | dept_size
------------+---------+-----------+-------------+----------+-----------
 Vikram     |       1 | 185000.00 |      110125 |   130750 |         4
 Neha       |       1 | 142000.00 |      110125 |   130750 |         4
 ...
 Daniel     |       2 | 150000.00 |      110125 |    89500 |         4
 Priya      |       2 |  72000.00 |      110125 |    89500 |         4
 ...
(8 rows)
```

`OVER ()` means all rows of the result (the 8 left after `WHERE`), so `company_avg` is constant; `OVER (PARTITION BY dept_id)` means "rows with my `dept_id`". Without windows each extra column needs a join to a grouped subquery or a correlated subquery.

How it works

After `FROM`, `WHERE`, `GROUP BY`, `HAVING`, rows are split into **partitions** (`PARTITION BY`) and sorted within them (`ORDER BY`); per row the function reads its window (the partition, or a **frame** in it — 8.3/8.4) and adds one value as a new column — row count unchanged. Then `DISTINCT`, the final `ORDER BY` and `LIMIT` run.

Two families work with `OVER`:

Family	Functions	Notes
Aggregate functions used as windows	`sum`, `avg`, `count`, `min`, `max`, `string_agg`, `array_agg`, `bool_and`, …	any normal aggregate becomes a window function when you add `OVER`
Dedicated window functions	`row_number`, `rank`, `dense_rank`, `percent_rank`, `cume_dist`, `ntile`, `lag`, `lead`, `first_value`, `last_value`, `nth_value`	only valid with `OVER`; the ranking ones take no arguments

Dialect notes

	Window functions available since	Frame support
PostgreSQL	8.4 (2009)	`ROWS`, `RANGE` with offsets, `GROUPS`, `EXCLUDE` (offsets/`GROUPS`/`EXCLUDE` since 11)
MySQL	8.0 (2018)	`ROWS`, `RANGE`; no `GROUPS`, no `EXCLUDE`
SQLite	3.25 (2018)	`ROWS`, `RANGE`; `GROUPS` and `EXCLUDE` since 3.28
SQL Server	ranking 2005; aggregate `ORDER BY` + frames 2012	`ROWS`; `RANGE` only with `UNBOUNDED`/`CURRENT ROW`; no `GROUPS`/`EXCLUDE`
Oracle	8i (1999)	full, plus extensions

8.2 PARTITION BY: per-group values next to each row

What it is

`PARTITION BY expr` splits rows into independent groups like `GROUP BY`, without collapsing them; the function restarts in each partition.

Typical uses

```
SELECT first_name, dept_id, salary,
       round(100.0 * salary / sum(salary) OVER (PARTITION BY dept_id), 1) AS pct_of_dept_payroll,
       salary - round(avg(salary) OVER (PARTITION BY dept_id))            AS vs_dept_avg
FROM employees
WHERE dept_id IN (1, 2)
ORDER BY dept_id, salary DESC;
```

Output:

```
 first_name | dept_id |  salary   | pct_of_dept_payroll | vs_dept_avg
------------+---------+-----------+---------------------+-------------
 Vikram     |       1 | 185000.00 |                35.4 |    54250.00
 Neha       |       1 | 142000.00 |                27.2 |    11250.00
 Rahul      |       1 |  98000.00 |                18.7 |   -32750.00
 Meera      |       1 |  98000.00 |                18.7 |   -32750.00
 ...
(8 rows)
```

"Percent of group total" and "difference from group average" are the two classic uses (`100.0 *` forces decimal division, chapter 3).

Partitioning rules

- Several expressions are allowed: `PARTITION BY store, date_trunc('month', day)` .
- `NULL` s form **one** partition (like `GROUP BY`):

```
SELECT first_name, dept_id, count(*) OVER (PARTITION BY dept_id) AS same_dept
FROM employees
WHERE dept_id IS NULL OR dept_id = 4
ORDER BY dept_id NULLS LAST;
```

Output:

```
 first_name | dept_id | same_dept
------------+---------+-----------
 Kiran      |       4 |         2
 Omar       |       4 |         2
 Asha       |  (null) |         2
 Rohan      |  (null) |         2
(4 rows)
```

- Window functions in one `SELECT` can use different partitions (8.1 mixed `OVER ()` and `OVER (PARTITION BY dept_id)`).

Common mistakes

- **Expecting fewer rows** — a window never removes rows; one row per group is `GROUP BY` (or filter afterwards, 8.8).
- **Non-grouped columns in a grouped query's window** — windows see the *grouped* rows, so arguments must be grouped columns or aggregates (8.7: `sum(sum(x)) OVER ()`).

8.3 ORDER BY inside OVER: running calculations and the default frame

What it is

`ORDER BY` inside `OVER` (1) sets the order for ranking and offset functions (`row_number`, `lag`, …) and (2) changes an **aggregate's** default window from the whole partition to "partition start **up to the current row**" — `sum` becomes a **running total**, `count` a running count, `max` a running maximum.

```sql
-- code/02_order_and_default_frame.sql
SELECT store, day, amount,
       sum(amount) OVER (PARTITION BY store ORDER BY day) AS running_total,
       count(*)    OVER (PARTITION BY store ORDER BY day) AS days_so_far
FROM daily_sales
WHERE store = 'Mumbai'
ORDER BY day;
```

Output:

```
 store  |    day     | amount  | running_total | days_so_far
--------+------------+---------+---------------+-------------
 Mumbai | 2025-03-01 | 1200.00 |       1200.00 |           1
 Mumbai | 2025-03-02 |  900.00 |       2100.00 |           2
 Mumbai | 2025-03-03 | 1500.00 |       3600.00 |           3
 ...
 Mumbai | 2025-03-07 | 1800.00 |       8700.00 |           7
(7 rows)
```

The default frame and the tie trap

With `ORDER BY` and no frame written, the default is:

```sql
RANGE BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
```

`RANGE` works on *values*: "current row" means the current row **and all its peers** (same `ORDER BY` value), so tied rows are summed together and share one "running" total:

```sql
SELECT first_name, salary,
       sum(salary) OVER (ORDER BY salary)                    AS default_range,
       sum(salary) OVER (ORDER BY salary
                         ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW) AS rows_frame,
       sum(salary) OVER (ORDER BY salary, emp_id)            AS unique_order
FROM employees
WHERE dept_id = 2
ORDER BY salary, emp_id;
```

Output:

```
 first_name |  salary   | default_range | rows_frame | unique_order
------------+-----------+---------------+------------+--------------
 Arjun      |  68000.00 |     136000.00 |   68000.00 |     68000.00
 Sara       |  68000.00 |     136000.00 |  136000.00 |    136000.00
 Priya      |  72000.00 |     208000.00 |  208000.00 |    208000.00
 Daniel     | 150000.00 |     358000.00 |  358000.00 |    358000.00
(4 rows)
```

- `default_range`: peers Arjun and Sara both see 68 000 + 68 000.
- `rows_frame`: physical rows, one at a time — but which tied row is "first" is **undefined**; Sara could get 68 000 next time.
- `unique_order`: a tie-breaker (`emp_id`) removes peers, so every frame type gives the same **deterministic** (identical every run) result — the best fix.

OVER (...) contains	Default window for aggregates
nothing / only `PARTITION BY`	the whole partition
`ORDER BY`	`RANGE BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW` (start → current row + its peers)

Common mistakes

- **Running totals that "jump"** on duplicates (the peer effect) — order by a unique key, or write `ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW`.
- **Confusing the window's `ORDER BY` with the query's** — the one in `OVER` controls only the calculation; output order needs the final `ORDER BY`.

8.4 Frames: ROWS, RANGE, GROUPS and EXCLUDE

What it is

The **frame** is the part of the partition that an aggregate (and `first_value` / `last_value` / `nth_value`) sees for the current row; it slides as the row moves. Ranking functions and `lag` / `lead` ignore it.

```
{ ROWS | RANGE | GROUPS } BETWEEN frame_start AND frame_end [EXCLUDE ...]

frame_start / frame_end:
    UNBOUNDED PRECEDING      -- first row of the partition
    n PRECEDING              -- n rows / n units of value / n peer groups before
    CURRENT ROW
    n FOLLOWING
    UNBOUNDED FOLLOWING      -- last row of the partition
```

Short form: `ROWS 2 PRECEDING` means `ROWS BETWEEN 2 PRECEDING AND CURRENT ROW`.

Mode	n PRECEDING means	Peers (ties)	Needs
`ROWS`	*n physical rows* before	treated as separate rows	any `ORDER BY`
`RANGE`	rows whose **value** is within *n* of the current value (`n` can be a number or an `interval`)	always included together	exactly one `ORDER BY` column of a numeric/date/time type when an offset is used
`GROUPS` (PostgreSQL 11+)	*n peer groups* before (a group = rows with equal `ORDER BY` values)	included together	an `ORDER BY`

Frame diagrams

`string_agg(...) OVER (frame)` lists the days in each frame — a trick to *see* a frame (the `frame_days` column).

`ROWS BETWEEN 2 PRECEDING AND CURRENT ROW` — a sliding window of up to 3 rows:

```
-- code/03_frames.sql
SELECT day, amount,
       string_agg(to_char(day, 'DD'), ',') OVER w AS frame_days,
       sum(amount) OVER w                         AS frame_sum
FROM daily_sales
WHERE store = 'Mumbai'
WINDOW w AS (ORDER BY day ROWS BETWEEN 2 PRECEDING AND CURRENT ROW)
ORDER BY day;
```

Output:

```
    day     | amount  | frame_days | frame_sum
------------+---------+------------+-----------
 2025-03-01 | 1200.00 | 01         |   1200.00
 2025-03-02 |  900.00 | 01,02      |   2100.00
 2025-03-03 | 1500.00 | 01,02,03   |   3600.00
 2025-03-04 | 1500.00 | 02,03,04   |   3900.00
 2025-03-05 |  700.00 | 03,04,05   |   3700.00
 2025-03-06 | 1100.00 | 04,05,06   |   3300.00
 2025-03-07 | 1800.00 | 05,06,07   |   3600.00
(7 rows)
```

The first two frames are *shorter* — no earlier rows. (`WINDOW w AS (...)` names a reusable window, 8.10.)

Centered: `ROWS BETWEEN 1 PRECEDING AND 1 FOLLOWING` — smoothing that looks both ways:

```
SELECT day, amount,
       string_agg(to_char(day, 'DD'), ',') OVER w AS frame_days,
       round(avg(amount) OVER w, 1)               AS centered_avg
FROM daily_sales
WHERE store = 'Mumbai'
WINDOW w AS (ORDER BY day ROWS BETWEEN 1 PRECEDING AND 1 FOLLOWING)
ORDER BY day;
```

Output:

```
    day     | amount  | frame_days | centered_avg
------------+---------+------------+--------------
 2025-03-01 | 1200.00 | 01,02      |       1050.0
 2025-03-02 |  900.00 | 01,02,03   |       1200.0
 ...
 2025-03-07 | 1800.00 | 06,07      |       1450.0
(7 rows)
```

`ROWS` vs `RANGE` when a day is missing. Pune has no 4 March, so "previous 2 rows" and "previous 2 days" differ:

```
SELECT day, amount,
       string_agg(to_char(day, 'DD'), ',') OVER (ORDER BY day
            ROWS BETWEEN 2 PRECEDING AND CURRENT ROW)                  AS rows_2,
       string_agg(to_char(day, 'DD'), ',') OVER (ORDER BY day
            RANGE BETWEEN INTERVAL '2 days' PRECEDING AND CURRENT ROW)  AS range_2_days
FROM daily_sales
WHERE store = 'Pune'
ORDER BY day;
```

Output:

```
    day     | amount |  rows_2  | range_2_days
------------+--------+----------+--------------
 2025-03-01 | 400.00 | 01       | 01
 2025-03-02 | 650.00 | 01,02    | 01,02
 2025-03-03 | 500.00 | 01,02,03 | 01,02,03
 2025-03-05 | 800.00 | 02,03,05 | 03,05
 2025-03-06 | 650.00 | 03,05,06 | 05,06
 2025-03-07 | 300.00 | 05,06,07 | 05,06,07
(6 rows)
```

`ROWS 2 PRECEDING` takes three rows whatever their dates; `RANGE INTERVAL '2 days' PRECEDING` takes dates from current − 2 days to current. For "average of the last 3 *calendar* days" `RANGE` is correct and `ROWS` silently reaches back to 2 March (or fill missing days with `generate_series` first — chapter 7).

`ROWS` vs `RANGE` vs `GROUPS` with ties. Ordering Pune by `amount` (two days have 650):

```sql
SELECT day, amount,
       string_agg(amount::int::text, ',') OVER (ORDER BY amount
            ROWS   BETWEEN 1 PRECEDING AND CURRENT ROW) AS rows_1,
       string_agg(amount::int::text, ',') OVER (ORDER BY amount
            RANGE  BETWEEN 150 PRECEDING AND CURRENT ROW) AS range_150,
       string_agg(amount::int::text, ',') OVER (ORDER BY amount
            GROUPS BETWEEN 1 PRECEDING AND CURRENT ROW) AS groups_1
FROM daily_sales
WHERE store = 'Pune'
ORDER BY amount, day;
```

Output:

```
    day     | amount | rows_1  |  range_150  |  groups_1
------------+--------+---------+-------------+------------
 2025-03-07 | 300.00 | 300     | 300         | 300
 2025-03-01 | 400.00 | 300,400 | 300,400     | 300,400
 2025-03-03 | 500.00 | 400,500 | 400,500     | 400,500
 2025-03-02 | 650.00 | 500,650 | 500,650,650 | 500,650,650
 2025-03-06 | 650.00 | 650,650 | 500,650,650 | 500,650,650
 2025-03-05 | 800.00 | 650,800 | 650,650,800 | 650,650,800
(6 rows)
```

- `ROWS 1 PRECEDING`: the previous physical row — the two 650 rows get *different* frames.
- `RANGE 150 PRECEDING`: amounts from current − 150 to current, **all peers included** (for 650: 500, 650, 650).
- `GROUPS 1 PRECEDING`: the current peer group plus the one before (650: {500}, {650, 650}; 800: {650, 650}, {800}).

`EXCLUDE` (PostgreSQL 11+) removes rows from the frame: `EXCLUDE CURRENT ROW`, `EXCLUDE GROUP` (current row and peers), `EXCLUDE TIES` (peers, not the current row), `EXCLUDE NO OTHERS` (default). Average of all *other* days:

```sql
SELECT day, amount,
       round(avg(amount) OVER (ORDER BY day ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING
                               EXCLUDE CURRENT ROW), 1) AS avg_of_other_days
FROM daily_sales
WHERE store = 'Mumbai'
ORDER BY day;
```

Output:

```
    day     | amount  | avg_of_other_days
------------+---------+-------------------
 2025-03-01 | 1200.00 |            1250.0
 2025-03-02 |  900.00 |            1300.0
 ...
 2025-03-07 | 1800.00 |            1150.0
(7 rows)
```

(Week total 8700; for 1 March: (8700 − 1200) / 6 = 1250.)

Choosing a frame

You want	Frame
Running total (deterministic)	`ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW` + unique `ORDER BY`
Last *N* rows (e.g. last 7 records)	`ROWS BETWEEN N-1 PRECEDING AND CURRENT ROW`
Last *N* days of time, with gaps in data	`RANGE BETWEEN INTERVAL 'N-1 days' PRECEDING AND CURRENT ROW`
Whole partition (e.g. for `last_value`)	`ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING` (or no `ORDER BY`)
Current value and the next distinct value(s)	`GROUPS BETWEEN CURRENT ROW AND 1 FOLLOWING`

Common mistakes

- **`ROWS 6 PRECEDING` for "last 7 days" with missing dates** — use `RANGE` with an interval.
- **Expecting `row_number`, `rank`, `lag`, `lead`, `ntile` to obey the frame** — they ignore it.
- **Frame start after frame end** (`BETWEEN CURRENT ROW AND 1 PRECEDING`) — an error.
- **Porting `GROUPS` / `EXCLUDE` / `RANGE n PRECEDING` to SQL Server or MySQL** — unsupported (8.1 dialect table).

In the interview

> **30-second answer:** "The frame says which rows around the current one an aggregate sees. `ROWS` counts physical rows, `RANGE` uses the ordering value — so ties are always included and you can say 'within 7 days' — and `GROUPS` counts groups of tied rows. The default with `ORDER BY` is `RANGE UNBOUNDED PRECEDING TO CURRENT ROW`, which is why running totals can repeat on ties."

- *Moving 7-day average with missing days?* → `avg(x) OVER (ORDER BY day RANGE BETWEEN INTERVAL '6 days' PRECEDING AND CURRENT ROW)`, or fill the calendar and use `ROWS 6 PRECEDING`.

Self-check

1. *Pune, current row 5 March: days in `ROWS 2 PRECEDING`? In `RANGE INTERVAL '2 days' PRECEDING`?* → 2, 3, 5 March; 3, 5 March.
2. *Which frame mode treats tied rows as separate?* → `ROWS`.

8.5 Ranking functions

What it is

Ranking functions number rows by position in the window's `ORDER BY`; they differ in **how they treat ties**.

Function	Returns	Ties
`row_number()`	1, 2, 3, … unique per row	tied rows get different numbers — **in no guaranteed order**
`rank()`	position with gaps ("Olympic" ranking)	same rank; next rank skips (1, 2, 2, 4)
`dense_rank()`	position without gaps	same rank; next rank is +1 (1, 2, 2, 3)
`percent_rank()`	(rank − 1) / (rows in partition − 1), from 0 to 1	same value
`cume_dist()`	(rows ordered before the current row + its peers) / rows in partition, from >0 to 1	same value
`ntile(n)`	bucket number 1…n, buckets as equal as possible	ties can be split across buckets

```sql
-- code/04_ranking.sql
SELECT first_name, salary,
       row_number()  OVER w AS row_number,
       rank()        OVER w AS rank,
       dense_rank()  OVER w AS dense_rank,
       round(percent_rank() OVER w :: numeric, 2) AS pct_rank,
       round(cume_dist()    OVER w :: numeric, 2) AS cume_dist,
       ntile(3)      OVER w AS ntile_3
FROM employees
WHERE dept_id IN (1, 2)
WINDOW w AS (ORDER BY salary DESC)
ORDER BY salary DESC, row_number;
```

Output:

```
 first_name |  salary   | row_number | rank | dense_rank | pct_rank | cume_dist | ntile_3
------------+-----------+------------+------+------------+----------+-----------+---------
 Vikram     | 185000.00 |          1 |    1 |          1 |     0.00 |      0.13 |       1
 Daniel     | 150000.00 |          2 |    2 |          2 |     0.14 |      0.25 |       1
 Neha       | 142000.00 |          3 |    3 |          3 |     0.29 |      0.38 |       1
 Rahul      |  98000.00 |          4 |    4 |          4 |     0.43 |      0.63 |       2
 Meera      |  98000.00 |          5 |    4 |          4 |     0.43 |      0.63 |       2
 Priya      |  72000.00 |          6 |    6 |          5 |     0.71 |      0.75 |       2
 Arjun      |  68000.00 |          7 |    7 |          6 |     0.86 |      1.00 |       3
 Sara       |  68000.00 |          8 |    7 |          6 |     0.86 |      1.00 |       3
(8 rows)
```

- Rahul and Meera tie at 98 000: `rank` 4, 4, then Priya **6** (5 is "used up"); `dense_rank` 4, 4, then **5**.
- `row_number` gave Rahul 4, Meera 5 — arbitrary; it could flip. Always add a tie-breaker (`ORDER BY salary DESC, emp_id`).
- `percent_rank` for Priya: (6 − 1) / (8 − 1) = 0.71. `cume_dist` for Rahul/Meera: 5 of 8 rows are ≥ 98 000 in this order → 0.63.
- `ntile(3)` over 8 rows: 8 / 3 = 2 remainder 2, so buckets of 3, 3, 2.

Ranking within groups — add `PARTITION BY`:

```sql
SELECT dept_id, first_name, salary,
       dense_rank() OVER (PARTITION BY dept_id ORDER BY salary DESC) AS rank_in_dept
FROM employees
WHERE dept_id IS NOT NULL
ORDER BY dept_id, rank_in_dept, first_name;
```

Output:

```
 dept_id | first_name |  salary   | rank_in_dept
---------+------------+-----------+--------------
       1 | Vikram     | 185000.00 |            1
       1 | Neha       | 142000.00 |            2
       1 | Meera      |  98000.00 |            3
       1 | Rahul      |  98000.00 |            3
 ...
       5 | Li         |  64000.00 |            1
(13 rows)
```

`ntile` **for quartiles** (four equal-sized quarters of the sorted values) — window in a subquery, `GROUP BY` outside:

```sql
SELECT quartile, count(*) AS people, min(salary), max(salary)
FROM (SELECT salary, ntile(4) OVER (ORDER BY salary) AS quartile FROM employees) t
GROUP BY quartile
ORDER BY quartile;
```

Output:

```
 quartile | people |    min    |    max
----------+--------+-----------+-----------
        1 |      4 |  30000.00 |  68000.00
        2 |      4 |  68000.00 |  98000.00
        3 |      4 |  98000.00 | 142000.00
        4 |      3 | 150000.00 | 250000.00
(4 rows)
```

68 000 and 98 000 each land in two buckets: `ntile` fills by row count and **splits ties**. To keep ties together use `percent_rank` / `cume_dist`, or `percentile_cont` (an ordered-set aggregate, reading input sorted, like a median) for exact quartile boundaries (chapter 4).

Which ranking function? (the question interviewers love)

Question	Use
"Exactly one row per …" / pagination (splitting results into pages) / dedup	`row_number()` with a unique tie-breaker
"Top 3 places, ties share a place, next place skips" (sports)	`rank()`
"The 2nd highest *distinct* salary" / "top 3 salary levels"	`dense_rank()`
"Top 10 %"	`percent_rank() <= 0.1` or `cume_dist() <= 0.1` (depending on how ties should count)
"Split into 4 equal groups"	`ntile(4)`

Nth highest value — the classic: `dense_rank` makes ties count once.

```sql
-- 2nd highest distinct salary (NULL if there isn't one)
SELECT max(salary) FROM (
    SELECT salary, dense_rank() OVER (ORDER BY salary DESC) AS dr FROM employees
) t WHERE dr = 2;
```

Output:

```
    max
-----------
 185000.00
(1 row)
```

Asha earns 250 000, so Vikram's 185 000 is second. `max()` returns one row even if several share that salary, and `NULL` rather than no rows when there is no 2nd value.

Self-check

1. *Salaries 100, 90, 90, 80: rank of 80? dense_rank?* → 4; 3.
2. `ntile(4)` *over 10 rows — bucket sizes?* → 3, 3, 2, 2.

8.6 Value (offset) functions: LAG, LEAD, FIRST_VALUE, LAST_VALUE, NTH_VALUE

What it is

Value functions return a value **from another row** of the window.

Function	Returns	Uses the frame?
`lag(expr [, offset [, default]])`	`expr` from `offset` rows **before** (default 1)	no — relative to the partition's order
`lead(expr [, offset [, default]])`	`expr` from `offset` rows **after**	no
`first_value(expr)`	`expr` from the frame's first row	yes
`last_value(expr)`	`expr` from the frame's last row	yes
`nth_value(expr, n)`	`expr` from the frame's *n*-th row (1-based), NULL if the frame is shorter	yes

When there is no such row (the first row has no "previous"), `lag` / `lead` return `NULL`, or the `default` you pass.

Day-over-day change:

```sql
-- code/05_value_functions.sql
SELECT store, day, amount,
       lag(amount)  OVER w          AS prev_day,
       amount - lag(amount) OVER w  AS change,
       lead(amount) OVER w          AS next_day,
       lag(amount, 2, 0) OVER w     AS two_days_ago_or_0
FROM daily_sales
WHERE day <= '2025-03-04'
WINDOW w AS (PARTITION BY store ORDER BY day)
ORDER BY store, day;
```

Output:

```
 store  |    day     | amount  | prev_day | change  | next_day | two_days_ago_or_0
--------+------------+---------+----------+---------+----------+-------------------
 Mumbai | 2025-03-01 | 1200.00 |   (null) |  (null) |   900.00 |                 0
 Mumbai | 2025-03-02 |  900.00 |  1200.00 | -300.00 |  1500.00 |                 0
 ...
 Mumbai | 2025-03-04 | 1500.00 |  1500.00 |    0.00 |   (null) |            900.00
 Pune   | 2025-03-01 |  400.00 |   (null) |  (null) |   650.00 |                 0
 Pune   | 2025-03-02 |  650.00 |   400.00 |  250.00 |   500.00 |                 0
 Pune   | 2025-03-03 |  500.00 |   650.00 | -150.00 |   (null) |            400.00
(7 rows)
```

`PARTITION BY store` stops `lag` crossing into the other store (Pune's first day has no previous). `WHERE` runs first, so `lead` on 4 March sees nothing (5 March was filtered). `lag` means previous **row**, not previous **day** — in Pune the row before 5 March is 3 March; fill gaps or check the date difference (8.9).

The `last_value` trap. With `ORDER BY`, the default frame ends at the current row, so `last_value` returns the current row:

```
SELECT day, amount,
       first_value(amount) OVER (ORDER BY day)    AS first_amt,
       last_value(amount)  OVER (ORDER BY day)    AS last_default_frame,
       last_value(amount)  OVER (ORDER BY day
            ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING) AS last_whole_partition,
       nth_value(amount, 2) OVER (ORDER BY day
            ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING) AS second_amt
FROM daily_sales
WHERE store = 'Pune'
ORDER BY day;
```

Output:

```
    day     | amount | first_amt | last_default_frame | last_whole_partition | second_amt
------------+--------+-----------+--------------------+----------------------+------------
 2025-03-01 | 400.00 |    400.00 |             400.00 |               300.00 |     650.00
 2025-03-02 | 650.00 |    400.00 |             650.00 |               300.00 |     650.00
 2025-03-03 | 500.00 |    400.00 |             500.00 |               300.00 |     650.00
 ...
 2025-03-07 | 300.00 |    400.00 |             300.00 |               300.00 |     650.00
(6 rows)
```

Fix: extend the frame to `UNBOUNDED FOLLOWING`, or reverse the order with `first_value(amount) OVER (ORDER BY day DESC)` (no frame needed). `first_value(first_name) OVER (PARTITION BY dept_id ORDER BY salary DESC)` puts each department's top earner beside every employee ("gap to the leader").

Time between events — days between a customer's orders (a retention metric: how often customers come back):

```
SELECT customer_id, order_id, placed_at::date AS placed,
       placed_at::date - lag(placed_at::date) OVER (PARTITION BY customer_id ORDER BY placed_at) AS
days_since_prev
FROM orders
WHERE customer_id IN (1, 2, 5)
ORDER BY customer_id, placed_at;
```

Output:

```
 customer_id | order_id |   placed   | days_since_prev
-------------+----------+------------+-----------------
           1 |      101 | 2025-01-03 |          (null)
           1 |      104 | 2025-02-11 |              39
           1 |      113 | 2025-06-02 |             111
           2 |      102 | 2025-01-17 |          (null)
           2 |      108 | 2025-04-01 |              74
 ...
(7 rows)
```

Subtracting two `date`s gives an integer number of days in PostgreSQL.

Dialect notes

- `IGNORE NULLS` (`lag(x) IGNORE NULLS OVER (...)`, "last non-null value") is standard and supported by Oracle, SQL Server 2022, BigQuery and Snowflake (cloud data warehouses). PostgreSQL gains it in **version 19** (in beta as of September 2026); PostgreSQL 14–18 and MySQL lack it — emulate with a running `count(x)` as group key plus `first_value` per group, or a correlated subquery.
- `nth_value(...) FROM LAST` (counting from the end of the frame) exists in Oracle and a few warehouses, not in PostgreSQL, MySQL or SQL Server.

Common mistakes

- `last_value` without a full frame; `lag` without `PARTITION BY` (crosses groups); `lag` treated as "previous period" when periods can be missing.

- The first row's `amount - lag(amount)` is `NULL`; pass a `default` or use `coalesce` if you want 0.

In the interview

> **30-second answer:** "`LAG` and `LEAD` read a column from a previous or next row in the window's order, per partition, which replaces a self-join on 'row n−1'. `FIRST_VALUE`, `LAST_VALUE` and `NTH_VALUE` read from positions in the frame — and because the default frame ends at the current row, `LAST_VALUE` needs `ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING`."

- *Year-over-year growth per product?* → `lag(revenue) OVER (PARTITION BY product ORDER BY year)`, then `(revenue - prev) / prev`.
- *How did people do this before window functions?* → A self-join on `t2.day = t1.day - 1` (breaks with gaps) or a correlated subquery `(SELECT ... WHERE t2.day < t1.day ORDER BY day DESC LIMIT 1)`.

Self-check

1. *`lag(x, 1, 0)` on the first row?* → 0.
2. *Which ignore the frame: `lag`, `first_value`, `lead`, `nth_value`?* → `lag` and `lead`.

8.7 Running totals, moving averages and growth

What it is

Everyday analytics = aggregate windows with the right frame:

Metric	Window
Running total	`sum(x) OVER (PARTITION BY g ORDER BY t ROWS UNBOUNDED PRECEDING)`
Moving average of last 3 rows	`avg(x) OVER (PARTITION BY g ORDER BY t ROWS BETWEEN 2 PRECEDING AND CURRENT ROW)`
Cumulative share	running total / `sum(x) OVER (PARTITION BY g)`
Share of total	`x / sum(x) OVER (PARTITION BY g)`
Growth vs previous period	`x / lag(x) OVER (ORDER BY t) - 1`

```sql
-- code/06_running_and_moving.sql
SELECT store, day, amount,
       sum(amount) OVER (PARTITION BY store ORDER BY day
                         ROWS UNBOUNDED PRECEDING)                        AS running_total,
       round(avg(amount) OVER (PARTITION BY store ORDER BY day
                         ROWS BETWEEN 2 PRECEDING AND CURRENT ROW), 1)    AS moving_avg_3,
       count(*) OVER (PARTITION BY store ORDER BY day
                         ROWS BETWEEN 2 PRECEDING AND CURRENT ROW)        AS rows_in_avg,
       round(100.0 * sum(amount) OVER (PARTITION BY store ORDER BY day ROWS UNBOUNDED PRECEDING)
                   / sum(amount) OVER (PARTITION BY store), 1)            AS pct_of_week_so_far
FROM daily_sales
ORDER BY store, day;
```

Output:

```
 store  |    day     | amount  | running_total | moving_avg_3 | rows_in_avg | pct_of_week_so_far
--------+------------+---------+---------------+--------------+-------------+--------------------
 Mumbai | 2025-03-01 | 1200.00 |       1200.00 |       1200.0 |           1 |               13.8
 Mumbai | 2025-03-02 |  900.00 |       2100.00 |       1050.0 |           2 |               24.1
 Mumbai | 2025-03-03 | 1500.00 |       3600.00 |       1200.0 |           3 |               41.4
 ...
 Mumbai | 2025-03-07 | 1800.00 |       8700.00 |       1200.0 |           3 |              100.0
 ...
 Pune   | 2025-03-03 |  500.00 |       1550.00 |        516.7 |           3 |               47.0
 Pune   | 2025-03-05 |  800.00 |       2350.00 |        650.0 |           3 |               71.2
 ...
 Pune   | 2025-03-07 |  300.00 |       3300.00 |        583.3 |           3 |              100.0
(13 rows)
```

The first two moving averages use only 1 and 2 rows (`rows_in_avg`) — a "warm-up" that can mislead on a chart; hide it with `CASE WHEN count(*) OVER w = 3 THEN avg(amount) OVER w END` (in the code file). Pune's 5 March average uses 2, 3, 5 March because the frame is `ROWS` (8.4 for `RANGE`).

Windows on top of GROUP BY

Windows run **after** `GROUP BY`, so they work on grouped rows — monthly revenue plus its running total in one query: the inner `sum` is per month, the outer `sum(...) OVER` adds those up:

```
SELECT date_trunc('month', o.placed_at)::date AS month,
       sum(oi.quantity * oi.unit_price * (1 - oi.discount))::numeric(10,2) AS revenue,
       sum(sum(oi.quantity * oi.unit_price * (1 - oi.discount)))
           OVER (ORDER BY date_trunc('month', o.placed_at)::date)::numeric(10,2) AS running_revenue
FROM orders o JOIN order_items oi USING (order_id)
WHERE o.status <> 'cancelled'
GROUP BY 1
ORDER BY 1;
```

Output:

```
   month    | revenue | running_revenue
------------+---------+-----------------
 2025-01-01 | 1871.90 |         1871.90
 2025-02-01 |  811.49 |         2683.39
 ...
 2025-06-01 | 1516.60 |         7403.62
(6 rows)
```

`sum(sum(...))` looks odd but is correct; often clearer is aggregating in a CTE (named subquery written with `WITH`, chapter 7) and windowing outside — here **month-over-month growth**:

```
WITH monthly AS (
    SELECT date_trunc('month', o.placed_at)::date AS month,
           sum(oi.quantity * oi.unit_price * (1 - oi.discount)) AS revenue
    FROM orders o JOIN order_items oi USING (order_id)
    WHERE o.status <> 'cancelled'
    GROUP BY 1
)
SELECT month, round(revenue, 2) AS revenue,
       round(lag(revenue) OVER (ORDER BY month), 2) AS prev_month,
       round(100.0 * (revenue / lag(revenue) OVER (ORDER BY month) - 1), 1) AS mom_growth_pct
FROM monthly
ORDER BY month;
```

Output:

```
   month    | revenue | prev_month | mom_growth_pct
------------+---------+------------+----------------
 2025-01-01 | 1871.90 |     (null) |         (null)
 2025-02-01 |  811.49 |    1871.90 |          -56.6
 2025-03-01 |  804.95 |     811.49 |           -0.8
 ...
 2025-06-01 | 1516.60 |     249.93 |          506.8
(6 rows)
```

If a month can have *no* orders, generate the months first (chapter 7) or `lag` compares with two months back. Guard with `NULLIF(prev, 0)` if the previous value can be zero.

Real-world examples

Account balance after each transaction (running `sum` ordered by time and id), 7-day moving average of daily active users, stock level as a running sum of `+received −shipped`.

8.8 Top-N per group and de-duplication

The problem

Top-N-per-group ("top 2 earners per department", "latest order per customer", "best seller per category") is the most common window interview problem. `LIMIT` limits the whole result, not each group; `GROUP BY` returns only the aggregate, not the rest of the row.

Why you need a subquery or CTE

Windows are computed *after* `WHERE`, so you can't filter on them at the same level:

```sql
-- code/07_top_n_per_group.sql
SELECT first_name, salary FROM employees
WHERE row_number() OVER (ORDER BY salary DESC) <= 3;
```

Output:

```
ERROR:  window functions are not allowed in WHERE
LINE 2: WHERE row_number() OVER (ORDER BY salary DESC) <= 3;
              ^
```

(Nor in `GROUP BY` or `HAVING`.) The pattern: **rank in a CTE or subquery, filter in the outer query.**

How it works

In a CTE add `row_number()` / `rank()` / `dense_rank()` `OVER (PARTITION BY group ORDER BY measure DESC)`; outside, `WHERE rank_col <= N`; choose the function by how ties should behave.

```sql
WITH ranked AS (
    SELECT d.name AS dept, e.first_name, e.salary,
           row_number() OVER (PARTITION BY e.dept_id ORDER BY e.salary DESC, e.emp_id) AS rn,
           rank()       OVER (PARTITION BY e.dept_id ORDER BY e.salary DESC)           AS rnk,
           dense_rank() OVER (PARTITION BY e.dept_id ORDER BY e.salary DESC)           AS drnk
    FROM employees e JOIN departments d USING (dept_id)
)
SELECT dept, first_name, salary, rn, rnk, drnk
FROM ranked
WHERE dept IN ('Engineering', 'Sales') AND rnk <= 3
ORDER BY dept, rn;
```

Output:

```
    dept     | first_name |  salary   | rn | rnk | drnk
-------------+------------+-----------+----+-----+------
 Engineering | Vikram     | 185000.00 |  1 |   1 |    1
 ...
 Engineering | Rahul      |  98000.00 |  3 |   3 |    3
 Engineering | Meera      |  98000.00 |  4 |   3 |    3
 Sales       | Daniel     | 150000.00 |  1 |   1 |    1
 Sales       | Priya      |  72000.00 |  2 |   2 |    2
 Sales       | Arjun      |  68000.00 |  3 |   3 |    3
 Sales       | Sara       |  68000.00 |  4 |   3 |    3
(8 rows)
```

With ties the three filters differ — Sales:

```sql
-- ranked = Sales only; then:
SELECT 'row_number <= 3' AS rule, string_agg(first_name, ', ' ORDER BY rn) AS kept FROM ranked WHERE rn <= 3
UNION ALL
SELECT 'rank <= 3',       string_agg(first_name, ', ' ORDER BY rn) FROM ranked WHERE rnk <= 3
UNION ALL
SELECT 'dense_rank <= 2', string_agg(first_name, ', ' ORDER BY rn) FROM ranked WHERE drnk <= 2;
```

Output:

```
      rule       |            kept
-----------------+----------------------------
 row_number <= 3 | Daniel, Priya, Arjun
 rank <= 3       | Daniel, Priya, Arjun, Sara
 dense_rank <= 2 | Daniel, Priya
(3 rows)
```

Filter	Meaning	Rows per group
`row_number() <= N`	exactly N rows; ties broken by your tie-breaker	exactly N (or fewer)
`rank() <= N`	everyone whose position is within the top N; ties included	can exceed N
`dense_rank() <= N`	everyone having one of the top N *distinct* values	can far exceed N

Ask in an interview: "If two people tie for 3rd, should both be returned?"

Latest row per key (de-duplication) — `N = 1`, newest first:

```sql
SELECT customer_id, order_id, placed_at, status
FROM (
    SELECT o.*, row_number() OVER (PARTITION BY customer_id ORDER BY placed_at DESC) AS rn
    FROM orders o
) t
WHERE rn = 1
ORDER BY customer_id;
```

Output:

```
 customer_id | order_id |      placed_at      |  status
-------------+----------+---------------------+-----------
           1 |      113 | 2025-06-02 10:30:00 | pending
           2 |      108 | 2025-04-01 12:00:00 | delivered
           3 |      111 | 2025-05-06 07:55:00 | cancelled
 ...
           8 |      109 | 2025-04-09 16:45:00 | delivered
(8 rows)
```

Duplicates to delete — number rows within each duplicate group; `rn > 1` are extra copies. The duplicate Anita Shah (messy email) after normalising:

```sql
WITH dups AS (
    SELECT customer_id,
           row_number() OVER (PARTITION BY lower(trim(email)) ORDER BY customer_id) AS rn
    FROM customers
    WHERE email IS NOT NULL
)
SELECT customer_id AS would_delete FROM dups WHERE rn > 1;
```

Output:

```
 would_delete
--------------
            8
(1 row)
```

Delete with `DELETE FROM customers WHERE customer_id IN (SELECT customer_id FROM dups WHERE rn > 1)` after re-pointing that customer's orders (chapter 9: `DELETE` and foreign keys — columns that must match a key in another table).

Alternatives

Approach	Top-1	Top-N	Notes
Window + filter (above)	yes	yes	portable, flexible about ties
`DISTINCT ON (group) ... ORDER BY group, measure DESC`	yes	no	PostgreSQL only, concise
`LATERAL (... ORDER BY ... LIMIT N)` per group (ch. 6)	yes	yes	fastest when an index (sorted lookup structure) on (group, measure) exists and groups are few
Correlated subquery `WHERE salary = (SELECT max(...) ...)`	yes (with ties)	awkward	classic pre-window approach
`QUALIFY rn <= N`	yes	yes	filter on windows without a subquery — Snowflake, BigQuery, DuckDB, Teradata; **not** PostgreSQL / MySQL / SQL Server

Top-1 shortcut: `SELECT DISTINCT ON (dept_id) dept_id, first_name, salary FROM employees WHERE dept_id IS NOT NULL ORDER BY dept_id, salary DESC, emp_id`.

In the interview

30-second answer: "Rank rows inside each group with a window function in a CTE — `PARTITION BY` the group, `ORDER BY` the measure descending — then filter `rank <= N` in the outer query, because window functions can't appear in `WHERE`. I'd choose `row_number`, `rank` or `dense_rank` depending on whether ties should be included, and add a tie-breaker for determinism."

- *Top 3 salaries per department with ties?* → `dense_rank() <= 3` (three distinct salary levels) or `rank() <= 3` (three places).
- *How to make it fast on a big table?* → Index `(dept_id, salary DESC)`; PostgreSQL can then read each partition pre-sorted; for few groups a `LATERAL ... LIMIT N` uses the index directly.

Self-check

1. *Keep only the newest row per `user_id`?* → `row_number() OVER (PARTITION BY user_id ORDER BY created_at DESC, id DESC) = 1` in a subquery.

8.9 Gaps and islands

What it is

An **island** is a run of consecutive values (login days, free seat numbers, readings above a threshold); a **gap** is a missing stretch between islands. Longest streak, missing invoice numbers and sessionizing events are all gaps-and-islands problems.

How it works — the row-number trick

In a run of consecutive dates both the date and `row_number()` rise by 1 per row, so their **difference is constant within a run** and changes at a skipped day — it labels the island. Compute `rn = row_number() OVER (PARTITION BY user_id ORDER BY login_date)`, `grp = login_date - rn` (date minus integer is a date), then `GROUP BY user_id, grp` with `min` / `max` / `count`.

```sql
-- code/08_gaps_and_islands.sql
SELECT user_id, login_date,
       row_number() OVER (PARTITION BY user_id ORDER BY login_date) AS rn,
       login_date - row_number() OVER (PARTITION BY user_id ORDER BY login_date)::int AS grp
FROM logins
WHERE user_id = 1
ORDER BY login_date;
```

Output:

```
 user_id | login_date | rn |    grp
---------+------------+----+------------
       1 | 2025-05-01 |  1 | 2025-04-30
       1 | 2025-05-02 |  2 | 2025-04-30
       1 | 2025-05-03 |  3 | 2025-04-30
       1 | 2025-05-06 |  4 | 2025-05-02
       1 | 2025-05-07 |  5 | 2025-05-02
       1 | 2025-05-10 |  6 | 2025-05-04
(6 rows)
```

`grp` has no meaning beyond being a shared label. (`::int` because `row_number()` is `bigint` and PostgreSQL defines `date - integer`, not `date - bigint`.)

Each island (streak):

```sql
WITH marked AS (
    SELECT user_id, login_date,
           login_date - row_number() OVER (PARTITION BY user_id ORDER BY login_date)::int AS grp
    FROM logins
)
SELECT user_id, min(login_date) AS streak_start, max(login_date) AS streak_end,
       count(*) AS days
FROM marked
GROUP BY user_id, grp
ORDER BY user_id, streak_start;
```

Output:

```
 user_id | streak_start | streak_end | days
---------+--------------+------------+------
       1 | 2025-05-01   | 2025-05-03 |    3
       1 | 2025-05-06   | 2025-05-07 |    2
       1 | 2025-05-10   | 2025-05-10 |    1
       2 | 2025-05-02   | 2025-05-05 |    4
       2 | 2025-05-09   | 2025-05-10 |    2
(5 rows)
```

Longest streak per user — one more step (`DISTINCT ON` keeps the top row per user):

```
WITH marked AS (...same as above...),
streaks AS (
    SELECT user_id, grp, count(*) AS days, min(login_date) AS from_day
    FROM marked GROUP BY user_id, grp
)
SELECT DISTINCT ON (user_id) user_id, days AS longest_streak, from_day
FROM streaks
ORDER BY user_id, days DESC, from_day;
```

Output:

```
 user_id | longest_streak |  from_day
---------+----------------+------------
       1 |              3 | 2025-05-01
       2 |              4 | 2025-05-02
(2 rows)
```

Duplicate dates per user break the trick — de-duplicate first or use `dense_rank`.

Gaps with `lead` — a difference above 1 to the next row is a gap:

```
SELECT user_id,
       login_date + 1              AS gap_start,
       next_login - 1              AS gap_end,
       next_login - login_date - 1 AS missing_days
FROM (
    SELECT user_id, login_date,
           lead(login_date) OVER (PARTITION BY user_id ORDER BY login_date) AS next_login
    FROM logins
) t
WHERE next_login - login_date > 1
ORDER BY user_id, gap_start;
```

Output:

```
 user_id | gap_start  |  gap_end   | missing_days
---------+------------+------------+--------------
       1 | 2025-05-04 | 2025-05-05 |            2
       1 | 2025-05-08 | 2025-05-09 |            2
       2 | 2025-05-06 | 2025-05-08 |            3
(3 rows)
```

The same query with integer ids finds missing invoice or ticket numbers.

The general method: flag and running sum

The row-number trick needs "consecutive" = "+1". For any other rule (events < 30 minutes apart, above a threshold, same status as previous): (1) **flag** each island's first row with `lag` (`CASE WHEN <does NOT continue the previous row> THEN 1 ELSE 0 END`); (2) a **running sum** of flags numbers the islands 1, 2, 3, ….

Sessionization — a new session after more than 30 minutes idle:

```
WITH ev(user_id, at) AS (VALUES
    (7, TIMESTAMP '2025-05-01 09:00'), (7, '2025-05-01 09:10'), (7, '2025-05-01 09:25'),
    (7, '2025-05-01 11:00'), (7, '2025-05-01 11:05'),
    (7, '2025-05-01 15:30')
), flagged AS (
    SELECT user_id, at,
           CASE WHEN at - lag(at) OVER (PARTITION BY user_id ORDER BY at) <= INTERVAL '30 minutes'
                THEN 0 ELSE 1 END AS new_session
    FROM ev
)
SELECT user_id, at, new_session,
       sum(new_session) OVER (PARTITION BY user_id ORDER BY at) AS session_no
FROM flagged
ORDER BY at;
```

Output:

```
 user_id |         at          | new_session | session_no
---------+---------------------+-------------+------------
       7 | 2025-05-01 09:00:00 |           1 |          1
       7 | 2025-05-01 09:10:00 |           0 |          1
       7 | 2025-05-01 09:25:00 |           0 |          1
       7 | 2025-05-01 11:00:00 |           1 |          2
       7 | 2025-05-01 11:05:00 |           0 |          2
       7 | 2025-05-01 15:30:00 |           1 |          3
(6 rows)
```

The first event's `lag` is `NULL`, so the comparison isn't true and `ELSE 1` starts a session, as intended. Group by `(user_id, session_no)` for start, end, length. (The `VALUES` CTE makes inline rows; the first `TIMESTAMP` literal sets the type.)

Real-world examples

App streaks and contribution graphs, web-analytics sessions (30 minutes idle is a common cut-off), periods of CPU load above 90 %, missing cheque or invoice numbers.

In the interview

30-second answer: "For consecutive dates or integers, subtract `row_number()` from the value; the difference is constant within each run, so grouping by it gives the islands with `min`, `max`, `count`. For general rules, flag each row that starts a new island using `lag`, then a running `sum` of the flags gives an island id. Gaps come from `lead(value) - value > 1`."

- *Users who logged in 3+ consecutive days?* → Islands query, then `HAVING count(*) >= 3`, select distinct users.
- *What if a user logs in twice on the same day?* → De-duplicate to one row per day first, or use `dense_rank`.

Self-check

1. *Missing numbers between consecutive ids?* → `lead(id) - id > 1` → gap from `id + 1` to `lead(id) - 1`.

8.10 Named windows, FILTER, evaluation order and performance

The WINDOW clause

Define a shared window once with `WINDOW name AS (...)` (after `HAVING`, before `ORDER BY`) and write `OVER name`. `OVER (w ROWS 2 PRECEDING)` adds a frame to named window `w` (which must have no frame of its own).

```sql
-- code/09_named_windows_filter_plan.sql
SELECT store, day, amount,
       sum(amount) OVER w_run    AS running_total,
       max(amount) OVER w_run    AS best_so_far,
       rank()      OVER w_amount AS amount_rank
FROM daily_sales
WHERE store = 'Pune'
WINDOW w_run    AS (PARTITION BY store ORDER BY day),
       w_amount AS (PARTITION BY store ORDER BY amount DESC)
ORDER BY day;
```

Output:

```
 store |    day     | amount | running_total | best_so_far | amount_rank
-------+------------+--------+---------------+-------------+-------------
 Pune  | 2025-03-01 | 400.00 |        400.00 |      400.00 |           5
 Pune  | 2025-03-02 | 650.00 |       1050.00 |      650.00 |           2
 ...
 Pune  | 2025-03-07 | 300.00 |       3300.00 |      800.00 |           6
(6 rows)
```

`max(...) OVER w_run` is a running maximum ("best day so far"). `WINDOW` is supported by PostgreSQL, MySQL 8, SQLite and SQL Server 2022 (not earlier).

FILTER on window aggregates

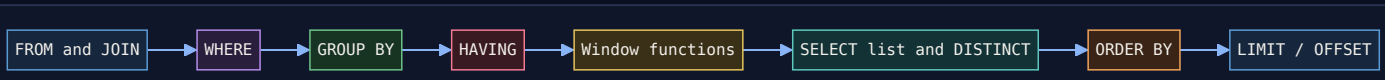
`FILTER (WHERE ...)` (chapter 4) works on aggregate windows too (PostgreSQL, SQLite; elsewhere `count(CASE WHEN cond THEN 1 END) OVER (...)`):

```sql
SELECT day, amount,
       count(*) FILTER (WHERE amount >= 1500) OVER (ORDER BY day) AS big_days_so_far
FROM daily_sales
WHERE store = 'Mumbai'
ORDER BY day;
```

Output:

```
    day     | amount  | big_days_so_far
------------+---------+-----------------
 2025-03-01 | 1200.00 |               0
 2025-03-02 |  900.00 |               0
 2025-03-03 | 1500.00 |               1
 2025-03-04 | 1500.00 |               2
 ...
 2025-03-07 | 1800.00 |               3
(7 rows)
```

Where windows fit in the logical order



Consequences:

- **`WHERE` changes the window**: with `WHERE dept_id = 1`, `rank() OVER (ORDER BY salary DESC)` ranks only Engineering (Vikram 1, Neha 2, Rahul and Meera 3). For company-wide ranks shown for Engineering, rank in a subquery **without** the filter, filter outside.
- Windows **can't** be used in `WHERE`, `GROUP BY` or `HAVING` (they don't exist yet) → wrap in a subquery/CTE (8.8).
- Windows **can** be used in `ORDER BY` (`ORDER BY rank() OVER (...)`) and can take aggregates as arguments in grouped queries (8.7).
- `DISTINCT` runs after windows, so `SELECT DISTINCT dept_id, count(*) OVER (PARTITION BY dept_id)` works (one row per department).
- `LIMIT` runs last, so `row_number() ... LIMIT 5` numbers *all* rows first.

Performance

PostgreSQL runs windows in a **WindowAgg** node (a step of the plan `EXPLAIN` prints) reading rows sorted by `PARTITION BY` + `ORDER BY` keys:

```
EXPLAIN (COSTS OFF)
SELECT store, day, sum(amount) OVER (PARTITION BY store ORDER BY day)
FROM daily_sales;

EXPLAIN (COSTS OFF)
SELECT first_name, rank() OVER (PARTITION BY dept_id ORDER BY salary DESC)
FROM employees;
```

Output:

```
                       QUERY PLAN
-------------------------------------------------------
 WindowAgg
   ->  Index Scan using daily_sales_pkey on daily_sales
(2 rows)

               QUERY PLAN
---------------------------------------
 WindowAgg
   ->  Sort
         Sort Key: dept_id, salary DESC
         ->  Seq Scan on employees
(4 rows)
```

- In the first plan the primary key `(store, day)` (uniquely identifying each row, backed by an index) already delivers the needed order — **no sort**. The second must sort by `dept_id, salary DESC`.
- **Index tip:** an index on `(partition columns, order columns)` can remove the sort — e.g. `CREATE INDEX ON employees (dept_id, salary DESC)` (chapter 12).
- **Different** `PARTITION BY` / `ORDER BY` need separate sorts; identical definitions share one pass — reuse named windows.
- Sliding `sum` / `count` / `avg` frames are computed incrementally (adjusting the previous result), but `min` / `max` over a moving frame (and `EXCLUDE` frames) may re-scan the frame — expensive for large frames.
- A window over millions of rows sorts them all; filter in `WHERE` first when logic allows.

Chapter 8 TL;DR

- **Window function** = `f(...) OVER (PARTITION BY … ORDER BY … frame)` : computed per row over related rows; **row count never changes**.
- `PARTITION BY` = groups without collapsing → share of total, difference from group average.
- `ORDER BY` **in** `OVER` makes aggregates running; the default frame `RANGE UNBOUNDED PRECEDING … CURRENT ROW` includes ties — use a unique order or `ROWS`.
- **Frames**: `ROWS` (physical rows), `RANGE` (value distance, intervals, ties included), `GROUPS` (peer groups), `EXCLUDE` — PostgreSQL 11+ for offsets/ `GROUPS` / `EXCLUDE` .
- **Ranking**: `row_number` (unique), `rank` (gaps), `dense_rank` (no gaps), `ntile` , `percent_rank` , `cume_dist` . Always decide how ties behave.
- **Value functions**: `lag` / `lead` (ignore frame), `first_value` / `last_value` / `nth_value` (use frame — give `last_value` a full frame). `IGNORE NULLS` only from PostgreSQL 19.
- **Patterns**: running total, moving average, growth vs `lag` , top-N per group (rank in CTE, filter outside), dedup (`row_number() = 1`), gaps & islands (`value − row_number()` , or flag + running sum).
- **Order of evaluation**: after `WHERE` / `GROUP BY` / `HAVING` , before `DISTINCT` / `ORDER BY` / `LIMIT` . No `QUALIFY` in PostgreSQL.
- **Performance**: one sort per distinct window; an index on (partition, order) columns can remove it.

Next: chapter 9 — DDL, DML and constraints: creating and changing tables, keys and constraints, and inserting, updating and deleting data safely.

Chapter 9 — DDL, DML and Constraints: Building Tables and Changing Data Safely

This chapter creates structures and changes data: **DDL — Data Definition Language** (`CREATE` , `ALTER` , `DROP` , `TRUNCATE` , `CREATE INDEX`); **keys and constraints** (primary and foreign keys, `UNIQUE` , `CHECK` , `NOT NULL` , `DEFAULT`), enforced on every change whichever application writes; **DML — Data Manipulation Language** (`INSERT` , `UPDATE` , `DELETE` , PostgreSQL's `ON CONFLICT` upsert and `RETURNING`); and **backup and restore**, run for real with `pg_dump` / `pg_restore` (command-line programs that write a database to a file and load it back). Table design (entities, relationships, normal forms — rules for keeping each fact in one place) is chapter 10 and chapter 11.

Sections: 9.1 Data types · 9.2 `CREATE` · 9.3 Keys and constraints · 9.4 Generated keys · 9.5 `ALTER` · 9.6 `DROP` and `TRUNCATE` · 9.7 Indexes (brief) · 9.8 `INSERT` · 9.9 `UPDATE` · 9.10 `DELETE` · 9.11 Upsert and `MERGE` · 9.12 Backup and restore.

Setup. The `company` database (chapter 1, 1.11); each file in `code/` starts from a fresh copy. Each `ERROR:` shown is the database deliberately refusing something.

9.1 Data types

What it is

A column's **data type** decides storage, valid operations, sort order and — just as important — which values are **rejected**: the first and cheapest constraint.

The common types

Kind	PostgreSQL type	Holds	Use for	MySQL / SQL Server notes
Integer	`smallint` (2 bytes)	−32 768 … 32 767	small ranges: ranks, ratings	same names
	`integer` / `int` (4 bytes)	about ±2.1 billion	counts, most ids	same
	`bigint` (8 bytes)	about ±9.2 × 10^{18}	large ids, populations, byte counts	same
Exact decimal	`numeric(p, s)` / `decimal(p, s)`	*p* digits in total, *s* after the point	**money**, anything that must add up exactly	same
Approximate	`real`, `double precision`	binary floating point (fast but approximate)	measurements, science, ML (machine-learning) features	`FLOAT` / `DOUBLE`; SQL Server `float`
Fixed text	`char(n)`	exactly *n* characters, padded with spaces	fixed codes (rarely worth it)	same
Variable text	`varchar(n)`	up to *n* characters	text with a real maximum length	SQL Server: `nvarchar(n)` for Unicode (text in any alphabet)
	`text`	any length	most text in PostgreSQL	MySQL `TEXT` has limits and index restrictions
Date / time	`date`	a calendar day	birthdays, due dates	same
	`time`	time of day	opening hours	same
	`timestamp`	date + time, **no** time zone	local "wall clock" times	MySQL `DATETIME`
	`timestamptz`	an exact instant (stored in UTC — the world's reference clock — shown in the session's time zone; a *session* is your current connection)	event times, created_at	MySQL `TIMESTAMP` (converted to UTC); SQL Server `datetimeoffset`
Year only	(none) — use `smallint` + `CHECK`, or `date`			MySQL has `YEAR`
Boolean	`boolean`	true / false / NULL	flags	MySQL `BOOLEAN` = `TINYINT(1)`; SQL Server `bit`
Other useful	`uuid`, `jsonb`, arrays, `inet`, ranges		see chapter 14	

Conversions, casts and per-type functions: chapter 3.

Seeing the rules in action:

```sql
-- code/01_data_types.sql
SET TimeZone = 'UTC';   -- make timestamptz output the same on every machine
CREATE TABLE type_demo (
    qty        integer,
    views      bigint,
    rank_no    smallint,
    price      numeric(10,2),
    ratio      double precision,
    code       char(5),
    name       varchar(10),
    notes      text,
    born_on    date,
    starts_at  time,
    created    timestamp,
    created_tz timestamptz,
    active     boolean,
    birth_year smallint CHECK (birth_year BETWEEN 1900 AND 2100)  -- no YEAR type in PostgreSQL
);
INSERT INTO type_demo VALUES
 (3, 9000000000, 7, 19.999, 0.1::float8 + 0.2::float8, 'AB', 'Tara', 'long text ...',
  '1994-09-22', '09:30', '2025-06-01 10:00', '2025-06-01 10:00+05:30', true, 1994);

SELECT price, ratio, '[' || code || ']' AS code_padded, length(code) AS len_code,
       octet_length(code) AS bytes_code, created, created_tz
FROM type_demo;
```

Output:

```
 price |        ratio        | code_padded | len_code | bytes_code |       created       |       created_tz
-------+---------------------+-------------+----------+------------+---------------------+------------------------
 20.00 | 0.30000000000000004 | [AB]        |        2 |          5 | 2025-06-01 10:00:00 | 2025-06-01 04:30:00+00
(1 row)
```

- `price` 19.999 was **rounded** to 20.00.
- `ratio`: binary 0.1 + 0.2 isn't exactly 0.3 (`0.1::float8` is a **cast** — conversion — to an 8-byte float; `0.1::numeric + 0.2::numeric = 0.3` is true) — why money is never `float`.
- `code` is padded to 5 characters (5 bytes) but `length` ignores a `char`'s trailing spaces and says 2 — confusing; prefer `text` / `varchar`.
- `created_tz`: 10:00 in India (+05:30) became an instant shown in UTC as 04:30. **What the types reject:**

```sql
INSERT INTO type_demo (name) VALUES ('Christopher Columbus');
INSERT INTO type_demo (rank_no) VALUES (40000);
INSERT INTO type_demo (price) VALUES (123456789.00);
INSERT INTO type_demo (born_on) VALUES ('2025-02-30');
INSERT INTO type_demo (birth_year) VALUES (1850);
```

Output:

```
ERROR:  value too long for type character varying(10)
ERROR:  smallint out of range
ERROR:  numeric field overflow
DETAIL:  A field with precision 10, scale 2 must round to an absolute value less than 10^8.
ERROR:  date/time field value out of range: "2025-02-30"
ERROR:  new row for relation "type_demo" violates check constraint "type_demo_birth_year_check"
...
```

`numeric(10,2)` allows 10 digits, 2 after the point, so $< 10^8$. MySQL in non-strict mode (errors become warnings) would silently truncate some of these; PostgreSQL always errors.

How to choose

1. **Money** → `numeric(p, s)` (or integer cents), never `float`. **Ids/counts** → `integer`, or `bigint` if it could ever pass 2 billion.
2. **Text** → `text` in PostgreSQL; `varchar(n)` only when the maximum is a real business rule (no speed difference). SQL Server: `nvarchar` for Unicode.

3. **Instants** → `timestamptz`; `timestamp` only for wall-clock times that must not shift (a shop's opening time). **Days** → `date`.
4. Pick the smallest type that safely fits — changing it later on a big table is expensive (9.5).

Common mistakes

Money in `float` (rounding errors); dates as `text` (`'03/04/2025'` — March or April? can't sort); `varchar(255)` everywhere by habit (a meaningless limit); `timestamp` for events across time zones (ambiguous instants).

9.2 CREATE: databases, schemas and tables

What it is

`CREATE` makes new objects:

Statement	Creates
`CREATE DATABASE name;`	a new, separate database on the server (the running database program)
`CREATE SCHEMA name;`	a namespace **inside** a database (like a folder); tables are then `schema.table`
`CREATE TABLE name (...);`	a table: columns, types, constraints
`CREATE TABLE name AS SELECT ...;`	a table filled with a query's result
`CREATE TABLE name (LIKE other INCLUDING ALL);`	an empty copy of another table's structure
`CREATE TEMP TABLE name (...);`	a table visible only to your session, dropped automatically at the end
`CREATE VIEW`, `CREATE INDEX`, `CREATE SEQUENCE`, `CREATE FUNCTION` …	covered in chapters 7, 9.7, 9.4, 14

How CREATE TABLE works

```
CREATE TABLE [IF NOT EXISTS] [schema.]table_name (
    column_name  data_type  [column constraints],   -- one line per column
    ...,
    [table constraints]                             -- e.g. PRIMARY KEY (a, b)
);
```

(Square brackets mark optional parts.) The name must be unique in its schema (default `public`); each column has a name, type, optional constraints (9.3) and `DEFAULT`; multi-column constraints follow the columns. `IF NOT EXISTS` turns "already exists" into a notice (not an error) — handy in scripts, but it does **not** check the existing table's columns.

```sql
-- code/02_create.sql
CREATE DATABASE school_demo;          -- a new database (then dropped again in the script)
CREATE SCHEMA school;

CREATE TABLE school.students (
    student_id   integer      PRIMARY KEY,
    first_name   varchar(50)  NOT NULL,
    last_name    varchar(50)  NOT NULL,
    roll_number  integer      UNIQUE,
    email        text,
    enrolled_on  date         NOT NULL DEFAULT current_date
);

CREATE TABLE IF NOT EXISTS school.students (student_id integer);

CREATE TABLE school.bad (
    id   integer PRIMARY KEY,
    name text,
);
```

Output:

```
NOTICE:  relation "students" already exists, skipping
ERROR:  syntax error at or near ")"
LINE 4: );
```

A trailing comma after the last column is a syntax error (PostgreSQL, MySQL, SQL Server).

Tables from queries and copies:

```sql
CREATE TABLE school.sales_staff AS
SELECT emp_id, first_name, salary FROM employees WHERE dept_id = 2;
SELECT * FROM school.sales_staff ORDER BY emp_id;
```

Output:

```
 emp_id | first_name |  salary
--------+------------+-----------
      6 | Daniel     | 150000.00
      7 | Priya      |  72000.00
 ...
(4 rows)
```

`CREATE TABLE … AS` copies names, types and data but **no constraints, defaults or indexes** (`SELECT … INTO new_table FROM …` does the same in PostgreSQL and SQL Server). For structure with constraints and defaults, use `LIKE`:

```sql
CREATE TABLE school.products_copy (LIKE products INCLUDING ALL);   -- 0 rows, same columns/constraints
```

Temporary tables (`CREATE TEMP TABLE scratch (id int, note text);`) live in a private per-session schema (the **catalog** — PostgreSQL's internal tables describing objects — shows a `pg_temp_N` schema and `relpersistence = 't'` instead of `'p'`) and vanish at session end, or transaction end with `ON COMMIT DROP`. Use them for multi-step work in one session (7.9).

> **Key term — transaction:** an all-or-nothing group of statements: `BEGIN`, then `COMMIT` (make permanent) or `ROLLBACK` (undo). Without `BEGIN`, each statement is its own transaction (chapter 13).

Dialect notes

	PostgreSQL	MySQL	SQL Server	SQLite
Database vs schema	database contains schemas	"schema" = "database" (synonyms)	database contains schemas (`dbo` default)	one file = one database
Switch database	reconnect (`\c db` in psql, PostgreSQL's command-line client)	`USE db;`	`USE db;`	open another file / `ATTACH`
`CREATE TABLE IF NOT EXISTS`	yes	yes	no (check `OBJECT_ID` first)	yes
Temporary table	`CREATE TEMP TABLE`	`CREATE TEMPORARY TABLE`	`CREATE TABLE #name`	`CREATE TEMP TABLE`
`CREATE DATABASE` inside a transaction	not allowed	implicit commit (silently ends the open transaction first)	not allowed	n/a

---

9.3 Keys and constraints

What they are

A **key** is a column (or set) whose values identify rows. A **constraint** is a rule checked on every write; a statement breaking it fails completely. Applications have bugs and there are usually several of them, but every write passes through the database — so the rules belong there.

The kinds of keys

For `members(member_id, email, phone, full_name)`:

Key	Definition	In `members`
Super key	any column set that is unique for every row (may include unnecessary columns)	`{member_id}`, `{member_id, full_name}`, `{email, phone}`, …
Candidate key	a *minimal* super key — drop any column and it stops being unique; a table can have several	`{member_id}`, `{email}`
Primary key (PK)	the candidate key chosen as main identifier: unique **and** `NOT NULL`, one per table	`member_id`
Alternate key	a candidate key that wasn't chosen as primary; enforced with `UNIQUE`	`email`
Unique key	columns declared `UNIQUE`: no duplicates, NULLs allowed (below); many per table	`email`, `phone`
Composite key	two or more columns; only the *combination* must be unique	`(club_id, member_id)` in `club_memberships`
Foreign key (FK)	column(s) whose values must match a key (usually the PK) of another (or the same) table	`club_memberships.member_id → members.member_id`
Natural vs surrogate	natural = a real-world value (email, ISBN); surrogate = a generated number/UUID (9.4)	`email` is natural, `member_id` is surrogate

Key theory (functional dependencies): chapter 10, chapter 11.

The constraints

Constraint	Rule	Written as
`NOT NULL`	a value is required	`col type NOT NULL`
`DEFAULT`	value used when an `INSERT` omits the column (not a check, but declared alongside)	`col type DEFAULT expr`
`UNIQUE`	no two rows share the value(s); NULLs don't count as equal	`col type UNIQUE` / `UNIQUE (a, b)`
`PRIMARY KEY`	`UNIQUE` + `NOT NULL`; one per table; creates a unique index (a lookup structure, 9.7)	`col type PRIMARY KEY` / `PRIMARY KEY (a, b)`
`CHECK`	a boolean expression must not be false	`CHECK (price >= 0)` / `CHECK (end_date >= start_date)`
`FOREIGN KEY`	value must exist in the referenced key (or be NULL)	`col type REFERENCES other(col)` / `FOREIGN KEY (a) REFERENCES other(x)`
`EXCLUDE` (PostgreSQL)	no two rows may "conflict" by an operator (e.g. overlapping bookings of one room)	see chapter 14

A **column constraint** sits on the column; a **table constraint** after the columns (required for multi-column rules). Name important ones (`CONSTRAINT name …`) — the name appears in errors and is needed to drop it; otherwise PostgreSQL generates names like `members_email_key`.

```sql
-- code/03_keys_and_constraints.sql
CREATE TABLE members (
    member_id  integer PRIMARY KEY,                       -- primary key
    email      text    NOT NULL UNIQUE,                   -- candidate key -> alternate key
    phone      text    UNIQUE,                            -- unique key, NULL allowed
    full_name  text    NOT NULL,
    plan       text    NOT NULL DEFAULT 'free'
                       CONSTRAINT plan_is_known CHECK (plan IN ('free', 'pro', 'team')),
    joined_on  date    NOT NULL DEFAULT current_date,
    left_on    date,
    CONSTRAINT left_after_joined CHECK (left_on IS NULL OR left_on >= joined_on)   -- table-level check
);

INSERT INTO members (member_id, email, phone, full_name, joined_on)
VALUES (1, 'ravi@mail.example', NULL, 'Ravi Kumar', '2025-01-10'),
       (2, 'lena@mail.example', NULL, 'Lena Ortiz', '2025-02-01');     -- two NULL phones: fine
SELECT member_id, email, phone, plan FROM members ORDER BY member_id;
```

Output:

```
 member_id |       email       | phone  | plan
-----------+-------------------+--------+------
         1 | ravi@mail.example | (null) | free
         2 | lena@mail.example | (null) | free
(2 rows)
```

`plan` was filled from its `DEFAULT`. Now each rule refusing bad data:

```sql
INSERT INTO members (member_id, email, full_name) VALUES (1, 'x@mail.example', 'Dup Id');
INSERT INTO members (member_id, email, full_name, joined_on) VALUES (NULL, 'y@mail.example', 'No Id', '2025-03-01');
INSERT INTO members (member_id, email, full_name) VALUES (3, 'ravi@mail.example', 'Dup');
INSERT INTO members (member_id, email, full_name, plan, joined_on) VALUES (4, 'z@mail.example', 'Z', 'gold', '2025-03-01');
INSERT INTO members (member_id, email, full_name, joined_on, left_on)
VALUES (5, 'w@mail.example', 'W', '2025-03-01', '2025-01-01');
```

Output:

```
ERROR:  duplicate key value violates unique constraint "members_pkey"
DETAIL:  Key (member_id)=(1) already exists.
ERROR:  null value in column "member_id" of relation "members" violates not-null constraint
...
ERROR:  duplicate key value violates unique constraint "members_email_key"
...
ERROR:  new row for relation "members" violates check constraint "plan_is_known"
...
ERROR:  new row for relation "members" violates check constraint "left_after_joined"
...
```

Two subtle rules:

- `CHECK` **passes when its expression is NULL** — it fails only on *false* (three-valued logic, chapter 2). Hence `left_on IS NULL` is allowed implicitly, and a required column also needs `NOT NULL`.
- `UNIQUE` **allows several NULLs** in PostgreSQL, MySQL, SQLite and Oracle (NULL ≠ NULL). SQL Server allows only **one** (use a filtered unique index — over rows matching a `WHERE` — for "unique when present"). PostgreSQL 15 added `UNIQUE NULLS NOT DISTINCT` (not run).

Foreign keys and referential actions

A foreign key keeps **referential integrity** (no child points to a missing parent). What happens when the *parent* is deleted or re-keyed is chosen with `ON DELETE` / `ON UPDATE`:

Action	When the referenced parent row is deleted (or its key updated)
`NO ACTION` (default)	error if children still reference it (checked at the end of the statement; can be deferred)
`RESTRICT`	error, checked immediately (can't be deferred)
`CASCADE`	delete (or update) the child rows too
`SET NULL`	set the child's FK column(s) to NULL
`SET DEFAULT`	set the child's FK column(s) to their default

```
CREATE TABLE clubs (
    club_id  integer PRIMARY KEY,
    name     text NOT NULL UNIQUE
);
CREATE TABLE club_memberships (
    club_id    integer REFERENCES clubs(club_id)     ON DELETE CASCADE,
    member_id  integer REFERENCES members(member_id) ON DELETE RESTRICT,
    role       text NOT NULL DEFAULT 'member',
    PRIMARY KEY (club_id, member_id)                                  -- composite key
);
CREATE TABLE club_events (
    event_id   integer PRIMARY KEY,
    club_id    integer REFERENCES clubs ON DELETE SET NULL,           -- "REFERENCES clubs" = its PK
    title      text NOT NULL
);
INSERT INTO clubs VALUES (10, 'Chess'), (20, 'Hiking');
INSERT INTO club_memberships (club_id, member_id) VALUES (10, 1), (10, 2), (20, 1);
INSERT INTO club_events VALUES (100, 10, 'Blitz night'), (101, 20, 'Sunrise trek');

INSERT INTO club_memberships (club_id, member_id) VALUES (10, 1);       -- composite PK duplicate
INSERT INTO club_memberships (club_id, member_id) VALUES (30, 1);       -- no club 30
DELETE FROM members WHERE member_id = 2;                                  -- RESTRICT
```

Output:

```
ERROR:  duplicate key value violates unique constraint "club_memberships_pkey"
...
ERROR:  insert or update on table "club_memberships" violates foreign key constraint
"club_memberships_club_id_fkey"
DETAIL:  Key (club_id)=(30) is not present in table "clubs".
ERROR:  update or delete on table "members" violates foreign key constraint "club_memberships_member_id_fkey" on
table "club_memberships"
DETAIL:  Key (member_id)=(2) is still referenced from table "club_memberships".
```

Deleting Chess triggers `CASCADE` (memberships go) and `SET NULL` (its event stays, orphaned on purpose):

```sql
DELETE FROM clubs WHERE club_id = 10;
SELECT * FROM club_memberships ORDER BY club_id, member_id;
SELECT * FROM club_events ORDER BY event_id;
```

Output:

```
 club_id | member_id |  role
---------+-----------+--------
      20 |         1 | member
(1 row)

 event_id | club_id |    title
----------+---------+--------------
      100 |  (null) | Blitz night
      101 |      20 | Sunrise trek
(2 rows)
```

Choosing: `CASCADE` for rows *part of* the parent (order lines, memberships); `RESTRICT` / `NO ACTION` for things that must not vanish silently (a customer with orders); `SET NULL` when the child should survive unlinked. Constraints are listed in `pg_constraint` (`contype` `p` / `u` / `f` / `c`).

Deferrable constraints. Normally checked after each statement; `DEFERRABLE INITIALLY DEFERRED` postpones to `COMMIT`, allowing temporary inconsistency — e.g. two rows referencing each other:

```sql
CREATE TABLE people (
    person_id  integer PRIMARY KEY,
    partner_id integer REFERENCES people DEFERRABLE INITIALLY DEFERRED
);
BEGIN;
INSERT INTO people VALUES (1, 2);   -- person 2 doesn't exist yet: allowed for now
INSERT INTO people VALUES (2, 1);
COMMIT;                             -- checked here: both exist
```

Both rows are committed. (`UNIQUE`, `PRIMARY KEY`, `REFERENCES` and `EXCLUDE` can be deferrable in PostgreSQL; `CHECK` and `NOT NULL` cannot.)

Dialect notes

(InnoDB is MySQL's default **storage engine**, the component storing rows; `PRAGMA` is SQLite's settings command.)

	PostgreSQL	MySQL (InnoDB)	SQL Server	SQLite
`CHECK` enforced	yes	since 8.0.16 (parsed but ignored before)	yes	yes
FK enforced	yes	yes	yes	only after `PRAGMA foreign_keys = ON`
Index auto-created on FK column	**no** — create one yourself	yes	no	no
Multiple NULLs in `UNIQUE`	yes	yes	no (one)	yes
Deferrable constraints	yes	no	no	FKs only

When not to rely on constraints alone

Constraints check single rows (and FK existence). Multi-row rules ("at most 5 members per club") need triggers (functions run automatically on row changes), `EXCLUDE` constraints or careful transactions (chapter 13, chapter 14).

Common mistakes

- **No index on the FK column**: in PostgreSQL each parent delete then scans the whole child table.
- `ON DELETE CASCADE` **everywhere**: one delete can wipe out large parts of the database.
- **Forgetting** `NOT NULL` next to `CHECK` or `UNIQUE` when a value is required.
- **A changeable natural key** (email, phone) as the PK — every FK must change with it. Use a surrogate PK plus `UNIQUE` on the natural key.

In the interview

30-second answer: "A candidate key is a minimal set of columns that identifies a row; the primary key is the chosen one — unique, not null, one per table; the rest are alternate keys enforced with `UNIQUE` (NULLs allowed). A composite key combines columns. A foreign key references another table's key, and `ON DELETE CASCADE / SET NULL / RESTRICT` decides what happens to children. `CHECK`, `NOT NULL` and `DEFAULT` complete the set."

- *Primary key vs unique key?* → PK: one per table, never NULL, the row's identity (often the clustered index — the one that stores the rows themselves in key order, 9.7 — in SQL Server/MySQL). UNIQUE: many per table, NULLs allowed.
- *Natural or surrogate PK?* → Usually a surrogate (`bigint` identity or UUID) for stability, plus `UNIQUE` on the natural key to keep the business rule.

Self-check

1. *Can a table have two primary keys?* → No; one (possibly composite), any number of unique keys.
2. *Is* `{member_id, email}` *a candidate key?* → No — a super key, not minimal (`member_id` alone is unique).

9.4 Generated keys: identity, sequences, serial, UUID, generated columns

What it is

Most tables use a generated **surrogate key**. PostgreSQL generates numbers with a **sequence** — an object that hands out increasing values (`nextval`); an **identity column** (SQL standard, PostgreSQL 10+) is a column tied to its own sequence.

Syntax	Meaning
`GENERATED ALWAYS AS IDENTITY`	database always generates the value; supplying one is an error (unless `OVERRIDING SYSTEM VALUE`)
`GENERATED BY DEFAULT AS IDENTITY`	generated when you don't supply one; you may supply your own
`serial` / `bigserial`	older PostgreSQL shorthand: integer column + sequence + `DEFAULT nextval(...)`
`uuid DEFAULT gen_random_uuid()`	random 128-bit id — a UUID, *universally unique identifier* (built in since PostgreSQL 13)
`GENERATED ALWAYS AS (expr) STORED`	a **generated column**: computed from other columns on every write (PostgreSQL 12+)

```sql
-- code/04_identity_and_sequences.sql
CREATE TABLE tickets (
    ticket_id  bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    subject    text NOT NULL,
    price      numeric(8,2) NOT NULL,
    qty        integer NOT NULL DEFAULT 1,
    total      numeric(10,2) GENERATED ALWAYS AS (price * qty) STORED
);
INSERT INTO tickets (subject, price) VALUES ('Printer jam', 10);
INSERT INTO tickets (subject, price, qty) VALUES ('VPN access', 5, 3), ('New laptop', 1200, 1)
RETURNING ticket_id, total;
```

Output:

```
 ticket_id |  total
-----------+---------
         2 |   15.00
         3 | 1200.00
(2 rows)
```

```sql
INSERT INTO tickets (ticket_id, subject, price) VALUES (99, 'Manual id', 1);
INSERT INTO tickets (subject, price, total) VALUES ('Fake total', 1, 500);
```

Output:

```
ERROR:  cannot insert a non-DEFAULT value into column "ticket_id"
DETAIL:  Column "ticket_id" is an identity column defined as GENERATED ALWAYS.
HINT:  Use OVERRIDING SYSTEM VALUE to override.
ERROR:  cannot insert a non-DEFAULT value into column "total"
DETAIL:  Column "total" is a generated column.
```

Sequences never go backwards — gaps are normal:

```sql
INSERT INTO tickets (ticket_id, subject, price) OVERRIDING SYSTEM VALUE VALUES (99, 'Manual id', 1);
BEGIN;
INSERT INTO tickets (subject, price) VALUES ('Will be rolled back', 1);
ROLLBACK;
INSERT INTO tickets (subject, price) VALUES ('After rollback', 2);
SELECT ticket_id, subject, total FROM tickets ORDER BY ticket_id;
```

Output:

```
 ticket_id |    subject     |  total
-----------+----------------+---------
         1 | Printer jam    |   10.00
 ...
         3 | New laptop     | 1200.00
         5 | After rollback |    2.00
        99 | Manual id      |    1.00
(5 rows)
```

Number 4 went to the rolled-back insert and is never reused: sequences are deliberately **not** transactional, so concurrent inserts never wait for each other. Never assume gap-free ids (legal invoice numbers need separate generation under a **lock** — a claim that makes other sessions wait).

Why identity beats `serial`: `serial` is just a default, so a manual id doesn't advance the sequence and a later one collides:

```
CREATE TABLE legacy (id serial PRIMARY KEY, label text);
INSERT INTO legacy (label) VALUES ('a'), ('b');       -- ids 1, 2
INSERT INTO legacy (id, label) VALUES (3, 'manual');  -- allowed
INSERT INTO legacy (label) VALUES ('c');              -- sequence gives 3 -> collision
```

Output:

```
ERROR:  duplicate key value violates unique constraint "legacy_pkey"
DETAIL:  Key (id)=(3) already exists.
```

`GENERATED ALWAYS` refuses the manual insert up front. (After loading your own ids, resync: `SELECT setval('legacy_id_seq', (SELECT max(id) FROM legacy));`.)

A standalone sequence (e.g. invoice numbers shared by several tables):

```
CREATE SEQUENCE invoice_no START 1000 INCREMENT 10;
SELECT nextval('invoice_no') AS first, nextval('invoice_no') AS second, currval('invoice_no') AS current;
-- 1000 | 1010 | 1010
```

`currval` returns the last value *this session* got from `nextval`.

UUIDs: `key_id uuid PRIMARY KEY DEFAULT gen_random_uuid()` gives 128-bit ids (36 characters as text), generated anywhere (even in the app) without coordination and not revealing row counts — but 16 bytes (vs 8 for `bigint`), and random values scatter index inserts. Time-ordered UUIDv7 (leading timestamp bits, so ids sort by creation) reduces that; PostgreSQL 18 adds `uuidv7()` (not run).

Dialect notes

	Auto-increment syntax
PostgreSQL	`GENERATED {ALWAYS | BY DEFAULT} AS IDENTITY` (10+), `serial` (legacy)
MySQL	`AUTO_INCREMENT`
SQL Server	`IDENTITY(1,1)`; `SEQUENCE` objects since 2012
SQLite	`INTEGER PRIMARY KEY` (alias of the rowid, SQLite's hidden row number); `AUTOINCREMENT` prevents reuse of deleted ids
Oracle	identity columns since 12c; sequences

Generated columns: PostgreSQL 12–17 `STORED` only; PostgreSQL 18 adds `VIRTUAL` (computed on read) and makes it the default when neither keyword is given (not run); MySQL 5.7+ supports both; SQL Server has "computed columns" (optionally `PERSISTED`).

9.5 ALTER: changing existing tables

What it is

`ALTER TABLE` changes an existing (usually populated) table's structure without recreating it.

Task	PostgreSQL
Add a column	`ALTER TABLE t ADD COLUMN c type [DEFAULT …] [NOT NULL];`
Drop a column	`ALTER TABLE t DROP COLUMN c;`
Rename a column / table	`ALTER TABLE t RENAME COLUMN a TO b;` / `ALTER TABLE t RENAME TO u;`
Change a type	`ALTER TABLE t ALTER COLUMN c TYPE newtype [USING expr];`
Set / drop a default	`ALTER TABLE t ALTER COLUMN c SET DEFAULT …;` / `DROP DEFAULT`
Set / drop NOT NULL	`ALTER TABLE t ALTER COLUMN c SET NOT NULL;` / `DROP NOT NULL`
Add / drop a constraint	`ALTER TABLE t ADD CONSTRAINT name …;` / `DROP CONSTRAINT name;`

How it works (with real data)

```sql
-- code/05_alter.sql
CREATE TABLE staff (id integer PRIMARY KEY, first_name text, last_name text);
INSERT INTO staff VALUES (1, 'Tara', 'Mendes'), (2, 'Omar', 'Mendes'), (3, 'Yuki', 'Tanaka');

ALTER TABLE staff ADD COLUMN salary integer;
ALTER TABLE staff ADD COLUMN country text NOT NULL DEFAULT 'US';
SELECT * FROM staff ORDER BY id;
```

Output:

```
 id | first_name | last_name | salary | country
----+------------+-----------+--------+---------
  1 | Tara       | Mendes    | (null) | US
  2 | Omar       | Mendes    | (null) | US
  3 | Yuki       | Tanaka    | (null) | US
(3 rows)
```

Existing rows get NULL or the default. Since PostgreSQL 11 a constant default is instant (no table rewrite — copying every row to a new file).

```sql
ALTER TABLE staff RENAME COLUMN first_name TO given_name;
ALTER TABLE staff RENAME TO team;

UPDATE team SET salary = 50000 + id * 1000;
ALTER TABLE team ALTER COLUMN salary TYPE numeric(10,2) USING salary::numeric(10,2);
ALTER TABLE team ALTER COLUMN last_name TYPE integer;                -- text -> integer?
```

Output:

```
ERROR:  column "last_name" cannot be cast automatically to type integer
HINT:  You might need to specify "USING last_name::integer".
```

`USING` says how to compute the new value; without it only automatic (implicit/assignment) casts — conversions PostgreSQL deems safe — are allowed.

`NOT NULL` on a column with NULLs fails — `ALTER TABLE team ADD COLUMN email text NOT NULL;` gives *ERROR: column "email" of relation "team" contains null values*. Add, backfill (fill existing rows), then constrain:

```sql
ALTER TABLE team ADD COLUMN email text;
UPDATE team SET email = lower(given_name) || '@corp.example';
ALTER TABLE team ALTER COLUMN email SET NOT NULL;
```

Live tables: `NOT VALID` then `VALIDATE`. A new `CHECK` or FK normally scans the table under a strong lock. Add it `NOT VALID` (enforced for new/changed rows only), fix old data, then `VALIDATE CONSTRAINT` (scans under a lighter lock):

```
ALTER TABLE team ADD CONSTRAINT team_email_uq UNIQUE (email);
INSERT INTO team (id, given_name, last_name, salary, email) VALUES (4, 'Leo', 'Brandt', -5, 'leo@corp.example');
ALTER TABLE team ADD CONSTRAINT salary_positive CHECK (salary > 0);            -- fails: Leo's row
ALTER TABLE team ADD CONSTRAINT salary_positive CHECK (salary > 0) NOT VALID;  -- accepted
ALTER TABLE team VALIDATE CONSTRAINT salary_positive;                          -- fails until fixed
UPDATE team SET salary = 48000 WHERE id = 4;
ALTER TABLE team VALIDATE CONSTRAINT salary_positive;                          -- succeeds
```

Output:

```
ERROR:  check constraint "salary_positive" of relation "team" is violated by some row
ERROR:  check constraint "salary_positive" of relation "team" is violated by some row
```

(`UNIQUE` / `PRIMARY KEY` can't be `NOT VALID` ; build a unique index `CONCURRENTLY` — without blocking writes, 9.7 — then `ADD CONSTRAINT … UNIQUE USING INDEX` .)

Several changes in one statement (one lock, one pass):

```
ALTER TABLE team
    DROP COLUMN country,
    ADD COLUMN hired_on date,
    ALTER COLUMN last_name SET NOT NULL;
SELECT conname, pg_get_constraintdef(oid) AS definition
FROM pg_constraint WHERE conrelid = 'team'::regclass ORDER BY conname;
```

Output:

```
     conname     |           definition
-----------------+--------------------------------
 salary_positive | CHECK ((salary > (0)::numeric))
 staff_pkey      | PRIMARY KEY (id)
 team_email_uq   | UNIQUE (email)
(3 rows)
```

The PK is still `staff_pkey`: renaming a table doesn't rename its constraints or indexes.

Locks and production safety

Most `ALTER TABLE` forms take an **ACCESS EXCLUSIVE** lock — no reads or writes until done. Metadata-only changes (catalog edits that don't touch rows: nullable column, constant default, rename, drop column — which only hides it) hold it briefly; **type changes that rewrite the table** can hold it for minutes. Production practice: set `lock_timeout` (give up rather than wait forever), use `NOT VALID` + `VALIDATE` and `CREATE INDEX CONCURRENTLY`, and "expand → migrate → contract" (add the new column, backfill in batches of a few thousand rows, switch the app, drop the old). Locks: chapter 13.

Dialect notes

Task	PostgreSQL	MySQL	SQL Server
Change type	`ALTER COLUMN c TYPE t`	`MODIFY COLUMN c t`	`ALTER COLUMN c t`
Rename column	`RENAME COLUMN a TO b`	`RENAME COLUMN a TO b` (8.0) / `CHANGE a b t`	`EXEC sp_rename 't.a', 'b', 'COLUMN'`
Rename table	`ALTER TABLE t RENAME TO u`	`RENAME TABLE t TO u`	`EXEC sp_rename 't', 'u'`
DDL inside a transaction (rollback-able)	yes	no — implicit commit	yes (mostly)

Common mistakes

`NOT NULL` without a default or backfill (fails); a type change on a huge table in business hours (long exclusive lock); renaming columns that application SQL still uses (views follow renames automatically

in PostgreSQL, application code does not).

9.6 DROP and TRUNCATE (vs DELETE)

What they are

- `DROP` removes an object entirely — definition, data, indexes, constraints, triggers, permissions (`DROP TABLE/DATABASE/SCHEMA/VIEW/INDEX …`): throw the notebook away.
- `TRUNCATE` removes **all rows** but keeps the table (structure, indexes, grants): a fresh empty notebook. (`TABLE` in `TRUNCATE TABLE t` is optional in PostgreSQL and MySQL, required in SQL Server and Oracle.)
- `DELETE` (DML, 9.10) removes rows one by one, optionally filtered by `WHERE`: erasing lines.

How TRUNCATE works

```
-- code/06_drop_and_truncate.sql
CREATE TABLE audit_log (log_id bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY, msg text NOT NULL);
INSERT INTO audit_log (msg) SELECT 'event ' || g FROM generate_series(1, 5) g;

TRUNCATE audit_log;
SELECT count(*) AS rows_after_truncate FROM audit_log;                  -- 0
INSERT INTO audit_log (msg) VALUES ('after truncate') RETURNING log_id;   -- 6: identity continues
TRUNCATE audit_log RESTART IDENTITY;
INSERT INTO audit_log (msg) VALUES ('after restart') RETURNING log_id;    -- 1
```

Output (the three results): `0`, then `log_id = 6`, then `log_id = 1`.

In PostgreSQL, `TRUNCATE` is **transactional** — you can roll it back:

```
BEGIN;
TRUNCATE audit_log;
SELECT count(*) AS inside_tx FROM audit_log;       -- 0
ROLLBACK;
SELECT count(*) AS after_rollback FROM audit_log;  -- 1
```

A table referenced by a foreign key can't be truncated alone (orphaned children); `CASCADE` truncates the referencing tables too:

```
TRUNCATE customers;
BEGIN;
TRUNCATE customers CASCADE;
SELECT (SELECT count(*) FROM customers) AS customers,
       (SELECT count(*) FROM orders) AS orders,
       (SELECT count(*) FROM order_items) AS items;
ROLLBACK;
```

Output:

```
ERROR:  cannot truncate a table referenced in a foreign key constraint
DETAIL:  Table "orders" references "customers".
HINT:  Truncate table "orders" at the same time, or use TRUNCATE ... CASCADE.
NOTICE:  truncate cascades to table "orders"
NOTICE:  truncate cascades to table "order_items"
 customers | orders | items
-----------+--------+-------
         0 |      0 |     0
(1 row)
```

How DROP works

After `DROP TABLE audit_log;`, querying gives *relation "audit_log" does not exist* and a second `DROP` errors; `DROP TABLE IF EXISTS` only notices (use it in scripts). `DROP` refuses while objects depend on the table (default `RESTRICT`); `CASCADE` drops dependent views and FKs *pointing at* it (the child table and rows stay):

```
CREATE VIEW big_spenders AS SELECT customer_id FROM orders GROUP BY customer_id HAVING count(*) > 2;
DROP TABLE orders;
BEGIN;
DROP TABLE orders CASCADE;
SELECT count(*) AS order_items_still_there FROM order_items;
ROLLBACK;
```

Output:

```
ERROR:  cannot drop table orders because other objects depend on it
DETAIL:  constraint order_items_order_id_fkey on table order_items depends on table orders
view big_spenders depends on table orders
HINT:  Use DROP ... CASCADE to drop the dependent objects too.
NOTICE:  drop cascades to 2 other objects
...
 order_items_still_there
-------------------------
                      22
(1 row)
```

`DROP DATABASE name;` removes a database (not while connected to it; in PostgreSQL not inside a transaction). `DROP SCHEMA s CASCADE;` removes a schema and its contents.

DELETE vs TRUNCATE vs DROP

	`DELETE FROM t [WHERE …]`	`TRUNCATE t`	`DROP TABLE t`
Kind	DML	DDL	DDL
Removes	chosen rows (or all)	all rows	rows **and** the table
`WHERE` possible	yes	no	no
Speed on big tables	slow: row by row, all logged (each change written to the transaction log)	fast: releases the data files	fast
Space returned to the OS	usually not: dead rows (old, deleted versions) are reused after `VACUUM` (PostgreSQL's cleanup process), the file rarely shrinks	yes	yes
Row-level `ON DELETE` triggers	fire	don't fire (PostgreSQL has `ON TRUNCATE` triggers)	n/a
FK checks	per row (or cascades)	refused if referenced, unless `CASCADE`	refused if referenced, unless `CASCADE`
Identity / auto-increment	continues	PostgreSQL: continues unless `RESTART IDENTITY` ; MySQL & SQL Server: reset	gone
Rollback	yes	inside a transaction in PostgreSQL & SQL Server; **not** in MySQL & Oracle (implicit commit)	same as `TRUNCATE`
Lock	row locks	exclusive table lock	exclusive table lock

Undoing `TRUNCATE` / `DROP` depends on the database: MySQL needs a backup (9.12); Oracle can undo `DROP TABLE` from its recycle bin (`FLASHBACK TABLE t TO BEFORE DROP` , if enabled) but not `TRUNCATE` . SQLite has no `TRUNCATE` ; `DELETE FROM t` without `WHERE` is optimized to do the same.

Common mistakes

- `DROP` / `TRUNCATE` on the wrong connection (production, the live system, instead of dev) — make production visually distinct; use reviewed migrations (versioned schema-change scripts).
- `… CASCADE` unread — try it in `BEGIN … ROLLBACK` first.
- `DELETE` to empty a 100-million-row table (hours, huge log) when `TRUNCATE` takes a second.
- Expecting `TRUNCATE` to be invisible to concurrent transactions: it isn't MVCC-safe (MVCC, *multi-version concurrency control*, gives each transaction a **snapshot** — a consistent view as of one moment; chapter 13), so an older-snapshot transaction reading afterwards sees it empty.

In the interview

> **30-second answer:** "`DELETE` is DML: selected rows, one at a time, fires triggers, keeps the table. `TRUNCATE` is DDL: all rows at once by deallocating storage — much faster, no `WHERE`, no delete triggers, often resets identity; rollback-able in PostgreSQL and SQL Server, not in MySQL/Oracle. `DROP` removes the table itself."

Self-check

1. *Empty a staging table (where incoming data lands before processing) fast and reset ids?* → `TRUNCATE staging RESTART IDENTITY;`
2. *`DROP TABLE orders CASCADE` — effect on `order_items`?* → Its FK constraint is dropped; table and rows remain.

9.7 Indexes (in brief)

An **index** is a separate structure (default a B-tree — balanced, always sorted) that finds rows by column values without reading the whole table. Only the DDL here; how indexes work, kinds, `EXPLAIN` (shows the query plan) and when they help: chapter 12.

```sql
-- code/07_indexes_brief.sql
CREATE INDEX idx_employees_last_name ON employees (last_name);
CREATE INDEX idx_orders_customer_placed ON orders (customer_id, placed_at DESC);   -- composite

-- A UNIQUE index can enforce rules a UNIQUE constraint can't, e.g. case-insensitive uniqueness:
CREATE UNIQUE INDEX idx_customers_email_ci ON customers (lower(trim(email)));
```

Output:

```
ERROR:  could not create unique index "idx_customers_email_ci"
DETAIL:  Key (lower(TRIM(BOTH FROM email)))=(anita.shah@mail.example) is duplicated.
```

The index found the duplicate Anita Shah; after `UPDATE customers SET email = NULL WHERE customer_id = 8` it succeeds.

PKs and unique constraints create indexes automatically (`pg_indexes`, the catalog view of indexes, lists `employees_pkey`, `employees_email_key`). Also: `DROP INDEX [IF EXISTS] name;`, and `CREATE/DROP INDEX CONCURRENTLY …`, which don't block writes on a live table (slower; not inside a transaction block).

Clustered vs non-clustered

	Clustered index	Non-clustered (secondary) index
What it is	the table's rows are **stored in the index's order** — the index *is* the table	a separate structure of (key → pointer to the row)
How many	at most one per table	many
SQL Server	`CREATE CLUSTERED INDEX ix ON t (col);` (the PK is clustered by default)	`CREATE [NONCLUSTERED] INDEX ix ON t (col);`
MySQL InnoDB	always the primary key (no PK → the first `UNIQUE NOT NULL` index, else a hidden row id)	every other index; it stores the PK value as the pointer
PostgreSQL	**none** — tables are unordered "heaps"; `CLUSTER t USING ix;` re-sorts the table once, but new rows aren't kept in order	every index is secondary

```
CLUSTER orders USING idx_orders_customer_placed;   -- one-time physical re-ordering in PostgreSQL
```

A clustered index makes range scans (every key between two values) fast — neighbouring keys are neighbouring rows — but a random key (random UUID) causes page splits on insert (a full page — the fixed-size storage block, 8 KB in PostgreSQL — split in two).

Cost: every index is maintained on inserts and updates of its columns (and cleaned after deletes) and takes disk. Index what queries filter, join and sort on — not every column.

9.8 INSERT

What it is

`INSERT` adds new rows to a table.

```
INSERT INTO table_name [(column1, column2, ...)]
VALUES (value1, value2, ...) [, (...), ...]
[ON CONFLICT ...]            -- 9.11
[RETURNING ...];             -- PostgreSQL: give back the inserted rows
-- or
INSERT INTO table_name [(columns)] SELECT ...;
```

How it works

1. The **column list** names supplied columns; omitted ones get their `DEFAULT`, else NULL (error if `NOT NULL` without default).
2. Without a list you supply **every** column in table order — fragile when columns are added.
3. If any row fails a type or constraint, the **whole statement** fails.

```sql
-- code/08_insert.sql
CREATE TABLE contacts (
    contact_id  integer GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    first_name  text NOT NULL,
    last_name   text NOT NULL,
    age         integer CHECK (age >= 0),
    city        text DEFAULT 'Pune',
    created_at  timestamp NOT NULL DEFAULT '2025-06-01 09:00'
);

INSERT INTO contacts (first_name, last_name, age) VALUES ('Tara', 'Mendes', 28);

INSERT INTO contacts (first_name, last_name, age, city) VALUES      -- several rows at once
    ('Omar', 'Mendes', 31, 'Mumbai'),
    ('Yuki', 'Tanaka', NULL, DEFAULT);                              -- DEFAULT keyword

INSERT INTO contacts VALUES (DEFAULT, 'Leo', 'Brandt', 40, 'Delhi', '2025-06-02 10:00');  -- all columns

INSERT INTO contacts (first_name, last_name) VALUES ('Ines', 'Silva')
RETURNING contact_id, city, created_at;
```

Output (of the `RETURNING`):

```
 contact_id | city |     created_at
------------+------+---------------------
          5 | Pune | 2025-06-01 09:00:00
(1 row)
```

`RETURNING` hands back generated values (ids, defaults) in the same round trip (request plus reply) — no second query, no race (a **race condition** is a bug whose outcome depends on concurrent sessions' timing).

`INSERT … SELECT` copies rows produced by any query:

```sql
INSERT INTO contacts (first_name, last_name, city)
SELECT first_name, last_name, city FROM customers WHERE country = 'UK'
RETURNING contact_id, first_name, city;
```

Output:

```
 contact_id | first_name |  city
------------+------------+--------
          6 | Ben        | London
          7 | George     | London
(2 rows)
```

Errors worth recognising:

```sql
INSERT INTO contacts (first_name, last_name) VALUES ("Ana", "Lima");
INSERT INTO contacts (first_name, last_name) VALUES ('Ana', 'Lima', 22);
INSERT INTO contacts (first_name, last_name, age) VALUES ('Ok', 'Row', 20), ('Bad', 'Row', -1);
SELECT count(*) AS still_7 FROM contacts;
```

Output:

```
ERROR:  column "Ana" does not exist
ERROR:  INSERT has more expressions than target columns
ERROR:  new row for relation "contacts" violates check constraint "contacts_age_check"
DETAIL:  Failing row contains (9, Bad, Row, -1, Pune, 2025-06-01 09:00:00).
 still_7
---------
       7
(1 row)
```

- In standard SQL (PostgreSQL, SQLite, SQL Server by default, Oracle) **double quotes name identifiers** (table, column names); strings use single quotes. MySQL accepts double-quoted strings, so its code often breaks elsewhere.

- The multi-row insert failed whole — 'Ok' was not kept (but consumed identity values 8 and 9: more gaps).

Bulk loading: multi-row `VALUES` or `INSERT … SELECT` beats one statement per row; for millions of rows use `COPY table FROM file` (psql: `\copy` for a client file), MySQL `LOAD DATA INFILE`, SQL Server `BULK INSERT` / `bcp`.

Dialect notes

	PostgreSQL	MySQL	SQL Server	SQLite
Multi-row `VALUES`	yes	yes	yes (max 1000 rows per statement)	yes
Get generated values back	`RETURNING`	`LAST_INSERT_ID()`	`OUTPUT inserted.*` / `SCOPE_IDENTITY()`	`RETURNING` (3.35+) / `last_insert_rowid()`
`INSERT … DEFAULT VALUES`	yes	`INSERT INTO t () VALUES ()`	yes	yes

9.9 UPDATE

What it is

`UPDATE` changes column values in existing rows.

```
UPDATE table_name
SET column1 = expr1, column2 = expr2, ...
[FROM other_tables]          -- PostgreSQL: use other tables to compute / filter
[WHERE condition]            -- which rows; WITHOUT it, EVERY row changes
[RETURNING ...];
```

How it works

`WHERE` picks rows (none = all); every `SET` expression reads the row's **old** values, then all assignments apply together; new rows must satisfy all constraints or nothing changes. PostgreSQL writes a new row version (MVCC — chapter 13); `VACUUM` later removes the old.

Raise Sales salaries by 10 %:

```
-- code/09_update.sql
UPDATE employees
SET salary = salary * 1.10
WHERE dept_id = 2
RETURNING emp_id, first_name, salary;
```

Output:

```
 emp_id | first_name |  salary
--------+------------+-----------
      6 | Daniel     | 165000.00
      7 | Priya      |  79200.00
 ...
(4 rows)
```

+10 % is `* 1.10`, +50 % `* 1.5`, doubling `* 2` — check the factor against the requirement.

All `SET` expressions see the old row:

```
UPDATE products
SET unit_price   = unit_price * 0.9,
    discontinued = unit_price * 0.9 < 40          -- uses the OLD unit_price
WHERE category = 'Furniture'
RETURNING name, unit_price, discontinued;
```

Output:

```
     name       | unit_price | discontinued
----------------+------------+-------------
 Standing Desk  |     449.10 | f
 Ergonomic Chair |     314.99 | f
 Desk Lamp      |      31.50 | t
(3 rows)
```

(In MySQL, assignments are applied left to right, so a later expression sees the *new* value of an earlier column — a real portability trap. PostgreSQL and SQL Server follow the standard.)

Different changes for different rows with `CASE`:

```
UPDATE orders
SET status = CASE WHEN shipped_at IS NULL THEN 'cancelled' ELSE 'delivered' END
WHERE status IN ('pending', 'shipped') AND placed_at < '2025-06-10'
RETURNING order_id, status;
```

Output:

```
 order_id |  status
----------+-----------
      107 | delivered
 ...
      113 | cancelled
(4 rows)
```

Using another table — PostgreSQL's `UPDATE … FROM` joins the target to other tables:

```
ALTER TABLE customers ADD COLUMN lifetime_orders integer NOT NULL DEFAULT 0;
UPDATE customers c
SET lifetime_orders = t.n
FROM (SELECT customer_id, count(*) AS n FROM orders GROUP BY customer_id) t
WHERE t.customer_id = c.customer_id;
SELECT customer_id, first_name, lifetime_orders FROM customers ORDER BY customer_id LIMIT 4;
```

Output:

```
 customer_id | first_name | lifetime_orders
-------------+------------+-----------------
           1 | Anita      |               3
           2 | Ben        |               2
 ...
(4 rows)
```

Each target row must match **at most one** `FROM` row, or PostgreSQL uses one unpredictably — hence aggregating first. Portable alternative: a correlated scalar subquery (one value, re-run per outer row) in `SET`:

```
UPDATE departments d
SET budget = (SELECT coalesce(sum(salary), 0) * 1.5 FROM employees e WHERE e.dept_id = d.dept_id)
WHERE d.dept_id = 5
RETURNING name, budget;                          -- Support | 96000.00
```

The classic accident — `UPDATE` without `WHERE` updates every row. Practise risky statements inside a transaction, check, then `COMMIT` or `ROLLBACK`:

```
BEGIN;
UPDATE employees SET salary = 1;
SELECT count(*) AS rows_now_paid_1 FROM employees WHERE salary = 1;
ROLLBACK;
SELECT count(*) AS rows_paid_1_after_rollback FROM employees WHERE salary = 1;
```

Output:

```
 rows_now_paid_1
-----------------
              15
(1 row)

 rows_paid_1_after_rollback
----------------------------
                           0
(1 row)
```

Also: run the `WHERE` as `SELECT count(*)` first; check psql's `UPDATE n`; use `RETURNING`; MySQL clients' `--safe-updates` refuses key-less updates. Constraints still apply (`dept_id = 99` fails the FK, `unit_price = -1` the `CHECK`).

9.10 DELETE

What it is

`DELETE` removes rows from a table.

```
DELETE FROM table_name
[USING other_tables]         -- PostgreSQL: join to other tables
[WHERE condition]            -- WITHOUT it, EVERY row is deleted
[RETURNING ...];
```

How it works

`WHERE` picks rows (none = all; the table remains). Deleting a parent with children fails or cascades per its FKs (9.3). PostgreSQL only marks rows dead; `VACUUM` reclaims space later.

```
-- code/10_delete.sql
DELETE FROM order_items
WHERE order_id = 105
RETURNING order_id, product_id, quantity;
```

Output:

```
 order_id | product_id | quantity
----------+------------+----------
      105 |          1 |        2
(1 row)
```

```
DELETE FROM customers WHERE customer_id = 1;
```

Output:

```
ERROR:  update or delete on table "customers" violates foreign key constraint "orders_customer_id_fkey" on table "orders"
DETAIL:  Key (customer_id)=(1) is still referenced from table "orders".
```

Deleting based on another table — PostgreSQL's `DELETE … USING`, or the portable subquery form:

```sql
DELETE FROM order_items oi
USING orders o
WHERE o.order_id = oi.order_id
  AND o.status = 'cancelled'
RETURNING oi.order_id, oi.product_id;          -- (111, 2)

DELETE FROM orders
WHERE status = 'cancelled'
  AND order_id NOT IN (SELECT order_id FROM order_items)
RETURNING order_id;                            -- 105, 111

DELETE FROM customers c
WHERE NOT EXISTS (SELECT 1 FROM orders o WHERE o.customer_id = c.customer_id)
RETURNING customer_id, first_name;
```

Output of the last statement:

```
 customer_id | first_name
-------------+------------
           4 | Divya
           9 | Hana
          10 | Ivan
(3 rows)
```

(Divya's only order, cancelled, was just removed.) Children first, then parents, as FKs demand. (`NOT IN` is safe here only because `order_items.order_id` can't be NULL; prefer `NOT EXISTS` — chapter 6.)

Delete everything — safely practised:

```sql
BEGIN;
DELETE FROM order_items;
SELECT count(*) AS items_inside_tx FROM order_items;       -- 0
ROLLBACK;
SELECT count(*) AS items_after_rollback FROM order_items;  -- 20
```

Soft delete — mark rows instead of physically deleting:

```sql
ALTER TABLE products ADD COLUMN deleted_at timestamp;
UPDATE products SET deleted_at = '2025-06-30 12:00' WHERE name = 'Desk Lamp';
CREATE VIEW active_products AS SELECT * FROM products WHERE deleted_at IS NULL;
SELECT count(*) AS all_products, (SELECT count(*) FROM active_products) AS active FROM products;
```

Output:

```
 all_products | active
--------------+--------
            9 |      8
(1 row)
```

	Hard delete	Soft delete
Undo / audit	only from backups or audit tables	trivial: clear `deleted_at`
Queries	simple	every query must filter (use a view, or row-level security — per-user rules that hide rows)
Unique constraints	simple	a deleted row still blocks its email — use a partial unique index (PostgreSQL's filtered index) `WHERE deleted_at IS NULL`
Privacy law ("right to be forgotten")	satisfied	must still purge eventually

Millions of rows: delete in batches (`DELETE … WHERE id IN (SELECT id … LIMIT 10000)` repeated) to keep transactions and locks short; with partitioned tables (one logical table as several physical pieces, e.g. per month) `DROP` an old partition instantly (chapter 15).

Dialect notes: UPDATE / DELETE using another table

	UPDATE	DELETE
PostgreSQL	`UPDATE t SET … FROM s WHERE t.id = s.id`	`DELETE FROM t USING s WHERE t.id = s.id`
SQL Server	`UPDATE t SET … FROM t JOIN s ON …`	`DELETE t FROM t JOIN s ON …`
MySQL	`UPDATE t JOIN s ON … SET t.col = …`	`DELETE t FROM t JOIN s ON …`
SQLite	`UPDATE t SET … FROM s WHERE …` (3.33+)	subquery only (`WHERE id IN (SELECT …)`)

9.11 Upsert: INSERT … ON CONFLICT, and MERGE

What it is

An **upsert** is "insert, or if the key exists, update (or skip)". `SELECT`-then-`INSERT` / `UPDATE` in app code races: two sessions both see "not there" and both insert. PostgreSQL (9.5+) does it atomically (one indivisible step) with `INSERT … ON CONFLICT`.

How it works

```
INSERT INTO t (cols) VALUES (...)
ON CONFLICT (conflict_target)          -- the unique/PK column(s), or ON CONSTRAINT name
DO NOTHING                             -- skip rows that would conflict
-- or
DO UPDATE SET col = EXCLUDED.col, ...  -- EXCLUDED = the row you tried to insert
         [WHERE condition];            -- update only when this is true
```

Each row is inserted, or — if it would violate the conflict target's unique index — the `DO` action runs instead of an error; atomically, even under concurrency. In `DO UPDATE`, `t.col` (or alias) is the **existing** row, `EXCLUDED.col` the **proposed** one.

```
-- code/11_upsert.sql
CREATE TABLE inventory (
    sku         text PRIMARY KEY,
    qty         integer NOT NULL CHECK (qty >= 0),
    updated_at  timestamp NOT NULL
);
INSERT INTO inventory VALUES ('MOUSE-1', 10, '2025-06-01 08:00'), ('HUB-2', 4, '2025-06-01 08:00');

INSERT INTO inventory VALUES ('MOUSE-1', 99, '2025-06-02 08:00'), ('LAMP-3', 7, '2025-06-02 08:00')
ON CONFLICT (sku) DO NOTHING
RETURNING sku, qty;
```

Output:

```
  sku   | qty
--------+-----
 LAMP-3 |   7
(1 row)
```

MOUSE-1 already existed, so it was skipped silently (and isn't in `RETURNING`).

Insert or add to the existing stock:

```
INSERT INTO inventory AS i VALUES
    ('MOUSE-1', 5, '2025-06-03 08:00'),
    ('KEYB-4',  2, '2025-06-03 08:00')
ON CONFLICT (sku) DO UPDATE
SET qty        = i.qty + EXCLUDED.qty,
    updated_at = EXCLUDED.updated_at
RETURNING sku, qty, updated_at, (xmax <> 0) AS was_update;
```

Output:

```
   sku   | qty |     updated_at      | was_update
---------+-----+---------------------+------------
 MOUSE-1 |  15 | 2025-06-03 08:00:00 | t
 KEYB-4  |   2 | 2025-06-03 08:00:00 | f
(2 rows)
```

(`xmax`, a hidden system column, `<> 0` is a common trick to tell updated from inserted rows — an internal detail; don't build business logic on it.)

Conditional update — only if incoming data is newer (common when syncing several sources):

```
INSERT INTO inventory AS i VALUES ('HUB-2', 1, '2025-05-01 00:00')    -- older than what we have
ON CONFLICT (sku) DO UPDATE
SET qty = EXCLUDED.qty, updated_at = EXCLUDED.updated_at
WHERE EXCLUDED.updated_at > i.updated_at
RETURNING sku, qty;
```

Output:

```
 sku | qty
-----+-----
(0 rows)
```

Nothing changed: HUB-2 keeps qty 4 from 1 June. **Rules and errors:**

- The conflict target must match a unique index or constraint (`ON CONFLICT (sku)` or `ON CONFLICT ON CONSTRAINT inventory_pkey`). `DO NOTHING` may omit the target (then any unique violation is skipped).
- One statement can't affect the same row twice:

```
INSERT INTO inventory VALUES ('PEN-5', 1, '2025-06-04'), ('PEN-5', 2, '2025-06-04')
ON CONFLICT (sku) DO UPDATE SET qty = EXCLUDED.qty;
```

Output:

```
ERROR:  ON CONFLICT DO UPDATE command cannot affect row a second time
HINT:  Ensure that no rows proposed for insertion within the same command have duplicate constrained values.
```

De-duplicate the input first (e.g. `SELECT DISTINCT ON (sku) …`).

- Other constraint violations (a `CHECK`, a different unique index) still raise errors.

MERGE (PostgreSQL 15+, SQL Server, Oracle)

`MERGE` is the standard "compare source with target and insert, update or delete accordingly". PostgreSQL added it in **version 15** (SQL Server 2008, Oracle 9i); on this book's 14 it is a syntax error. Shape (not run):

```
MERGE INTO inventory AS t
USING incoming AS s ON t.sku = s.sku
WHEN MATCHED AND s.qty = 0 THEN DELETE
WHEN MATCHED THEN UPDATE SET qty = s.qty, updated_at = s.updated_at
WHEN NOT MATCHED THEN INSERT (sku, qty, updated_at) VALUES (s.sku, s.qty, s.updated_at);
```

	`INSERT … ON CONFLICT`	`MERGE`
Can delete	no	yes (`WHEN MATCHED … THEN DELETE`)
Needs a unique index	yes	no — any join condition
Safe against concurrent inserts of the same key	yes (that's its purpose)	not by itself: two sessions can both take the `NOT MATCHED` branch → unique violation; retry or lock
`RETURNING`	yes	PostgreSQL 17+ (which also added `WHEN NOT MATCHED BY SOURCE` for deleting target rows missing from the source)

Rule of thumb in PostgreSQL: **upserts of single rows/batches →** `ON CONFLICT`**; synchronising a table from a staging table with deletes →** `MERGE` (15+).

Dialect notes

	Upsert syntax
PostgreSQL	`INSERT … ON CONFLICT … DO NOTHING / DO UPDATE SET … EXCLUDED.col` (9.5+); `MERGE` (15+)
MySQL	`INSERT … ON DUPLICATE KEY UPDATE col = VALUES(col)` (or row alias `AS new … new.col`, 8.0.19+); `INSERT IGNORE` ≈ do nothing
SQLite	`INSERT … ON CONFLICT(col) DO UPDATE SET col = excluded.col` (3.24+); `INSERT OR IGNORE` / `INSERT OR REPLACE`
SQL Server	`MERGE`, or `UPDATE` + `INSERT … WHERE NOT EXISTS` with locking hints
Oracle	`MERGE`

In the interview

30-second answer: "In PostgreSQL, `INSERT … ON CONFLICT (key) DO UPDATE SET col = EXCLUDED.col` — atomic and concurrency-safe because it relies on the unique index; `DO NOTHING` skips duplicates. `MERGE` (PostgreSQL 15+, SQL Server, Oracle) is the standard multi-branch statement that can also delete, but doesn't by itself protect against concurrent inserts."

Self-check

1. *What does* `EXCLUDED` *refer to?* → The row that was proposed for insertion.
2. *What does* `ON CONFLICT` *need?* → A unique index/constraint matching the conflict target.

9.12 Backup and restore

What it is

A **backup** is a copy to rebuild from; a **restore** rebuilds. Backups cover what replication (live copies — **replicas** — on other servers; System Design Fundamentals) can't: a mistaken `DELETE` / `DROP` (replicas copy it instantly), corrupting bugs, ransomware (malware encrypting files for payment), losing a data centre. Strategy is set by:

- **RPO — recovery point objective**: how much recent data you can afford to lose (e.g. 5 minutes).
- **RTO — recovery time objective**: how long you can afford to be down while restoring (e.g. 1 hour).

Kinds of backups

Kind	What is copied	Tools (PostgreSQL)	Good for	Limits
Logical	SQL / an archive of schema + data	`pg_dump`, `pg_dumpall`	small/medium DBs, single tables, moving across versions	slow for huge DBs; restore rebuilds indexes
Physical	the database files themselves	`pg_basebackup`, file-system snapshots, pgBackRest, Barman	large DBs, fast full restore	whole cluster (in PostgreSQL: one server instance with all its databases) only; same major version (e.g. 16.x)
Continuous (PITR)	a base backup **plus** the write-ahead log (WAL) stream	WAL archiving + base backup	restore to *any moment* (just before the bad `DELETE`)	more setup and storage

> **Key term — WAL and PITR:** the **write-ahead log (WAL)** records every change *before* it reaches the tables (chapter 1). **Point-in-time recovery (PITR)** restores a **base backup** (full physical copy) then replays archived WAL up to any moment — e.g. one second before a bad `DELETE`.

Also: **full** (everything), **differential** (since the last full), **incremental** (since the last backup of any kind; PostgreSQL 17 added incremental `pg_basebackup`), **hot/online** (taken while in use — all of the above).

pg_dump and pg_restore, for real

`pg_dump` takes a **consistent** snapshot of one database while others keep working (one repeatable-read transaction — an isolation level seeing data as of its start; chapter 13). Formats:

Format flag	Output	Restore with	Notes
`-F p` (plain, default)	a `.sql` script	`psql -f`	readable, editable; replayed statement by statement (`--single-transaction` for all-or-nothing)
`-F c` (custom)	one compressed archive	`pg_restore`	pick objects to restore, reorder, parallel restore (`-j`)
`-F d` (directory)	a folder, one file per table	`pg_restore`	the only format that can **dump** in parallel (`-j`)
`-F t` (tar)	a tar archive (a standard Unix file bundle)	`pg_restore`	like custom, but no compression or parallel restore

A full round trip on the book's server (temporary directory, deleted afterwards):

```
# code/12_backup_restore.sh (key lines; CONN="-h /tmp -p 5544 -U postgres")
pg_dump $CONN -d shop -F p -f "$WORK/shop.sql"          # plain SQL script
pg_dump $CONN -d shop -F c -f "$WORK/shop.dump"         # custom archive
pg_dump $CONN -d shop -F d -j 2 -f "$WORK/shop_dir"     # directory format, 2 parallel jobs
pg_restore -l "$WORK/shop.dump"                         # list the archive's contents
psql $CONN -d shop -c "DROP TABLE order_items"          # the "disaster"
pg_restore $CONN -d shop -t order_items "$WORK/shop.dump"      # bring back one table
pg_restore $CONN -d shop_copy -j 2 "$WORK/shop.dump"    # full restore into a new, empty DB
psql $CONN -d shop_plain -f "$WORK/shop.sql"            # plain dumps are replayed by psql
pg_dump $CONN -d shop -t products --schema-only         # structure only (--data-only: rows only)
```

Output (run with `bash chapters/09-ddl-dml-and-constraints/code/12_backup_restore.sh` from the book root):

```
== 1. a database to protect
shop: 10 customers, 14 orders, 22 order_items
...
-- plain dump contains 8 COPY blocks (one per table with data)
...
== 4. disaster: someone drops a table
ERROR:  relation "order_items" does not exist
== 5. restore just that table (definition + data) from the archive
shop: 10 customers, 14 orders, 22 order_items
-- note: -t restores the table and its rows, not its PK/FK constraints:
0 constraints on order_items
== 6. full restore into a NEW database, 2 parallel jobs
shop_copy: 10 customers, 14 orders, 22 order_items
3 constraints on order_items
== 7. a plain dump is restored with psql
shop_plain: 10 customers, 14 orders, 22 order_items
== 8. schema-only and data-only dumps of one table
CREATE TABLE statements in schema-only dump: 1, in data-only dump: 0
...
```

1. A plain dump is SQL: `CREATE TABLE`s, a `COPY … FROM stdin` block per table (`stdin` , *standard input*: rows inline in the script), then constraints and indexes (added *after* the data — faster).
2. A custom archive has a **table of contents** (`pg_restore -l`); `-t` , `-n` (schema), `-L` (edited list) choose what to restore.
3. `-t` restores definition and rows **but not constraints or indexes** (separate entries) — re-add via `-L` or the DDL.

Options: `--clean --if-exists` (drop before recreating), `--create` (create the database), `--no-owner` (restoring as another user), `-j N` . **Roles and tablespaces** (users/groups, named disk locations) are cluster-wide — also run `pg_dumpall --globals-only` .

Backup and restore in other databases

	Full backup	Restore	Notes
PostgreSQL	`pg_dump` (logical), `pg_basebackup` + WAL (physical, PITR)	`pg_restore` / `psql` ; restore base backup + replay WAL to a target time	command-line tools, not SQL statements
MySQL	`mysqldump` , MySQL Shell dump utilities (logical); Percona XtraBackup / MySQL Enterprise Backup (physical)	`mysql < dump.sql` ; binary logs (MySQL's log of every change) for point-in-time	
SQL Server	`BACKUP DATABASE db TO DISK = '… \db.bak'` (also `WITH DIFFERENTIAL` , `BACKUP LOG` , `TO URL` for cloud storage)	`RESTORE DATABASE db FROM DISK = '…' [WITH NORECOVERY / RECOVERY, STOPAT = …]`	backup and restore are T-SQL (SQL Server's SQL dialect) statements; the others mostly use separate tools
SQLite	`.backup file` in the shell, `VACUUM INTO 'file'`	copy the file back / `.restore`	the database is one file

Two tutorial errors corrected (SQL Server, not run):

- `BACKUP DATABASE` has no `WITH SCHEDULE = …` ; schedule via SQL Server Agent jobs or maintenance plans (elsewhere cron/systemd timers — Linux job schedulers — or a backup tool).
- There is no `RESTORE TABLE` : restore under **another database name** (`RESTORE DATABASE db_old FROM DISK = … WITH MOVE …`) and copy the table across — like `pg_restore -t` or `shop_copy` .

Best practices

- **3-2-1 rule**: 3 copies, on 2 kinds of storage, 1 off-site.
- **Test restores regularly** — an untested backup is a hope; automate a restore to a scratch server with sanity queries (row counts).

- **Encrypt** backups and restrict who can read them.
- **Match RPO/RTO**: nightly `pg_dump` alone risks 24 h of loss; PITR gets it to seconds.
- **Replication is not a backup** — replicas faithfully copy `DROP TABLE` (chapter 15).
- Managed databases (Amazon RDS, Cloud SQL, Azure) provide automated snapshots + PITR within a retention window (how long backups are kept).

In the interview

> **30-second answer:** "Logical backups (`pg_dump`) export schema and data — flexible, single tables, portable across versions, slow for huge databases. Physical backups copy the data files; with archived WAL they allow point-in-time recovery. Choose by RPO/RTO, keep 3-2-1 copies, encrypt, test restores — and replication doesn't protect against a bad `DELETE` ."

- *`DELETE` without `WHERE` at 14:03 — recovery?* → PITR: restore the last base backup to a new server, replay WAL to 14:02:59, copy the lost rows back (or fail over — switch the app to it).
- *Does pg_dump block a busy database?* → No (consistent snapshot), but its light locks block DDL like `DROP` / `ALTER` on dumped tables meanwhile.

Self-check

1. *Which formats does `pg_restore` read?* → custom, directory, tar (plain is replayed with `psql`).
2. *What does `pg_dumpall --globals-only` add?* → Roles and tablespaces.

Chapter 9 TL;DR

- **Types** validate as well as store: `numeric` for money, `integer` / `bigint` for ids, `text` , `timestamptz` , `date` , `boolean` .
- **`CREATE`** databases, schemas, tables (`IF NOT EXISTS` , `AS SELECT` = data without constraints, `LIKE … INCLUDING ALL` = structure, `TEMP`).
- **Keys**: super ⊇ candidate → one primary (unique + not null) + alternate keys (`UNIQUE`); composite keys; foreign keys with `ON DELETE CASCADE | SET NULL | RESTRICT | NO ACTION` . `CHECK` rejects only false; `UNIQUE` allows several NULLs (one in SQL Server). Name constraints; index FK columns.
- **Generated keys**: `GENERATED ALWAYS AS IDENTITY` (not `serial`), sequences have gaps, UUIDs for distributed ids, `GENERATED … STORED` for derived columns.
- **`ALTER`** : add/drop/rename columns, `TYPE … USING` , `SET NOT NULL` after backfilling, `NOT VALID` + `VALIDATE` on big tables; watch exclusive locks.
- **`DROP`** removes the object, **`TRUNCATE`** empties it fast (transactional in PostgreSQL, `RESTART IDENTITY` , `CASCADE`), **`DELETE`** removes chosen rows.
- **Indexes**: `CREATE [UNIQUE] INDEX [CONCURRENTLY]` ; clustered vs non-clustered; details in chapter 12.
- **DML**: `INSERT` (list columns, multi-row, `INSERT … SELECT`), `UPDATE` (`SET` sees old values, `FROM`), `DELETE` (`USING` , children first), all with `RETURNING` . Practise destructive statements in `BEGIN … ROLLBACK` .
- **Upsert**: `INSERT … ON CONFLICT … DO NOTHING / DO UPDATE SET … = EXCLUDED.…` ; `MERGE` needs PostgreSQL 15+.
- **Backups**: `pg_dump` (plain/custom/directory) + `pg_restore` (selective, parallel); physical + WAL for PITR; RPO/RTO, 3-2-1, test restores.

Next: chapter 10 — data modeling and ER diagrams, which decides *which* tables and keys to create in the first place.

Chapter 10 — Data Modeling and ER Diagrams

In real work nobody hands you tables: someone describes a business ("we rent bikes to people") and you decide what to store, how the pieces connect and which rules the database enforces. That is **data modeling**, and it matters more than any query, because a fact left out of the model is a question no SQL can ever answer.

The path: a plain-language brief → a drawing business people can check (an **ER diagram**, in **Chen notation** and **crow's-foot notation** — the one `mermaid erDiagram` draws; Mermaid turns short text into diagrams) → precise tables with keys and constraints (the **relational model**, §10.11) → `CREATE TABLE` on PostgreSQL. Covered: the three model levels, a seven-step process and best practices, entities, the five kinds of attribute, relationships (degree, cardinality, participation, attributes), weak entities, a worked employee example, relations, domains, `NULL`, keys, integrity constraints, and ER-to-table mapping. Chapter 11 then tests tables for repeated data; Chapter 17 applies everything to nine case studies.

Code in `code/`: `00_schema.sql` builds a small bike-rental database; `01`–`04` each run on a fresh database (Chapter 1 server). Outputs are real psql output.

10.1 What data modeling is

What it is

Data modeling is deciding three things before you write any SQL:

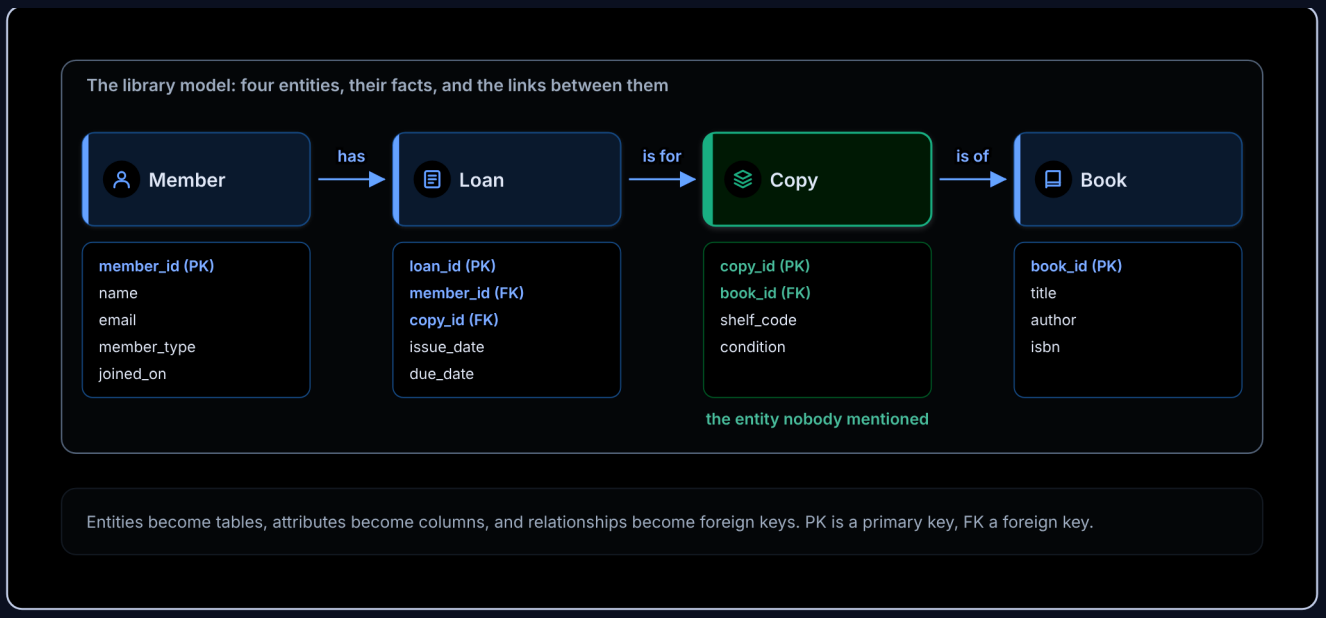
Part	Plain meaning	Usually becomes
Entity	a *thing* you keep data about (Customer, Bike, Rental) — almost always a noun	a table
Attribute	one *fact* about an entity (a customer's email, a bike's frame number)	a column
Relationship	a *link* between entities (a customer *rents* a bike)	a foreign key (a column holding another table's key), or a table of its own

A **data model** pictures those three things and nothing else — no indexes (lookup structures that speed up searches), `VARCHAR` lengths or product choice: the architect's floor plan before anyone picks bricks.

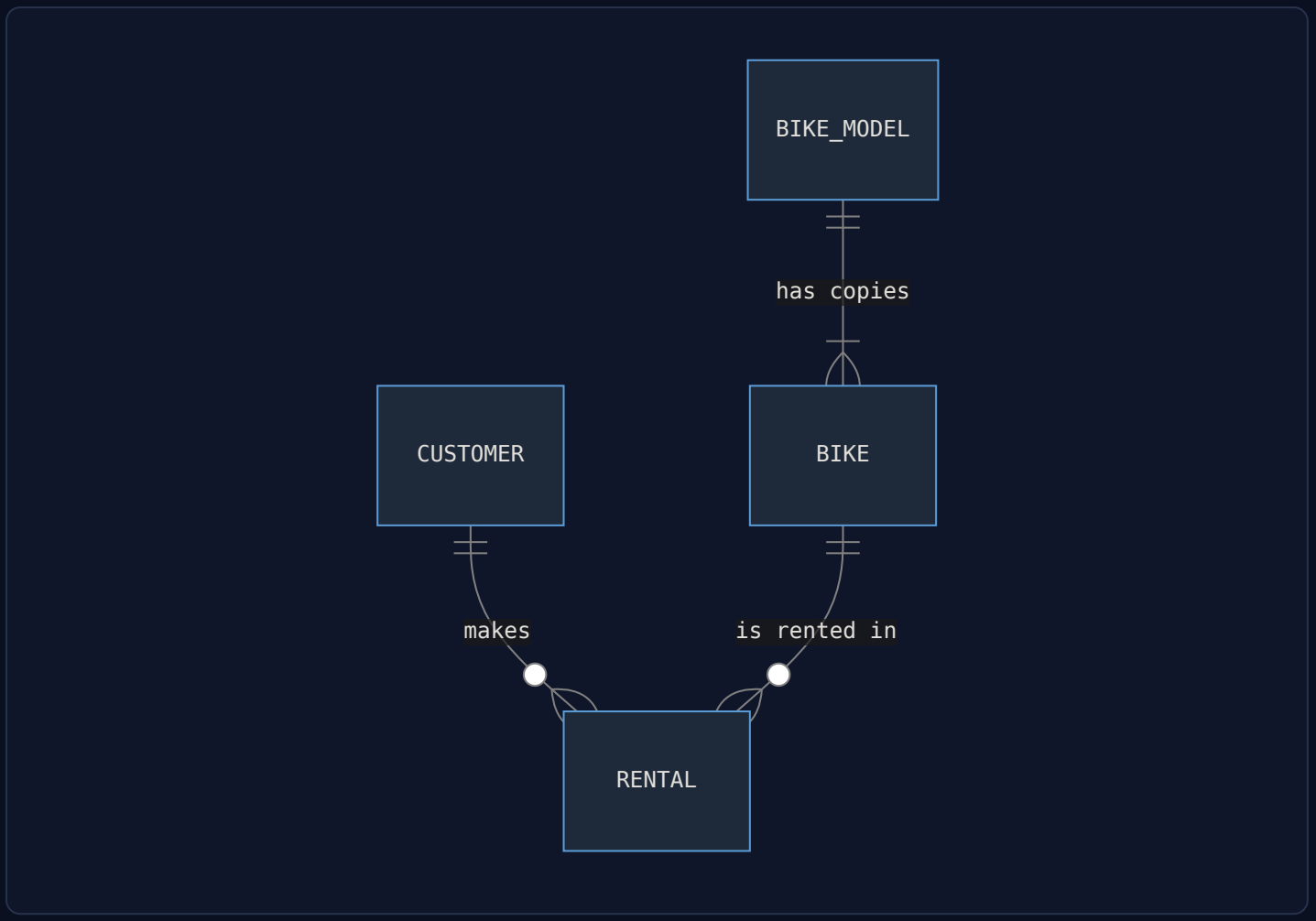
The problem it solves — the question you cannot answer

"We rent bikes to customers" sounds like `customer` and `bike`. Then the owner asks: *"We own three 'Urban 3' bikes. One came back with a bent wheel — which one, and who had it?"* If `bike` describes a **model** (name, type, hourly rate), it cannot answer: the physical object with a frame number differs from the product it exemplifies. Add an entity for the physical bike and the question is a short join — the database simply records one more fact.

Figure — the same idea in the course's library example: a Book (a title) is not a Copy (the object on the shelf) (course diagram)



The bike shop's model therefore has four entities: `customer`, `bike_model` (the "title"), `bike` (the physical object) and `rental` (who had which bike, when).



Who reads a data model

Reader	What they check
Business people (shop owner, registrar, librarian)	"Is anything missing? Are the rules right?" — they spot a forgotten entity in seconds
Developers	which tables exist and what each one means
Database administrators	storage, indexes, backups (mostly at the physical level)
Analysts	where each fact lives, so reports are correct

Non-technical readers must follow it, so describe it in everyday language first, detail later — hence §10.2's three levels.

What data modeling is *not*

- **Not writing SQL** — that comes last and is nearly mechanical (§10.15).
- **Not performance tuning** — indexes and query plans (the database's chosen execution strategy) come later (Chapter 12); quickly answering the wrong question helps nobody.
- **Not application code** — very different apps can share one model.

Why it matters

- **Fewer wrong answers** — rules in the model become constraints that every program must obey.
- **Less repeated data** — each fact stored once cannot contradict itself.
- **Cheaper change** — a model that matches reality absorbs new requirements; one that doesn't fights each of them.
- **Shared vocabulary** — when the team says "rental", everyone means the same thing.

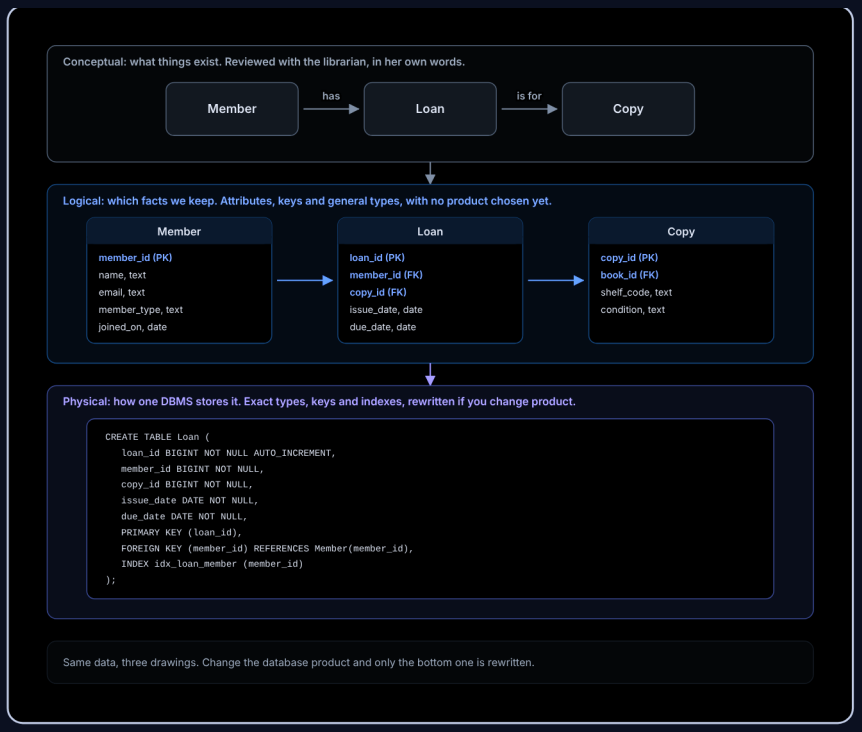
10.2 The three levels of a data model

What it is

The same model drawn three times, with rising detail, for different audiences:

	Conceptual	Logical	Physical
Shows	entities and relationships only	+ attributes, keys, general types (text/number/date)	+ exact types, indexes, partitioning (splitting a big table into pieces), storage options
Detail	low	medium	high
Main audience	business people	designers, developers	DBAs (database administrators), developers
Tied to a DBMS (database management system, e.g. PostgreSQL) product?	no	no	**yes**
Question it answers	"Is every thing present, and is each link correct?"	"Which facts are recorded, and which keys join the tables?"	"How do we store this well on *this* database?"
Bike-shop example	Customer *rents* Bike; Bike *is of* BikeModel	`rental(rental_id, customer_id → customer, bike_id → bike, out_at, due_at, back_at)`	`CREATE TABLE rental (... timestamp ..., INDEX on customer_id)`

Figure — the same data at the three levels (course diagram)



How it works

1. **Conceptual** — boxes and lines in the users' words; valid for a relational database, a document store (JSON-like documents instead of tables) or a paper ledger.
2. **Logical** — attributes, primary/foreign keys, general types; **normalization** (removing repeated data, Chapter 11) happens here. Still product-neutral.
3. **Physical** — for one DBMS: `bigint` vs `int`, `numeric(6,2)`, `timestamp`, indexes for known queries, identity columns (auto-numbered keys), partitioning.

The physical version of the rental table from `code/00_schema.sql`:

```
CREATE TABLE rental (
  rental_id   bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  customer_id bigint      NOT NULL REFERENCES customer(customer_id),
  bike_id     bigint      NOT NULL REFERENCES bike(bike_id),
  out_at      timestamp   NOT NULL,
  due_at      timestamp   NOT NULL,
  back_at     timestamp,                          -- NULL = bike still out (meaningful NULL)
  CHECK (due_at > out_at),
  CHECK (back_at IS NULL OR back_at >= out_at)
);
CREATE INDEX rental_customer_idx ON rental(customer_id);   -- "rentals of this customer"
CREATE INDEX rental_bike_idx     ON rental(bike_id);       -- "who has this bike"
```

Entity from the conceptual model, columns and keys from the logical; only exact types, the identity column and the indexes are physical — and identity syntax differs by product:

Auto-numbered key	Syntax
PostgreSQL 10+	`bigint GENERATED ALWAYS AS IDENTITY` (older: `bigserial`)
MySQL	`BIGINT AUTO_INCREMENT`
SQLite	`INTEGER PRIMARY KEY` (becomes the rowid, SQLite's built-in row number)
SQL Server	`BIGINT IDENTITY(1,1)`

Why bother with three levels

- **Each reader gets a suitable version** — an owner reviewing three boxes spots a missing entity; shown `CREATE TABLE`, nothing.

- **Mistakes caught cheaply** — a missing entity costs a conversation at the conceptual level, a migration (script changing an existing schema and data) at two million rows.
- **Technology isolated** — MySQL → PostgreSQL rewrites only the physical model: **data independence** (Chapter 1) at design time.

Working out the logical model often shows the conceptual one was wrong; going back is cheap because the drawing is small.

Common mistakes

- Showing column types to business reviewers (they stop reading).
- Calling a diagram with attributes and keys "conceptual" — once keys appear it is logical.
- Treating logical and physical as the same: *logical = what the business needs; physical = how one product stores it.*

Self-check

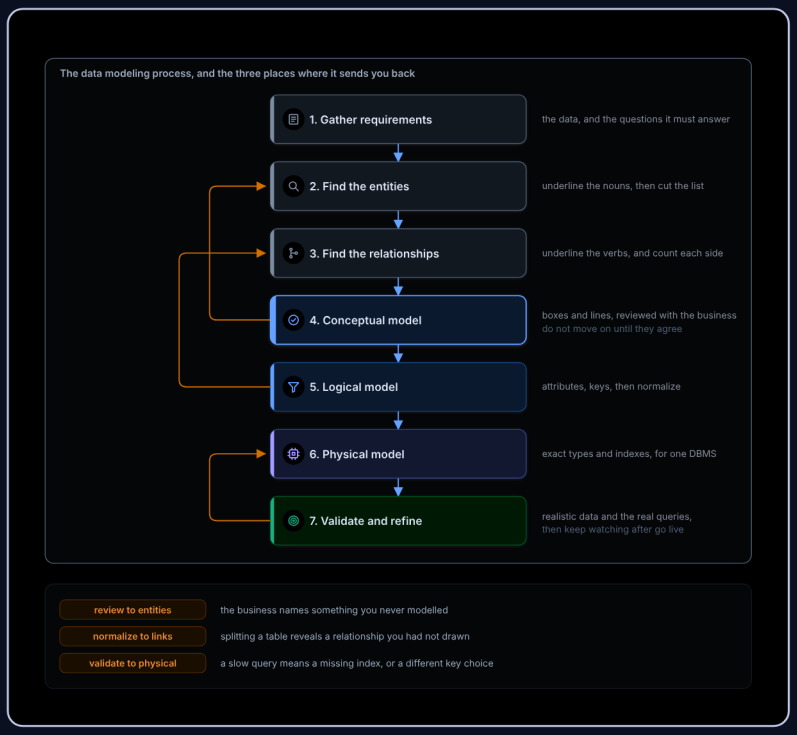
1. Which level first shows primary keys, and hosts normalization? → **Logical.**
2. Migrating SQL Server → PostgreSQL changes which levels? → **Only the physical.**

10.3 The data modeling process

What it is

A fixed order of work, each step producing something reviewable. Starting with tables is the classic mistake — it fixes the answer before you understand the question.

Figure — seven steps and the three places the work loops back (course diagram)



How it works — the bike shop, step by step

1. **Gather requirements — including the questions** the system must answer:
    - Customers are casual or members; members may keep a bike longer.
    - The shop owns several physical bikes of each model.
    - A customer takes a bike at a time and must return it by a due time.
    - Late returns cost 2.00 per started hour.
    - Staff must see which bikes are out and who has them.

An unanswerable question is a design bug, free to fix now.

2. **Find the entities: underline the nouns** (customer, member, model, bike, time, fee, staff). Keep a noun only if it has *several facts* and *many instances*:
   - customer, bike_model, bike, rental → entities.
   - time → one value → an attribute.
   - casual/member → two kinds of customer → start with a `customer_type` attribute.
   - late fee → calculable from times → a **derived** value (§10.6), not stored.
   - staff → users of the system, not necessarily data in it.
3. **Find the relationships: underline the verbs** (customer *rents* bike; bike *is of* bike_model); ask "how many on each side?" (cardinality, §10.7) and write it in words.
4. **Review the conceptual model** with the owner, read back as sentences. She will correct you ("customers can also *reserve* a bike") — a new entity for the price of one box. Don't continue until the business agrees.
5. **Logical model** — attributes, keys, general types; normalize.
6. **Physical model** — exact types, an index for every step-1 question (`rental.customer_id`, `rental.bike_id`).
7. **Validate, implement, refine** — realistic volumes (enough for slowness to show), run the step-1 questions. After go-live keep watching: right for 10 thousand rentals may need a new index at 10 million.

The loops back are part of the process

Trigger	Sends you back to
Stakeholder (anyone with a say in the system) review reveals something new (reservations)	step 2 (entities)
Normalization splits a table and exposes a hidden link	step 3 (relationships)
Validation finds a slow query	step 6 (index or key choice)

Looping back is the method working: every loop costs less than releasing the error.

Trade-offs to keep in mind throughout

- **Normalize vs denormalize** — normalize first (each fact once); add redundancy (deliberate copies) only for a measured slow query (Chapter 11).
- **Growth** — plan for rows in three years; it changes key and index choices.
- **Access control** — mark sensitive data early (emails, payments).
- **Integrity** — each step-1 rule belongs in a constraint, not an application `if`.

The model answering the step-1 questions (`code/01_bike_questions.sql`)

```sql
-- Q1: which bikes are out right now, and who has them?
SELECT b.frame_no, m.model_name, c.full_name, r.due_at
FROM rental r
JOIN bike b       ON b.bike_id = r.bike_id
JOIN bike_model m ON m.model_id = b.model_id
JOIN customer c   ON c.customer_id = r.customer_id
WHERE r.back_at IS NULL
ORDER BY b.frame_no;
-- Q2: bikes never rented (NOT EXISTS anti-join)   Q4: late fee, derived at read time
SELECT r.rental_id, c.full_name,
       ceil(extract(epoch FROM (r.back_at - r.due_at)) / 3600) * 2.00 AS late_fee
FROM rental r JOIN customer c USING (customer_id)
WHERE r.back_at > r.due_at;
```

Output:

```
--- Q1: which bikes are out right now, and who has them?
 frame_no | model_name |  full_name   |       due_at
----------+------------+--------------+---------------------
 FR-2001  | Ridge 29   | Meera Nair   | 2026-09-27 18:00:00
 FR-3001  | Volt City  | Lena Fischer | 2026-09-27 10:30:00
(2 rows)

--- Q2: which physical bikes have never been rented?
 frame_no | model_name
----------+------------
 FR-1002  | Urban 3
 FR-1003  | Urban 3
(2 rows)

--- Q3: how many bikes of each model exist, and how many are out?
 model_name | bikes | out_now
------------+-------+---------
 Ridge 29   |     1 |       1
 Urban 3    |     3 |       0
 Volt City  |     1 |       1
(3 rows)

--- Q4: late returns, with a late fee of 2.00 per started hour (fee is derived, not stored)
 rental_id | full_name  | late_fee
-----------+------------+----------
         2 | Tom Okafor |     2.00
(1 row)
```

Q2 and Q3 work only because `bike` (the object) is separate from `bike_model`.

In the interview

30-second answer: "I start from requirements *and the questions the system must answer*. Nouns give candidate entities, verbs give relationships. I draw a conceptual model and review it with the business, then add attributes and keys and normalize (logical), then choose types and indexes for the target database (physical), and validate with realistic data and the real queries. I expect to loop back."

- *How do you decide whether a noun is an entity or an attribute?* → Entity if it carries several facts and has many instances; a single value (a date, a colour) is an attribute.
- *When do you add indexes?* → At the physical level, for the queries written down in requirements — not "on every column".

10.4 Best practices in data modeling

Two models that both work on day one can differ enormously a year later because of small early decisions:

Practice	Rule	Why
Questions first	Write the list of questions before drawing.	A model is only "correct" relative to the questions it must answer.
Rules in the database	Every stated rule → a `PRIMARY KEY`, `FOREIGN KEY`, `NOT NULL`, `UNIQUE` or `CHECK`.	A constraint protects every program that ever connects, including tomorrow's ad-hoc (one-off, written for a single task) script; an `if` protects only the program it is in.
One naming convention	Decide once (`snake_case` = lower case with underscores, singular *or* plural tables) and never mix. Same thing = same name everywhere (`customer_id`, never `cust_no` elsewhere). Name FKs after their target.	Joins read naturally; nobody has to guess.
Types on purpose	Dates as `date` / `timestamp`, never text; money as `numeric`, never `float` / `double`; text sized for real data. Prefer the narrowest type that fits.	Text dates accept "31 February" and sort wrongly; floating point cannot represent most decimals exactly, so totals drift; narrower rows = more rows per page (the fixed-size block the database reads from disk) = fewer disk reads.
Decide what NULL means	`NOT NULL` whenever the fact must exist; allow `NULL` only when "missing" is meaningful (`back_at IS NULL` = bike still out).	`NULL` means unknown/not applicable, is not 0 or '' and is not equal to anything (§10.12).
Keys deliberately	Surrogate primary key (a generated id with no business meaning) + `UNIQUE` on the natural key (a real-world identifier: `email`, `frame_no`). Keep keys narrow.	Natural values change; a PK is copied into every FK and index.
Normalize, then denormalize with evidence	Default to 3NF (third normal form, the usual target of normalization); denormalize only after a measured slow query that an index could not fix — and document why.	Redundant copies can disagree (Chapter 11).
Index for real queries	Index what the requirement questions filter/join on; nothing else.	Every index slows every write and costs disk.
Design for predictable growth	Estimate rows in 3 years; don't build tables for imagined requirements.	Speculative tables become the parts nobody understands.
Document meaning	For each entity/column: what it means, allowed values, units.	Anyone can see `status` is text; only docs say which 4 values it holds.

The bike schema's `CHECK (due_at > out_at)` and `CHECK (back_at IS NULL OR back_at >= out_at)` (§10.2) apply "rules in the database" for every client.

Common mistakes

- **A list in one column** (`phones = '555-1, 555-2'`) — can't be searched, counted or constrained → separate table.
- **Numbered columns** (`phone1, phone2, phone3`) — a list across columns; breaks at the fourth phone.
- **One huge table** — 60 mostly-empty columns are several unseparated entities.
- **Over-normalizing** — seven joins for an everyday question is a real cost; 3NF is the usual stop.
- **Rules only in code** — another program will break them.

Real-world examples

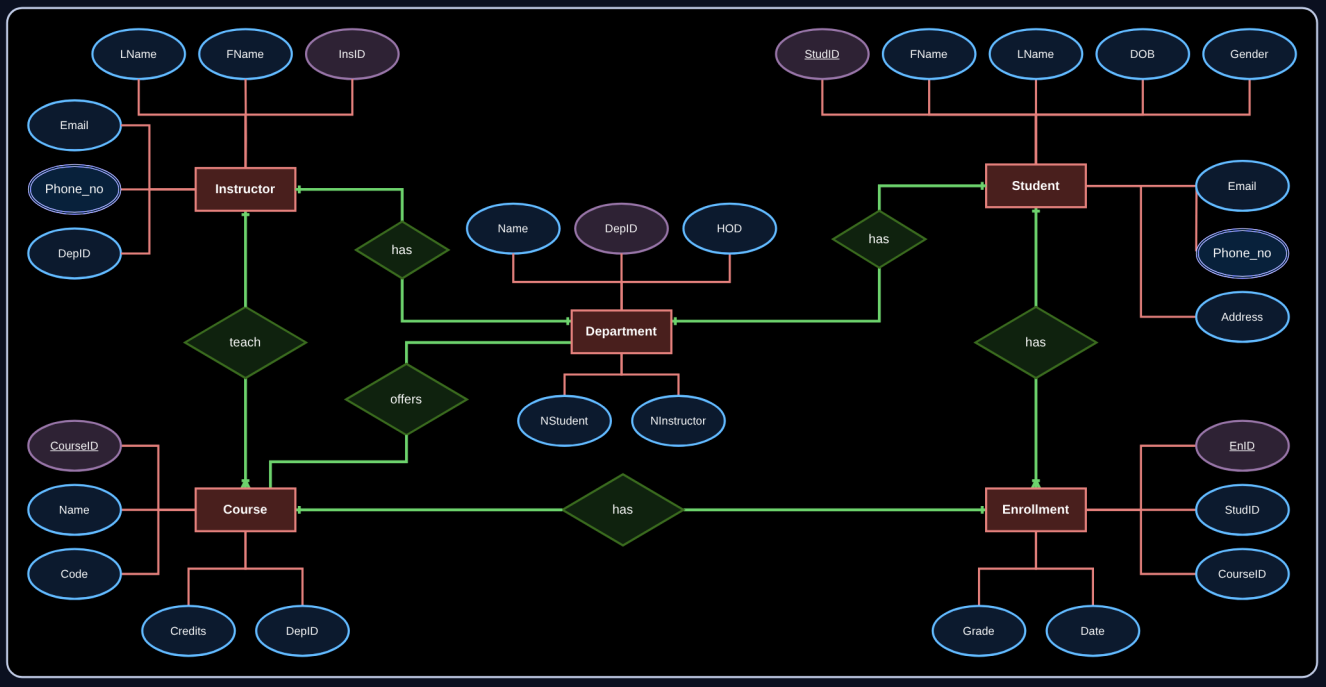
- **Stripe's API** (the payments company's programming interface) stores amounts as integers in the smallest currency unit (cents) — "never float for money".
- **ISO country codes** (two-letter standard codes such as `IN`, `GB`) are `char(2)` — "size text for the data".

10.5 The Entity-Relationship (ER) model and its notations

What it is

The **Entity-Relationship model** (ER model), proposed by Peter Chen in 1976, sketches entities, attributes and relationships before any table exists. An **ER diagram (ERD)** draws the conceptual model, then the logical one (plus attributes, keys, cardinalities); it never names a DBMS.

Figure — a complete Chen-notation ER diagram for a student management system; by §10.10 you can read every symbol (course diagram)



Two notations for the same model

	Chen notation	Crow's-foot notation (IE / Martin)
Entity	rectangle	rectangle with a title bar
Attribute	ellipse attached by a line	a row listed *inside* the rectangle
Relationship	diamond labelled with a verb	the line itself, labelled with a verb
Cardinality	`1`, `N`, `M` written near the diamond (or `(min,max)` pairs)	the *shape* of each line end (see table below)
Participation	double line = total, single = partial	inner symbol: bar = at least one, circle = may be zero
Used by	textbooks, exams, teaching	nearly every design tool (dbdiagram, MySQL Workbench, ERwin, Lucidchart), and `mermaid erDiagram`
Strength	shows the concepts explicitly	compact; fits 40 tables on a screen

Neither is "more correct": learn Chen for the concepts, then crow's foot reads itself. This book shows concepts in the course's Chen figures and every schema in mermaid crow's foot.

Reading crow's-foot line ends — *outer* symbol = maximum, *inner* = minimum:

Mermaid syntax (left end / right end)	Picture	Meaning (min, max)
`|o` / `o|`	circle + bar	zero or one `(0,1)`
`||` / `||`	bar + bar	exactly one `(1,1)`
`}o` / `o{`	circle + crow's foot	zero or more `(0,N)`
`}|` / `|{`	bar + crow's foot	one or more `(1,N)`

`CUSTOMER ||--o{ RENTAL : makes` reads: *each rental belongs to exactly one customer; a customer makes zero or more rentals*. Strictly, mermaid's solid `--` marks an **identifying** relationship (the child's key includes the parent's — a weak entity, §10.9) and dashed `..` a non-identifying one; this book draws all lines solid, so spot a weak entity by a PK containing the owner's key.

Strengths and limits of an ERD

- **Good at** showing things, links and counts so even non-technical reviewers notice gaps.
- **Not** a table diagram (one M:N diamond becomes *three* tables, §10.7), a performance document (no indexes or storage), or a process diagram (a student enrols in a course — not which screens or approvals).

10.6 Entities and the five kinds of attribute

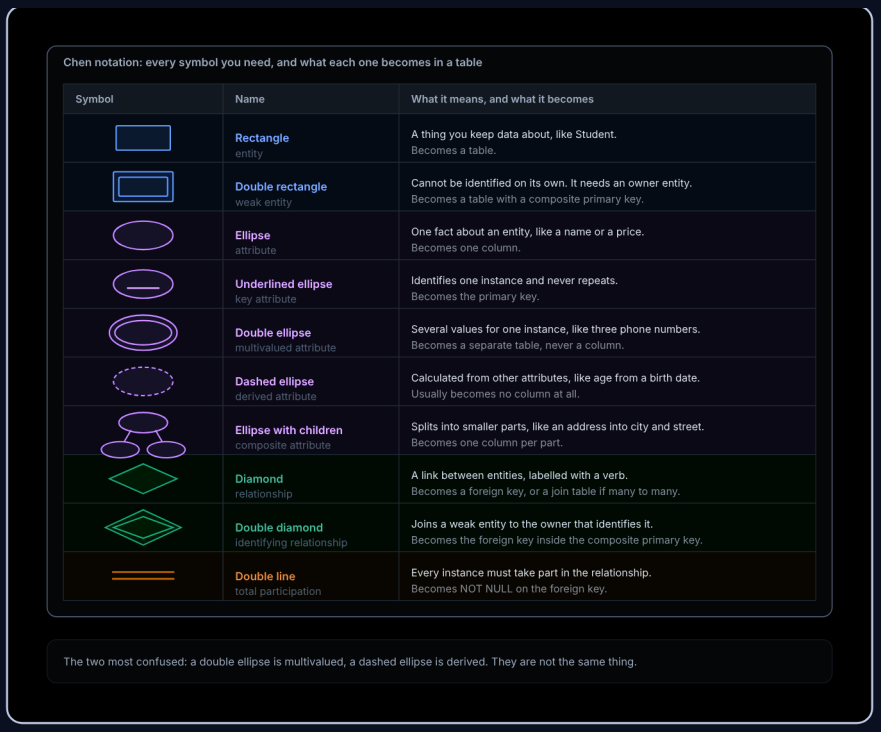
Entities: type, instance, set

An **entity** is an object or concept you record data about and can distinguish from others like it. Three loosely used words:

Term	Meaning	Example
Entity type	the general kind; what the rectangle is labelled	`Employee`
Entity instance	one particular member	employee #2, Ravi Menon
Entity set	all instances that exist right now	every row in the `employee` table today

You *draw* types; the database *stores* sets.

Figure — every Chen symbol and its meaning (course diagram)



The five kinds of attribute

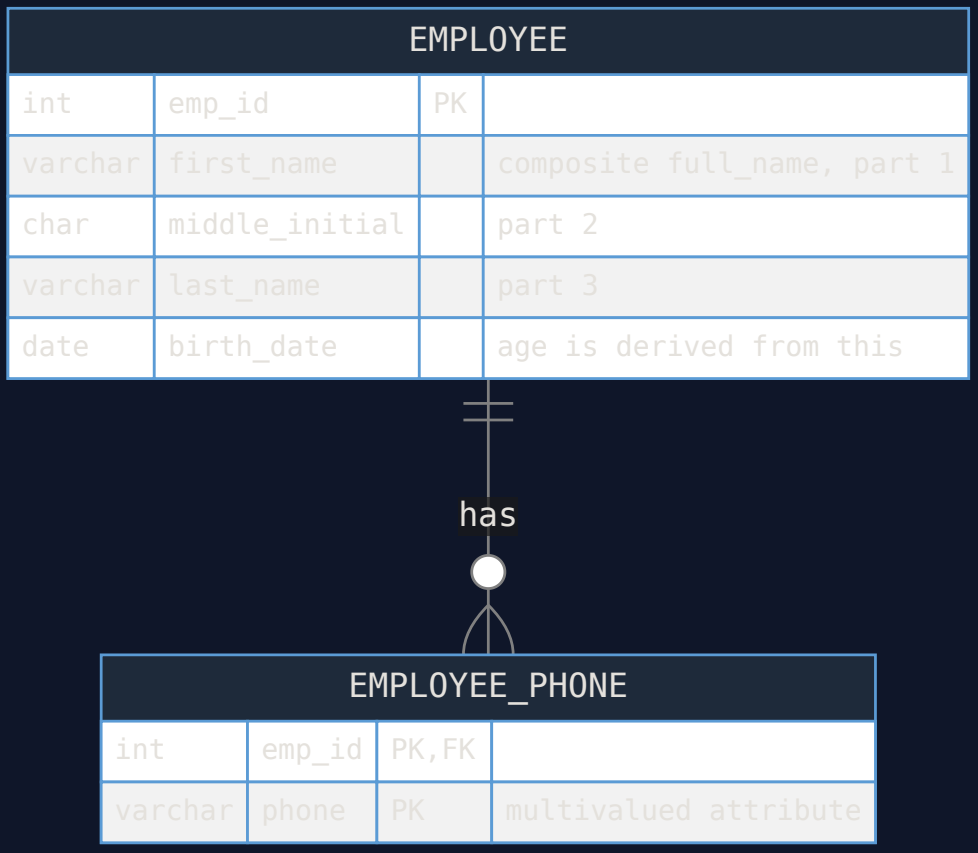
Kind	Chen symbol	Example	Becomes in tables
Key	ellipse, name **underlined**	`emp_id`	the primary key
Simple (atomic)	plain ellipse	`birth_date`	one column
Composite	ellipse with child ellipses	`full_name` = first + middle initial + last	one column **per part**
Multivalued	**double** ellipse	`phone` (an employee may have several)	a **separate table** (owner key + value)
Derived	**dashed** ellipse	`age` (from `birth_date`)	usually **no column** — computed on read

The two most confused: **double ellipse = multivalued**, **dashed ellipse = derived**.

(All three special shapes appear together in the employee diagram of §10.10.) Why each rule holds:

- **Composite → split** — an address in one column can't be searched by city (if you *never* query the parts, one column is fine).
- **Multivalued → own table** — a cell holds one value (§10.12); `phone1, phone2, phone3` is the mistake the double ellipse prevents.
- **Derived → compute** — a stored `age` is wrong a year later. Store only if expensive *and* kept fresh, or **frozen in time** (an order total at purchase-day prices isn't derivable from *today's* data).

Crow's foot has no special shapes: composites are already columns, multivalued attributes separate boxes, derived ones omitted or commented:



Self-check

1. `age` next to `birth_date` — which kind, what column? → **Derived**; no column, compute it.

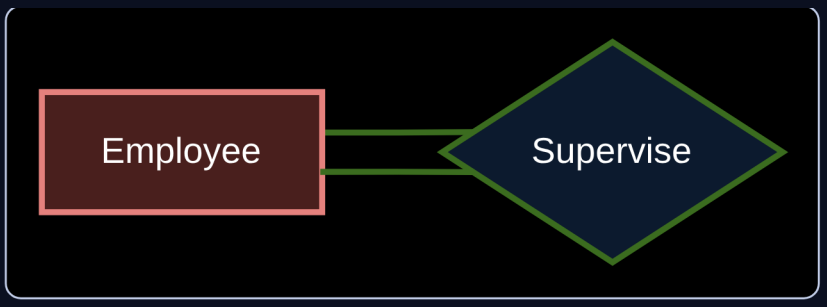
10.7 Relationships: degree, cardinality and participation

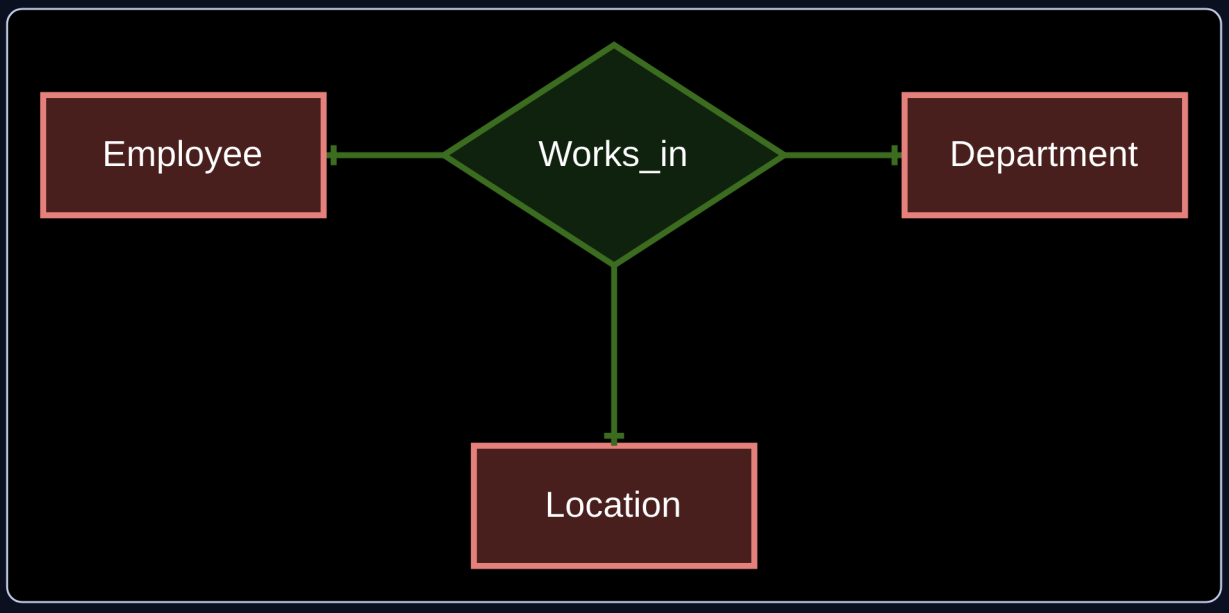
A **relationship** is an association between entities — a Chen diamond with a verb that reads well aloud ("Employee *works in* Department", not "*has*"). The **relationship type** is what you draw; the **relationship set** the actual stored pairs. Three separate questions describe it: **degree** (how many entity *types*), **cardinality** (how many *instances* per side), **participation** (is taking part *required*).

Degree

Degree	Name	Example	Tables
1	**Unary** (recursive)	an employee *supervises* employees	self-referencing FK: `employee.supervisor_id → employee.emp_id`
2	**Binary**	employee *works in* department	the usual case
3	**Ternary**	supplier *supplies* part *to* site	one table with three FKs, usually all three in the PK

Figure — unary and ternary relationships (course diagrams)





A ternary relationship records what three binary ones cannot: *which* combination is true. `code/04_ternary_and_weak_entity.sql` stores three supplier-part-site facts, keeps only binary pairs and joins them back:

```
WITH sp AS (SELECT DISTINCT supplier_id, part_id FROM supplies),
     ss AS (SELECT DISTINCT supplier_id, site_id FROM supplies)
SELECT ... FROM sp JOIN ss USING (supplier_id) ...   -- rebuild from pairs
```

Output:

```
--- the ternary fact: who supplies what, where
 supplier | part  | city
----------+-------+-------
 Acme     | bolt  | Leeds
 Acme     | panel | Pune
 Bolt Co  | bolt  | Pune
(3 rows)

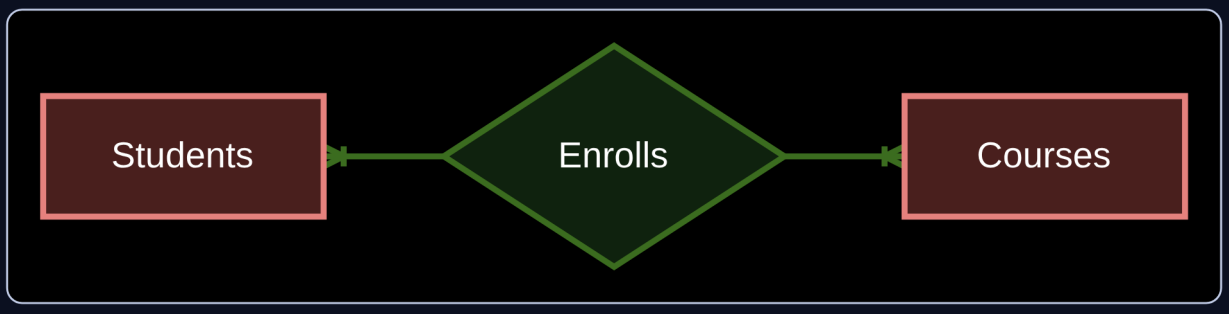
--- keep only binary pairs (supplier-part, supplier-site) and join them back:
 supplier | part  | city  | really_true
----------+-------+-------+-------------
 Acme     | bolt  | Leeds | t
 Acme     | bolt  | Pune  | f
 Acme     | panel | Leeds | f
 Acme     | panel | Pune  | t
 Bolt Co  | bolt  | Pune  | t
(5 rows)
```

From the pairs you can't tell whether Acme sends bolts to Pune. Ternaries are rare — check the fact truly needs all three (Chapter 11's 5NF, §11.13, formalises this).

Cardinality (mapping cardinality)

Cardinality	Meaning	Example
1:1	each side links to at most one of the other	employee ↔ payroll record
1:N	one on the left, many on the right	a department employs many employees
N:1	1:N viewed from the opposite side	many employees work in one department
M:N	many on both sides	employees work on many projects; a project has many employees

Figure — many-to-many in Chen notation (course diagram)



How each cardinality becomes tables

Cardinality	Rule	Why
1:N / N:1	FK on the **many** side (`employee.dept_id`)	the "one" side would need a list in a cell
M:N	a new **junction table** (also *join*, *bridge* or *associative* table) holding both FKs, which together form its PK (`works_on(emp_id, project_id)`)	there is no single side to put it on
1:1	FK on **either** side **+** `UNIQUE` ; prefer the side with total participation so it can be `NOT NULL`	without `UNIQUE` it silently becomes 1:N
Unary	FK to the same table	

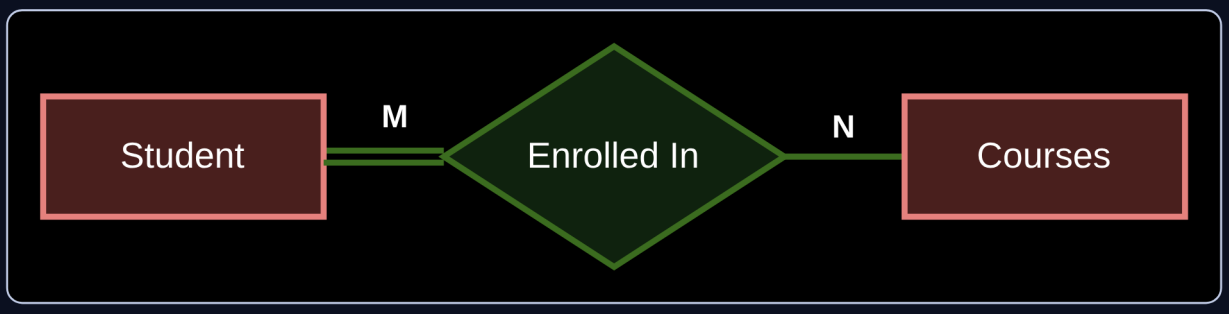
Memorise: **every M:N link costs three tables in SQL.**

Participation

Participation says whether every instance *must* take part.

	Chen drawing	Meaning	Becomes
Total (mandatory)	double line	every employee must work in a department	FK `NOT NULL`
Partial (optional)	single line	an employee may have no supervisor	FK allows `NULL`

Figure — total participation (double line) (course diagram)



Skipping participation is easy and expensive: it decides whether a column can be empty, affecting every query (outer vs inner joins, `IS NULL` checks).

The (min, max) notation

Tools often label each end **(min, max)** — fewest and most times one instance takes part:

Pair	Meaning	Participation	Crow's foot
`(0,1)`	at most once, maybe never	partial	`o|`
`(1,1)`	exactly once	total	`||`
`(0,N)`	any number, including none	partial	`o{`
`(1,N)`	at least once	total	`|{`

Minimum = participation; maximum = cardinality — one label says both. Careful: Chen's `(min,max)` is usually written next to the entity it describes ("each employee works in (1,1) departments") — the *opposite* end from crow's foot. Check the legend.

In the interview

30-second answer: "Degree counts the entity types involved — one (unary), two (binary) or three (ternary). Cardinality is how many instances link: 1:1, 1:N or M:N. Participation is whether taking part is mandatory. 1:N puts the FK on the many side; M:N needs a junction table; 1:1 is an FK plus UNIQUE; total participation makes the FK NOT NULL."

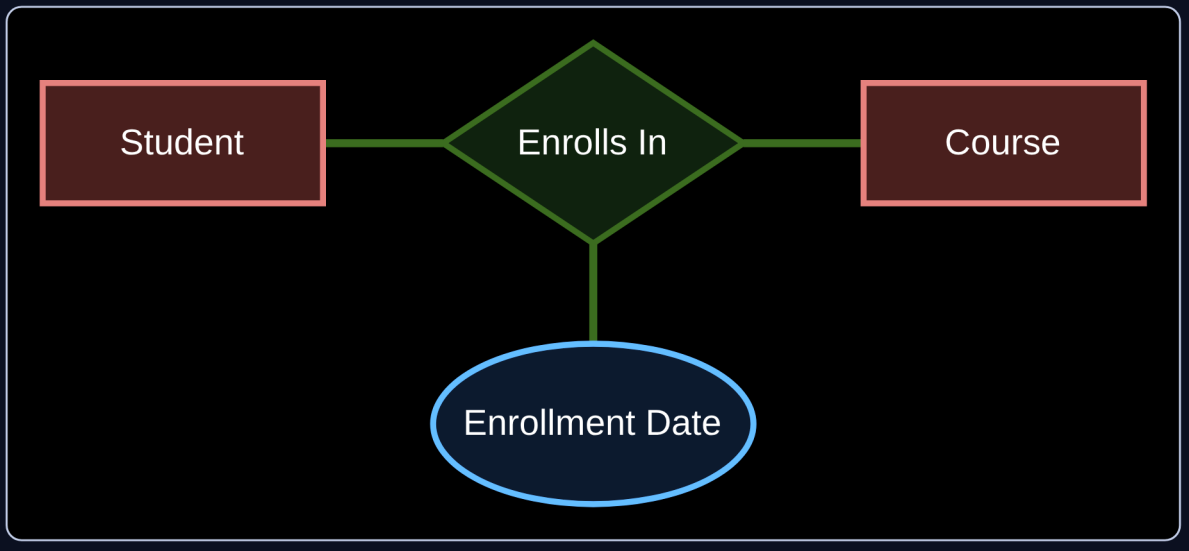
- *Where does the FK go in a 1:N?* → Many side. *Why not the one side?* → It would need a list in one cell, which violates 1NF (first normal form: one value per cell).
- *How do you enforce 1:1 in SQL?* → FK + `UNIQUE` on it (or share the same PK in both tables).
- *Enforce "every department has at least one employee" with a plain FK?* → No; an FK constrains the child side. Use a deferred constraint trigger (code run automatically on a change, checked at commit) or application logic.

10.8 Attributes of relationships

What it is

Some facts describe the **link**: an employee's hours on a project depend on *both*. In Chen such an attribute hangs off the diamond.

Figure — an attribute on a relationship (course diagram)



The test: *would you need to know both entities to know the value?*

Attribute	Needs	Belongs on
`weekly_hours`	employee **and** project	the relationship *works on*
`start_date` of an assignment	employee **and** project	the relationship
`title`	project only	Project
`last_name`	employee only	Employee

Where it lands in tables

Put the attribute wherever the relationship itself ended up:

- **M:N** → junction table column: `works_on(emp_id, project_id, weekly_hours)`.
- **1:N** → the many side beside the FK (`role_in_dept` on `employee` next to `dept_id`).
- **1:1** → wherever the FK went.

On an entity it forces a false rule: `weekly_hours` on `employee` allows one number however many projects.

When a relationship should become an entity

A link with many facts (rental: out, due, return times, fee) becomes an entity — same tables, one more box for a noun people already use. **Test:** if the business names the link ("rental", "loan", "enrolment", "booking"), make it an entity — why `rental` is an entity, not a diamond.

10.9 Weak entities

What it is

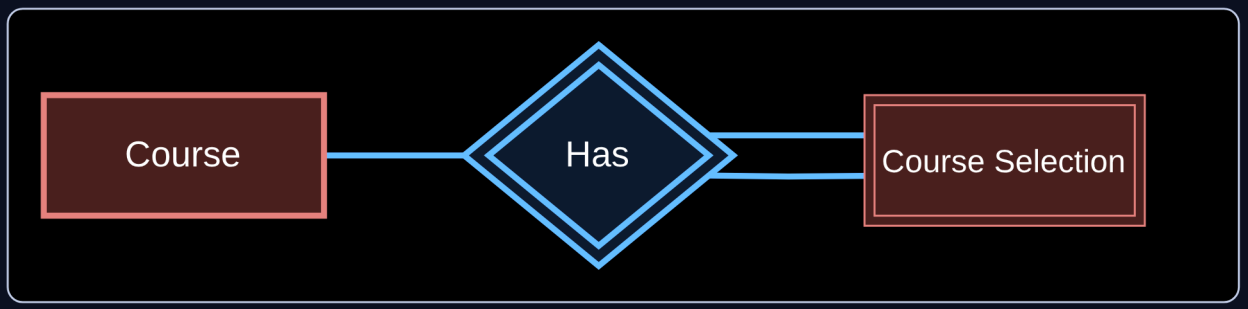
A **weak entity** has nothing of its own that singles it out: "Room 101" is ambiguous; only *(Main building, room 101)* identifies it. It depends on its **owner** (identifying / strong entity). Four properties:

1. **No key of its own** — nothing inside it is unique across all instances.
2. **A partial key** (also called a **discriminator**) — unique *within one owner* (`room_no`).
3. **Existence dependency** — delete the building and its rooms mean nothing.
4. **Total participation in the identifying relationship** — no room belongs to no building.

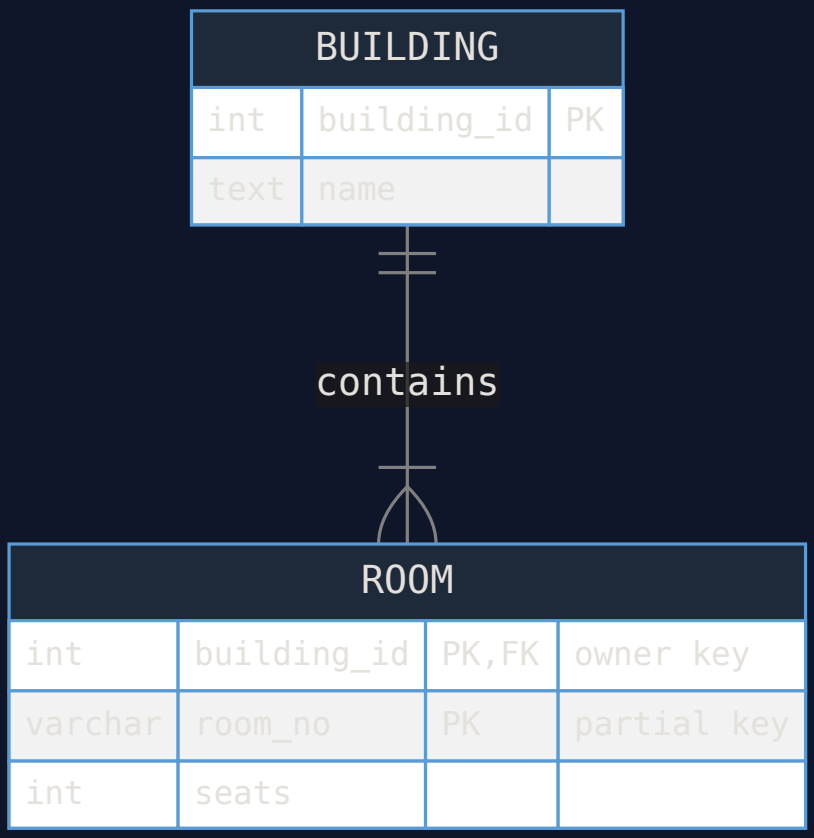
How it is drawn

Chen symbol	Meaning
double rectangle	the weak entity
double diamond	the **identifying relationship**
dashed underline	the partial key
double line (weak side)	total participation

Figure — a weak entity in Chen notation (course diagram)



In crow's foot (mermaid), a weak entity is shown by an **identifying (solid) line** and a PK that includes the owner's key:



What it becomes in SQL — two valid options

```
-- Option A: composite key = owner key + partial key
CREATE TABLE room_a (
  building_id int NOT NULL REFERENCES building ON DELETE CASCADE,
  room_no     varchar(6) NOT NULL,                -- partial key (discriminator)
  seats       int,
  PRIMARY KEY (building_id, room_no)
);
-- Option B: surrogate key + UNIQUE on (owner key, partial key)
CREATE TABLE room_b (
  room_id     int GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  building_id int NOT NULL REFERENCES building ON DELETE CASCADE,
  room_no     varchar(6) NOT NULL,
  seats       int,
  UNIQUE (building_id, room_no)                   -- without this, duplicates slip in
);
```

Output (both options refuse a duplicate room, and cascade when the owner goes):

```
--- both options refuse a second room 101 in building 1
NOTICE:  room_a: duplicate key value violates unique constraint "room_a_pkey"
NOTICE:  room_b: duplicate key value violates unique constraint "room_b_building_id_room_no_key"
--- existence dependency: delete the Annex, its rooms go too
 room_a_rows | room_b_rows
-------------+-------------
           1 |           1
(1 row)
```

	Composite PK (A)	Surrogate PK (B)
Enforces "no duplicate partial key per owner"	automatically	only with the extra `UNIQUE` (easy to forget)
Referencing it from other tables	two columns	one column
Key width copied into child FKs/indexes	wider	narrow

Examples: order → order line (`(order_id, line_no)`), building → room, invoice → invoice line. **Weak or not is a design decision**: per-account transaction numbers are weak, a global transaction id strong. Default: option B *with* the unique constraint.

Self-check

1. Which SQL clause expresses existence dependency? → `ON DELETE CASCADE` on the owner FK.
2. Why is the owner FK `NOT NULL` ? → Participation in the identifying relationship is total.

10.10 Worked example — an employee-management ER diagram

The brief

A company wants to:

1. Record each employee: name, date of birth, one or more phone numbers, and the department they belong to (every employee has exactly one department).
2. Keep one payroll record per employee (monthly salary, cash advance, tax); no payroll record is shared by two employees. Net pay is salary − advance − tax.
3. Record who supervises whom (the CEO has no supervisor).
4. Record which projects employees work on and for how many hours a week (many-to-many).
5. Record each employee's dependants (children, spouse) for insurance; a dependant is identified by name *within* an employee.

Step 1 — entities (nouns)

Employee, Department, Payroll, Project, Dependant ("record", "information", "number" are words about data, not things).

Step 2 — attributes, classified

Entity	Attribute	Kind	Reason
Employee	`emp_id`	key	issued once, never repeats
	`full_name`	composite	first name, middle initial, last name
	`birth_date`	simple	one value
	`phone`	**multivalued**	"one or more"
	`age`	**derived**	from `birth_date`
Payroll	`payroll_id`	key	
	`monthly_salary`, `cash_advance`, `tax`	simple, **stored**	typed in by a person — nothing computes them
	`net_pay`	**derived**	salary − advance − tax
Department	`dept_id` (key), `dept_name`		
Project	`project_id` (key), `title`		
Dependant	`dep_name`	**partial key**	unique only within one employee
	`relationship`	simple	

The trap is salary vs net pay: **typed in → stored; computable from stored attributes → derived.**

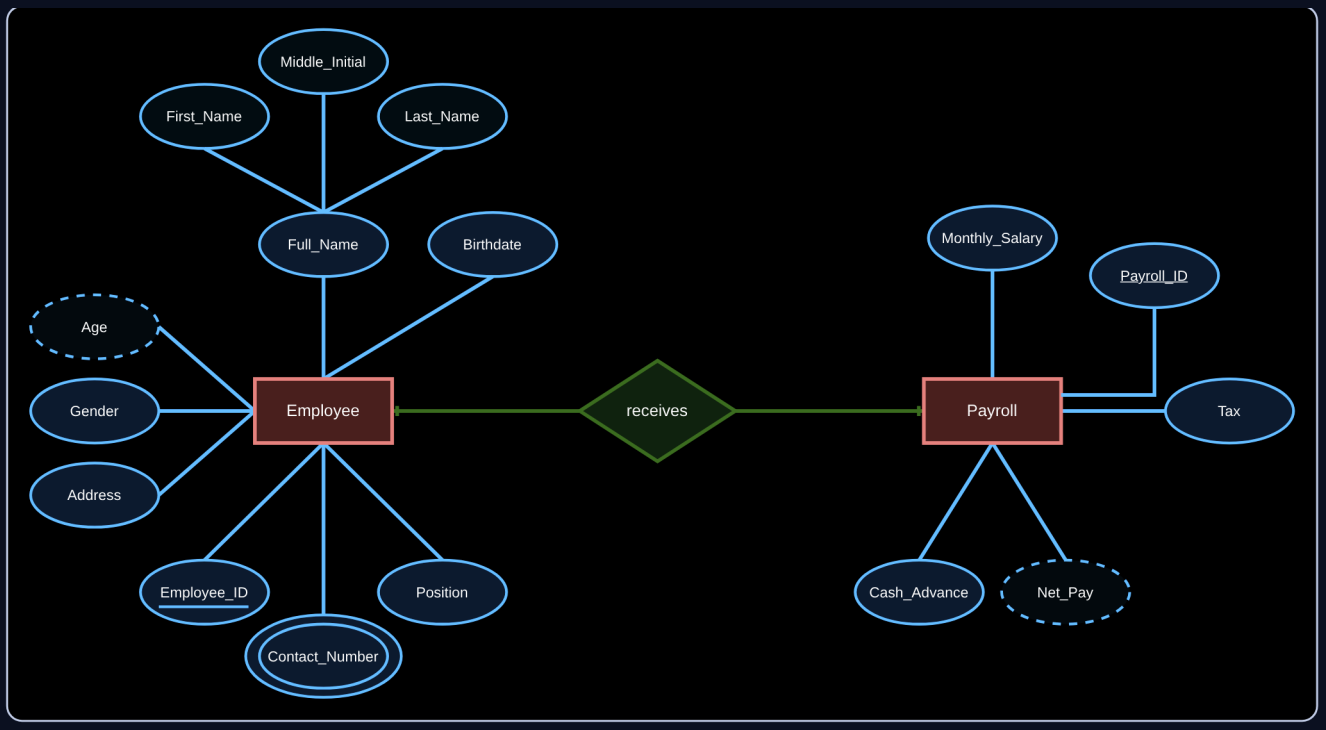
Steps 3–4 — relationships with degree, cardinality and participation

Relationship	Degree	Cardinality	Participation
Employee *works in* Department	binary	N:1	employee total (brief says "exactly one")
Employee *is paid through* Payroll	binary	1:1	both total
Employee *supervises* Employee	unary	1:N	partial (CEO has none)
Employee *works on* Project, attribute `weekly_hours`	binary	M:N	partial
Employee *has* Dependant (identifying)	binary	1:N	dependant total

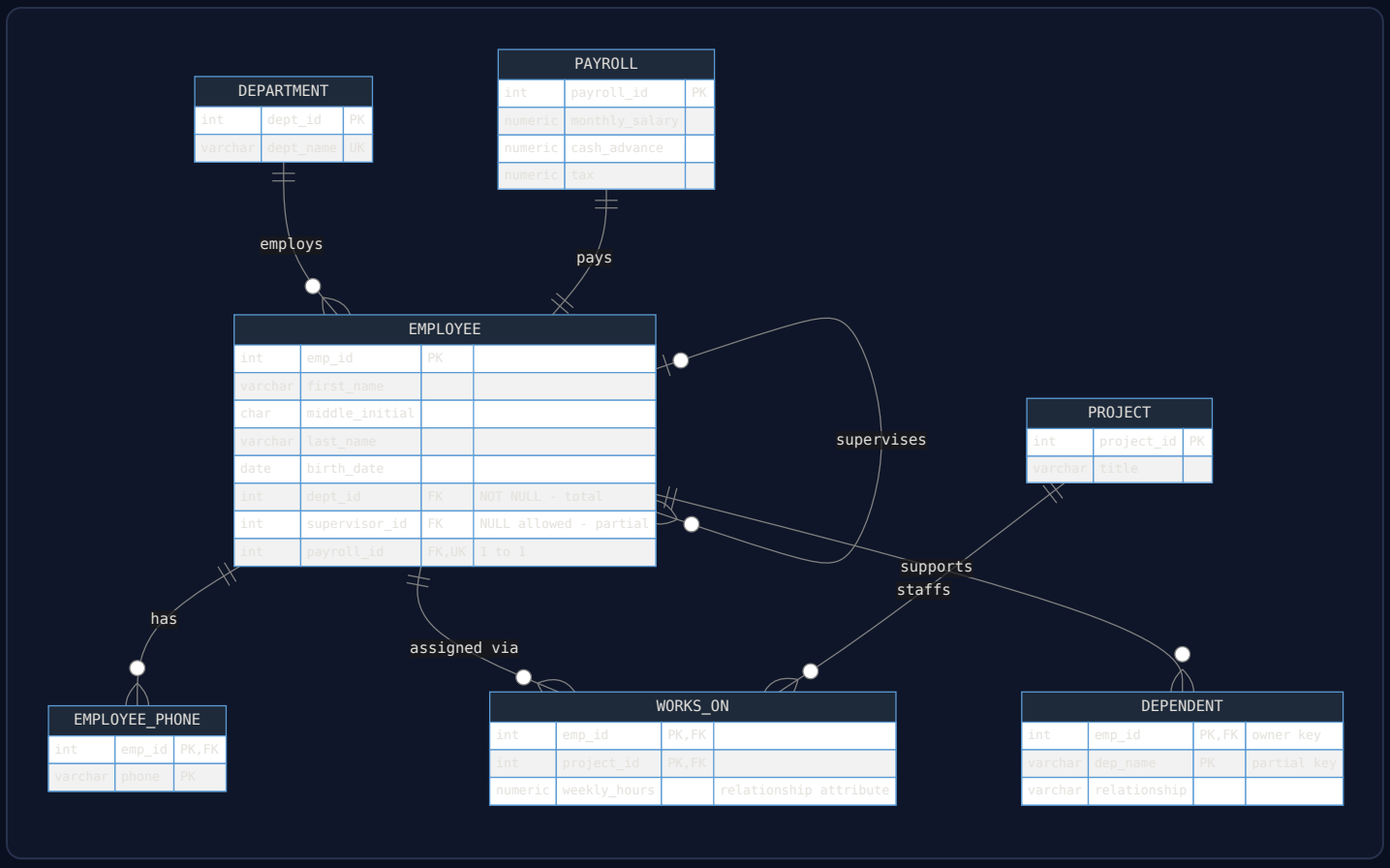
Step 5 — the diagram

The course's Chen diagram of the core (Employee + Payroll) shows every special symbol at once:

Figure — Employee and Payroll in Chen notation: underlined keys, composite Full_Name, double-ellipse Contact_Number, dashed Age and Net_Pay (course diagram)



The full design in crow's-foot notation:



Read it back as sentences — if one sounds wrong, the diagram is: "Each employee works in exactly one department; a department employs zero or more (the brief doesn't require staff). Each employee has exactly one payroll record and vice versa. An employee may supervise several and has at most one supervisor…"

Design check: does Payroll deserve its own table?

A 1:1 total on **both** sides usually is one thing in two places. Keep two tables for **access control** (`GRANT SELECT ON payroll TO hr` — `GRANT` gives a role a permission, Chapter 14), **row width** (keep the hot `employee` row narrow) or **optional data** (no row instead of several `NULL`s). Merge to avoid the join and make

"employee without payroll" impossible. Either is fine in an interview **if you notice the question and justify it**; here, access control wins. §10.15 converts and runs it.

10.11 The relational model

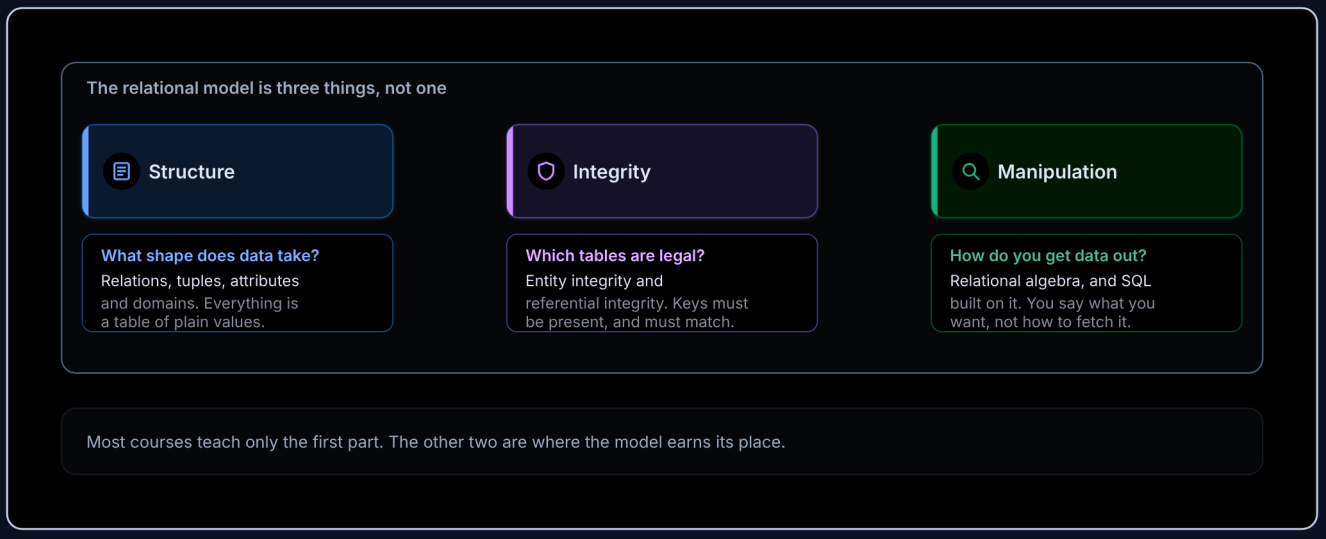
What it is

A database stores **relations** (tables). The **relational model** (**Edgar F. Codd, 1970**): store all data as tables of values; programs *describe* what they want and the system works out how. It replaced hierarchical and network databases (records linked in trees or webs), whose programs walked stored pointers (disk addresses) and broke on every reorganization (Chapter 1).

The three parts of the model

Part	Question it answers	Content
Structure	What shape is the data?	Only relations: a name, named columns, rows of single values. No nested records, arrays or pointers.
Integrity	Which tables are legal?	Keys and constraints declared once on the table; the DBMS refuses any change that breaks them.
Manipulation	How do I get data out?	Declarative queries (SQL: you state *what* you want, not how to fetch it), built on **relational algebra**: operations (select, project, join, union, difference …) that take relations and return a relation.

Figure — structure, integrity, manipulation (course diagram)



Links are values, not pointers

`employee.dept_id = 10` links to department 10 — the shared *value* is the whole link. It survives copying to another server, reloading in another order or rebuilding indexes, and the DBMS can **check** it (an FK refuses `dept_id = 99` if no department 99 exists), which a raw pointer can't.

What you get

- **One rule reaches every program** — constraints live with the data.
- **Physical data independence** — queries name *what*, not *how*, so storage and indexes change freely.
- **Some logical data independence** — adding a column or hiding a split behind a view (a saved query acting as a table) breaks nothing; *not* total — renaming a column or shrinking its type still breaks users.

What it does not promise

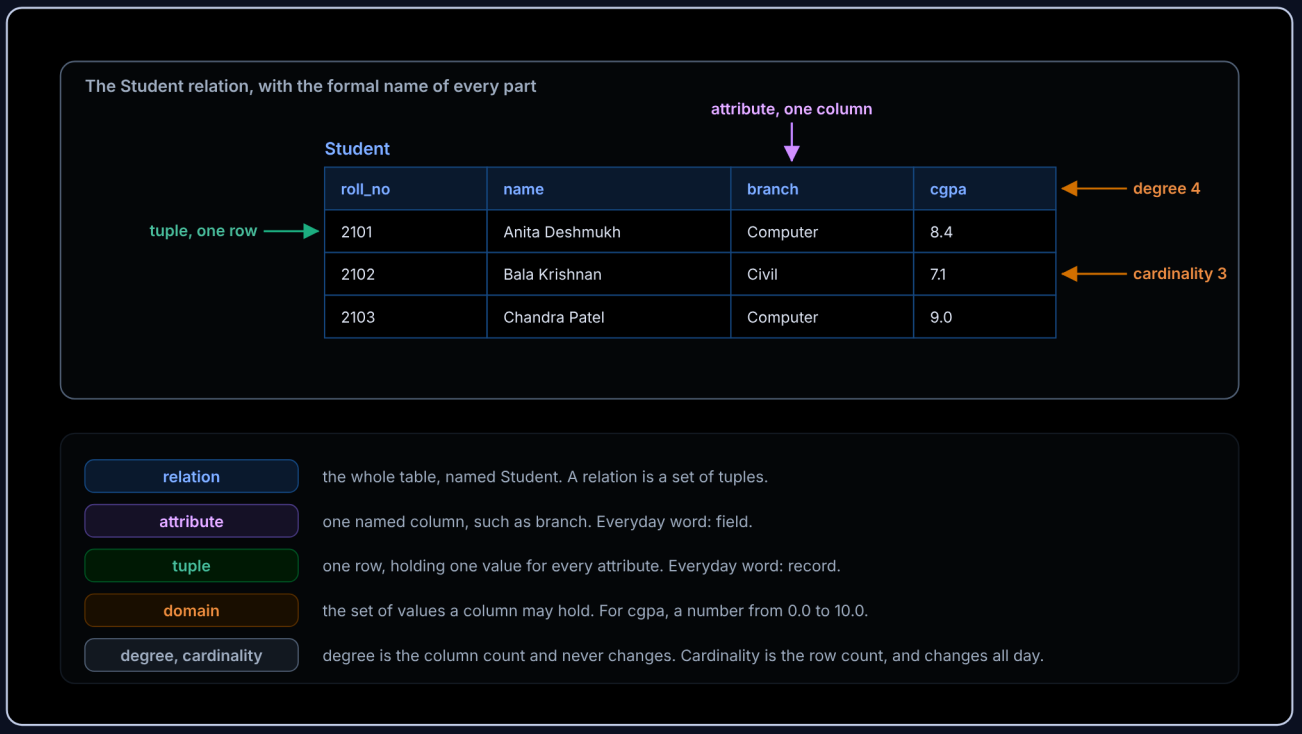
- **Nothing about storage** — files, pages, indexes belong to the storage engine (the DBMS part reading and writing files; Chapter 12).
- **Joins aren't free** — work grows with table size.
- **A table isn't always an entity** — also M:N relationships (`works_on`) and multivalued attributes (`employee_phone`).

10.12 Relations, tuples, attributes — the precise vocabulary

The core words

Formal term	SQL term	Older file term	Meaning
relation	table	file	named set of rows with fixed named columns
tuple	row	record	one value for each attribute
attribute	column	field	a named column
domain	data type + constraints	field type	the set of allowed values for an attribute
degree (arity)	number of columns	number of fields	a property of the schema
cardinality	number of rows	number of records	a property of the current data

Figure — anatomy of a relation (course diagram)



- **Domain** is what you *mean*, more than a type: `int` is a type; "integers 1–500" is room capacity's domain, enforced with `CHECK` .
- **Schema vs instance.** The **relation schema** is the design (`room(room_id, code, capacity)`), rarely changing; the **relation instance** is the current rows. A key belongs to the *schema* — true for any legal data; current rows prove nothing.
- **Two meanings of "cardinality"**: instances per link (1:N, §10.7) vs row count here.

Five properties of every relation

1. **One value per cell** (atomic) — why multivalued attributes need a table; the heart of 1NF (§11.7).
2. **Rows have no inherent order** — ask with `ORDER BY` .

3. **Columns have no inherent order** — referred to by name (SQL tables remember declaration order for `SELECT *`; still list columns explicitly).
4. **Attribute names are unique within a relation.**
5. **No duplicate tuples** — *SQL differs*: identical rows are accepted unless a `PRIMARY KEY` or `UNIQUE` forbids them.

NULL

`NULL` marks a **missing** value — *unknown* (grade not computed) or *not applicable* (no middle name); not zero or ''. Under **three-valued logic** comparing with `NULL` gives *unknown* and `WHERE` keeps only *true* (Chapter 2). From `code/03_keys_and_integrity.sql`:

```
SELECT (SELECT count(*) FROM lecturer WHERE room_id = NULL)  AS eq_null,
       (SELECT count(*) FROM lecturer WHERE room_id IS NULL) AS is_null,
       count(room_id) AS count_col, count(*) AS count_star
FROM lecturer;
```

Output:

```
 eq_null | is_null | count_col | count_star
---------+---------+-----------+------------
       0 |       2 |         1 |          3
(1 row)
```

`= NULL` matches nothing, `IS NULL` works, `count(col)` skips nulls, `count(*)` counts rows — every query on a nullable column must decide how to treat missing values.

Self-check

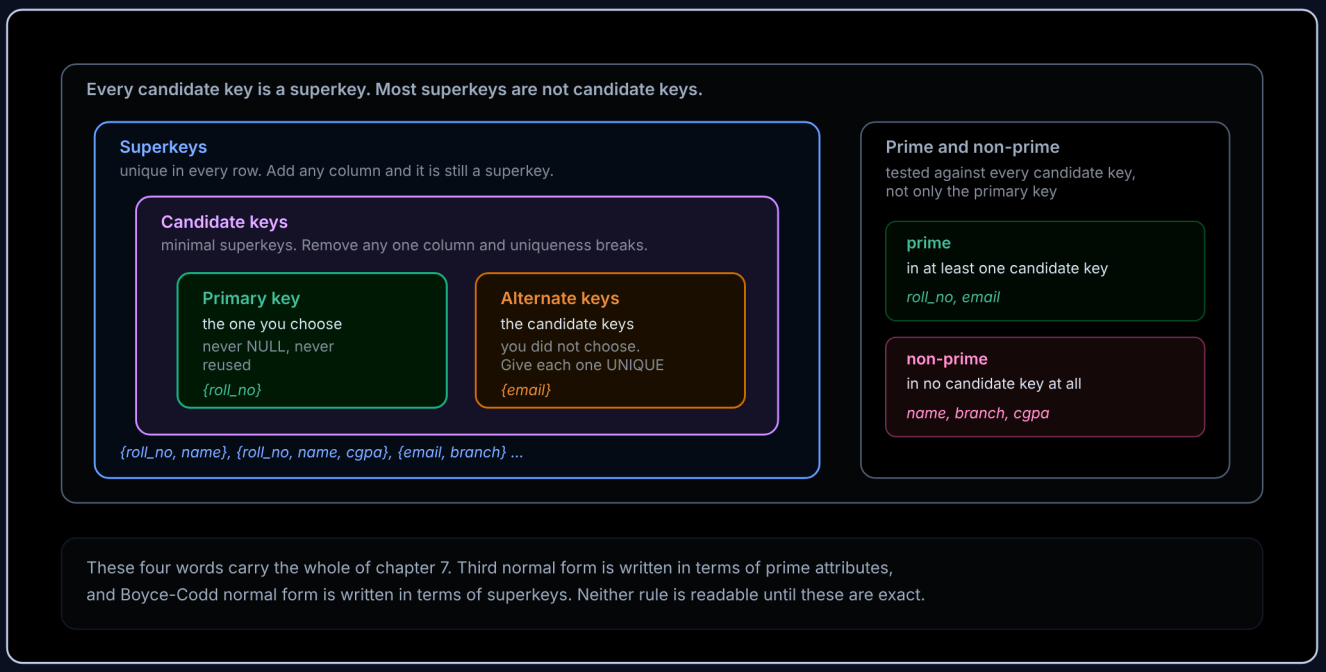
1. `student(roll_no, name, cgpa, dept)` with 8,000 rows: degree, cardinality? → **4**, **8,000**.
2. Why can't rows prove `{name}` is a key? → Keys are schema properties; rows can only *disprove* one (two equal names).

10.13 Keys in relational databases

Two identical-looking rows and a ticket to change "that student" have no answer without **keys**, which make identity exist. Chapter 11's rules use exactly these definitions (§9.3 shows them as DDL). Example: `lecturer(lecturer_id, email, name, room_id)`, ids issued once, emails unique.

Key	Definition	Example
Superkey	a set of attributes that can **never** repeat in two rows (in any legal instance)	`{lecturer_id}`, `{email}`, `{lecturer_id, name}`, all four columns …
Candidate key	a **minimal** superkey — remove any attribute and it stops being unique	`{lecturer_id}`, `{email}`
Primary key	the one candidate key you *choose*; unique and never `NULL`	`lecturer_id`
Alternate key	any candidate key other than the chosen primary one — declare it `UNIQUE`	`email`
Composite key	any key with 2+ attributes (a shape, not a separate kind)	`booking(room_id, day, slot)`
Foreign key	attributes whose values must match a key in a (possibly the same) table, or be `NULL`	`lecturer.room_id → room.room_id`
Natural key	made of real-world data	email, ISBN, frame number
Surrogate key	system-generated, no business meaning	identity integer, UUID

Figure — superkeys ⊇ candidate keys ⊇ primary key, with prime and non-prime attributes (course diagram)



Key facts to be exact about:

- **Adding columns to a superkey keeps it a superkey**, so there are many; candidate keys have nothing spare. Minimality is *removability*, not size — a 3-column candidate key is fine if none can go. All candidate keys are superkeys, rarely the reverse.
- **Prime attribute** = in *some* candidate key (any, not only the primary); **non-prime** = in none. In `lecturer`: `lecturer_id`, `email` prime; `name`, `room_id` non-prime. 3NF uses these words (§11.9).
- **Composite**: `enrolment(student_id, course_id, term, grade)` — retakes mean only `{student_id, course_id, term}` identifies a row.
- **Foreign keys** may be `NULL` (no link yet — not a violation), may reference any `PRIMARY KEY` **or** `UNIQUE` set, and may reference their own table (`employee.supervisor_id` — a unary relationship as a column).
- **Surrogate vs natural**: surrogates are narrow and never change; natural keys are readable and may save a join but change with the business (a new roll-number format updates every FK). **Default: surrogate PK + `UNIQUE` natural key** (as `customer` in `00_schema.sql`).

In the interview

30-second answer: "A superkey uniquely identifies rows; a candidate key is a minimal superkey; the primary key is the candidate I pick — unique and not null; the other candidates are alternate keys and get UNIQUE constraints. A foreign key references a primary or unique key and may be null. My default is a generated surrogate PK, with the natural key kept unique."

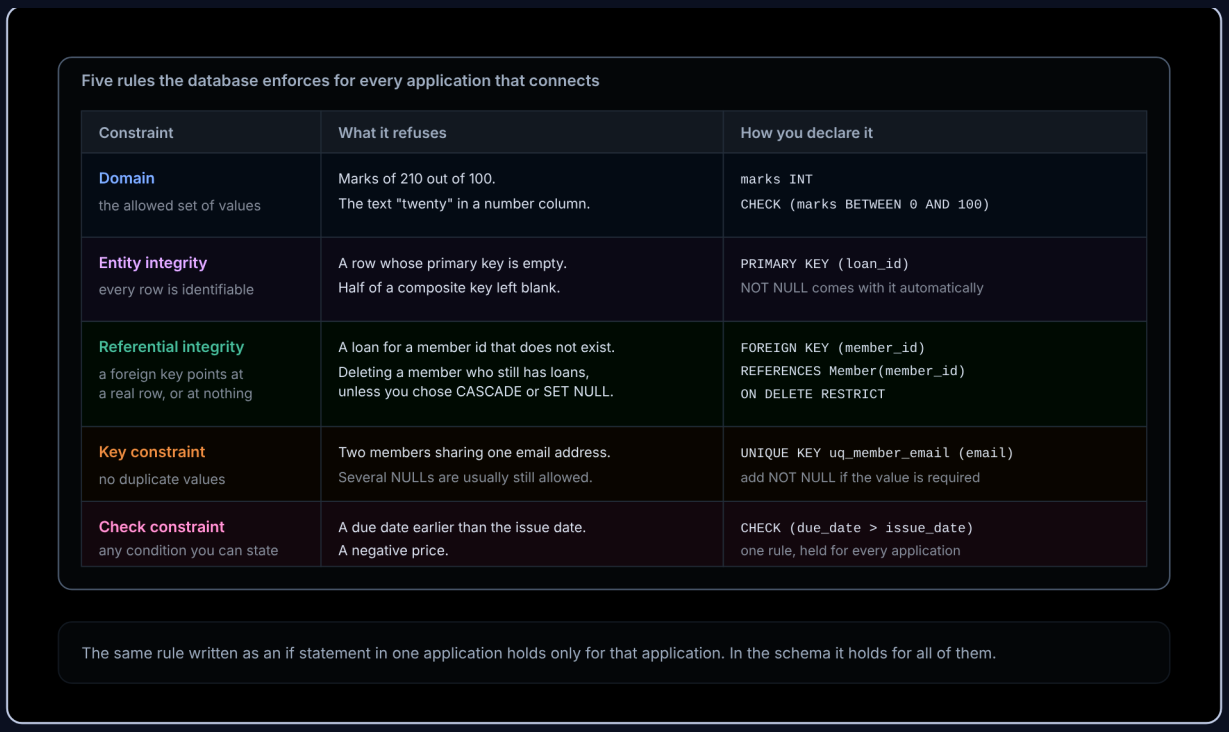
- *Is `{id, name}` a candidate key?* → A superkey, not a candidate key: `name` can be dropped.
- *UUID vs bigint surrogate?* → See §9.4: coordination-free but 16 bytes vs 8, and random UUIDs scatter B-tree inserts; v7 reduces that.
- *Is an attribute of an alternate key prime?* → Yes — prime means "in any candidate key".

10.14 Relational integrity constraints

What it is

An **integrity constraint** is a condition stored data may never break; the DBMS tests it on every `INSERT`, `UPDATE`, `DELETE` from any program and refuses violations.

Figure — the integrity constraints and the SQL that declares each (course diagram)



Constraint	Rule	SQL
Domain	a value must be in the attribute's domain	the data type, plus `CHECK (capacity BETWEEN 1 AND 500)`
Entity integrity	no part of a primary key may be `NULL`	`PRIMARY KEY` (implies `NOT NULL`)
Key	candidate keys are unique	`PRIMARY KEY`, `UNIQUE`
Referential integrity	an FK value matches an existing referenced row, or is `NULL` — no **orphan rows**	`FOREIGN KEY … REFERENCES …`
(also declared the same way)	must always have a value / default when omitted	`NOT NULL`, `DEFAULT` (DEFAULT never refuses anything, so it is not strictly a constraint)

Entity integrity: a PK identifies a row, and a row of unknown (`NULL`) identity can't be distinguished; in composite keys *every* column must be present.

What happens when a referenced (parent) row is deleted or its key updated

The `ON DELETE` / `ON UPDATE` actions are tabulated in §9.3: `NO ACTION` (default; error at statement end, can be `DEFERRABLE` to transaction end), `RESTRICT` (immediate error, not deferrable), `CASCADE` (for weak entities and pure child data — phones, order lines — dangerous elsewhere), `SET NULL` (optional relationships; column must be nullable), `SET DEFAULT` (rare). Dialects: MySQL/InnoDB treats `NO ACTION` as `RESTRICT` and rejects `SET DEFAULT`; SQL Server has no `RESTRICT` keyword; SQLite enforces FKs only after `PRAGMA foreign_keys = ON`; MySQL enforces `CHECK` only from 8.0.16.

Running it (`code/03_keys_and_integrity.sql`)

Each failing statement runs in a `DO` block (one-off PostgreSQL procedural code) that prints the error as a `NOTICE`, so the script continues.

```
CREATE TABLE room (
  room_id  int PRIMARY KEY,                                  -- entity integrity: unique + NOT NULL
  code     varchar(10) NOT NULL UNIQUE,                      -- alternate (candidate) key
  capacity int NOT NULL CHECK (capacity BETWEEN 1 AND 500)   -- domain constraint
);
CREATE TABLE lecturer (
  lecturer_id int PRIMARY KEY,
  name        varchar(60) NOT NULL,
  email       varchar(120) UNIQUE,                           -- UNIQUE but nullable
  room_id     int REFERENCES room(room_id) ON DELETE SET NULL  -- optional link
);
CREATE TABLE booking (
  room_id  int  NOT NULL REFERENCES room(room_id) ON DELETE RESTRICT,
  day      date NOT NULL,
  slot     int  NOT NULL CHECK (slot BETWEEN 1 AND 8),
  PRIMARY KEY (room_id, day, slot)                           -- composite key
);
-- each violation is attempted inside:  DO $$ BEGIN ... EXCEPTION WHEN ... THEN RAISE NOTICE ... END $$;
```

Output:

```
--- 1. domain: capacity 900 is outside the allowed values
NOTICE:  new row for relation "room" violates check constraint "room_capacity_check"
--- 2. entity integrity: a primary key may not be NULL
NOTICE:  null value in column "room_id" of relation "room" violates not-null constraint
--- 3. key constraint: the alternate key code must be unique
NOTICE:  duplicate key value violates unique constraint "room_code_key"
--- 4. referential integrity: no orphan rows
NOTICE:  insert or update on table "booking" violates foreign key constraint "booking_room_id_fkey"
--- 5. UNIQUE lets several rows have NULL (Dr Silva and Dr Wu both have no email)
   name   | email
----------+-------
 Dr Silva |
 Dr Wu    |
(2 rows)

--- 6. ON DELETE RESTRICT: room 1 still has bookings
NOTICE:  update or delete on table "room" violates foreign key constraint "booking_room_id_fkey" on table
"booking"
--- 7. ON DELETE SET NULL: deleting room 2 empties Dr Silva's room_id
   name   | room_id
----------+---------
 Dr Park  |       1
 Dr Silva |
 Dr Wu    |
(3 rows)
```

UNIQUE and NULL — a classic surprise

`NULL` ≠ `NULL`, so most databases let a `UNIQUE` column hold **many** nulls (output 5) — add `NOT NULL` if "unique when present" isn't wanted. PostgreSQL, MySQL and SQLite allow many (PostgreSQL 15+ `UNIQUE NULLS NOT DISTINCT` allows one); SQL Server allows **only one** (use a filtered unique index `WHERE col IS NOT NULL`) — see §9.3.

Why rules belong in the schema

They **hold for everybody** (web app, mobile backend, nightly import, a midnight hand fix), **can't drift** (one copy), and **document meaning**. Costs — a little work per write, and a thoughtless `CASCADE` wiping far more than meant — are small next to one central enforcement point.

Common mistakes

- Undeclared alternate keys → two customers with one email.
- `ON DELETE CASCADE` on a "real" relationship (deleting a customer silently deletes invoices).
- Assuming a nullable `UNIQUE` means "at most one missing".
- FKs left off "for speed" — orphans appear, joins silently lose rows.

10.15 Converting an ER diagram to tables

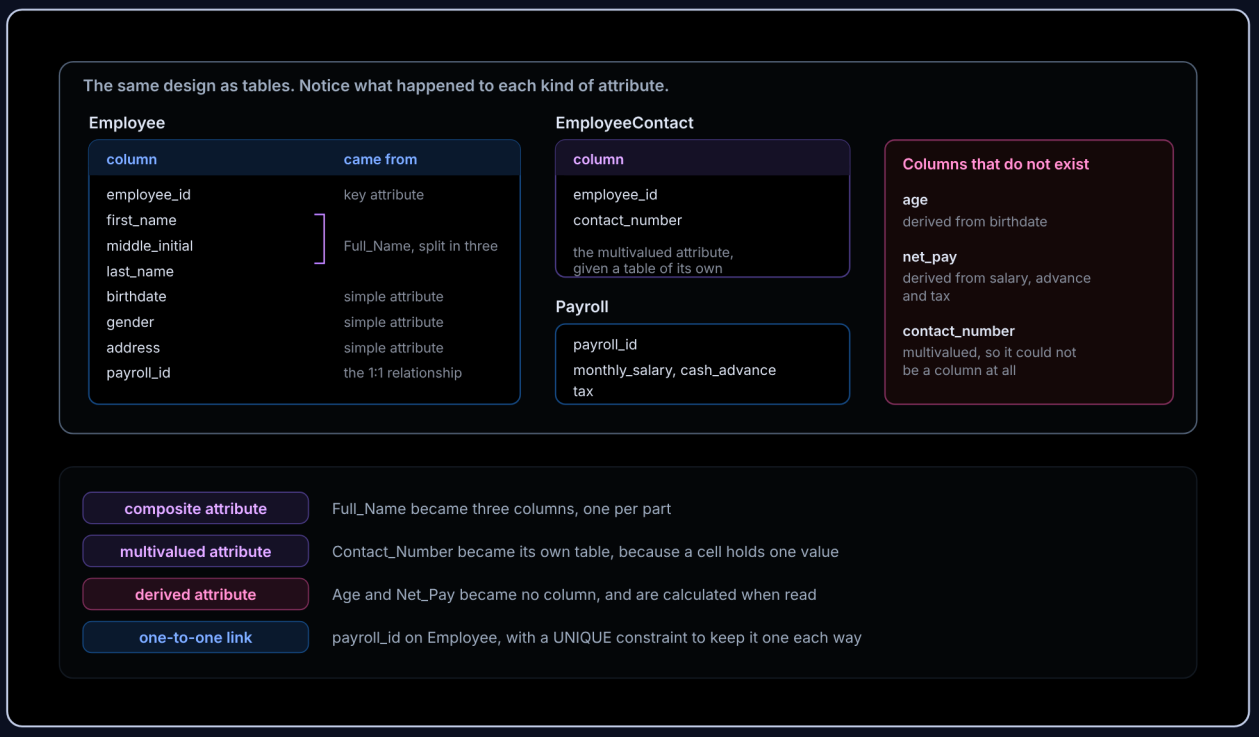
The procedure

Once the diagram is right, conversion is mechanical:

#	ER construct	Relational result
1	Strong entity	a table; key attribute → primary key
2	Simple attribute	a column
3	Composite attribute	one column per part (no column for the whole)
4	Derived attribute	no column (compute in the query or a view)
5	Multivalued attribute	own table: owner key + value, both in the PK, `ON DELETE CASCADE`
6a	1:N / N:1 relationship	FK on the many side
6b	M:N relationship	junction table: both FKs, together the PK
6c	1:1 relationship	FK + `UNIQUE` on either side, prefer the total-participation side
6d	Unary relationship	FK to the same table
6e	Ternary relationship	table with three FKs (PK usually all three)
7	Relationship attribute	follows the relationship: junction table (M:N) or many side (1:N)
8	Weak entity	table with PK = owner key + partial key; owner FK `NOT NULL`, usually `ON DELETE CASCADE`
9	Participation	total → FK `NOT NULL`; partial → FK nullable

Create tables in **dependency order** — parents first, since an FK can only point at an existing table.

Figure — the course's Employee/Payroll example as tables: composite split, multivalued attribute a table, derived attributes gone (course diagram)



Running the procedure on §10.10 (`code/02_employee_er_to_tables.sql`)

```
CREATE TABLE department (                       -- strong entity -> table
  dept_id   int PRIMARY KEY,
  dept_name varchar(60) NOT NULL UNIQUE
);
CREATE TABLE payroll (                          -- strong entity -> table
  payroll_id     int PRIMARY KEY,
  monthly_salary numeric(10,2) NOT NULL CHECK (monthly_salary >= 0),  -- stored (typed in)
  cash_advance   numeric(10,2) NOT NULL DEFAULT 0,
  tax            numeric(10,2) NOT NULL DEFAULT 0
  -- net_pay is DERIVED -> no column (computed in the view below)
);
CREATE TABLE employee (
  emp_id         int PRIMARY KEY,                             -- key attribute
  first_name     varchar(40) NOT NULL,                        -- composite attribute full_name
  middle_initial char(1),                                     --   split into
  last_name      varchar(40) NOT NULL,                        --   one column per part
  birth_date     date NOT NULL,                               -- simple; age is DERIVED
  dept_id        int  NOT NULL REFERENCES department(dept_id),  -- 1:N, total participation -> NOT NULL
  supervisor_id  int  REFERENCES employee(emp_id),            -- unary 1:N, partial -> NULL allowed
  payroll_id     int  NOT NULL UNIQUE REFERENCES payroll(payroll_id)  -- 1:1 -> FK + UNIQUE
);
CREATE TABLE employee_phone (                   -- multivalued attribute -> own table
  emp_id int         NOT NULL REFERENCES employee(emp_id) ON DELETE CASCADE,
  phone  varchar(20) NOT NULL,
  PRIMARY KEY (emp_id, phone)
);
CREATE TABLE project (project_id int PRIMARY KEY, title varchar(80) NOT NULL);
CREATE TABLE works_on (                         -- M:N -> junction table
  emp_id     int NOT NULL REFERENCES employee(emp_id),
  project_id int NOT NULL REFERENCES project(project_id),
  weekly_hours numeric(4,1) NOT NULL CHECK (weekly_hours > 0),  -- relationship attribute
  PRIMARY KEY (emp_id, project_id)
);
CREATE TABLE dependent (                        -- weak entity
  emp_id       int NOT NULL REFERENCES employee(emp_id) ON DELETE CASCADE,  -- owner key
  dep_name     varchar(40) NOT NULL,                                       -- partial key
  relationship varchar(20) NOT NULL,
  PRIMARY KEY (emp_id, dep_name)                                           -- owner key + partial key
);
-- Derived attributes live in a view (fixed "as of" date so the output is repeatable):
CREATE VIEW employee_card AS
SELECT e.emp_id,
       concat_ws(' ', e.first_name, e.middle_initial, e.last_name)   AS full_name,
       date_part('year', age(date '2026-09-28', e.birth_date))::int  AS age,
       p.monthly_salary - p.cash_advance - p.tax                    AS net_pay,
       (SELECT string_agg(phone, ', ' ORDER BY phone) FROM employee_phone ph
         WHERE ph.emp_id = e.emp_id)                                AS phones
FROM employee e JOIN payroll p USING (payroll_id);
```

Output:

```
--- employee_card (derived: full_name, age, net_pay; multivalued: phones)
 emp_id |  full_name   | age | net_pay |              phones
--------+--------------+-----+---------+----------------------------------
      1 | Aisha Khan   |  41 | 4900.00 | +44 20 7946 0001
      2 | Ravi K Menon |  33 | 4250.00 | +91 80 4000 0003, +91 98450 00002
      3 | Sofia Rossi  |  28 | 3100.00 |
(3 rows)

--- the unary relationship: who reports to whom
 employee | supervisor
----------+------------
 Aisha    |
 Ravi     | Aisha
 Sofia    | Aisha
(3 rows)

--- 1:1 is enforced by UNIQUE: a second employee on payroll 501 is refused
NOTICE:  refused: duplicate key value violates unique constraint "employee_payroll_id_key"
--- weak entity: deleting the owner removes its dependents (ON DELETE CASCADE)
 dependents_left | phones_left
-----------------+-------------
               0 |           0
(1 row)
```

(Before deleting employee 2 the script removes their `works_on` rows and clears `supervisor_id` pointing at them — those FKs use default `NO ACTION`, so the database would otherwise refuse: referential integrity at work.)

Checking a conversion

1. **Count:** 5 entities + 1 multivalued attribute + 1 M:N = 7 tables. ✓
2. **Every relationship produced something** (FK, UNIQUE FK or table) — otherwise it was missed.
3. **No derived attribute survived** as a column (or a comment says why).
4. **No multivalued attribute left in its parent** — a list column violates 1NF (Chapter 11).

10.16 Best practices for ER → relational mapping

Large diagrams need judgement in a few places:

Situation	Recommendation	Why / exception
Composite attribute	Store the parts; do **not** also store the whole. Build the whole in a query or view.	Two copies disagree after the first partial update. Exception: never query the parts → one column is fine.
Multivalued attribute	Always a separate table with a composite PK (owner key + value). Never numbered columns.	The composite PK stops the same phone being stored twice for one person.
Derived attribute	Compute on read. Store only if expensive *or* it must be frozen in time (order total at purchase prices) — and comment the column.	Otherwise the next engineer deletes it as a "mistake".
1:1 with total participation both sides	Probably one table. Keep two only for access control, row width, or mostly-absent optional data.	Two tables always joined 1:1 are cost without benefit.
M:N	The junction table is mandatory; relationship attributes live there; give it a business name (`enrolment`, `booking`) not `student_course`.	Add a surrogate key only if other tables must reference a single row — and keep `UNIQUE` on the pair so nobody enrols twice.
Weak entity with a surrogate key	Keep `UNIQUE (owner_key, partial_key)`.	Otherwise the rule the weak entity expressed is enforced nowhere.
Naming	One style for tables and columns; FKs named after their target; **role names** when two FKs hit the same table (`origin_airport_id`, `destination_airport_id`, not `airport_id_1/2`).	Readable joins.
Normalization	Run the normal forms over the result (Chapter 11); expect one or two further splits.	A careful ERD lands *near* 3NF, rarely exactly on it.
Large diagrams	Create parent tables before children and finish one area (say orders) before starting the next (inventory); record each choice you made that the rules left open (merged 1:1s, stored derived columns, surrogate keys on weak entities).	Those are the choices someone will question later.

Common mistakes

- Converting an M:N with an FK on one side (only works if it was really 1:N).
- Keeping `full_name` *and* `first_name` / `last_name`.
- Surrogate key on a junction table without the pair `UNIQUE`.
- Creating child tables before parents in a migration script.

In the interview

30-second answer (for "turn this ERD into tables"): "Each strong entity becomes a table with its key as PK. Composite attributes split into columns, derived ones are computed, multivalued ones get their own table. 1:N puts the FK on the many side, M:N gets a junction table holding both keys plus any relationship attributes, 1:1 is an FK with UNIQUE. A weak entity's PK combines its owner's key with its partial key, with cascade delete, and total participation means NOT NULL. Then I check the result against the normal forms."

- *Where does `grade` go in student–course?* → In the enrolment junction table.
- *Would you merge a 1:1?* → Yes unless access control, row width or sparse optional data justify splitting.

Self-check

1. A junction table has a surrogate PK. What else must it have? → `UNIQUE` on the two FK columns.
2. Is the converted schema automatically in 3NF? → No; check the normal forms.

Key terms

Defined where first used: data model (conceptual / logical / physical), data independence, entity type / instance / set, key / simple / composite / multivalued / derived attribute, relationship type / set, degree, cardinality (ER: 1:1, 1:N, M:N; relation: row count), participation (total / partial), (min, max), junction table, weak entity / owner / partial key (discriminator) / identifying relationship, Chen vs crow's foot, relation / tuple / attribute / domain, schema vs instance, superkey / candidate / primary / alternate / composite / foreign / natural / surrogate key, prime / non-prime attribute, domain / entity / key / referential integrity, orphan row (child whose FK points at nothing), RESTRICT / NO ACTION / CASCADE / SET NULL / SET DEFAULT. Glossary: chapter 20.

Chapter 10 TL;DR

- **Model before tables.** Entities (nouns) → tables, attributes → columns, relationships (verbs) → FKs or junction tables. A fact you do not model is a question you cannot answer.
- **Three levels:** conceptual (for the business), logical (attributes, keys, normalization), physical (types, indexes, one DBMS).
- **Process:** requirements + questions → entities → relationships → conceptual review → logical → physical → validate; loop back freely.
- **Best practices:** constraints in the schema, one naming style, dates as dates, money as `numeric`, explicit `NOT NULL`, surrogate PK + unique natural key, 3NF by default, index real queries.
- **ER notation:** Chen (rectangle, ellipse, diamond; double ellipse = multivalued, dashed = derived, double rectangle/diamond = weak) vs crow's foot (`||` , `o|` , `o{` , `|{`), which mermaid draws.
- **Relationships:** degree, cardinality, participation are three separate questions; (min,max) = (participation, cardinality).
- **Relational model (Codd 1970):** relations of atomic values, links by value, declarative queries; SQL tables allow duplicates unless keyed; `NULL` needs `IS NULL` .
- **Keys:** superkey ⊇ candidate (minimal) ⊇ primary; alternates get `UNIQUE` ; prime = in any candidate key.
- **Integrity:** domain, entity, key, referential (+ ON DELETE choices); UNIQUE usually allows many NULLs.
- **Mapping:** composite → parts, multivalued → table, derived → view, 1:N → FK on many, M:N → junction, 1:1 → FK + UNIQUE, weak → owner key + partial key + CASCADE, total → NOT NULL. Then normalize — Chapter 11.

Next: Chapter 11 — Normalization, which checks a design for redundancy and splits tables safely.

Chapter 11 — Normalization

Chapter 10 turned a business description into tables. This chapter asks: **does each fact live in exactly one place?** If not, the table can contradict itself (a customer with two cities) and some facts cannot be stored or are lost by accident. **Normalization** is the rule-based way to find such tables and split them safely.

Everything here rests on one exact tool, the **functional dependency**, so "is this table in 3NF?" has a checkable yes/no answer. `code/fd_tools.py` (Python 3: `python fd_tools.py`) computes closures, candidate keys (minimal column sets that identify a row) and normal-form violations; every worked answer was verified with it. The SQL files (`00_schema.sql`, one wide order table; `01`–`07`, each on a fresh database) show the anomalies on PostgreSQL and their disappearance after decomposition (splitting one table into several). All "Output:" blocks are real.

What you'll learn: FDs and their types · Armstrong's rules, closure, finding every candidate key, minimal covers · insert/update/delete anomalies, live · 1NF → 5NF: rule, violating table, fix, proof nothing was lost · lossless joins and dependency preservation, and why 3NF (not BCNF) is the usual stopping point · four exercises · when to **denormalize**.

11.1 Functional dependencies

What it is

A **functional dependency (FD)** `X -> Y` ("X determines Y") says: take any two rows; if they agree on every column of `X`, they must agree on every column of `Y`. One pair that agrees on `X` but differs on `Y` proves the FD false. (An ID number determines a birthday; a birthday does not determine an ID.)



The running example

One deliberately bad table from a small online shop (`code/00_schema.sql`):

order_id	product_id	order_date	customer_id	customer_name	customer_city	product_name	list_price	qty
O1	P1	2026-09-01	C1	Nadia Lopez	Lisbon	Kettle	29.00	1
O1	P2	2026-09-01	C1	Nadia Lopez	Lisbon	Mug	6.50	4
O2	P1	2026-09-03	C2	Jonas Berg	Oslo	Kettle	29.00	1
O3	P3	2026-09-05	C1	Nadia Lopez	Lisbon	Teapot	18.00	2
O4	P4	2026-09-06	C3	Priya Das	Kochi	Tea tin	9.00	3

The shop owner confirms four rules — these are the FDs:

#	FD	In words
F1	`order_id -> order_date, customer_id`	an order is placed on one date by one customer
F2	`customer_id -> customer_name, customer_city`	a customer has one name and one city
F3	`product_id -> product_name, list_price`	a product has one name and one current list price
F4	`{order_id, product_id} -> qty`	a product appears once per order, with one quantity

Vocabulary: the left side is the **determinant** (`customer_id` in F2; several normal forms are phrased in terms of determinants), the right side the **dependent**; a multi-column side is **composite** (`{order_id, product_id}` in F4).

An FD is a rule, not a summary of today's rows

Every `customer_name` above is different, so `customer_name -> customer_city` *happens* to hold — but two customers called Nadia Lopez can live in different cities. Rows can **disprove** an FD but never **prove** one; FDs are constraints on every legal state, like `NOT NULL`.

- **Where FDs come from:** ask the data owners, one rule at a time — *"If I tell you the X, do you always know the Y?"* Write every answer down before designing tables.
- **Direction matters:** `X -> Y` says nothing about `Y -> X` (an id and a unique email determine each other, but each needs its own check).

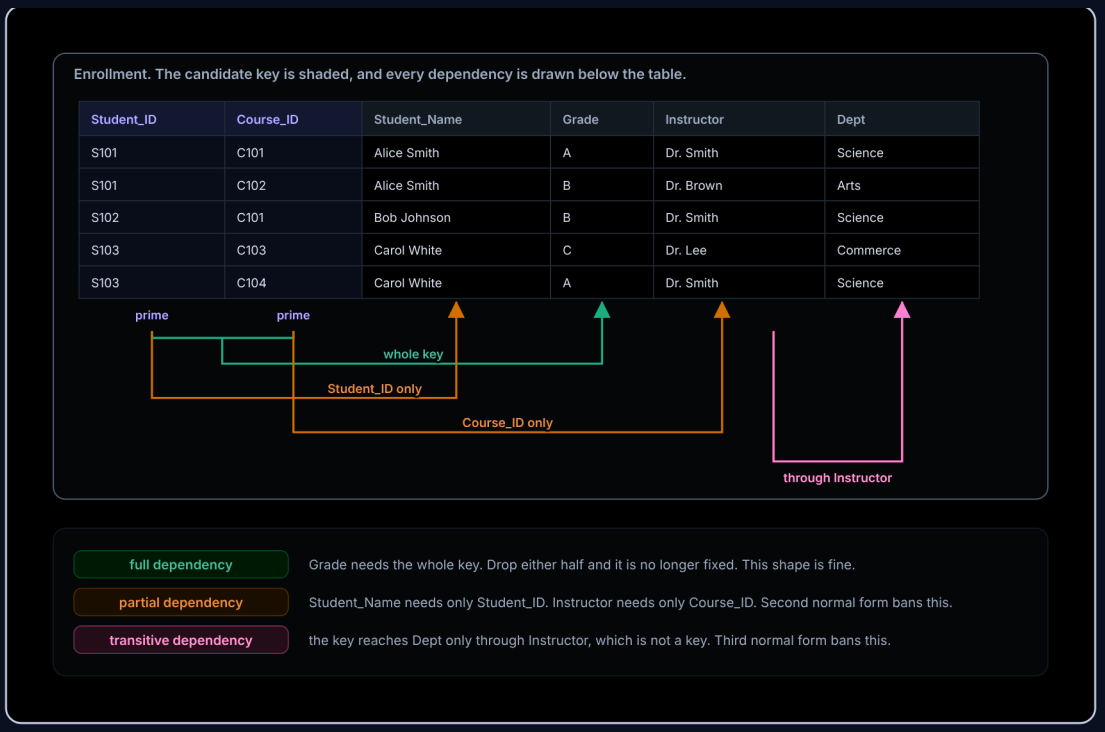
FDs and keys are the same idea

A **superkey** (§10.13) is a set that never repeats: **`X` is a superkey exactly when `X` determines all columns** (rows equal on X would be the same row). A **candidate key** is a superkey from which nothing can be removed. So closure (§11.4), the tool that tests FDs, also finds keys — and each normal form is a statement about which FDs may exist, which makes "is this normalized?" a fact, not an opinion.

> **TL;DR**
>
> `X -> Y`: rows equal on X are equal on Y. One counter-pair disproves it; data never proves it — FDs come from business rules. X is a superkey iff X determines all columns.

11.2 Types of functional dependencies

In the running example the only candidate key is `{order_id, product_id}`, so the **prime attributes** (members of some candidate key) are `order_id` and `product_id`; all others are **non-prime**.



Type	Definition	Example (running table)	Banned by
Trivial	right side contained in the left: `{order_id, product_id} -> product_id`	always true; says nothing	never a problem — every rule says "non-trivial"
Non-trivial	right side adds a column the left lacks	`customer_id -> customer_city`	—
Completely non-trivial	left and right share no column	`product_id -> list_price`	—
Full	needs its *whole* (composite) determinant: remove any column and it fails	`{order_id, product_id} -> qty` (O1 has quantities 1 and 4; P1 appears in O1 and O2)	—
Partial	a *proper part* of a candidate key determines a non-prime column	`order_id -> order_date`, `product_id -> product_name`	**2NF**
Transitive	`X -> Y` and `Y -> Z` give `X -> Z`, where `Y` is not a superkey and `Z` is non-prime	`order_id -> customer_id -> customer_city`	**3NF**
Determinant not a superkey	any non-trivial `X -> Y` with X not a superkey	all of F1–F3 here	**BCNF**
Multivalued (`X ->> Y`)	for one X, the *set* of Y values is independent of the other columns	a restaurant's dishes vs its delivery zones (§11.12)	**4NF**

Precision points:

- **Full vs partial needs a composite determinant** — a one-column key has no "part".
- **Both conditions on "transitive" matter:** without "Y is not a superkey", every table would be full of transitive dependencies (key → key → everything).
- **Partial and transitive can overlap:** `order_id -> customer_city` is both.
- **Every FD is also an MVD** (multivalued dependency), not vice versa (§11.12).

Self-check: `{order_id, product_id} -> order_id`? → trivial. Why is `product_id -> product_name` partial? → `product_id` is a proper part of the only key and `product_name` is non-prime. With a single-column key, which can exist? → only transitive.

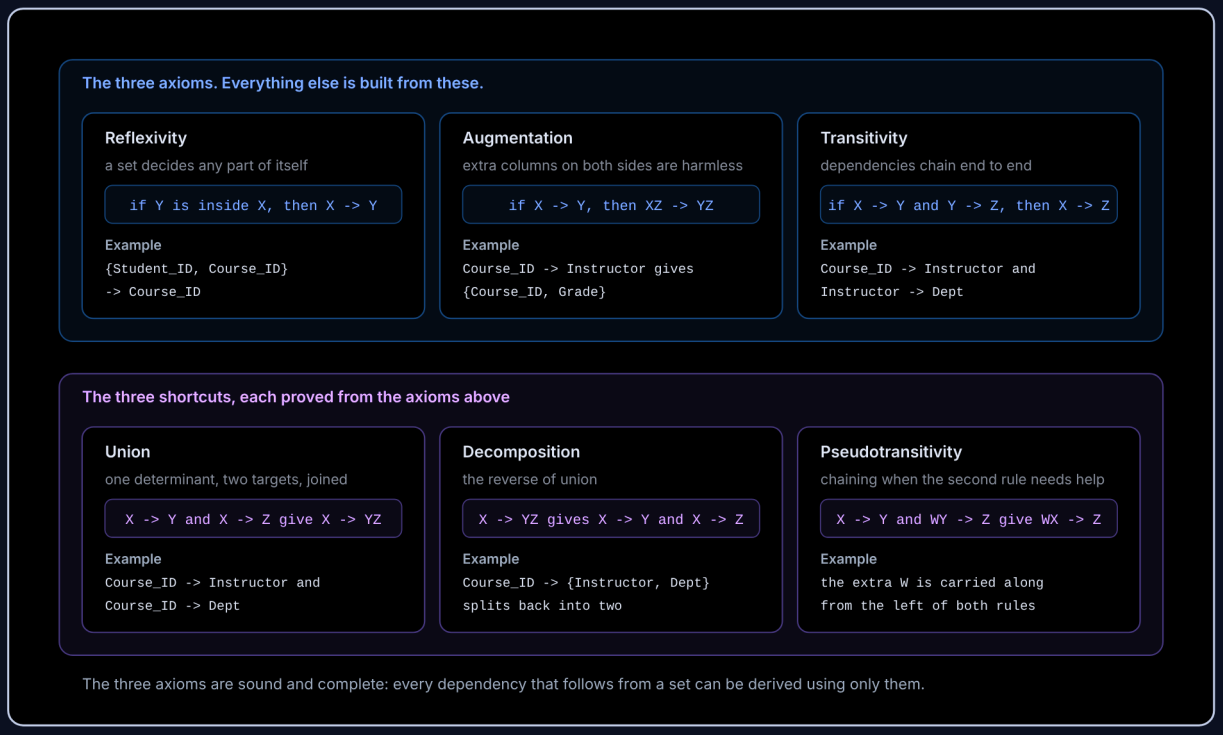
TL;DR

Trivial (always true) · full (needs whole key) · partial (part of a key → non-prime; kills 2NF) · transitive (key → non-key → non-prime; kills 3NF) · non-superkey determinant (kills BCNF) · multivalued (kills 4NF).

11.3 Armstrong's inference rules

What it is

A set of FDs implies more than it lists: nobody wrote `order_id -> customer_city`, yet F1 and F2 force it. The given FDs `F` plus everything they imply is the **closure of F**, **F⁺**. The derivation rules are **Armstrong's axioms** (W. W. Armstrong, 1974).



Axiom	Rule	Example	Why it is true
Reflexivity	if `Y ⊆ X` then `X -> Y`	`{order_id, product_id} -> product_id`	rows equal on X are equal on any part of X (the trivial FDs)
Augmentation	if `X -> Y` then `XZ -> YZ`	from F2: `{customer_id, product_id} -> {customer_city, product_id}`	rows equal on X and Z are equal on Y (FD) and Z (assumption)
Transitivity	if `X -> Y` and `Y -> Z` then `X -> Z`	F1 + F2: `order_id -> customer_city`	chain the two guarantees

The axioms are **sound** (never derive an FD false in some table satisfying F) and **complete** (derive every FD that holds in all such tables). The derived rules add convenience, not power:

Rule	Pattern	Example
Union	`X -> Y` and `X -> Z` ⟹ `X -> YZ`	`order_id -> {order_date, customer_city}`
Decomposition (split)	`X -> YZ` ⟹ `X -> Y` and `X -> Z`	F3 ⟹ `product_id -> list_price`
Pseudotransitivity	`X -> Y` and `WY -> Z` ⟹ `WX -> Z`	if `{customer_id, product_id} -> review_stars` (one review per product), F1 with W = product_id gives `{order_id, product_id} -> review_stars`

Decomposition is why worked examples first rewrite every FD with a single column on the right.

Deriving union from the axioms (common exam question): (1) augment `X -> Y` with X: `X -> XY` (since XX = X); (2) augment `X -> Z` with Y: `XY -> YZ`; (3) transitivity: `X -> YZ`. ∎ Decomposition = reflexivity (`YZ -> Y`) + transitivity; pseudotransitivity = augment `X -> Y` with W, then transitivity.

They matter because keys and normal-form claims depend on implied FDs; in practice you apply them via attribute closure (next). `fd_tools.py` confirms the derivations:

```
order_id -> customer_city follows? True
customer_id -> order_id follows?   False
pseudotransitivity: order_id -> customer_id and {customer_id, product_id} -> review_stars
  so {order_id, product_id} -> review_stars? True
```

TL;DR

F^+ = all FDs implied by F. Axioms: reflexivity, augmentation, transitivity — sound and complete. Shortcuts: union, decomposition, pseudotransitivity.

11.4 Attribute closure and finding candidate keys

What it is

The **closure of an attribute set X**, **X^+**, is every attribute X determines, directly or through chains (always including X).

The algorithm

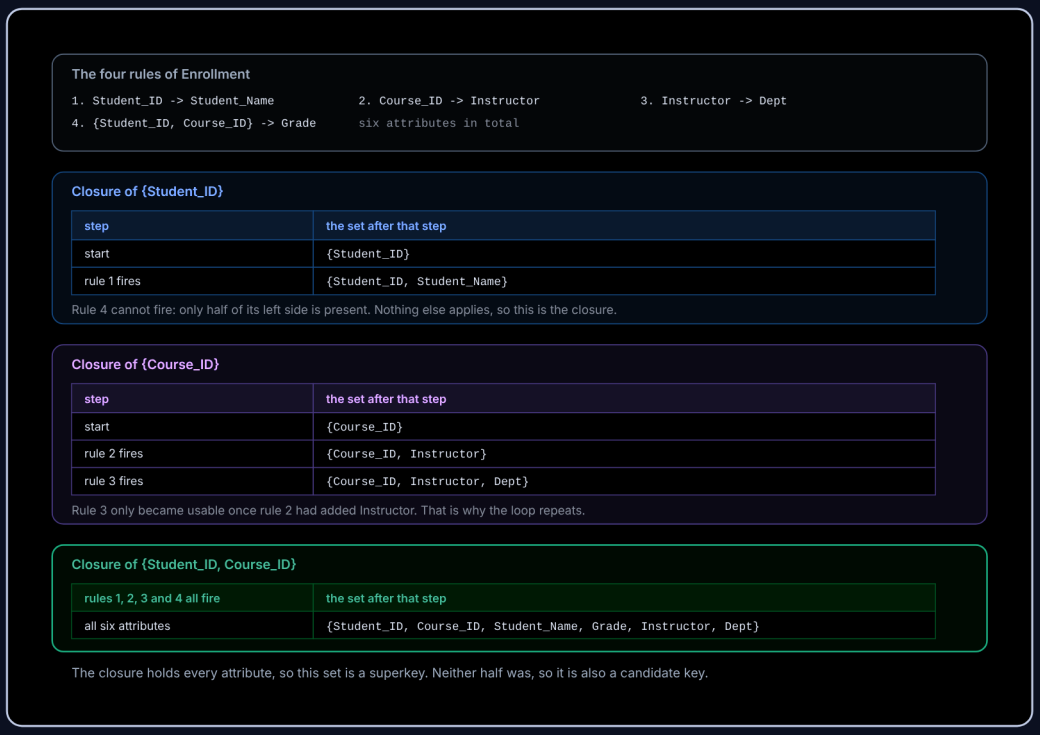
1. Start with `result = X`.
2. For any FD whose **entire** left side is inside `result`, add its right side.
3. Repeat full passes until a pass adds nothing.

The classic mistake is firing an FD whose left side is only *partly* present. Repeated passes matter because an FD can become usable once an earlier one adds the column it needs. The core of `fd_tools.py`:

```python
def closure(attrs, fds, trace=False):
    """X+ : keep adding the right side of any FD whose WHOLE left side is already in the set."""
    result = set(attrs)
    changed = True
    while changed:
        changed = False
        for lhs, rhs in fds:
            if lhs <= result and not rhs <= result:   # lhs fully present, something new to add
                result |= rhs
                changed = True
    return frozenset(result)
```

{order_id}⁺: F1 fires → add `order_date, customer_id`; now F2 fires → add `customer_name, customer_city`. F3 needs `product_id` (absent); F4 needs *both* key columns — **skip**.

```
closure trace for {order_id}:
    step 1: {order_id} -> {customer_id, order_date} fires, set = {customer_id, order_date, order_id}
    step 2: {customer_id} -> {customer_city, customer_name} fires, set = {customer_city, customer_id,
customer_name, order_date, order_id}
{order_id}+ = {customer_city, customer_id, customer_name, order_date, order_id}
{product_id}+ = {list_price, product_id, product_name}
{order_id, product_id}+ = {customer_city, customer_id, customer_name, list_price, order_date, order_id,
product_id, product_name, qty}
```



Uses of closure

Question	Test
Is X a **superkey**?	X^+ contains **all** attributes
Is X a **candidate key**?	superkey **and** no proper subset is (enough to test subsets with one attribute removed)
Does **X -> Y** follow from F?	$Y \subseteq X^+$

Here `{order_id, product_id}+` is all nine columns, while `{order_id}+` (5) and `{product_id}+` (3) are not → **`{order_id, product_id}` is a candidate key**. `customer_id -> order_id` does **not** follow (`order_id ∉ {customer_id}+` — a customer has many orders).

Finding *all* candidate keys systematically

1. **Classify attributes:** never on a right side → **in every key**; only on right sides → **in no key**; both → maybe.
2. Close the "must" set. If it is everything, it is the *only* candidate key (the running example: `order_id`, `product_id` never appear on a right side — which is why `fd_tools.candidate_keys` starts from that "core").
3. Otherwise add "maybe" attributes one at a time, then two, …, skipping supersets of keys already found.

Harder example (Exercise 3): R(A, B, C, D, E), `A -> B`, `BC -> D`, `D -> A`, `C -> E`. `C` is never on a right side; `C+ = {C, E}`. Adding one attribute:

```
{C}+ = {C, E}
{A, C}+ = {A, B, C, D, E}
{B, C}+ = {A, B, C, D, E}
{C, D}+ = {A, B, C, D, E}
{C, E}+ = {C, E}
```

Three candidate keys: `{A,C}`, `{B,C}`, `{C,D}`; prime A, B, C, D; non-prime E. Overlapping keys like these are where BCNF differs from 3NF (§11.11).

In the interview

30-second answer: "To test whether X is a key, compute X^+: start with X and repeatedly add the right side of any FD whose left side is fully contained, until nothing changes. If X^+ is the whole relation, X is a superkey; if no smaller subset is, it's a candidate key. Attributes that never appear on a right-hand side must be in every key, which is where I start."

- *How do you check whether an FD is implied?* → Close its left side; is the right side inside?
- *Complexity?* → A naive closure loop is $O(|F|^2 \cdot |attrs|)$; finding all keys can be exponential in the worst case (a relation can have exponentially many keys), but class-sized problems are tiny.

TL;DR

X^+ = everything X determines (fire only FDs whose whole LHS is present; loop until stable). Superkey $\iff$ X^+ = all attributes; candidate key = minimal superkey; X -> Y implied $\iff$ $Y \subseteq X^+$. Never-on-the-right attributes are in every key.

11.5 Minimal (canonical) cover

A **minimal cover** (**canonical cover**, F_c) is an equivalent FD list with no waste:

1. every right side is a **single** attribute;
2. no left side has an **extraneous** attribute (droppable without changing F^+);
3. no FD is **redundant** (derivable from the others).

It is the input to 3NF synthesis (§11.14). **Worked example:** F = { `A -> BC`, `B -> C`, `AB -> D` }

1. Split: `A -> B`, `A -> C`, `B -> C`, `AB -> D`.
2. In `AB -> D`, B is extraneous: `A+` = {A, B, C, D} contains D → `A -> D`.
3. `A -> C` follows from `A -> B`, `B -> C` → remove.

```
=== minimal cover of {A->BC, B->C, AB->D}: {A}->{B}, {B}->{C}, {A}->{D}
```

A minimal cover is not always unique (removal order can matter), but every one implies exactly F^+.

TL;DR

Minimal cover = singleton right sides, no extraneous left attributes, no redundant FDs; equivalent to F. Needed for 3NF synthesis.

11.6 Anomalies — why normalization exists

The problem it solves

The wide table stores "Nadia lives in Lisbon" three times and "P1 is a Kettle at 29.00" twice. That causes three failures:

Anomaly	What goes wrong	In the running table
Insertion	cannot record one fact without inventing an unrelated one	a product nobody has ordered has no row — the key needs an `order_id`
Update	one real-world change must be repeated on many rows; miss one and the table contradicts itself	Nadia moves: 3 rows must change
Deletion	deleting one fact silently deletes another	cancel Priya's only order and the shop forgets Priya *and* the tea tin



`code/01_anomalies.sql` :

```sql
-- INSERT: a new product with no order
INSERT INTO order_line_wide (order_id, product_id, ..., product_name, list_price, qty)
VALUES (NULL, 'P5', ..., 'Milk jug', 12.00, 1);              -- inside a DO block that catches the error
-- UPDATE: Nadia moves, but a careless WHERE edits only one of her rows
UPDATE order_line_wide SET customer_city = 'Porto' WHERE customer_id = 'C1' AND order_id = 'O3';
-- DELETE: cancel O4
DELETE FROM order_line_wide WHERE order_id = 'O4';
```

Output:

```
--- the same facts are stored many times
 customer_id | customer_name | customer_city | rows_holding_it
-------------+---------------+---------------+-----------------
 C1          | Nadia Lopez   | Lisbon        |               3
 C2          | Jonas Berg    | Oslo          |               1
 C3          | Priya Das     | Kochi         |               1
(3 rows)

--- INSERT anomaly: a new product nobody has ordered yet has nowhere to go
NOTICE:  refused: null value in column "order_id" of relation "order_line_wide" violates not-null constraint
--- UPDATE anomaly: Nadia moves to Porto, but the update misses one of her rows
 customer_id | cities_on_file
-------------+----------------
 C1          | {Lisbon,Porto}
(1 row)

--- DELETE anomaly: cancel order O4 and customer C3 and product P4 vanish with it
 rows_about_priya | rows_about_tea_tin
------------------+--------------------
                0 |                  0
(1 row)
```

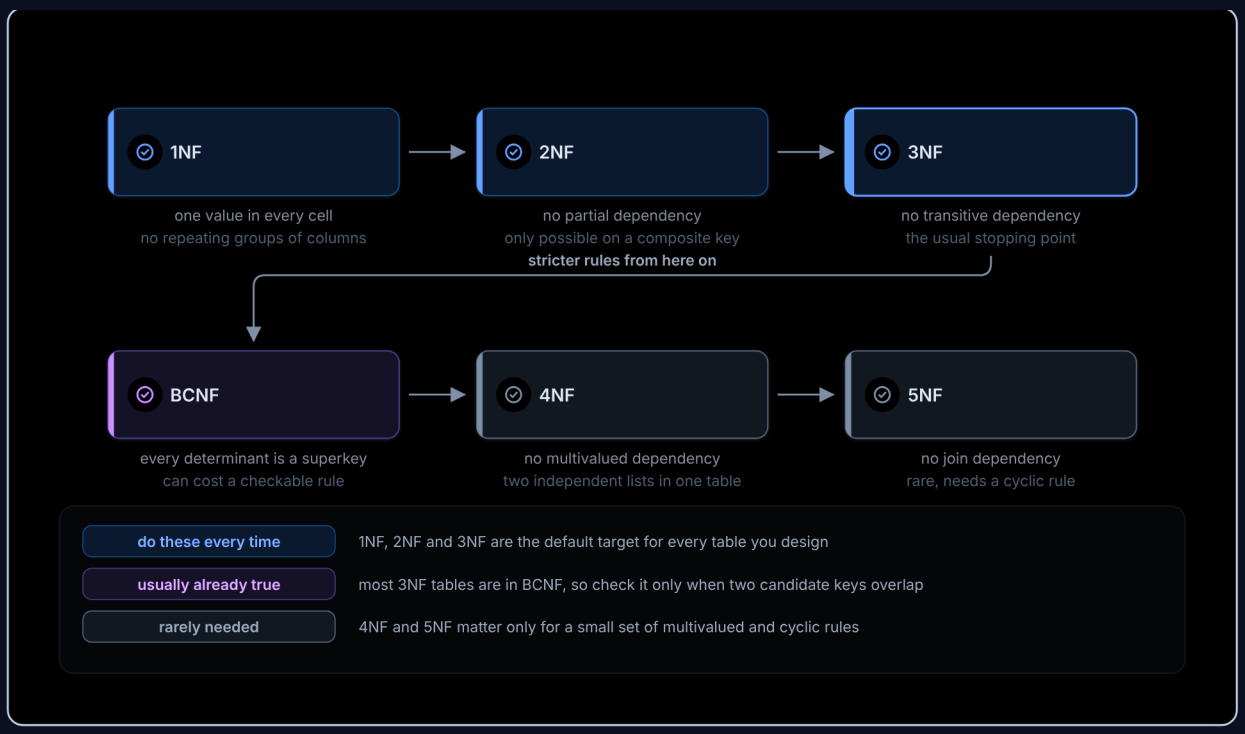
What normalization does

Normalization splits a table along its FDs so **every fact has exactly one home** — one row to update, nothing unrelated to delete, every fact insertable on its own. It uses **decomposition** into projections (a *projection* = some of the columns, duplicates removed) that must be **lossless** (§11.10).

The ladder of normal forms

Each **normal form** is a pass/fail test; each assumes the previous one and is stricter:

Form	Removes	One-line rule	Origin
1NF	lists and repeating groups in cells	every cell holds one atomic value	Codd 1970
2NF	partial dependencies	no non-prime attribute depends on part of a candidate key	Codd 1971
3NF	transitive dependencies	no non-prime attribute depends on a non-superkey	Codd 1971
BCNF	remaining FD redundancy	every non-trivial FD's determinant is a superkey	Boyce & Codd 1974
4NF	independent multivalued facts	every non-trivial MVD's determinant is a superkey	Fagin 1977
5NF	join dependencies not implied by keys	the table cannot be split losslessly except via its keys	Fagin 1979



(Domain-key NF and 6NF — used in temporal databases, which record how values change over time, and some data-warehouse styles — are beyond everyday design.)

You do not always climb to the top. Each step removes redundancy and usually adds a join. **3NF is where most designs stop**: it removes the redundancy behind the worst anomalies and always has a safe decomposition (§11.14). Putting redundancy back on purpose is **denormalization** (§11.16).

TL;DR

Redundancy → insertion, update and deletion anomalies. Normalization decomposes tables along FDs so each fact is stored once. 1NF → 2NF → 3NF → BCNF → 4NF → 5NF, each stricter; most designs stop at 3NF.

11.7 First normal form (1NF)

What it is

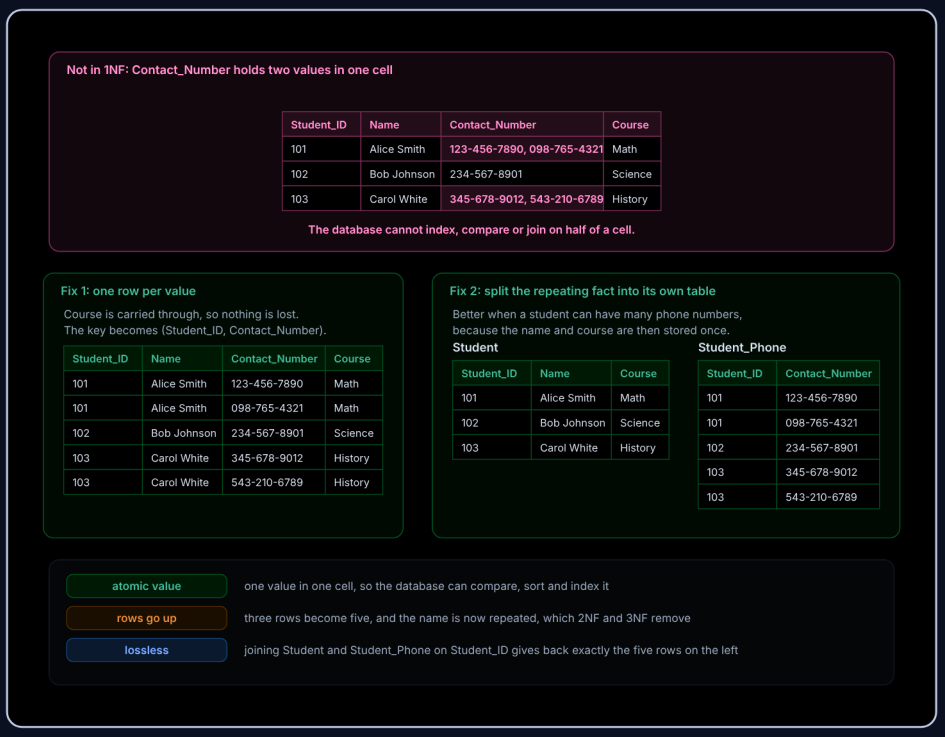
A table is in **1NF** when (1) every cell holds a **single, indivisible (atomic)** value — no lists, sets or nested records; (2) there are **no repeating groups** (`phone1, phone2, phone3`); (3) rows are distinguishable — it has a key. It is purely about the **shape of a cell**.

The problem it solves

```
vol_id | name  | languages
     1 | Hana  | Japanese, English
     3 | Ama   | Twi, English, French
```

"Who speaks English?" needs `LIKE '%English%'`, which also matches "Old English" or a typo; you cannot count, index usefully or constrain spelling.

How to fix it — two routes



1. **One row per value** (key becomes `(vol_id, language)`) — reaches 1NF but copies the other columns onto every row.
2. **A separate table** — `volunteer(vol_id, name)` + `volunteer_language(vol_id, language)`. This is what 2NF would produce from route 1 anyway, and what §10.6 predicted for a multivalued attribute.

`code/03_first_normal_form.sql` repairs both kinds of violation (*unpivoting* turns columns into rows; `CROSS JOIN LATERAL` lets `VALUES` read the current row):

```
-- a list in one cell -> one row per value (string_to_table: PostgreSQL 14+)
CREATE TABLE volunteer_language AS
SELECT vol_id, trim(lang) AS language
FROM volunteer_packed, string_to_table(languages, ',') AS lang;
ALTER TABLE volunteer_language ADD PRIMARY KEY (vol_id, language);

-- a repeating group phone1..phone3 -> rows (unpivot with LATERAL VALUES)
SELECT g.guest_id, g.name, p.phone
FROM guest_sideways g
CROSS JOIN LATERAL (VALUES (g.phone1), (g.phone2), (g.phone3)) AS p(phone)
WHERE p.phone IS NOT NULL
ORDER BY g.guest_id, p.phone;
```

Output:

```
--- repair: one row per value (string_to_table needs PostgreSQL 14+)
 vol_id | language
--------+----------
      1 | English
      1 | Japanese
      2 | Spanish
      3 | English
      3 | French
      3 | Twi
(6 rows)

--- now counting and exact matching are ordinary SQL
 ...
--- unpivot the phone columns into rows; guests without a phone produce no rows
 guest_id | name  |  phone
----------+-------+----------
       10 | Ivo   | 555-0101
       10 | Ivo   | 555-0102
       12 | Malik | 555-0301
       12 | Malik | 555-0303
(4 rows)
```

Dialect note (not run): PostgreSQL < 14 `unnest(string_to_array(languages, ','))` ; MySQL 8 `JSON_TABLE` or a recursive CTE; SQL Server `STRING_SPLIT` and `CROSS APPLY (VALUES …)`.

Misconceptions and borderline cases

- **1NF does not require unique column values** — only keys are unique.
- **1NF does not reduce redundancy — route 1 increases it.** Removing repetition is 2NF/3NF's job.
- A normalization step **never drops a column**; if one disappears, information was lost.
- **Structured text** (a whole address in one column) is a 1NF problem only if you query the parts.
- **JSON / arrays** are strictly non-atomic. PostgreSQL has first-class `jsonb` (parsed binary JSON) and indexable arrays (Chapter 14); storing a document *on purpose* (sparse attributes most rows lack, payloads you never filter on) is legitimate. If you search, join or constrain on what is inside, it belongs in rows.
- **Common mistakes:** comma-separated ids (`tag_ids = '3,7,9'`) — *SQL Antipatterns*' **Jaywalking**: no joins, no FKs; numbered columns (`item1…item5`); believing 1NF means "no duplicate values".

TL;DR

1NF = atomic cells, no repeating groups, a key. Fix lists with one row per value or (better) a child table. 1NF does not remove redundancy and does not require unique values.

11.8 Second normal form (2NF)

What it is

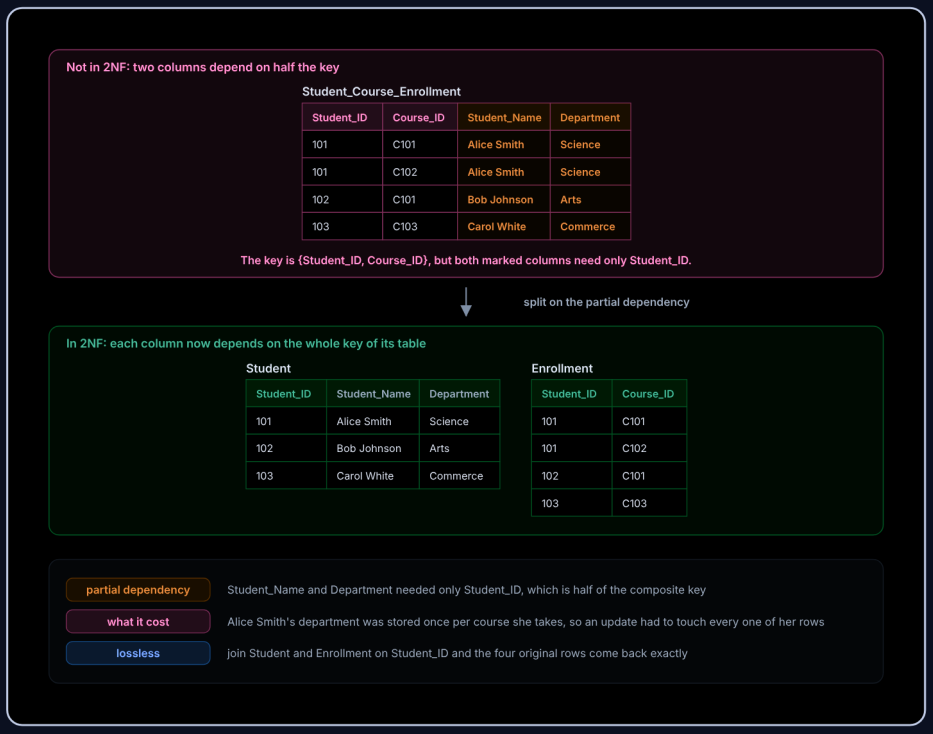
2NF = 1NF **and no non-prime attribute is partially dependent on any candidate key** — every non-key column depends on the *whole* key. A table whose candidate keys all have **one column** is automatically in 2NF.

Finding the violation

Key `{order_id, product_id}` :

Column	Needs	Verdict
`qty`	order **and** product	full ✓
`order_date` , `customer_id`	`order_id` only	partial ✗
`customer_name` , `customer_city`	`order_id` only (via customer_id)	partial ✗
`product_name` , `list_price`	`product_id` only	partial ✗

```
candidate keys: {order_id, product_id}
2NF violations: ['{order_id} -> {customer_city, customer_id, customer_name, order_date}', '{product_id} ->
{list_price, product_name}']
```



How to fix it

One new table per key part that has dependents (dependents grouped by the part that determines them), plus a table for the full-key columns:

- `order_header(order_id, order_date, customer_id, customer_name, customer_city)`
- `product(product_id, product_name, list_price)`
- `order_item(order_id, product_id, qty)`

`order_header` still repeats Nadia's city once per order — a *transitive* dependency, invisible to 2NF.

Common mistakes: moving only one part's dependents (still not 2NF); checking only the primary key — 2NF concerns **every candidate key** and **non-prime** attributes (Exercise 1).

TL;DR

2NF = 1NF + no non-prime column determined by part of a candidate key. Only composite keys can violate it. Fix: one table per key part with its dependents, plus the full-key table.

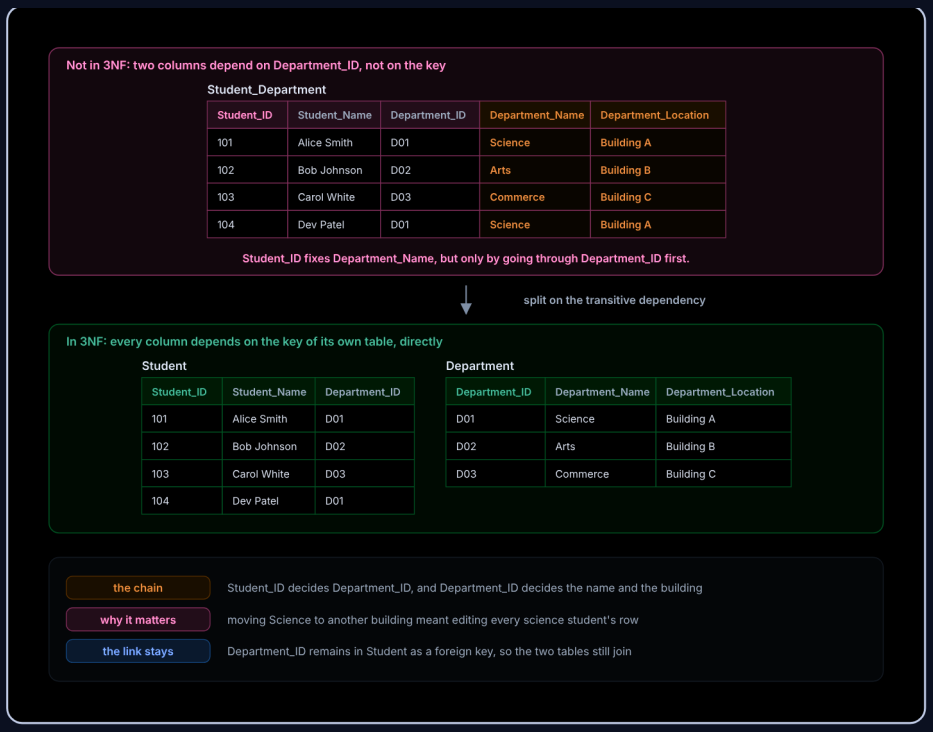
11.9 Third normal form (3NF)

What it is

3NF = 2NF **plus no non-prime attribute transitively dependent on a candidate key** — "the key, the whole key, and nothing but the key" (mnemonic usually credited to Bill Kent). The precise form (Zaniolo's), worth knowing for interviews: **for every non-trivial FD `X -> A`, either X is a superkey, or A is prime.** It implies 2NF.

Finding the violation

`order_header` has a single-column key (so 2NF). But `customer_id -> customer_city`: is `customer_id` a superkey here? No (Nadia has three orders). Is `customer_city` prime? No. Both escapes fail → **not 3NF**.



How to fix it

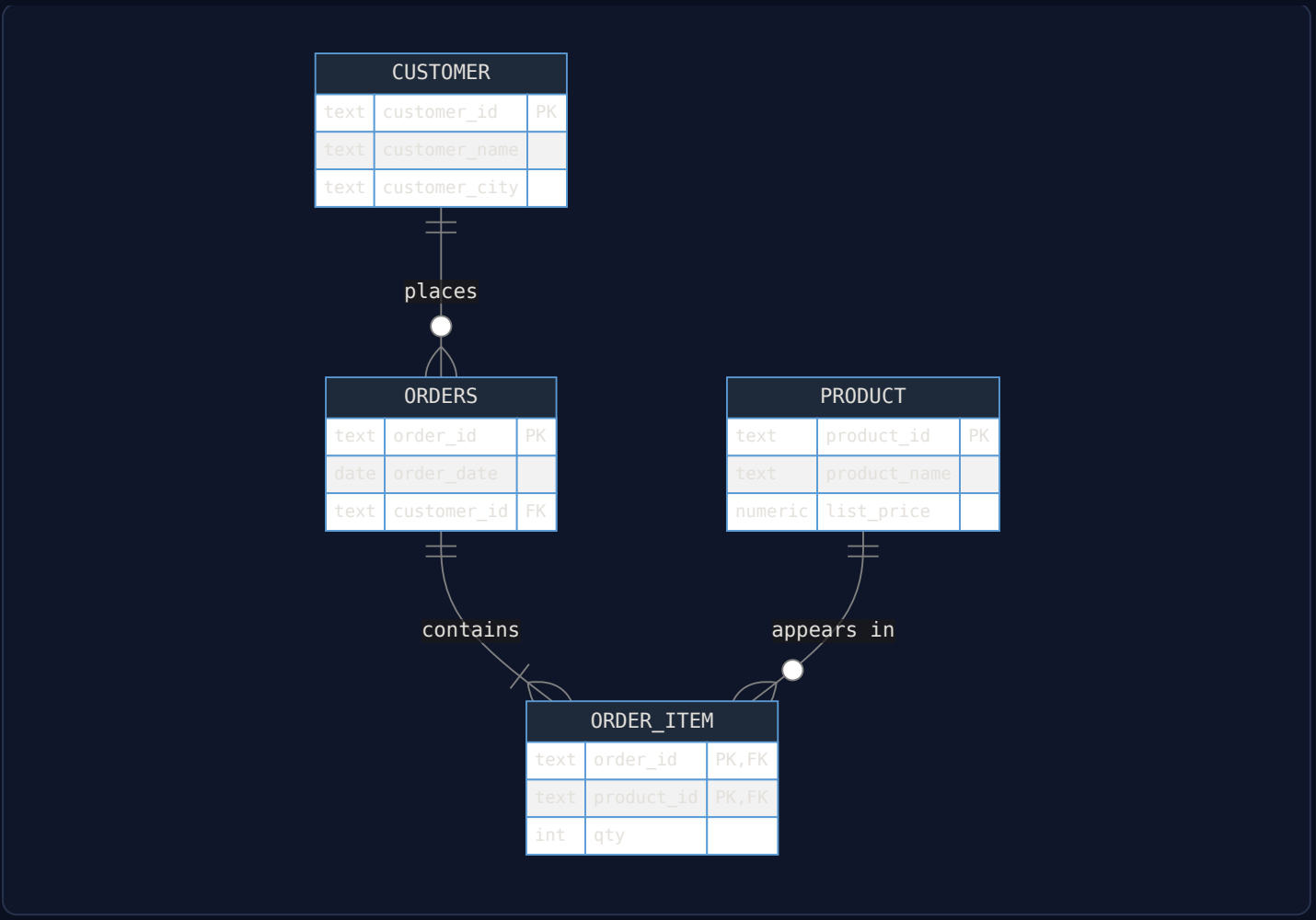
Move the **far end of the chain** into a table keyed by the **middle** column, which stays behind as an FK (otherwise nothing to join on):

- `customer(customer_id, customer_name, customer_city)`
- `orders(order_id, order_date, customer_id → customer)`

	Dependency runs through	Possible when
2NF violation	**part** of a candidate key	the key is composite
3NF violation	a **non-key** column	any key shape, including one column

So 3NF must be checked on every table; 2NF only with composite keys. The 3NF synthesis algorithm (§11.14) produces exactly our four tables:

```
3NF synthesis: ['{customer_id, order_date, order_id}', '{customer_city, customer_id, customer_name}',
'{list_price, product_id, product_name}', '{order_id, product_id, qty}']
```



In the interview

30-second answer: "1NF: atomic values. 2NF: no non-key attribute depends on part of a composite key. 3NF: no non-key attribute depends on another non-key attribute — formally, for every non-trivial FD X → A, X is a superkey or A is prime. I normalize to 3NF by default because it removes the update anomalies and there's always a lossless, dependency-preserving 3NF decomposition."

- *A 3NF violation with a single-column key?* → `employee(emp_id, dept_id, dept_name)` : `dept_id -> dept_name`.
- *Is a table with only key columns in 3NF?* → Yes (no non-prime attributes); check BCNF separately.

TL;DR

3NF = 2NF + no non-key → non-key dependency (formally: every FD's LHS is a superkey or RHS is prime). Fix: split the chain's tail into a table keyed by the middle column, which stays behind as an FK.

11.10 Decomposition done right: lossless joins and dependency preservation

Lossless (non-additive) join — mandatory

Joining the pieces must return **exactly** the original rows. A lossy split usually *adds* rows (**spurious tuples**); what is lost is which combinations were true. **Two-way test:** splitting R into R1, R2 is lossless iff `R1 ∩ R2` is a **superkey of R1 or of R2** (`R1 ∩ R2 -> R1` or `-> R2` is in F^+).

`code/04_lossy_join.sql` : `staff(emp, dept, city)` with `emp -> dept, city` :

```sql
-- bad: (emp, city) + (dept, city); city is a key of neither piece
WITH ec AS (SELECT DISTINCT emp, city FROM staff),
     dc AS (SELECT DISTINCT dept, city FROM staff)
SELECT ec.emp, dc.dept, ec.city,
       (ec.emp, dc.dept, ec.city) IN (SELECT emp, dept, city FROM staff) AS real_row
FROM ec JOIN dc USING (city) ORDER BY 1, 2;
```

Output:

```
--- bad split: (emp, city) and (dept, city). The shared column city is a key of neither piece.
 emp |  dept   | city  | real_row
-----+---------+-------+----------
 Ana | Sales   | Leeds | t
 Ana | Support | Leeds | f
 Ben | Sales   | Leeds | f
 Ben | Support | Leeds | t
 Cai | Sales   | York  | t
(5 rows)

--- good split: (emp, dept) and (emp, city). The shared column emp is a key of both.
 invented_rows
---------------
             0
(1 row)
```

Dependency preservation — strongly desirable

Every original FD should be checkable **inside one table** (so a `PRIMARY KEY` / `UNIQUE` enforces it), or be implied by FDs that are. Otherwise enforcement needs a join — a trigger (code the database runs on every change) or application code. §11.11 shows a lossless split that loses one.

The running example decomposed (`code/02_decompose_and_fix.sql`)

```sql
CREATE TABLE customer AS SELECT DISTINCT customer_id, customer_name, customer_city FROM order_line_wide;
CREATE TABLE product  AS SELECT DISTINCT product_id, product_name, list_price     FROM order_line_wide;
CREATE TABLE orders   AS SELECT DISTINCT order_id, order_date, customer_id        FROM order_line_wide;
CREATE TABLE order_item AS SELECT order_id, product_id, qty                       FROM order_line_wide;
-- then ADD PRIMARY KEY to each, and FOREIGN KEYs from orders and order_item (see the file)
-- lossless check: (original EXCEPT rejoined) and (rejoined EXCEPT original) must both be empty
```

Output:

```
--- row counts after the split
 customers | products | orders | order_items
-----------+----------+--------+-------------
         3 |        4 |      4 |           5
(1 row)

--- lossless? join the pieces back and compare with the original, both directions
 missing_rows | invented_rows
--------------+---------------
            0 |             0
(1 row)

--- INSERT: a product with no orders is now fine
--- UPDATE: Nadia moves once, in one row
 customer_city
---------------
 Porto
(1 row)

--- DELETE: cancel O4; Priya and the tea tin survive
 priya_rows | tea_tin_rows | o4_items
------------+--------------+----------
          1 |            1 |        0
(1 row)
```

All three anomalies are gone, and each of F1–F4 lives inside one table. `SELECT DISTINCT` + `ADD PRIMARY KEY` is the standard way to split live data — if `ADD PRIMARY KEY` fails, the data already contradicts the assumed FD.

TL;DR

Lossless: shared columns must be a key of one side (else the join invents rows). Dependency-preserving: each FD checkable in one table. Verify a split with `EXCEPT` both ways.

11.11 Boyce–Codd normal form (BCNF)

What it is

BCNF: the left side of **every non-trivial FD `X -> Y` is a superkey** — "every determinant is a (super)key". It drops 3NF's second escape ("…or Y is prime"), so every BCNF table is in 3NF, not vice versa; sometimes called "3.5NF". **3NF passes but BCNF fails only with several composite candidate keys that overlap** — with a single candidate key they coincide.

A 3NF table that is not in BCNF

Clinic rules: each doctor has one specialty (`doctor -> specialty`); a patient sees one doctor per specialty (`{patient, specialty} -> doctor`).

patient	specialty	doctor
Ali	Cardiology	Dr Osei
Ali	Dermatology	Dr Lind
Beth	Cardiology	Dr Osei
Beth	Neurology	Dr Tan
Chen	Cardiology	Dr Moreau

```
=== visit (3NF, not BCNF)
candidate keys: {doctor, patient}, {patient, specialty}
prime: {doctor, patient, specialty}  non-prime: {}
2NF violations: none
3NF violations: none
BCNF violations: ['{doctor} -> {specialty}']
highest normal form: 3NF
BCNF decomposition: ['{doctor, specialty}', '{doctor, patient}']
FDs no longer checkable in one piece: ['{patient, specialty} -> {doctor}']
```

Both keys are composite and overlap on `patient`, so every attribute is prime: `doctor -> specialty` passes 3NF (specialty is prime) but fails BCNF (doctor is not a superkey). The redundancy is real (`code/05_bcnf.sql`):

```
--- the fact "Dr Osei is a cardiologist" is stored once per patient
  doctor   |  specialty  | copies
-----------+-------------+--------
 Dr Osei   | Cardiology  |      2
 ...
--- nothing stops the copies disagreeing (update anomaly)
 doctor  |      specialties
---------+----------------------
 Dr Osei | {Cardiology,Oncology}
(1 row)
```

Decomposing, and the cost

Split on the offending FD: `doctor_specialty(doctor PK, specialty)` + `patient_doctor(patient, doctor)` (PK both). Shared `doctor` is the key of `doctor_specialty` → lossless (`invented_rows 0, missing_rows 0`). But `{patient, specialty} -> doctor` now spans two tables, so no single key enforces it:

```
INSERT INTO patient_doctor VALUES ('Ali', 'Dr Moreau');     -- accepted!
```

Output:

```
--- the cost: {patient, specialty} -> doctor is no longer checked by any one table
 patient | specialty  |         doctors
---------+------------+-------------------------
 Ali     | Cardiology | {"Dr Moreau","Dr Osei"}
(1 row)
```

Ali now has two cardiologists. **General result:** a **3NF** decomposition that is both lossless and dependency-preserving always exists; for **BCNF** it is not guaranteed — why practitioners usually stop at 3NF.

Practical middle way (also in `05_bcnf.sql`): keep the 3NF table (`{patient, specialty}` stays PK), add a master `doctor_specialty` table and a **composite foreign key** `(doctor, specialty) → doctor_specialty(doctor, specialty) ON UPDATE CASCADE`. Copies can no longer disagree, and one update propagates:

```
NOTICE:  refused: insert or update on table "visit_3nf" violates foreign key constraint
"visit_3nf_doctor_specialty_fkey"
 patient |    specialty    | doctor
---------+-----------------+---------
 Ali     | Cardiac surgery | Dr Osei
 Beth    | Cardiac surgery | Dr Osei
(2 rows)
```

The trap: "looks like BCNF, is really 2NF"

`enrol(student, course, instructor)` , `course -> instructor` , key `{student, course}` : `instructor` is **non-prime** and hangs off **half** the key → not even 2NF (`2NF violations: ['{course} -> {instructor}'] … highest normal form: 1NF`). **Rule of thumb:** non-prime dependent of a proper subset of a key → 2NF problem; only when every attribute involved is prime and a determinant still is not a superkey is it a genuine BCNF problem.

In the interview

30-second answer: "BCNF says every determinant must be a superkey. It's stricter than 3NF, which also allows X → A when A is prime. The two only differ for tables with several composite candidate keys that overlap. BCNF decompositions are always lossless but not always dependency-preserving, so sometimes I stay at 3NF and enforce the rule with a composite foreign key."

- *Example of 3NF-not-BCNF?* → Patient/specialty/doctor with doctor → specialty.
- *What's lost in the BCNF split?* → {patient, specialty} → doctor can't be enforced by one table's key.

TL;DR

BCNF: every non-trivial FD's LHS is a superkey. Differs from 3NF only with overlapping composite candidate keys. Split on the violating FD (lossless), but you may lose dependency preservation.

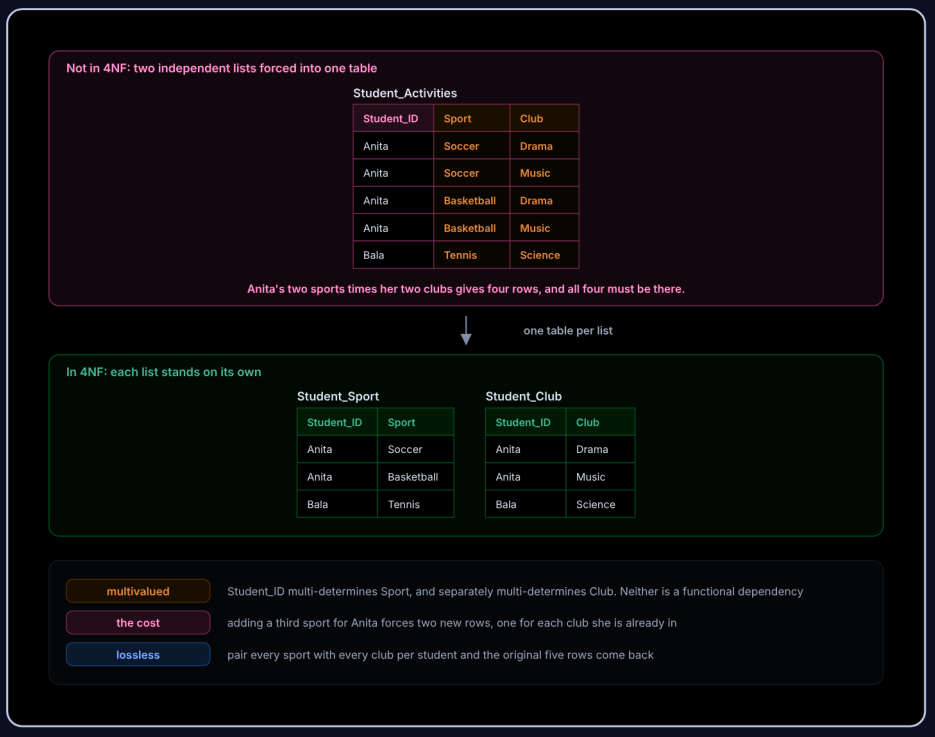
11.12 Fourth normal form (4NF) — independent multivalued facts

What it is

4NF handles a non-FD redundancy: two **independent lists** about one thing in one table. A restaurant's dishes and its separately chosen delivery zones, stored together, force every dish to pair with every zone:

restaurant	dish	zone
Saffron	Biryani	North
Saffron	Biryani	Centre
Saffron	Dal	North
Saffron	Dal	Centre
Nori	Ramen	East

`restaurant` does not determine `dish`; what holds is a **multivalued dependency (MVD)** `restaurant ->> dish`: for a restaurant, the set of dishes is the same whatever zone you look at (and symmetrically `restaurant ->> zone` — MVDs come in complementary pairs). The only key is all three columns, so the table is in BCNF — and still redundant.



Rule: 4NF = BCNF and **for every non-trivial MVD `X ->> Y`, X is a superkey** — in practice, **one table per independent list**.

The signature and the damage (`code/06_4nf_5nf.sql`)

A **full cross product** per key value (2 dishes × 2 zones = 4 rows). Adding a dish needs one row *per zone*; forget one and the table lies:

```
--- restaurant ->> dish | zone: every dish is paired with every zone (a full cross product)
 restaurant | dishes | zones | rows
------------+--------+-------+------
 Nori       |      1 |     1 |    1
 Saffron    |      2 |     2 |    4
(2 rows)

--- adding one dish to Saffron needs one row PER ZONE; forget one and the table lies
 restaurant |     dish     | zones_listed
------------+--------------+--------------
 Saffron    | Biryani      |            2
 Saffron    | Dal          |            2
 Saffron    | Paneer tikka |            1
(3 rows)

--- 4NF split: one table per independent list; the join rebuilds the original exactly
 dish_rows | zone_rows | invented_rows
-----------+-----------+---------------
         3 |         3 |             0
(1 row)
```

Five rows became 3 + 3, and each new fact is one row. The split is lossless because the lists are independent (an MVD `X ->> Y` is exactly the condition for splitting R into XY and X(R−Y) losslessly). **Caution:** if a dish is only delivered to some zones, the three-column table is correct and there is no MVD — the business rule decides, not the rows.

TL;DR

MVD `X ->> Y`: for each X, the set of Y values is independent of the other columns (visible as a full cross product). 4NF: every non-trivial MVD's LHS is a superkey → store each independent list in its own table.

11.13 Fifth normal form (5NF) — facts that need three tables

What it is

5NF (**project-join normal form, PJ/NF**): a table with no FD/MVD problems that can still be split into **three (or more)** projections that join back exactly, while **no two** of them can. The constraint is a **join dependency (JD)**. Classic shape — a **cyclic business rule**: *if a supplier supplies a part, the part is used in a project, and the supplier supplies that project, then the supplier supplies that part to that project.*

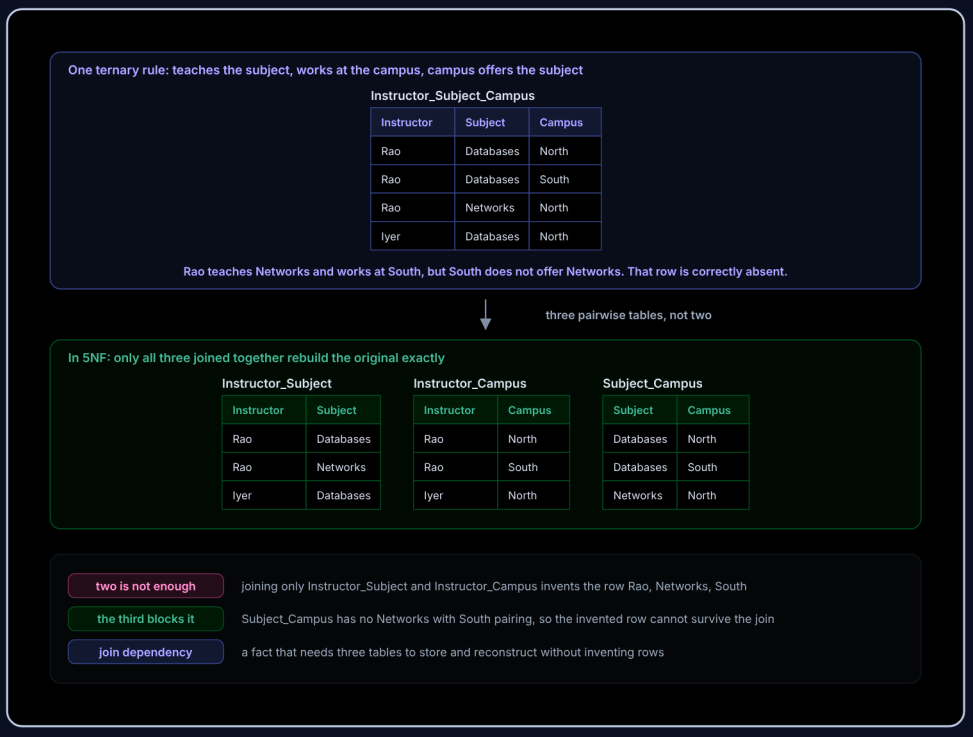
supplier	part	project
Acme	bolt	Bridge
Acme	bolt	Tower
Acme	nut	Tower
Brix	bolt	Tower

Of the pairwise projections supplier-part (`sp`), part-project (`pj`), supplier-project (`sj`), **every** two-way join invents a false row; the three-way join does not:

```
--- rows INVENTED by each two-way join (none of these is true)
 pieces | supplier | part | project
--------+----------+------+---------
 pj+sj  | Brix     | nut  | Tower
 sp+pj  | Brix     | bolt | Bridge
 sp+sj  | Acme     | nut  | Bridge
(3 rows)

--- the three-way join invents nothing: 5NF stores sp, pj and sj instead of spj
 invented_rows
---------------
             0
(1 row)
```

E.g. `sp+pj` claims Brix supplies bolts to Bridge; `sj` rules it out (Brix does not supply Bridge at all).



Rule: 5NF = every non-trivial join dependency is implied by the candidate keys. Practically: for a real cyclic rule, keep the **three two-column tables** and derive the combination by a three-way join; inserting one pairwise fact then implies the right combinations.

How often? Rarely — independent lists usually get separate tables from the ER diagram, and three-way rules are often simpler as a constraint or code. Know 4NF/5NF to **name** the problem and recognise the cross-product and invented-row signatures; proof is always the same — join the pieces, get exactly the original rows.

TL;DR

5NF (PJ/NF): no join dependency except those implied by keys. Signature: every two-way join of the projections invents rows but the three-way join doesn't → store the three pairwise tables.

11.14 Putting it together — a normalization procedure

1. **Write down the FDs** from business rules (not sample rows); split right sides to single attributes.
2. **Find all candidate keys** with closure (§11.4); mark prime / non-prime.
3. **1NF:** atomic cells, no repeating groups.
4. **2NF:** non-prime attribute determined by part of a candidate key? → split per key part.
5. **3NF:** FD `X -> A` with X not a superkey and A non-prime? → split the chain.

6. **BCNF** (only with overlapping composite keys): non-superkey LHS? → split, and check what you lose.
7. **4NF/5NF** (only on cross products / three-way rules).
8. **Verify every split:** lossless (shared columns key one side; `EXCEPT` both ways empty), ideally dependency-preserving, no column dropped.

Two textbook algorithms (both in `fd_tools.py`)

	3NF synthesis (Bernstein)	BCNF decomposition
Steps	1. minimal cover · 2. one table per group of FDs with the same LHS · 3. if no table contains a candidate key, add one · 4. drop tables contained in others	while some table has an FD X → Y with X not a superkey *of that table*: replace it by `X⁺` (within the table) and `(table − X⁺) ∪ X`
Lossless?	always	always
Dependency-preserving?	**always**	not always
Redundancy left?	possibly (only the 3NF-not-BCNF kind)	none from FDs

On Exercise 3's R(A,B,C,D,E), `A -> B, BC -> D, D -> A, C -> E`:

```
BCNF decomposition: ['{A, B}', '{C, E}', '{A, D}', '{C, D}']
FDs lost by it: ['{B, C} -> {D}']
3NF synthesis: ['{A, B}', '{B, C, D}', '{A, D}', '{C, E}']
```

BCNF loses `BC -> D`; 3NF keeps it (in `{B, C, D}`) at the cost of some redundancy — the whole BCNF-vs-3NF story in one line.

> **TL;DR**
>
> FDs → keys → check 1NF, 2NF, 3NF (BCNF/4NF/5NF when their signatures appear) → split → verify lossless + dependency-preserving. 3NF synthesis always preserves dependencies; BCNF decomposition may not.

11.15 Exercises (with solutions)

Try each on paper first. Every answer was checked with `fd_tools.py` (output quoted).

Exercise 1 — workshop sign-ups (target: 2NF, and beyond)

attendee_id	workshop_id	attendee_name	attendee_email	workshop_title	room	seat_no
A1	W1	Kofi	kofi@mail.example	Intro to SQL	R12	4
A1	W2	Kofi	kofi@mail.example	Indexing	R07	9
A2	W1	Mei	mei@mail.example	Intro to SQL	R12	5
A3	W3	Lars	lars@mail.example	Data modeling	R12	1

Rules: an attendee has one name and one email; **emails are unique per attendee**; a workshop has one title and one room; each attendee has one seat per workshop. Is it in 2NF? Normalize to 3NF.

Solution. FDs: `attendee_id -> attendee_name, attendee_email`; `attendee_email -> attendee_id`; `workshop_id -> workshop_title, room`; `{attendee_id, workshop_id} -> seat_no`. **The twist:** email identifies an attendee, so there are **two** candidate keys:

```
candidate keys: {attendee_email, workshop_id}, {attendee_id, workshop_id}
prime: {attendee_email, attendee_id, workshop_id}  non-prime: {attendee_name, room, seat_no, workshop_title}
2NF violations: ['{attendee_email} -> {attendee_name}', '{attendee_id} -> {attendee_name}', '{workshop_id} -> {room, workshop_title}']
highest normal form: 1NF
```

`attendee_email` is prime, so `attendee_id -> attendee_email` is not itself a 2NF problem — but `attendee_name`, `workshop_title`, `room` are non-prime and depend on halves of keys. **Not in 2NF.** 3NF synthesis gives three tables:

- `attendee(attendee_id PK, attendee_name, attendee_email UNIQUE)` — both keys enforced.
- `workshop(workshop_id PK, workshop_title, room)`
- `signup(attendee_id FK, workshop_id FK, seat_no, PK (attendee_id, workshop_id))`

All three are in BCNF; lossless because `signup` shares the key of each other table. **Common wrong answer:** splitting off only `attendee` (still partial on `workshop_id`). **Beyond the question:** if one person per seat, declare `UNIQUE (workshop_id, seat_no)` — a rule no normal form would tell you to add.

Exercise 2 — flights (target: 3NF, and question the assumption)

flight_id	flight_date	tail_number	aircraft_type	seat_capacity
9001	2026-10-01	G-EZAA	A320	186
9002	2026-10-01	G-EZBB	A320	180
9003	2026-10-02	G-EZAA	A320	186
9004	2026-10-02	EI-FJK	B737-800	189

Key `flight_id`. A flight uses one aircraft (tail number); each aircraft has one type and one capacity. 2NF? 3NF? Normalize.

Solution. 2NF: yes, automatically (single-column key, atomic cells). FDs: `flight_id -> flight_date, tail_number` ; `tail_number -> aircraft_type, seat_capacity` ; chain `flight_id -> tail_number -> aircraft_type`, and `tail_number` is not a key here (G-EZAA flies 9001 and 9003) → **transitive** → **not 3NF**.

```
2NF violations: none
3NF violations: ['{tail_number} -> {aircraft_type}', '{tail_number} -> {seat_capacity}']
highest normal form: 2NF
BCNF decomposition: ['{aircraft_type, seat_capacity, tail_number}', '{flight_date, flight_id, tail_number}']
```

Tables: `aircraft(tail_number PK, aircraft_type, seat_capacity)`, `flight(flight_id PK, flight_date, tail_number FK)`. A cabin refit is one update; a new aircraft can be recorded before its first flight.

Question the assumption: `aircraft_type -> seat_capacity` (a third table `aircraft_type(type, capacity)`) is refuted by the data — G-EZAA and G-EZBB are both A320s with 186 and 180 seats; capacity depends on each airframe's layout. Had the business said "every A320 has 186 seats", the third table would be right: **a dependency is a claim about the world** — in an interview, state which assumptions your answer depends on.

Exercise 3 — abstract relation (keys, closure, BCNF vs 3NF)

R(A, B, C, D, E), F = { `A -> B`, `BC -> D`, `D -> A`, `C -> E` }. (a) All candidate keys. (b) Highest normal form. (c) BCNF decomposition and what it loses. (d) 3NF decomposition.

Solution. (a) `C` never on a right side; `C⁺ = {C, E}` ; `{A,C}⁺`, `{B,C}⁺`, `{C,D}⁺` are all of R → **keys AC, BC, CD**. (b) Prime {A, B, C, D}, non-prime {E}; `C -> E` is partial → **only 1NF**:

```
candidate keys: {A, C}, {B, C}, {C, D}
2NF violations: ['{C} -> {E}']
3NF violations: ['{C} -> {E}']
BCNF violations: ['{A} -> {B}', '{D} -> {A}', '{C} -> {E}']
highest normal form: 1NF
```

`A -> B` and `D -> A` are fine for 3NF (B and A are prime) but violate BCNF. (c) `{A,B}`, `{C,E}`, `{A,D}`, `{C,D}` — lossless, but **`BC -> D` is lost**. (d) `{A,B}`, `{B,C,D}`, `{A,D}`, `{C,E}` — `{B,C,D}` contains key BC (lossless) and every FD lives in one table.

Exercise 4 — quick-fire: keys and highest normal form

Find the keys and highest normal form of Q1–Q4 (relations and FDs in the first columns below), and of Q5: Tutor(tutor, subject, language), where a tutor's subjects and teaching languages are independent.

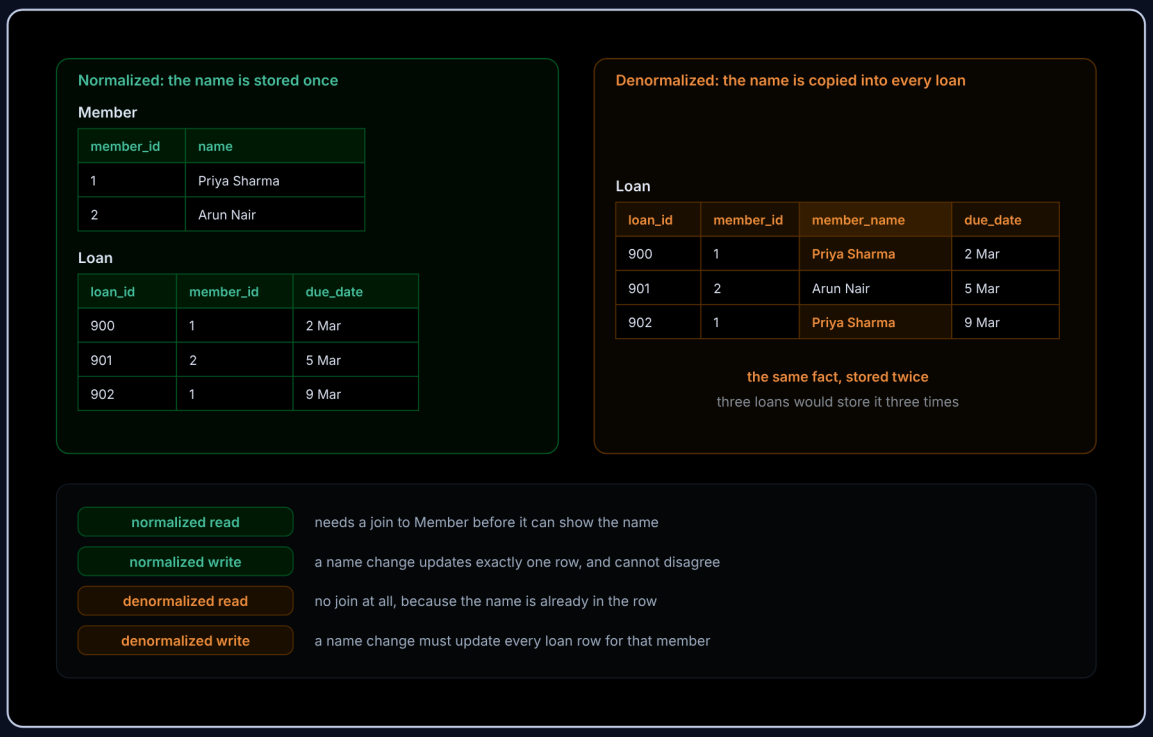
```
Q1 R(A,B,C)  A->B, B->C          keys: {A}              highest NF: 2NF
Q2 R(A,B,C)  AB->C, C->B         keys: {A, B}, {A, C}   highest NF: 3NF
Q3 R(A,B,C,D)  AB->C, B->D       keys: {A, B}           highest NF: 1NF
Q4 R(A,B,C)  A->B, B->A, A->C    keys: {A}, {B}         highest NF: BCNF
```

- Q1: transitive A → B → C (single key, so 2NF holds).
- Q2: the textbook 3NF-not-BCNF shape — `C -> B`, B prime, C not a superkey.
- Q3: `B -> D` is partial.
- Q4: A and B are both keys; every determinant is a key.
- Q5: no non-trivial FDs, key = all three → BCNF; but `tutor ->> subject` (and `->> language`) has a non-superkey LHS → **BCNF, not 4NF**. Split into `tutor_subject` and `tutor_language`.

11.16 Denormalization — when and how to break the rules

What it is

Denormalization deliberately adds redundancy back into a normalized design to make important reads faster or simpler, accepting extra write work and a consistency risk. It is a measured trade, not "skipping normalization": you know exactly which redundancy you add and how it stays correct.



	Normalized	Denormalized
Each fact stored	once	more than once
Writes	touch one row; cannot disagree	touch several places; can drift if not maintained
Reads	may need joins / aggregation	fewer joins; precomputed answers
Storage	smaller	larger
Best for	OLTP (online transaction processing: many small writes)	read-heavy, reporting, analytics (OLAP, online analytical processing)

Common techniques

Technique	Example	Keeping it correct
Stored summary / counter column	`orders.item_count`, a post's `like_count`	trigger in the same transaction, or batch recompute
Materialized view (a saved query result)	nightly sales-by-product report	`REFRESH MATERIALIZED VIEW` (stale in between; `CONCURRENTLY` refreshes without blocking readers, needs a unique index)
Duplicated column to avoid a join	`customer_name` copied onto `orders`	trigger or application code on every change — the riskiest option
Pre-joined / wide tables	**star schema** dimensions in a warehouse (Kimball-style dimensional modeling: a narrow *fact* table of events surrounded by wide *dimension* tables)	rebuilt by **ETL** (extract, transform, load: batch copy-reshape-load into the warehouse), not updated in place
Caches outside the DB	Redis (an in-memory store) copy of a product page	expiry / invalidation (System Design Fundamentals)

Running it (`code/07_denormalization.sql`)

```
CREATE TABLE orders (order_id text PRIMARY KEY, customer text NOT NULL,
                     item_count int NOT NULL DEFAULT 0);   -- DENORMALIZED: derivable from order_item
CREATE TABLE order_item (
  order_id   text REFERENCES orders ON DELETE CASCADE,
  product_id text REFERENCES product,
  qty        int  NOT NULL CHECK (qty > 0),
  price_paid numeric(8,2) NOT NULL,   -- NOT redundancy: the price at order time is its own fact
  PRIMARY KEY (order_id, product_id)
);
-- AFTER INSERT/UPDATE OF qty/DELETE trigger adjusts orders.item_count in the same transaction
CREATE MATERIALIZED VIEW sales_by_product AS
SELECT p.name, sum(oi.qty) AS units, sum(oi.qty * oi.price_paid) AS revenue
FROM order_item oi JOIN product p USING (product_id) GROUP BY p.name;
```

Output:

```
--- stored item_count vs the value recomputed from order_item
 order_id | stored | recomputed
----------+--------+------------
 O1       |      4 |          4
 O2       |      2 |          2
(2 rows)

--- the list price changes later; price_paid on old orders must not
 order_id |  name  | price_paid | todays_price
----------+--------+------------+--------------
 O1       | Kettle |      29.00 |        31.00
 O2       | Kettle |      27.00 |        31.00
(2 rows)

--- after a new sale, before REFRESH (stale copy)
  name  | units | revenue
--------+-------+---------
 Kettle |     3 |   83.00
 Mug    |     3 |   19.50
(2 rows)

--- after REFRESH
  name  | units | revenue
--------+-------+---------
 Kettle |     3 |   83.00
 Mug    |    13 |   84.50
(2 rows)
```

Not everything that looks redundant is. `price_paid` looks like a copy of `list_price`, but its FD is `{order_id, product_id} -> price_paid` — the price *on that day*, a different fact. A real shop stores both. Ask: *the same fact, or a snapshot of it?*

When to denormalize — and when not

1. Normalize first (3NF).
2. Find a specific, important query too slow **on realistic data**, after indexes, query rewrites and caching failed.
3. Add the smallest redundancy that fixes it, **with an enforced maintenance path** (trigger, MV refresh, ETL), documenting which query it serves.

Not: on write-heavy tables (every write pays), when data must be exactly consistent and no reliable maintenance exists, or "because joins are slow" — joins on indexed keys are what relational databases are built for (Chapter 12).

Real-world examples: cloud warehouses (Snowflake, BigQuery, Redshift) use star schemas because data is batch-loaded and read by many analytical queries; social feeds store like/follower counters; wide-column and document ("NoSQL", Chapter 16) stores are built around query-shaped, denormalized data.

In the interview

30-second answer: "I normalize to 3NF by default so every fact has one home. I denormalize only for a measured read bottleneck that indexing didn't solve — typically a counter column maintained by a trigger, a materialized view for reports, or a star schema for analytics — and I make sure the redundant copy has a single, enforced maintenance path."

- *Is storing the order total denormalization?* → If recomputable from current data, yes; if it freezes historical prices, it is its own fact.
- *Materialized view vs trigger-maintained table?* → MV: cheap to read, simple, **stale until refreshed**; trigger: always exact, but adds a write per item change and a row lock on the order (other transactions changing it wait).

TL;DR

Denormalize = add controlled redundancy for reads, after normalizing and measuring. Use counters with triggers, materialized views or warehouse star schemas; every copy needs an enforced maintenance path. Snapshots (price paid) are facts, not redundancy.

Chapter 11 TL;DR

- **FDs are the foundation.** `X -> Y` is a business rule; data can disprove it, never prove it. X is a superkey iff X^+ is everything.
- **Tools:** Armstrong's axioms (sound, complete), attribute closure (fire only fully-present LHS; loop), candidate keys (start with attributes never on a right side), minimal covers.
- **Anomalies** (insert/update/delete) come from storing a fact more than once; normalization decomposes tables so each fact lives in one place.
- **1NF** atomic cells · **2NF** no non-prime on part of a key · **3NF** every FD's LHS is a superkey or RHS is prime · **BCNF** every determinant is a superkey · **4NF** no independent lists (MVDs) · **5NF** no join dependencies beyond keys.
- **Every split must be lossless** (shared columns = key of one side; check with `EXCEPT` both ways) and ideally dependency-preserving. 3NF can always have both; BCNF sometimes loses an FD — which is why 3NF is the default stopping point.
- **Denormalize** only for a measured read problem, with an enforced way to keep copies correct; snapshots such as `price_paid` are separate facts, not redundancy.
- All terms are defined in the glossary. Next: Chapter 12 makes the joins normalization creates fast; Chapter 17 applies Chapters 10–11 to full designs.